目录

基本概念.....2
报表.....2
报表设计器.....2
报表选项.....3
报表页面.....4
页面管理.....5
页面属性.....6
区域.....9
设计器区域.....10
配置区域.....11
打印栏位.....12
栏位属性.....13
报表对象.....13
通用对象属性.....14
文本.....15
多行文本.....20
图片.....21
线.....23
图形.....24
条码.....25
复选框.....28
表格.....28
交叉表.....29
图表.....30
邮编.....31
蜂窝文本.....31
第一个报表.....32
手动创建报表.....32
示例 2：用向导创建报表.....34
创建报表.....37
选择报表数据源.....37
动态布局.....37
CanGrow, CanShrink 属性.....38
ShiftMode 属性.....38
GrowToBottom 属性.....39
Anchor 属性.....39
Dock 属性.....40
格式.....40
边框和填充.....41
文本格式.....42

样式.....42
数据格式.....43
隐藏零值.....47
Hide duplicate 属性.....48
突出显示奇/偶行.....48
只有一个“数据”区域的报表.....49
连接区域到数据源.....49
打印文本.....49
数据分类.....49
数据过滤.....50
数据页首和页尾.....51
Breaking data and keeping it together.....52
打印空数据行.....54
打印“无数据”文本.....56
分层打印.....56
主-从报表.....59
主-主表.....61
主-细-子细报表.....63
分栏报表.....64
页面分栏.....64
数据区域分栏.....65
"册子"-类型报表.....66
添加页面.....66
页面设置.....66
打印奇偶页.....67
分组与合计.....68
创建分组.....70
数据排序.....72
嵌套分组.....73
管理分组.....74
打印合计.....75
重复分组页眉和分组页脚.....76
分组属性.....77
嵌套报表.....78
打印模式.....78
边-对-边嵌套报表.....79
嵌套子报表.....79
表格类型报表.....79
配置列.....79
管理列宽.....80
配置行.....80
管理行高.....80

配置单元格.....81
合并拆分单元格.....81
在单元格中插入对象.....81
打印表格.....82
打印复合表头.....84
使用合计.....86
设计表格.....86
范例.....87
交叉报表.....93
一些理论.....93
配置交叉报表.....95
配置表头.....96
配置单元格.....96
交叉表样式.....97
管理行和列的大小.....98
范例.....98
报表的交互性.....112
超链接.....112
配置超链接.....112
链接到 URL.....113
链接到页码.....113
链接到书签.....114
链接到明细报表.....114
链接到具体页面.....115
自定义链接.....116
报表大纲.....117
范例.....118
继承报表.....130
创建报表.....130
更改基表.....131
限制.....132
图表报表.....132
图表元素.....132
图表编辑器.....133
交互式图表.....144
数据.....146
“数据”窗口.....146
数据源.....147
创建数据源.....148
创建 SQL 查询.....152
查询参数.....154
将值传递给参数.....155

编辑连接.....156
编辑数据源.....156
别名.....156
数据源层次.....157
关系.....157
创建关系.....160
编辑关系.....161
系统变量.....161
函数.....162
数学.....163
文本.....166
日期 & 时间.....170
格式.....174
转换.....179
Program Flow.....184
合计.....185
创建合计.....185
合计条件.....187
运行合计.....188
页面合计.....188
在页首打印合计.....188
报表参数.....190
创建一个参数.....191
在报表中使用参数.....191
表达式.....192
表达式编辑器.....192
引用报表对象.....193
引用 .Net 函数.....193
引用数据元素.....194
引用数据源.....195
引用系统变量.....195
引用合计值.....196
引用报表参数.....196
脚本.....196
总说明.....198
事件处理程序.....199
报表事件.....199
使用 .Net 对象.....200
引用报表对象.....201
报表和引擎对象.....202
引用数据源.....205
引用系统变量.....205

引用合计值.....205
引用报表参数.....206
范例.....206
例 1 更改对象的外观.....206
例 2.突出显示区域的偶数行.....207
例 3.数据过滤.....208
例 4.计算合计.....209
例 5.移动打印位置.....209
对话框形式.....210
控件.....211
在代码中引用控件.....212
数据过滤.....213
自动过滤-它是如何工作的.....213
过滤器的操作.....214
在报表中添加过滤.....215
过滤数据范围.....216
关联数据字段的过滤.....216
使用级联列表过滤.....216
以代码控制过滤.....217
范例.....217
例 1."Hello, Report!".....217
例 2.　　向用户要一个文本.....218
例 3.处理对话框控件.....219
例 4.处理报表对象.....220
例 5.　简单过滤.....221
例 6.　自动过滤.....222
例 7.通过范围自动过滤.....223
例 8 通过关联数据字段过滤.....224
Preview, print, export.....226
编辑报表.....228
打印报表.....229
输出报表.....231
Saving in FPX format.....231
Export to Adobe Acrobat (PDF).....232
Export to Word (RTF).....233
Export to HTML.....234
Export to MHT (web archive).....235
Export to Excel (XML).....236
Export to Excel 2007.....237
Export to PowerPoint 2007.....237
Export to OpenOffice Calc.....238
Export to CSV.....239

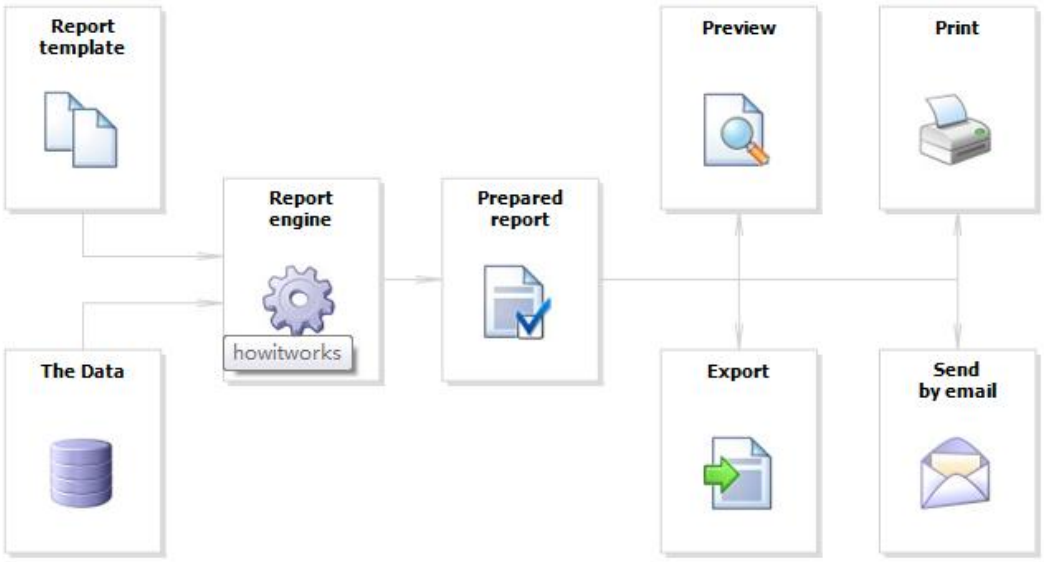
Export to TXT.....239
Export to picture.....240
Report design recommendations.....241
通过邮件发送报表.....243

基本概念

在本章中，我们将学习 Report 的工作原理。我们也将对报表元素进行全面的了解，如报告页面，区域和报告对象。

报表

报表创建过程如下图所示:



报表模板(后报告） - 这是，我们在设计器中看到的。报表保存在扩展名为.FRX 的文件中。报表也可以使用设计器或以编程的方式来创建。

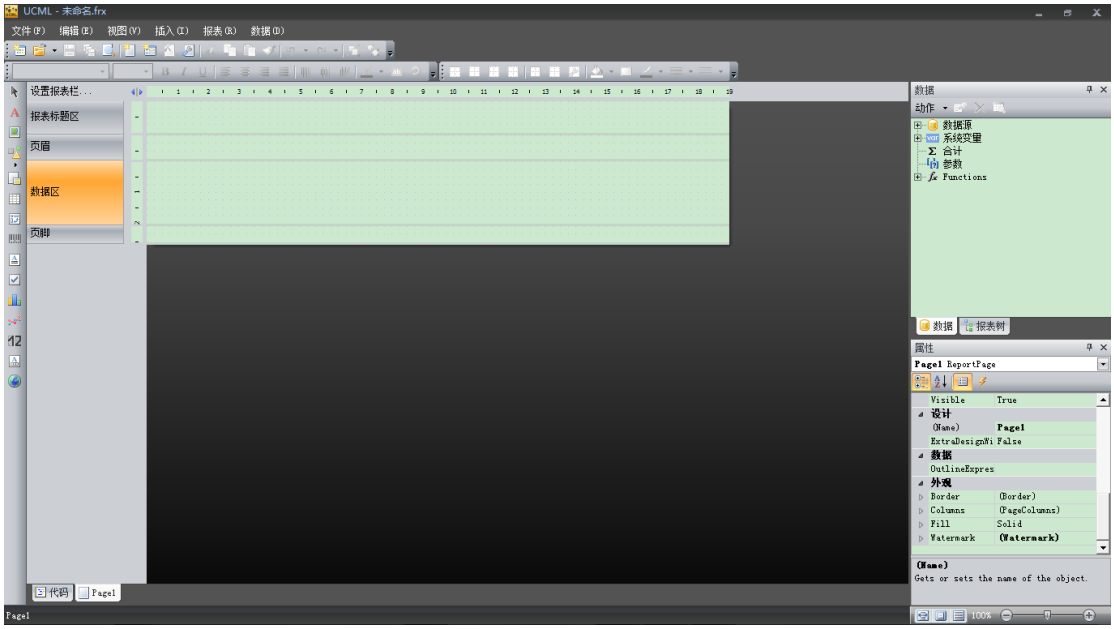
数据类型可以是：定义在程序中或关系型数据库中的数据。Report 也可以使用业务逻辑对象做为报表数据。

准备报表-这就是我们在预览窗口中看到的。准备好的报表，可以预览，打印并保存成指定格式（DOC，XLS，PDF 和其他），也可以通过电子邮件发送。

报表设计器

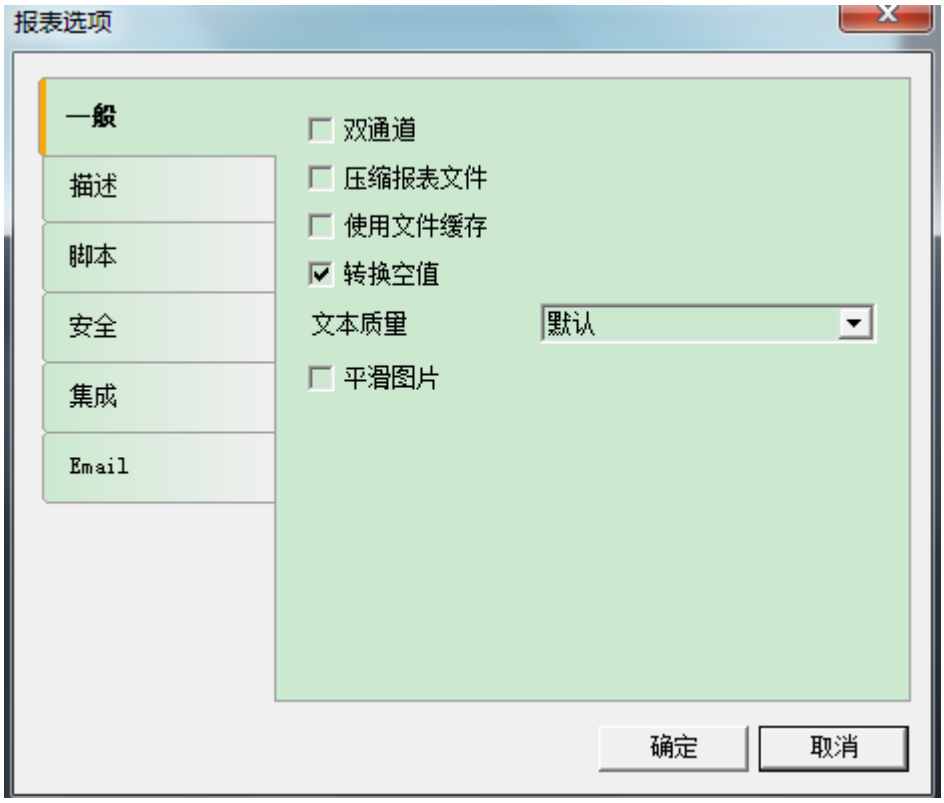
可以使用报表设计器来创建一个模板。设计者为用户提供了舒适的设施，设计报告。用户可以很方便的使用设计器来设计报表，并支持同时预览报表。

报告设计器是 Report 的一部分，不依赖于开发环境（例如，微软的 Visual Studio)。如果你是一个软件开发人员，您可以把报表设计器集成到你的程序里。这样可以使您的程序的用户具备更改现存报表或创建一个新报表的能力。



报表选项

在菜单中点击“报表—>选项”报表选项界面。你可以看到有六个选项卡的对话框窗体。



在一般选项卡中，你可以设置下面的报表参数：

- ″双通道″可以使两个报表同步。这在你使用“总页数”系统变量时是必须的。

- ″压缩报表文件″在保存报表时以指定格式进行压缩. 在 Report 中，压缩格式为.ZIP，这使得你可以使用所有的压缩软件提取原始文件。
- ″使用文件缓存″参数用于在创建的报表，节约内存。 在创建很多页报表时使用这个参数。
- ″转换空值″根据列的数据类型将空值转换为默认值 (0,空字符串，false);
- ″文本质量″对文本在报表中显示的模式进行设置. 这个参数的设定不会影响打印报表的效果。
- ″平滑图片″使以平滑模式画图像（线，边，图片).

在“描述”选项卡中，你可以对报表进行描述。这些参数并不是必须的:



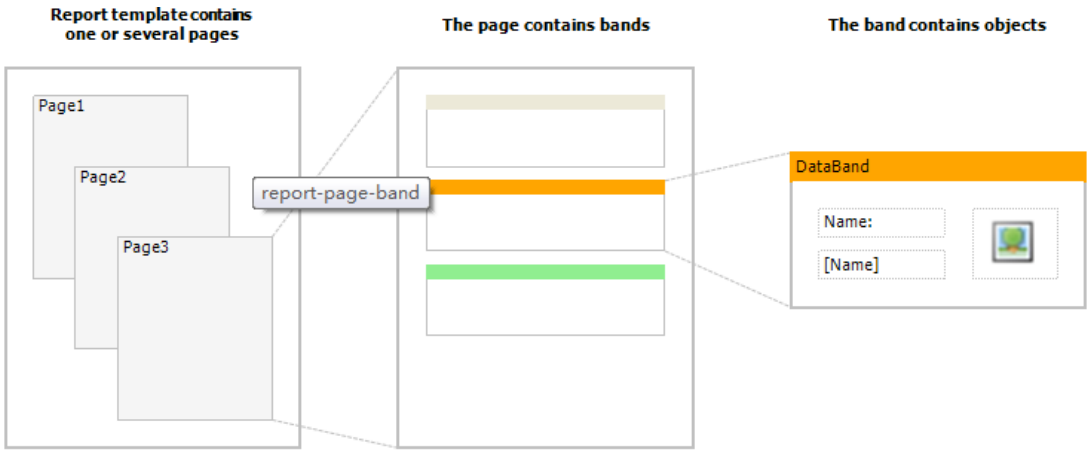
在“脚本”选项卡，你可选择在报表中使用的脚本语言。详细信息可以查看“脚本”章节。

在“安全”选项卡中，你可以设置打开报表时的密码。一个有密码的报表，以加密格式保存，所以不要忘记你的密码！如果丢了密码是不可能恢复的。

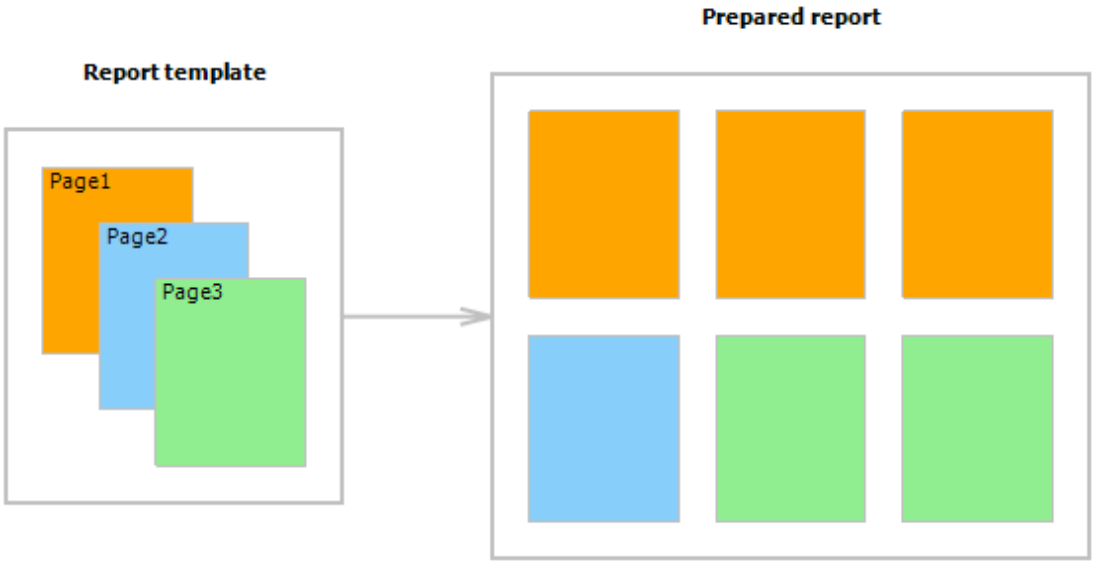
在“集成”选项卡，你可以控制报表的继承关系。这个功能可以在后面看到。

报表页面

模板由一个（大多数情况下）或几个报表页面组成。而报表页又包含报表区域，各个报表区域上面又放置着像文本、图片和其它的对象:



报表模板可以包含许多的页面。例如，你可以创建一个包含标题页和一个数据页的模板。当你创建一个报表时，第一个页先被打印，然后第二个页再打印，依此类推。模板的每一个页可以生成一个或多个预先准备好的报表-这由它包含的数据决定：

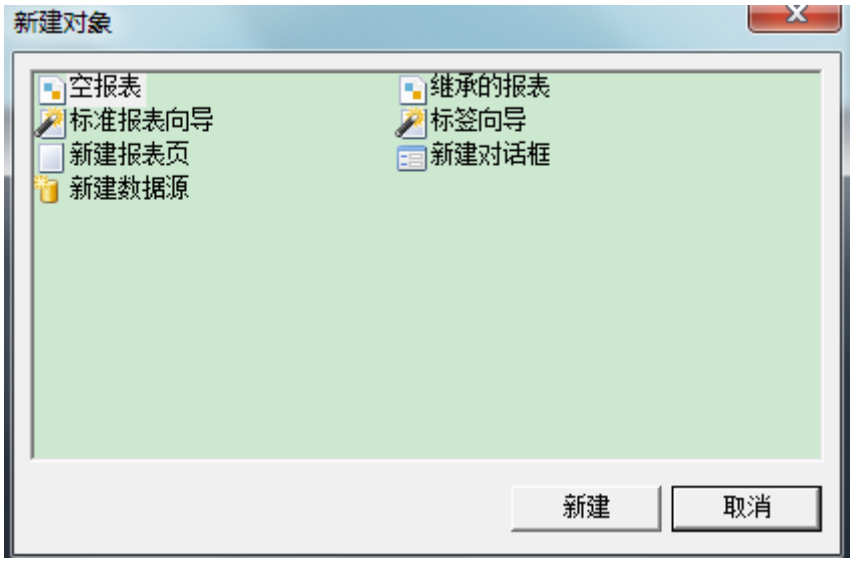


报表页也可以在子报表中使用。和其它报表的生成不一样的是，在 Report 中子报表保存在一个独立的模板页中，但并不在一个独立的文件中。

除了报表页以外，模板包含一个或多个对话框窗体。可以在创建报表前获取参数时使用。有关对话框窗体的具体信息可以查看“对话框窗体”章节。

页面管理

在创建一个报表时，它已经在页面上包含了几个区域。添加页面，可以点击  按钮。也可以点击  按钮，在弹出的窗体中选择“空报表”来创建报表页。



类似的，对话框窗体可以通过按钮来添加。

模板页以选项卡的形式在设计器中显示：



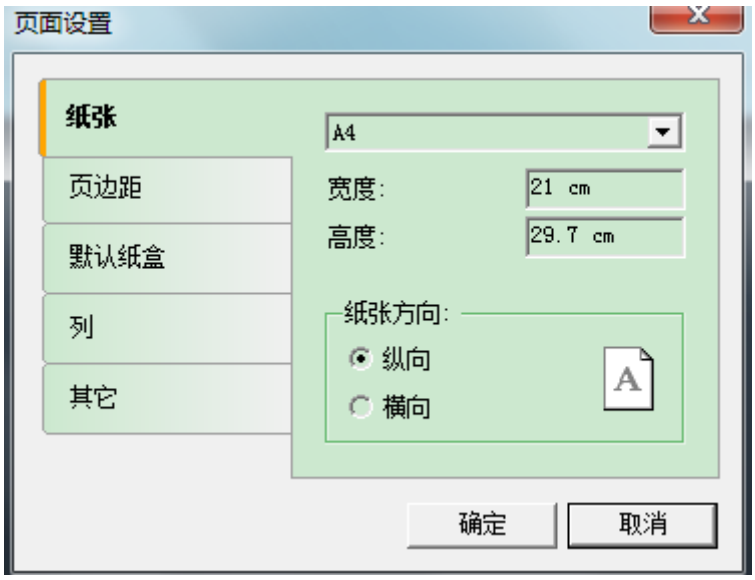
第一个选项卡是报表脚本页。它不可移动也不能删除。

只要点击选项卡，就可以切换到相应页面。可以用鼠标拖拽来改变除代码选项卡外的其它选项卡顺序。

点击按钮来删除页面，该按钮在只有一个页面时不可用。

页面属性

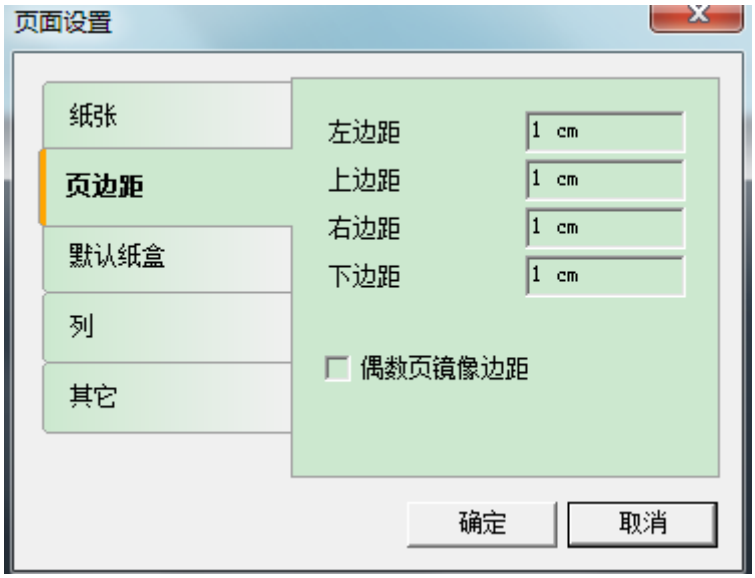
可以针对每个报表页面进行诸如页面尺寸，方向（横向或纵向），边缘，栏位，纸张来源等其它的设定。报表模板可以包含许多不同方向及尺寸的页面。可以通过点击菜单中的“文件→页面”或点击按钮来打开页面设置窗体。



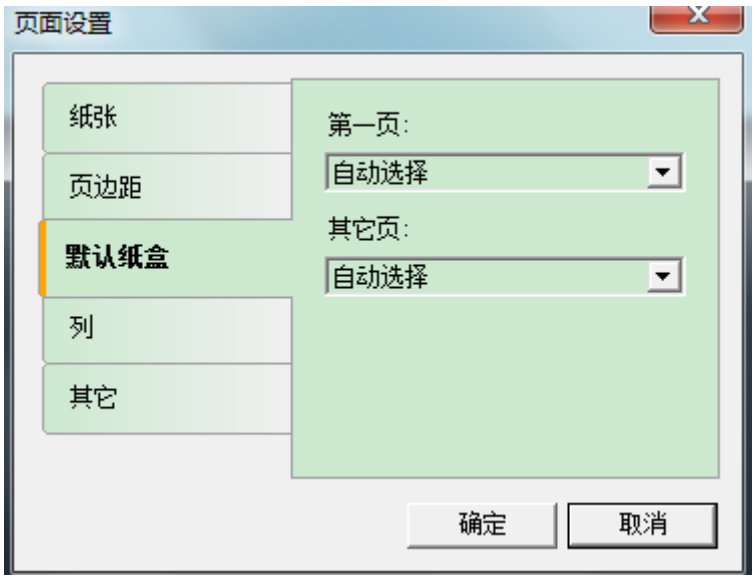
“纸张”可以设置纸张的尺寸与方向。下拉列表中列出了指定打印机上支持的指张尺寸选项。

注：当前打印机可以通过使用“文件→打印机”来进行设置。

页边距选项卡可以设置页面的页边距。“偶数页镜像边距”选项可以用于打印小册子。

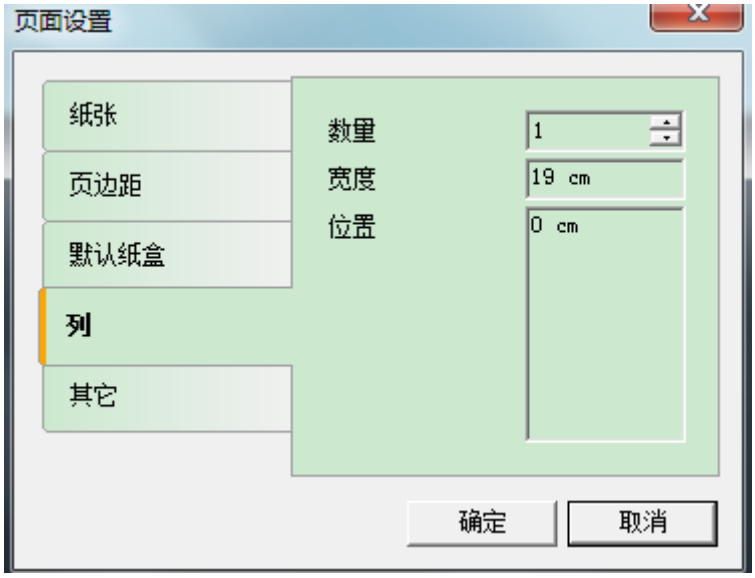


“默认纸盒”用于选择纸张来源。注意来源可能不同。要打印的报表第一页和其它页可能使用不同的纸盒。

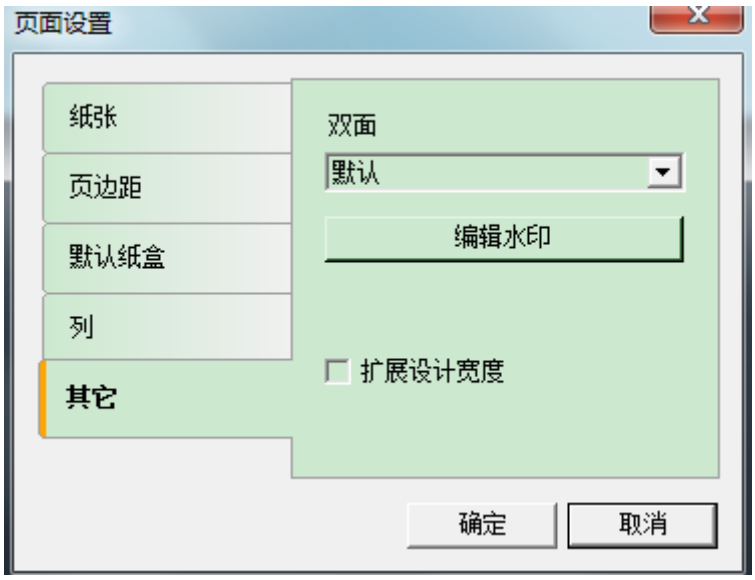


注：来源也可以在打印机上进行选择。

“列”选项卡可以设定多列报表的打印，通过设置这个选项，来对所有列的列数与列宽进行设置。



“其它”选项卡提供了很实用的属性。支持双面打印，当然这需要您的打印机支持。还可以支持水印，这将在报表打印时显示。



扩展设计宽度复选框允许增加页面宽度的设计模式，当你使用表或交叉表时，它将是有用的。

区域

区域在报表页面上，同时它也作为像“文本”，“图片”等其它对象的一个容器。总之，Report 提供了 13 种不同类型的区域。它们分别位于报表的不同位置。

区域	打印方式
报表标题	它将在报表的开头打印一次。你可以选择打印的顺序-通过设置“标题在页眉之前”属性来使其显示在页面页眉区域之前或之后，这个属性位于属性窗体。默认情况下，这个属性值为真，即报表标题在报表页眉之前。
报表摘要	报表结尾打印一次，位于数据记录之后，但在页脚区域之前。
报表页眉	在报表的每个页面的最上面打印。
报表页脚	在报表的每个页面的最下面打印。
字段页眉	这个区域用于打印多字段报表,它打印在报表页面页眉区域之后的最上面。
字段页脚	显示在每列的位于页脚区域之前的最下面。
数据	这个区域用于连接数据源并且在数据源中多次打印。
数据页眉	该区域链接到数据区域。并且在数据第一行之前。
数据页脚	该区域链接到数据区域，并且在最后一行数据之后打印。
分组页眉	该区域被打印在每个分组的开始位置，
分组页脚	该区域被打印在每个分组的结束位置。
子	这个区域可以连接到任何区域，包括其他子区域，它在其父区域后立即打印

覆盖	在每个报表页面中作为背景打印

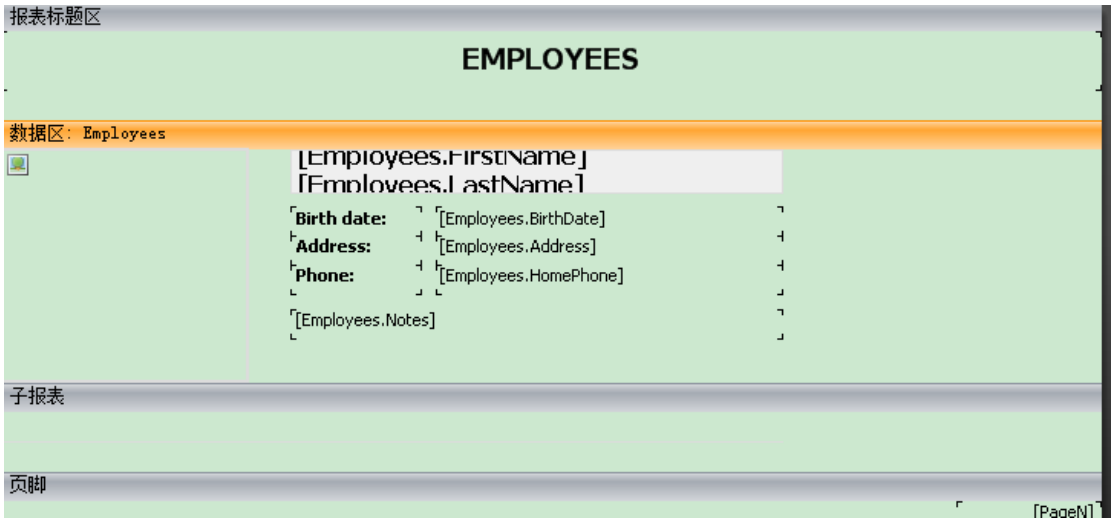
设计器区域

区域在设计器中以矩形来显示。像许多的报表对象一样，设计器区域可以有一个边框和填充（默认情况下是不可用的）。此外，每个区域都显示一个网格。可以通过在菜单“视图→选项”或“报表页面”中来设置网格模式。同时，你也可以通过“视图”菜单设置网格可用或不可用。

有三种设置区域高度的方法：

- 将鼠标置于区域最下端，光标形状变为水平分隔样式，你可以重新设置区域大小。
- 拖动区域左侧的标尺
- 使用“属性”窗体来设定区域的高度属性

设计器有两种显示区域的模式，你可以在任何时候对两者进行切换。第一种模式中，每个区域都有一个头，包含区域的标题和相关的信息（例如，连接数据源的名称）。

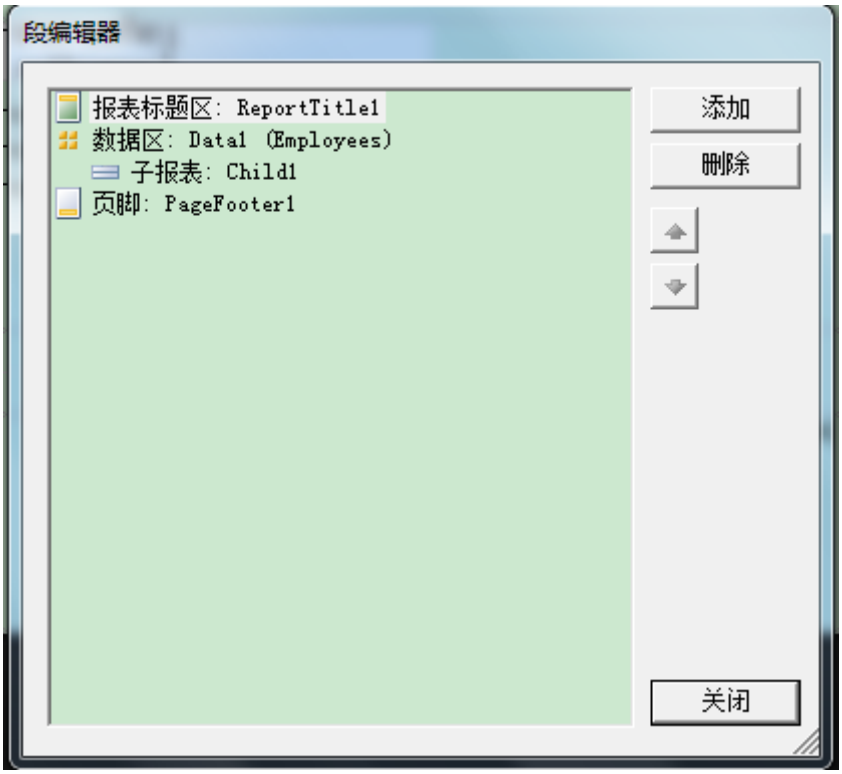


在第二种模式中，区域没有头，报表区域的结构显示在窗体的左边栏处。这种模式可以使你更易于理解报表的结构，尤其该报表不是由你创建的情况下。

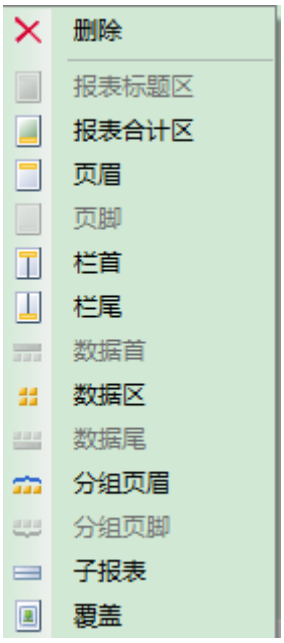


配置区域

你可以在“设置报表栏”窗体中对区域进行配置。你可以从菜单的“报表→设置报表栏”或“设置报表栏”工具按钮来打开该窗体。

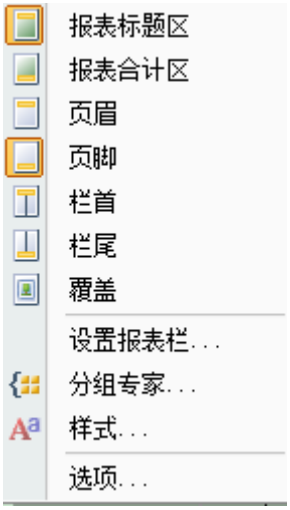


在该窗体中，你可以向报表添加、删除或改变一个栏位。添加可以通过点击“添加按钮”或在栏位树上右键实现。将会弹出一个可操作栏位的列表菜单。不能添加的栏位将不可用。



可添加的栏位类型依赖于你选择了报表树中的哪个节点。例如，要添加“数据首”和“数据尾”栏位，只能先选中数据栏位。

还有第二种方法从菜单“报表”来配置一些栏位。



删除一个栏位，可以选择该栏位，并点击“删除”按钮。

在配置栏位时，Report 不允许做任何导致建立一个错误报告模板的操作。例如，你不能删除一个和组相连的数据栏位，除非你先删除组。删除数据栏位时，栏首与栏尾会自动删除，如果报表页面中只有一个栏位，那么是不能进行删除的。

打印栏位

在下面这个报表中，内含几个栏位，报表将怎么准备这样一个报表，让我们来看看下面的例子。



报表标题区将第一个打印，随后是页眉区。而数据栏位将根据其连接的数据源的记录数来打印多次。在数据区数据打印完成后，将进行报表摘要栏位的打印，而后在报表的底部进行页脚的打印。

在打印报表的时候，报表要对当前页面的打印空间进行检测，如果空间不足，将可能引发下列情况：

- 页脚被打印
- 新页的添加
- 页头被打印
- 继续打印在上页中不匹配的栏位

栏位属性

每个栏位都有许多可用的属性，每个都影响打印操作。它们可以在栏位的弹出菜单中进行配置。在栏位的空白区域上右键或双击栏位或选中栏位，在属性窗口中进行属性修改。

属性名称	描述
CanGrow, CanShrink	该属性决定栏位中的对象是否可以增大或缩小，如果两个属性都禁用，栏位将以设计者指定的大小显示。
CanBreak	如果该属性可用，将会在可用空间打印栏位中的一部分内容，即“分解”栏位。
StartNewPage	从新的报表页开始打印，该属性可在打印分组时使得每个分组都在一个新页面中打印。
PrintOnBottom	在页面的底部报表页脚前打印，在打印报表时，可以设置该属性来使其打印在报表的底部。
RepeatOnEveryPage	在数据页眉、页脚，组页眉、页脚中有这个属性，这种类型的栏位将会打印在每页上

报表对象

可以在报表中使用的多种对象。

图	名称	描述

标		
	"文本"	显示一个或多个文本行
	"图片"	显示一个图片
	"线"	显示一条线，可以为横向、纵向、斜向
	"图形"	显示一个几何图形-矩形、椭圆、二角形
	"多行文本框"	显示格式化文本（RTF 格式）
	"条形码"	显示一个条形码
	"复选框"	显示有两种状态-可用和不可用的复选框
	"表格"	显示包含行、列、单元格的表格
	"交叉表"	显示交叉表，也叫矩阵表
	"图表"	显示一个图表
	"邮政编码" (ZipCodeObject)	显示一个邮政编码
	"蜂窝文本"	文本中的每个字符都显示在独立的单元格中

对象可以显示信息（文本对象）或美化报表的展现（“图片”，线，图形等对象）。表格或交叉表一类复杂的对象可以包含其它简单的对象。

通用对象属性

所有的报表对象都是从类 ReportComponentBase 继承而来，所以包含一组通用的属性。在学习具体对象之前，我们将向您展示这些属性。

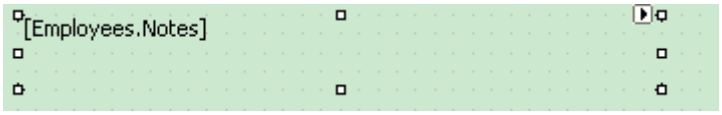
你可以通过属性窗口来改变这些属性的值。一些属性也可以通过对象的弹出菜单或工具栏来进行更改（例如：边框与填充）

属性	描述
Left, Top, Width, Height	通常来说，报表对象都是一个矩形区域，它有位置（左，上）与尺寸（宽，高）属性
Anchor	这个属性决定对象在其容器控件放大或缩小时如何更新其位置与尺寸。在这种模式下，其可以与容器控件共同移动。
Dock	这个属性确定对象将停靠在容器控件的哪个边上。

Border, Fill	这个属性控制控件的边与填充，可以通过工具栏上的边框与填充来进行改变。
CanGrow, CanShrink	这两个属性用来自动设置对象的高度
ShiftMode	设置后可以上下移动
GrowToBottom	该属性处于启用状态，将被拉伸到栏位底部。
CanBreak	对象“文本”和“富文本”有这样的属性。用以确定对象的内容是否可以被分解成几部分。
PrintOn	这个属性决定哪些页面可以打印对象
Cursor	该属性决定当鼠标移动到对象上时的形状
Visible	这个属性决定对象是否显示在报表中，这种对象不会在预览窗口中显示，也不会被打印
Printable	该属性确定报表是否打印，如果禁用该属性，那么对象显示在预览窗口中，但不会被打印
Hyperlink	此属性为报表增加互动性
Bookmark	该属性与 Hyperlink 一起使用，它可以包含任何表达式，在报表中，表达式的值可以通过它的名称来进行使用。
Restrictions	用来限制某些操作，如移动、调整大小、删除等
Style	用此属性来设置对象的显示样式

文本

“文本”对象是在报表中我们经常使用到的主要的对象。它的外观是这样的：

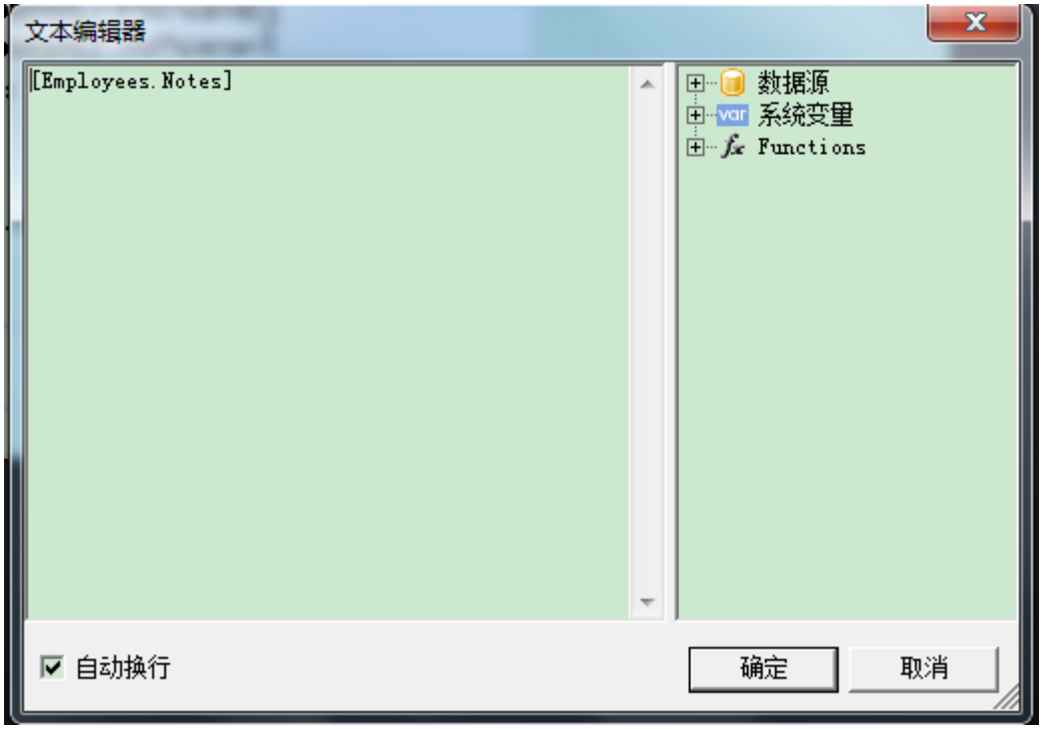


可以显示任何文本数据，尤其是：

- 一行或多行文本
- 数据列
- 报表参数
- 合计值
- 表达式
- 上面各项的组合

一、 文本编辑

双击对象即可对对象文本进行编辑，你将看到一个文本编辑框：



在编辑器右边有一个数据树，在其上的元素都可以添加到文本中。你可以用鼠标通过拖拽的方式把树元素添加到需要的位置处。另一种添加元素的方法，就是双击，就会把元素 添加到鼠标所在的位置处。

可以通过点击“确定”按钮或使用 Ctrl+Enter 快捷键来保存改变并关闭编辑窗体。

另一个就近编辑文本的方法是，选择文本对象，按下回车键，完成编辑后，点击文本对象外部或使用 Ctrl+Enter 快捷键，点击 ESC 键将不保存修改文本。

二、 显示表达式

文本对象可以包含混合表达式的纯文本，例如：

今天是 [Date]

当打印这样一个对象时，在其中的表达式将会被计算，所以结果会像这样：

今天是 12.09.2010

如果想让表达式以其它的方式进行包含，那么可以通过设置 Brackets 属性来进行更改，例如：″<,>″，或者 ″<!,!>″

今天是 <!Date!>

除此之外，可以禁用所有表达式。要做到这一点，设置 AllowExpressions “属性设置为 false。在这种情况下，文本将被原样显示。

在方括号内，你可以使用任何有效的表达。例如，以下文字：

2 * 2 = [2 * 2]

将被打印成：

2 * 2 = 4

常见的错误 - 试图在方括号外写表达。注意，它被认为是一种表达，并得到执行，这是位于方括号内。例如：

2 * 2 = [2] * [2]

文本将会以下面的方式被打印：

2 * 2 = 2 * 2

有可能是元素需要置于自己的方括号表达式内。例如，它可能是一个系统变量（参见“表达式”一章，详细）。让我们来看看下面的例子:

The next page: [[Page] + 1]

文本中包含表达式[] +1。它是一个返回当前报告页面的数量系统变量。它包含在自己的括号内。这里无论在“brackets”对象设置没有，都必须是在方括号中。

严格的讲，我们在上面的例子中使用“日期”系统变量时，应该使用两个双方括号:

Today is [[Date]]

然而，如果在表达式里只有一个成员的情况下，允许去除不必要的一对括号.

三、 显示数据字段

你可以用下面这种方式来打印数据字段：

[Datasource name.Column name]

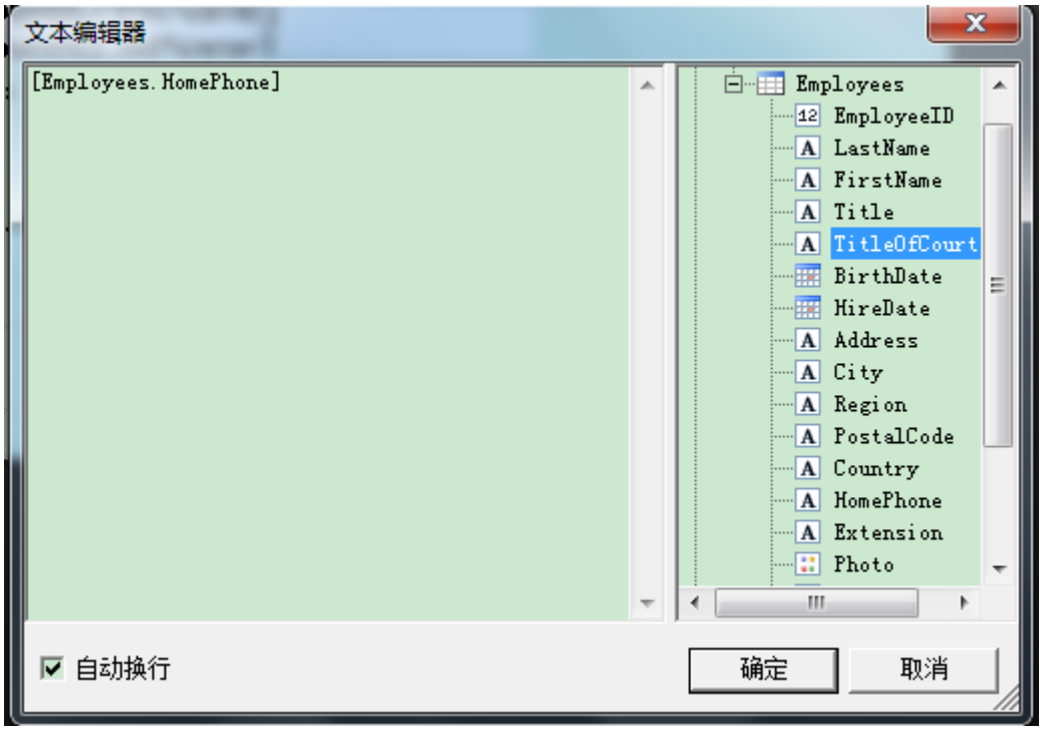
这里面使用了一对中括号。数据源名称和数据字段名称被.分隔，例如：

[Employees.FirstName]

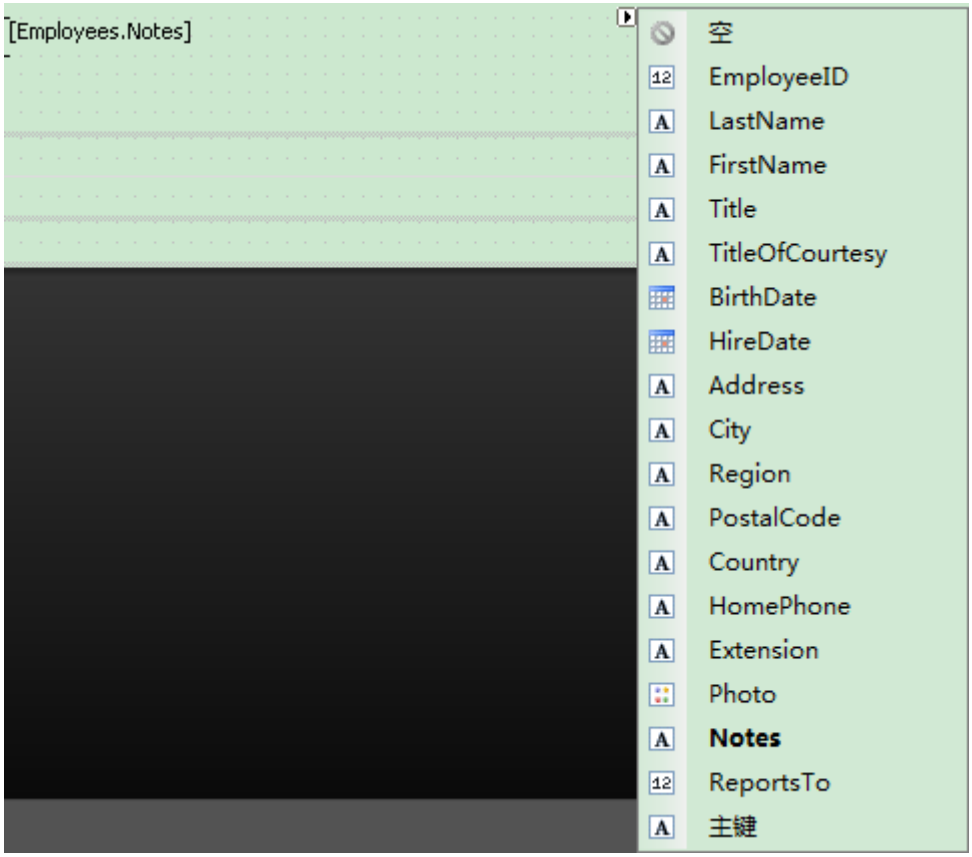
有以下几种方法向文本对象中插入数据字侧面。

1. 在文本对象编辑器中，手工写入数据字段名称。这种方法最不方便，因为它很容易犯错误。

2. 在该对象的编辑器中，选择所需要的数据列，将其拖放到文本:



3. 点击对象右上角的小按钮，并从列表中选择数据列：



4. 从“数据”窗口拖放的数据列到报告页面。在这种情况下，“文本”对象自动被创建，它包含一个链接。

四、 HTML 标记

在“文本”对象您可以使用一些简单的 HTML 标签。默认情况下，标签被禁用，可以在“属性”窗口中，将 “HtmlTags”属性设置为 true。下面是一个支持的标签列表:

标签	描述
...	粗体
...	斜体
<u>...</u>	下划线
<strike>...</strike>	删除线
_{...}	下标
^{...}	上标
...	字体颜色

下面的例子演示了如何使用这些标签.

text bold text text in italic bold and in italic

E = mc²

A₁ = B²

this is regular text, and this is a red one

this is regular text, and this is an orange one

这段文本将以下面的方式展现：



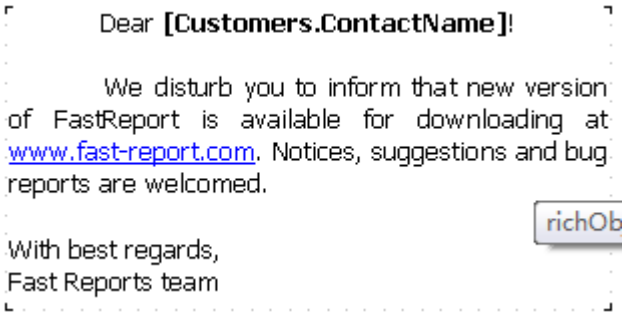
五、 对象属性

属性	描述
AllowExpressions	决定文本对象是否可以包括表达式，默认情况为 true。
Angle	此属性指示文本旋转度。
AutoShrink	这个属性允许自动缩小字体大小或字体宽度以适应文本。
AutoShrinkMinSize	这个属性决定在使用 AutoShrink 属性后，最小的字体大小和最小字体宽度比
AutoWidth	决定是否自动调整文本的宽度。
Brackets	此属性包含一对用于指定文本为表达式的符号。
BreakTo	有了这个属性，你可以组织文本流从一个文本对象到另一个。例如，我们有“A”和“B”的文本对象。“A”不适合长文本。您可以设置 A.BreakTo 到 B。将会在 B 中显示文字其余部分
Clip	这个属性决定是否有必要在对象范围之外剪辑文本，默认情况下允许。
Duplicates	这个属性决定如何将打印重复值
FirstTabOffset	这个属性决定第一个 TAB 的偏移量，以像素为单位
FontWidthRatio	使用这个属性来使字体更宽或更窄。默认情况下，属性设置为 1。为了使字体较宽，设置属性值>1。为了让字体更窄，设置属性值在 0 和 1 之间。
HideValue	此属性是字符串类型。它允许隐藏与属性值相同的字符
HideZeros	数字前面的 0 不显示。
Highlight	这个属性允许设置高亮的条件。
HorzAlign, VertAlign	这些属性决定了文本的对齐方式。
HtmlTags	决定文本对象的内容是否可使用 HTML 标签
LineHeight	这个属性允许显式地设置文本行的高度。缺省情况下，它被设置为 0

NullValue	替换空值为设置的文本。您还需要在“报告/选项...”菜单中取消选中“转换空值”选项。
Padding	这个属性允许设置填充，以像素为单位。
RightToLeft	此属性指示是否应显示文本从右到左的顺序。
TabWidth	这个属性决定 TAB 符号的宽度，以像素为单位。
Text	此属性为显示的文本
TextFill	这个属性决定了文本填充。使用这个属性可选择不同的文本填充类型。
Trimming	这个属性决定是否处理文本两侧的空格。只有当“WordWrap”属性设置为 false 时生效。
Underlines	这个属性使文本有下划线。只有在文本 top-aligned 时生效。
WordWrap	这个属性决定文本是否自动换行
Wysiwyg	此属性改变显示模式，“文本”对象相匹配的屏幕和最终打印输出。此模式也使用，如果你使用的证明对齐或非标准的行高度。

多行文本

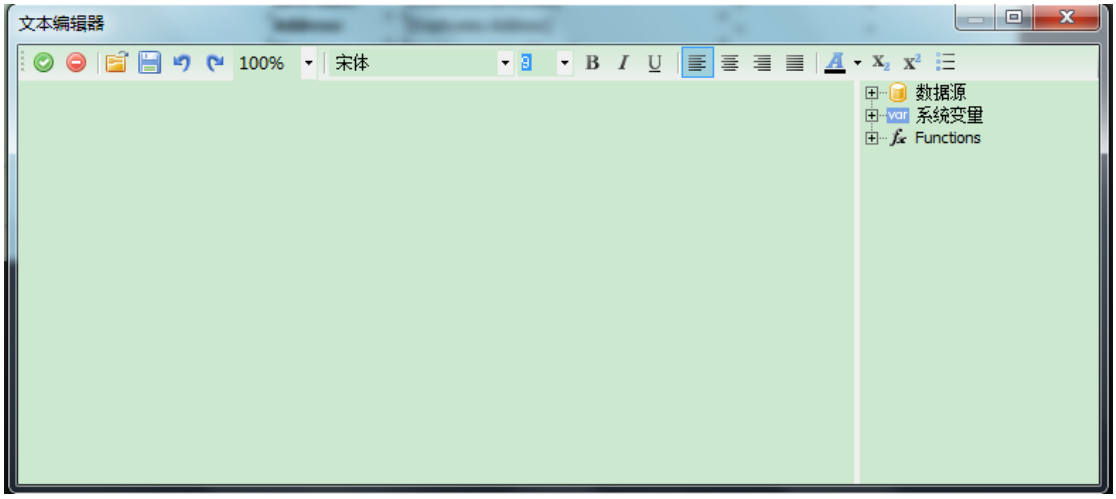
此对象显示格式化文本（RTF 格式）像这样:



当使用文本对象显示文本时，导出报表为不同文本格式时，"Rich Text"将输出一个图片。

该对象只支持实心填充。

要编辑对象，双击就行了。你将看到在编辑器窗口:



您也可以使用 Microsoft Word 中创建一个文本，而后保存为 RTF 格式。接下来，点击按钮  打开并加载.RTF 文件。

该对象并不支持所有的 Microsoft Word 格式功能.

在这个对象，可以通过以下方式显示数据:

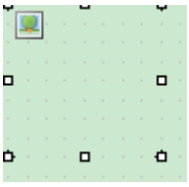
- 您可以像你在“文本”中插入表达式一样。将数据列插入到文本;
- 使用“DataColumn 的”属性绑定到列。

对象具有以下属性：

属性	描述
AllowExpressions	决定文本对象是否可以包括表达式。
Brackets	此属性包含对指定表达式匹配符号。
DataColumn	绑定到数据列。
Text	此属性包含 RTF 文本。
Padding	填充，以像素为单位。

图片

可以显示以下图形格式：BMP，PNG，JPG，GIF，TIFF，ICO，EMF，WMF。使用“图片”对象，您可以在报表上打印自己公司的标志，雇员或任何图形信息的照片。对象在编辑器中看起来像这样:

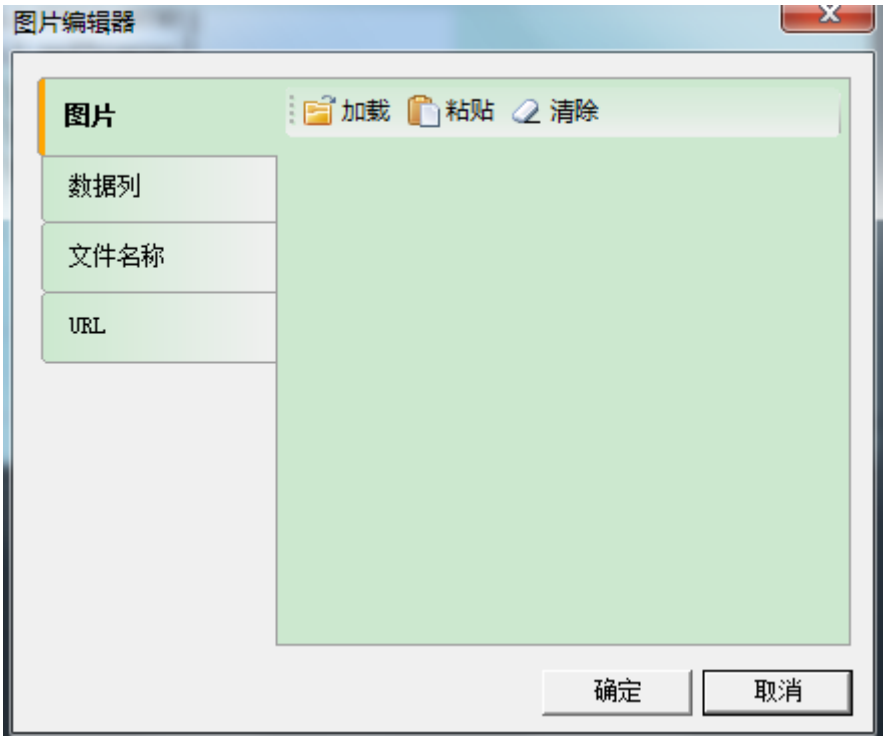


可以从下面的数据源显示数据:

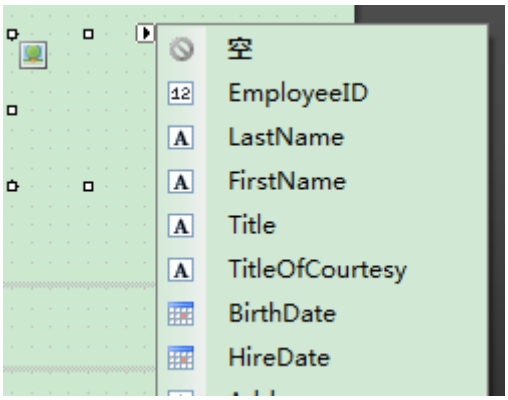
数据源	描述
图 片	从文件加载图片，并保存在报表。图片存

文件	储在“image”属性。
数据字段	图片从一个数据列加载。列的名称存储在“DataColumn”属性。
文件名称	加载图片从一个文件中给定的名称。文件的名称被存储在“ImageLocation”属性中。
URL	每次创建报告时从互联网加载图片。图像不会存储在报表中。 URL 存储在“ImageLocation”属性。

为了调用一个图片编辑器，双击对象。在编辑器中，你可以选择图片的数据源



为了将对象绑定到数据列上，可以点击对象右上角的小按钮，并从列表中选择数据列：



您还可以从“数据”窗口拖放数据列到报表页面。在这种情况下得到的“图片”对象包含一个链接列。您指定的数据列的数据类型应为“byte[]”.

在弹出菜单中可以选择的尺寸模式:

- 自动大小. 自动获取图像

大小.

- 居中. 图片在对象

中间.

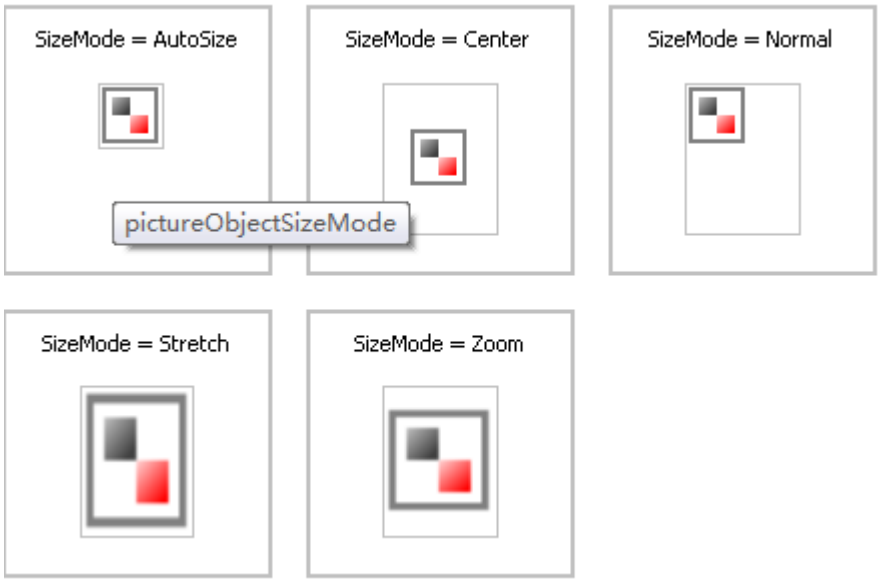
- 正常. 图片以原始大小在左侧显示.
- 拉抻. 图片拉伸至对象大小尺寸

显示.

- 放大. 根据纵横比的图像被拉伸的对象的

大小.

以上五种不同模式显示如下:

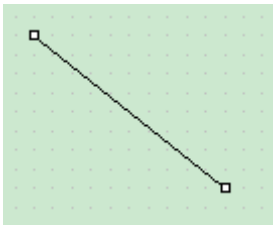


"Picture" 有如下属性:

属性	描述
Angle	旋转角度，单位为度。此属性的值可能是 0，90，180，270。
SizeMode	尺寸模式.
Transparency	图像的透明度的程度。该属性可为 0 和 1 之间的值。值为 0（默认）表示，图片不透明.
TransparentColor	显示图像时，颜色将是透明的。
Image	图像
DataColumn	图像绑定的数据列
ImageLocation	这个属性可以包含的文件名称或 URL。创建报表时，从该位置加载图片。
Padding	填充，以像素为单位。
ShowErrorImage	在找不到图片时显示“的图片，当图片是空的。

线

线有横、纵、对角三类，在设计器中显示如下:



如果可能的话，最好使用对象的边界来代替“线”。这将简化报表，并避免由于导出不同格式时可能出现的问题.

设计器可以很便捷的画线。为了向报表插入一条线，单击“图形”工具栏上的按钮，在菜单中选择“线”或“对角线”。将鼠标移至栏位。然后左键单击并按住鼠标，以划出区域。

与对角线不同的是，一条普通的线可以切换为直线与竖线。

线有以下属性:

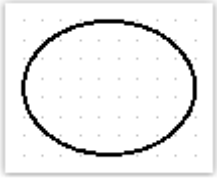
属性	描述
Diagonal	该属性决定线是否为对角线，启用该属性可以将一条普通线转换为对角线
StartCap, EndCap	这些属性允许设置的线帽. 你可以选择下面的值: ellip se; rectan gle; diam ond; ar row. 线帽尺寸可以通过 Width 和 Height 属性来设置。

图形

图形控件可以用于显示下面的图形：

- 矩形;
- 圆角矩形;
- 椭圆;
- 三角形
- 菱形

在设计器中显示如下:



可以点击在工具栏上的按钮 选择需要的图形来向报表插入图形。

图形，像其它的所有报表对象一样，都有边框和填充。与“文本”对象不一样的是，你不能控制每条边线，所以，不能使用双线样式。

如果可以最好用对象的边框来代替矩形。

图形具有下面的属性:

属性	描述
Shape	该属性决定图形的类别
Curve	该属性用于圆角矩形，用以确定其的圆角弧度

条码

该控件可以在报表中显示，样子如下：



该控件提供下列几种类型的条形码:

编号	长度	允许的符号
2 of 5 Interleaved		0-9
2 of 5 Industrial		0-9
2 of 5 Matrix		0-9
Codabar		0-9, -$:/.+
Code128		128 ASCII chars
Code39		0-9,A-Z, -. *$/+%
Code39 Extended		128 ASCII chars
Code93		0-9,A-Z, -. *$/+%
Code93 Extended		128 ASCII chars
EAN8	8	0-9
EAN13	13	0-9
MSI		0-9
PostNet		0-9
UPC A	12	0-9
UPC E0	6	0-9
UPC E1	6	0-9

2-Digit Supplement	2	0-9
5-Digit Supplement	5	0-9
PDF417		any
Datamatrix		any

条形码的详细信息描述可以在下面的网址找到:

http://www.barcodeisland.com

控件中的二维码数据是文本类型。这个文本可以包含所有字符。你可以通过在条形码控件的上下文菜单中选择“条形码”对象。

你可以使用下面的方法连接对象数据：

- 在“Text”属性中设置条形码数据;
- 使用“DataColumn”属性将控件绑定到数据列表。
- 在“表达式”属性中设置表达式返回条形码数据。

条形码拥有以下属性：

属性	描述
条形码	这个属性包含特殊条形码的配置。扩展这个属性来进行配置。
角	此属性确定条形码的旋转度数。您可以使用下列值之一：0，90，180，270。
放大	此属性允许放大条形码。它伴随“自动调整大小”属性而使用。
自动调整大小	如果这个属性是打开的，该对象将延伸，以显示整个条码。如果此属性是关闭的，条形码将延伸到对象的边界。
显示文本	此属性决定是否有必要显示人类可读的文本。
数据列	该对象绑定的数据字段。
表达式	该表达式返回条形码数据。
文本	条形码数据
填充	以像素为单位填充

下面的属性对于条形码类型是特殊的,可以通过选择“条形码－属性”窗口，扩展“条形码”属性，来改变它们。

Property	Description
WideBarRatio	此属性特定于所有线形条形码。它决定了条形码的宽窄比例，对于大多数的条码类型，此属性的值应该在 2 和 3 之间。
CalcCheckSum	该属性特定于所有线形条形码。它决定是否有必要对校验和进行自动计算。如果此属性是关闭的，您需要在条形码数据中提供校验位。
AutoEncode	该属性特定于 Code128 条形码。该代码有 3 个

<table>
<tr><td></td><td>不同的编码-A,B,C。您可以选择设置编码明确的条形码数据或将该属性设置为“true”。在这种情况下，编码将自动选择。
在条形码数据中使用以下控制代码：
<table>
<tr><th>Code</th><th>Meaning</th></tr>
<tr><td>&A;</td><td>START A / CODE A</td></tr>
<tr><td>&B;</td><td>START B / CODE B</td></tr>
<tr><td>&C;</td><td>START C / CODE C</td></tr>
<tr><td>&S;</td><td>SHIFT</td></tr>
<tr><td>&1;</td><td>FNC1</td></tr>
<tr><td>&2;</td><td>FNC2</td></tr>
<tr><td>&3;</td><td>FNC3</td></tr>
<tr><td>&4;</td><td>FNC4</td></tr>
</table>
当你将"AutoEncode"属性设置为 true 时，所有控制代码都将失去作用。
使用控制代码的示例：
&C;1234&B;ABC</td></tr>
<tr><td>AspectRatio</td><td>此属性特定于 PDF417 条码。它决定了高度与宽度的纵横比，并用于自动计算条形码的大小（通常情况下“列”和“行”的属性没有被设置）。</td></tr>
<tr><td>CodePage</td><td>此属性特定于 PDF417 和 Datamatrix 条形码。它决定了哪些是用来转换非 ASCII 字符的代码页。例如，默认的 Windows 代码页 1251。</td></tr>
<tr><td>Columns, Rows</td><td>这些属性特定于 PDF417 条码。它们决定于条形码的行和列的数目。如果两者的属性都设置为 0，则条码的大小将被自动计算。在这种情况下，会同时使用“AspectRatio”的属性。</td></tr>
<tr><td>CompactionMode</td><td>此属性特定于 PDF417 条码。它决定了 PDF417 的数据压缩模式。</td></tr>
<tr><td>ErrorCorrection</td><td>此属性是特定于 PDF417 条码。它决定了纠错等级。</td></tr>
<tr><td>PixelSize</td><td>此属性特定于 PDF417 条码。它决定了条形码元素的大小，以屏幕像素为单位。作为一项规则，</td></tr>
</table>

	该元素的高度应大于元素的宽度 3 倍以上。
Encoding	该属性特定于 Datamatrix 条形码。它决定了 Datamatrix 的数据编码。
PixelSize	该属性特定于 Datamatrix 条形码。它确定条形码元素的大小，以像素为单位。
SymbolSize	该属性是特定于 Datamatrix 条形码。它决定条形码符号的大小。

复选框

该控件显示报表中的复选框。样子如下：



该控件可以显示两种状态：“选中”和“未选中”。使用以下方法控制该控件的状态：

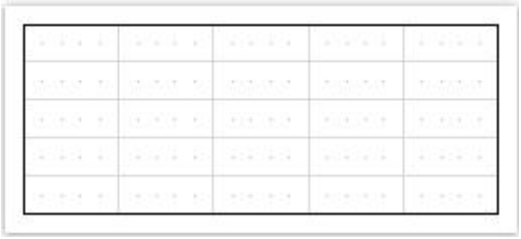
- 在"Checked" 属性中设置状态；
- 使用"DataColumn" 将该控件绑定到数据列表；
- 使用"DataColumn" 将该控件绑定到数据列表;

复选框拥有以下属性：

Property	Description
CheckedSymbol, UncheckedSymbol	这些特性决定了显示在该对象上的标志，这取决于该对象的状态。
CheckColor	该属性决定了选中标志的颜色。
CheckWidthRatio	使用此属性来设置选框的的宽度比例。选框的宽度取决于控件的大小，你可以在 0.2-2 之间进行选择。
HideIfUnchecked	在控件未选中的状态下，此属性允许隐藏控件。
Checked	此属性控制该控件的状态。
DataColumn	该控件绑定的数据字段，字段类型应该是布尔型或整数型。
Expression	该表达式返回 true 或 false。

表格

“表”由行、列和单元格组成。它是 Microsoft Excel 表的简化模拟。它看起来像：



你可以在“创建报表”章节学到更多关于该控件的内容。

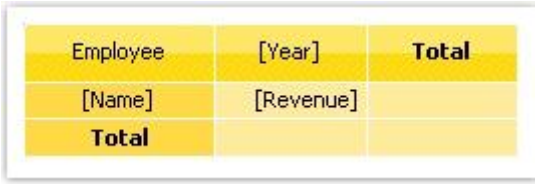
“表”拥有以下属性：

Property	Description
ColumnCount	使用此属性可以快速设置列数。如果表中的列数偏少，它将得到补充，否则会被删除。
RowCount	使用此属性可以快速设置行数。如果表中的行数偏少，它将得到补充，否则会被删除。
FixedColumns	该属性决定表的固定列数。固定列形成表头。表头的打印由 “RepeatHeaders”属性控制。
FixedRows	该属性决定表的固定行数。固定行形成表头。表头的打印由 “RepeatHeaders”属性控制。
RepeatHeaders	该属性允许打印没一张新页面上的表头，此属性只适用于动态形成的表。

交叉表

“交叉表”类似于“表”，也是由行，列和单元格组成。同时，交叉表的行和列取决于它连接的数据。

该控件样子如下：



你会在“创建报表”章节学到你更多关于该控件的内容。

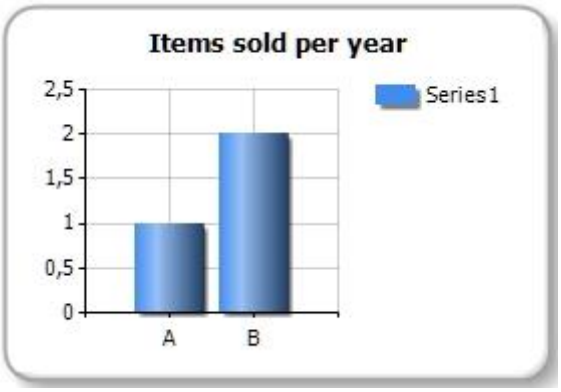
“交叉表“拥有以下属性：

Property	Description
RepeatHeaders	如果该交叉表被划分到不同的页面，该属性允许打印每个新页面的表头。
CellsSideBySide	该属性决定了当交叉表拥有数个数据单元水平时，交叉表如何定位。可能的方式如下： ● 单元格并排显示

	● 单元格上下显示
Style	使用这个属性，你可以为整个交叉表设置样式。你可以选择一个预定义的样式。
AutoSize	这个属性允许自动计算交叉表的大小。如果你想手动控制该控件大小的话可以将其禁用。
DataSource	该属性允许交叉表连接数据源，当你将数据字段拖拽到该交叉表时，该属性会自动建立连接。但是，如果你在单元格中使用了表达式，请检查该属性是否正确建立连接。
Filter	此属性包含数据过滤的表达式，它适用于交叉表的数据源连接（请参阅“DataSource”属性）。

图表

"MS Chart"允许显示图标。可用系列超过 3 0 种不同类型－条形图、柱形图、区域图、线形图、气泡图、饼图、圆环图、financial、棱锥图、范围图。样子如下：



你会在“创建报表”章节学到更多关于该控件的内容。

“图表”拥有以下属性：

Property	Description
Chart	参考 Microsoft Chart 对象
AlignXValues	该属性允许将不同图表系列上的 X 值排成一线（通过插入空值）。它适用于包含两种或两种以上的图表系列。
AutoSeriesColumn, AutoSeriesColor, AutoSeriesSortOrder	这些属性允许设置自动创建系列。在“创建报表”章节阅读更多关于这些属性的内容。
DataSource	该属性允许连接到数据源。
Filter	此属性包含数据过滤的表达式，它将适用于

	图表与数据源的连接（请参阅“DataSource”属性）。

邮编

“邮编”允许在信封上打印邮编号码。它可以显示数字字符（0 — 9）如下图所示：



您可以通过以下方法将该控件与数据连接：

- 在“文本”属性设置邮政编码数据;
- 使用"DataColumn" 属性将该控件绑定到一个数据列表
- 在"表达式" 设置表达式，返回邮政编码数据。

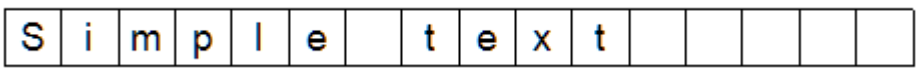
“邮政编码”拥有以下属性：

Property	Description
SegmentCount	代码段中的分段数量，此属性默认设置为 6。
SegmentWidth, SegmentHeight	一个代码分段的大小，默认大小为 0.5×1 厘米。
Spacing	此属性决定两个分段起点之间的距离，默认值是 0.9 厘米。
ShowGrid	确定是否有必要显示网格。
ShowMarkers	决定是否有必要显示标记（邮编上方的粗体水平线）。
DataColumn	该对象绑定的数据字段。
Expression	返回邮政编码数据的表达式。
Text	包含邮政编码的文本

蜂窝文本

该控件可以将每个字符显示在单个单元格中，它经常用于打印金融应用方面的表格。

样子如下：



事实上，这个控件直接继承了“文本”控件。你可以用文本方法将它连接到数据。例如，你可以调用该控件的编辑器，然后键入以下文本：

[Employees.FirstName]

“蜂窝文本”拥有以下属性：

Property	Description
CellWidth, CellHeight	这些属性决定了单个单元格的大小，如果两者的值均为 0（默认），单元格大小将根据所用文字大小，自动进行设置。
HorzSpacing, VertSpacing	这些属性决定相邻单元格之间的水平和垂直间距。

第一个报表

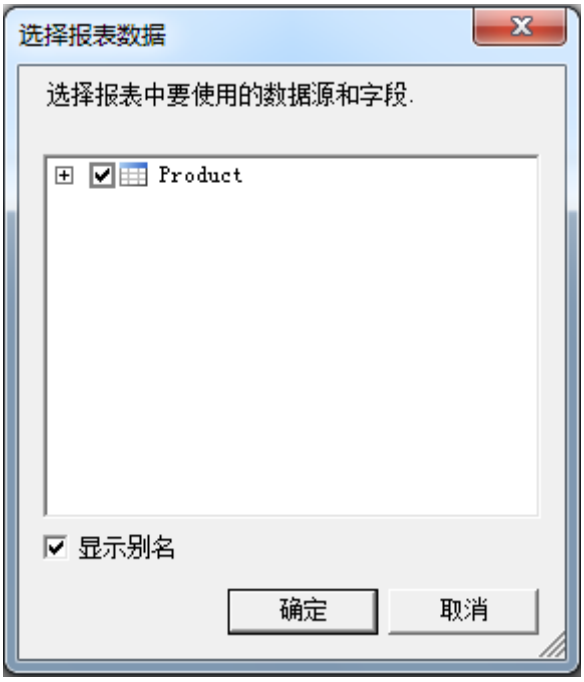
让我们创建一个简单的报表，打印产品清单。我们将使用产品表作为我们的数据源，它可以在演示数据的基础上中被找到。

假设，你将要以下面演示程序中所写的代码执行调用报表设计器的操作。

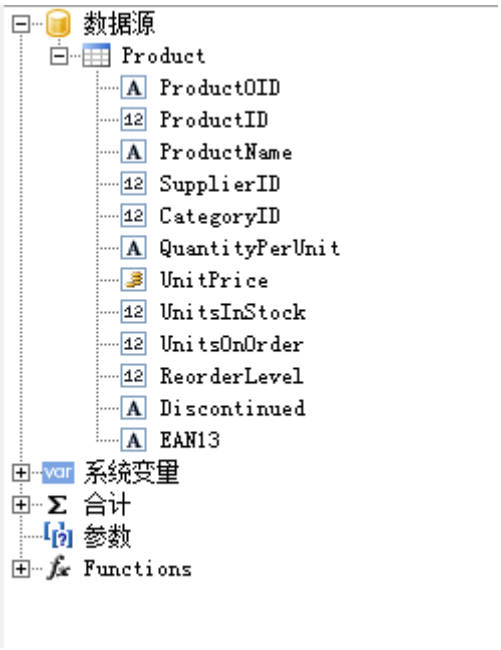
手动创建报表

在这个例子中，我们将手动创建一个报表。为此，我们将做如下操作：

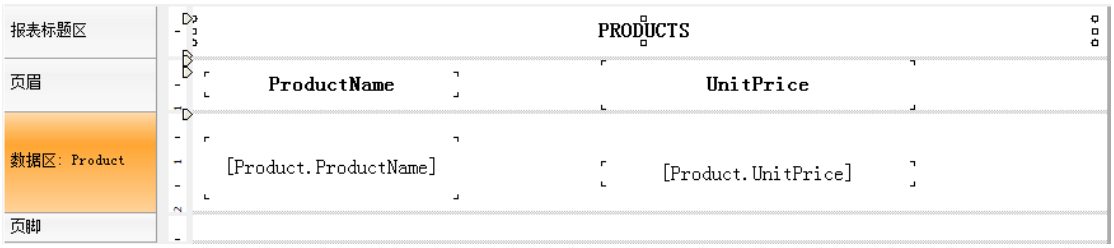
- 点击工具栏中的 按钮，在“添加新项目”窗口，选择“空报表”。
- 在“数据”菜单中，选择“选择数据源”选项，勾选“Product”数据来源。



切换到“数据”服务窗口（如果它不在屏幕上，可以通过选择“数据－显示数据源”将其显示在屏幕上），展开“数据来源”，选中“Product”。



- 将“ProductName”数据字段拖拽到“数据”栏位，Report 会创建一个“文本”对象，实现与该字段和表头的连接。
- 我们将用另一种方法创建“UnitPrice”字段，在“对象”工具栏中，点击“文本”按钮。
- 离开鼠标并拖动指针到“数据”栏位-你会看到 Report 提供了可插入的控件。选择合适的位置，然后单击鼠标插入控件。
- 将鼠标指针放置在控件上方，点击该控件右上方的小按钮，你会看到一列数据字段，选择“UnitPrice”字段。
- 创建一个“文本”控件-作为“单价”字段的表头，并将它放置在“页首”区域。双击该控件，在文本框中输入“单价”。
- 创建“文本”控件-作为报表的标题。将它放置在“报表标题区”区域，在文本框中输入“Product”。
- 将“报表标题区”和“页首”区域中的对象的字体类型设置为粗体。为此，按住 shift 键，选择所有对象，然后点击“文本”工具栏中的  按钮。在此之后，报表将如下图所示：



为了运行报表，可点击工具栏上的  按钮。报表即被创建并在预览窗口显示。

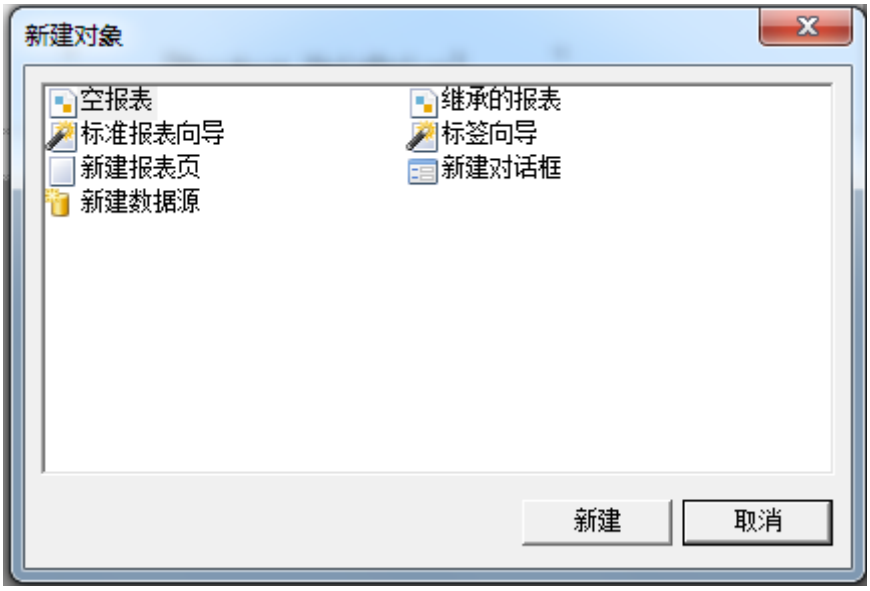
PRODUCTS

ProductName	Unit Price
Chai	18
Chang	19
Aniseed Syrup	10
Chef Anton's Cajun Seasoning	22
Chef Anton's Gumbo Mix	21,35
Grandma's Boysenberry Spread	25
Uncle Bob's Organic Dried	30
Northwoods Cranberry Sauce	40
Mishi Kobe Niku	97
Ikura	31

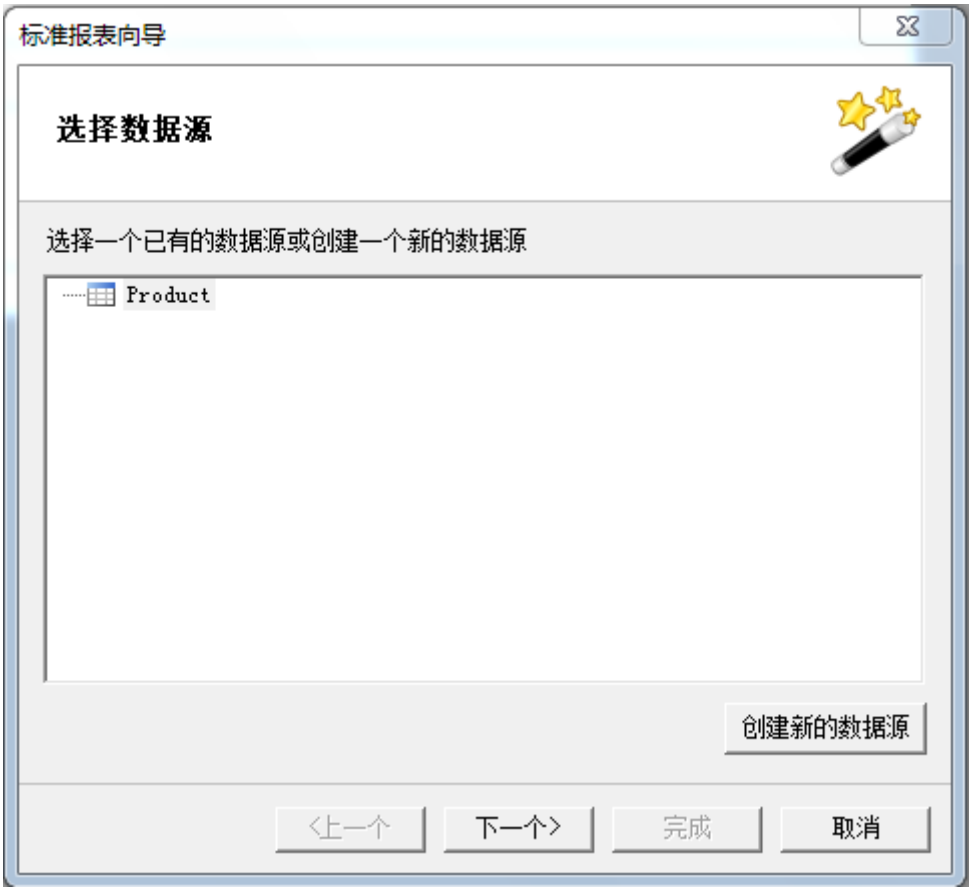
示例 2：用向导创建报表

在这个例子中，我们将用“标准报表向导”创建一个报表。为此，请执行以下操作：

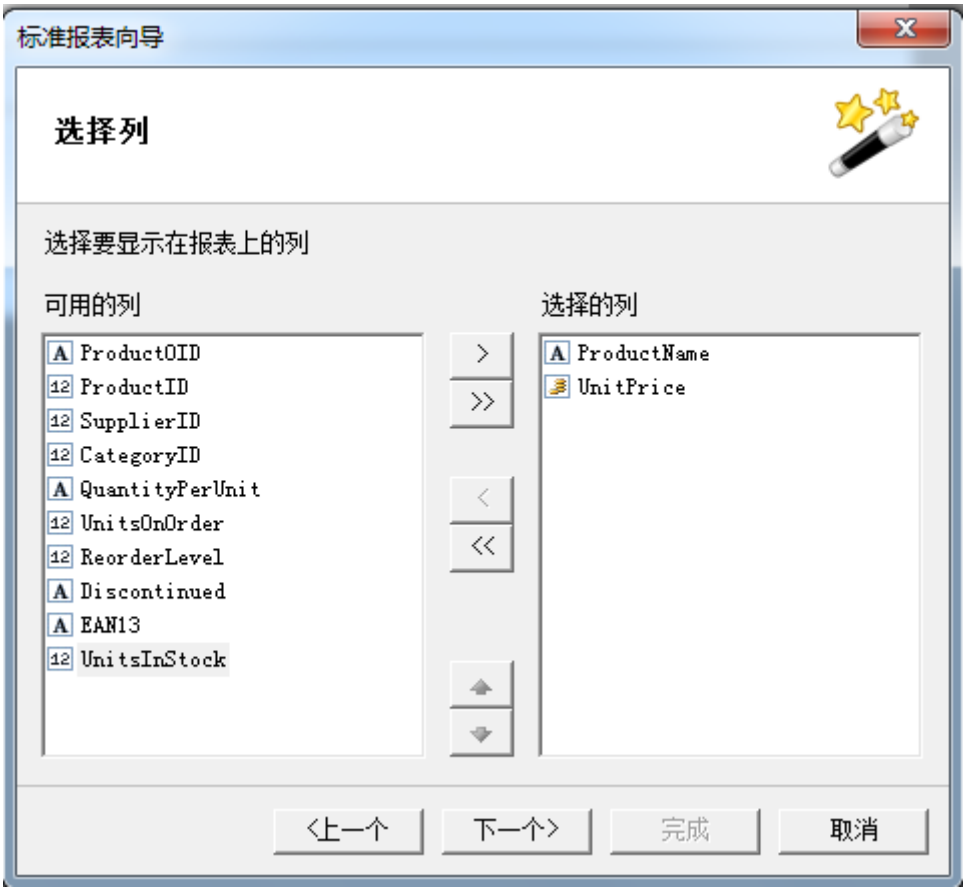
- 点击工具栏上的 按钮，在“新项目”窗口中，选择“标准报表向导”：



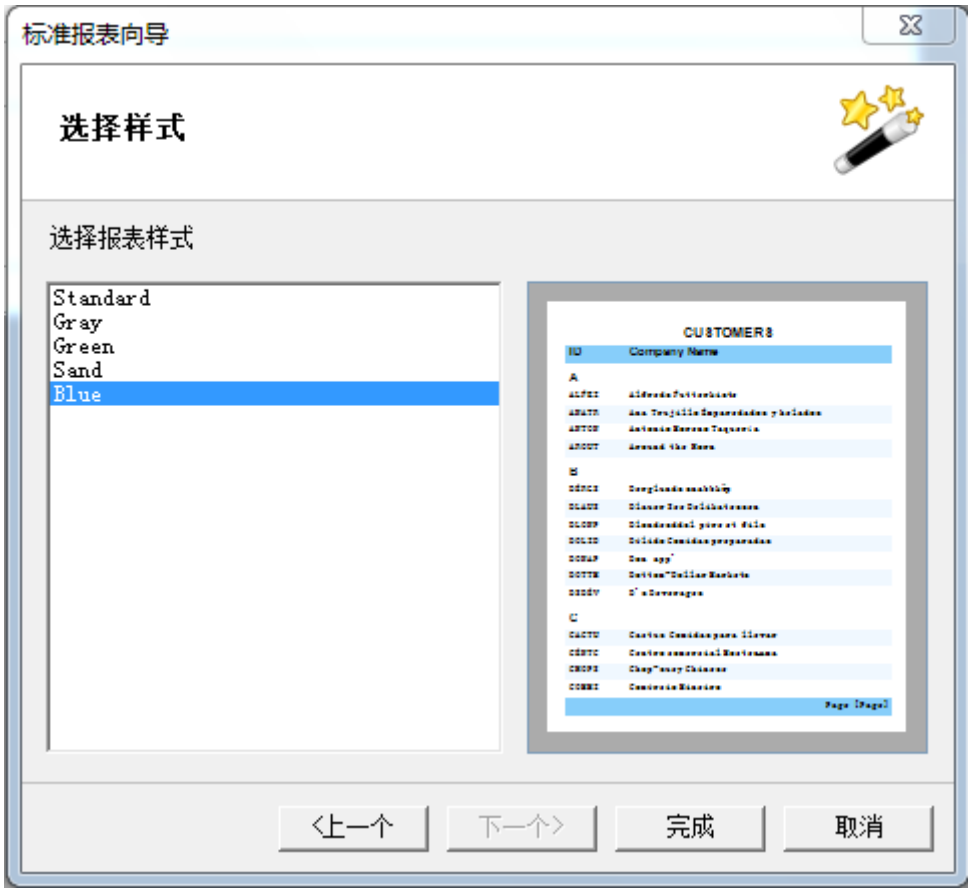
- 在向导的第一步，选择“Product”表，点击“下一个”按钮：



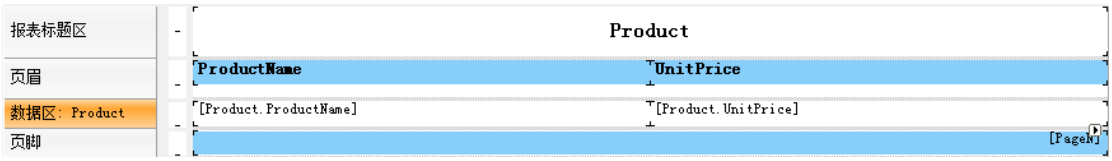
- 第二步，选择“ProductName”和“UnitPrice”字段：



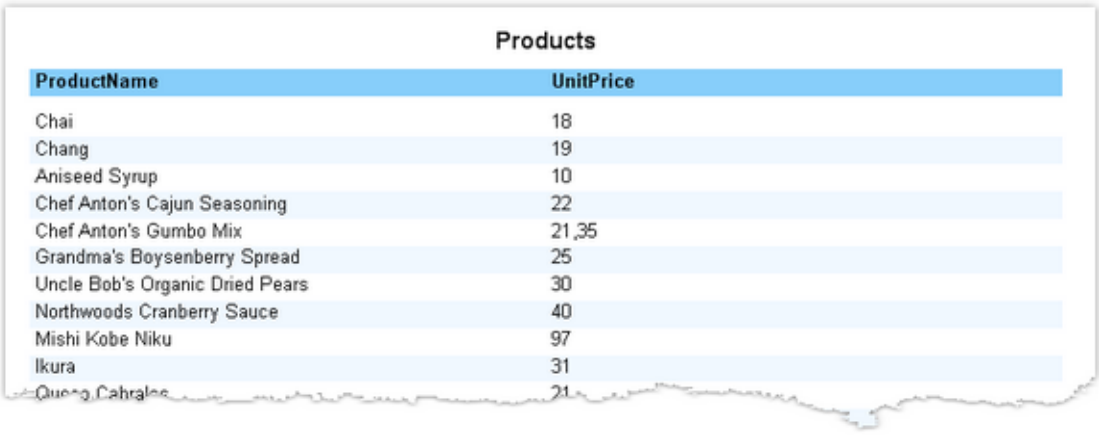
- 其它步骤可以跳过，点击“下一个”：
- 向导的最后一步，选择“Blue”报表样式，然后点击“完成”按钮：



Report 将创建以下报表：



可以通过点击工具栏中的  按钮运行此报表。报表即被创建并在预览窗口显示：



Products

ProductName	UnitPrice
Chai	18
Chang	19
Aniseed Syrup	10
Chef Anton's Cajun Seasoning	22
Chef Anton's Gumbo Mix	21,35
Grandma's Boysenberry Spread	25
Uncle Bob's Organic Dried Pears	30
Northwoods Cranberry Sauce	40
Mishi Kobe Niku	97
Ikura	31
Queso Cabrales	21

创建报表

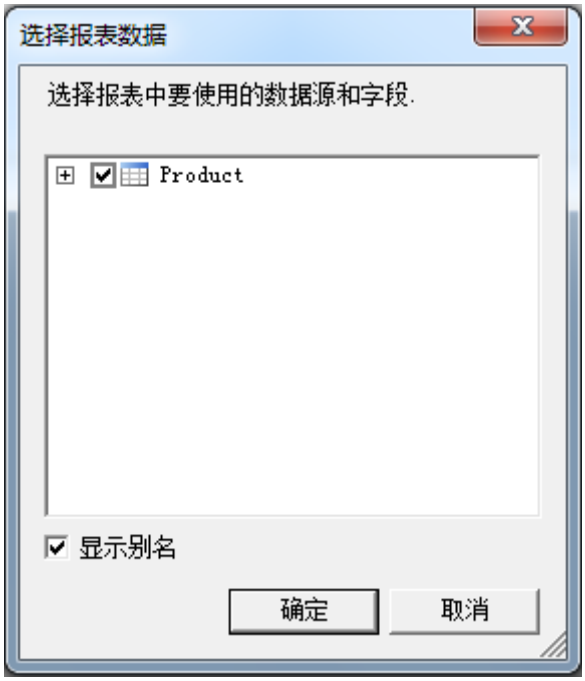
在这一章你将学习创建普通类型报表的方法。通常，创建任何报表，你都需要进行如下的操作：

1. 选择报表需要使用到的数据。
2. 通过向报表中添加需要的区域来创建报表结构。
3. 将报表区域绑定到数据源。
4. 在区域中放置文本控件用于显示打印数据。
5. 设置外观与格式。

选择报表数据源

在创建一张报表之前，你需要选择要在报表中打印的数据。有如下两种方法：

- 选择菜单“数据|选择数据源”来选择一个或多个数据源。



- 选择菜单“数据|添加数据源”来添加一个或多个数据源。

在“数据源”章节了解更多关于数据源的内容。

在你选择了数据源后，就可以在数据窗口看到。这时你就可以在报表中使用该数据源。许多报表只使用一个数据源。而对于“主-从”类型的报表，你需要选择两个数据源，并关联它们。在报表中也可能需要多个相关的数据源来打印数据。

动态布局

在构建一张报表时，经常会碰到要打印文本的长度不知道的情况。例如，可能是一个商品的描述。这种情况下，需要解决如下问题：

- 计算控件的高度，以便整个文本可以完全显示
- 计算栏位高度，使所有控件可以显示出来

- 移动或改变一个栏位内的其它对象的高度，以便于报表设计

这些问题可以通过设置控件与栏位的属性来解决：

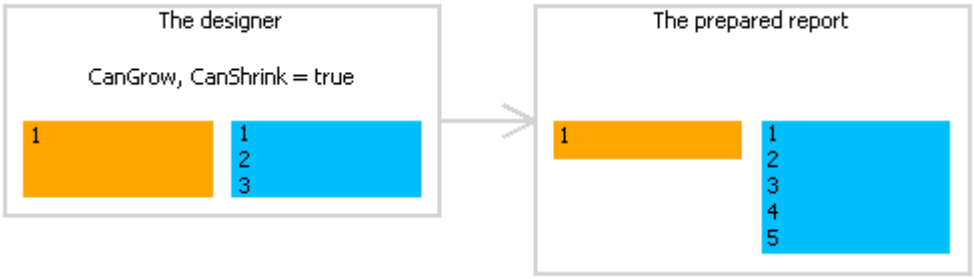
- CanGrow 和 CanShrink 属性允许自动计算对象的高度
- ShiftMode 属性允许移动位于底部的对象
- GrowToBottom 使控件与栏位的底部对齐
- Anchor 与 Dock 属性依据栏位的尺寸来控制控件尺寸

所有这些属性都可以在后面找到。

CanGrow, CanShrink 属性

每个栏位与报表控件都有这两个属性。通过设置这两个属性可以决定控件的尺寸是否随其内容的多少来自动变大与缩小。如果这两个属性都不可用，那么该控件将始终保持指定的尺寸。

在你打印不知道尺寸大小的文本时，这两个属性很有用。为了让控件去适应全部的文本，需要设置 CanGrow 和 CanShrink 为可用状态。

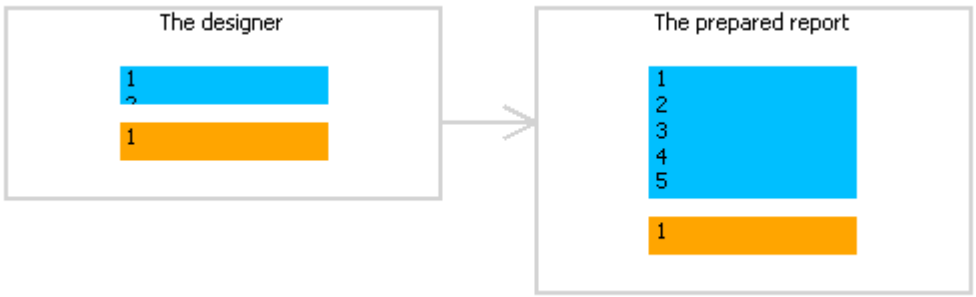


下面的控件可以影响到栏位的高度：

- 文本
- 多行文本
- 图片（使 AutoSize 为可用状态）
- 表格

ShiftMode 属性

每个报表控件都有这个属性。该属性只能在属性窗体中设置，如果 ShiftMode 属性为可用状态，如果对象可以放大或缩小，那么将可上下移动。



ShiftMode 属性可以设置为如下值：

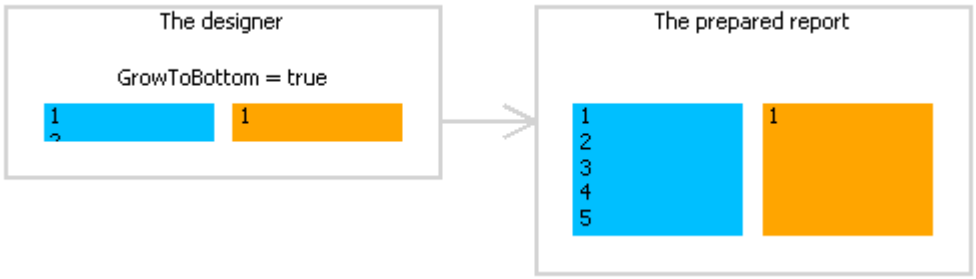
- Always（默认）。表示对象总是需要按 SHIFT

- Never。对象不需要 SHIFT
- WhenOverlapped。意味着该对象在位于扩展对象的正上方时，需要 shift（即两个对象在水平方向上重叠）。

当多个单元格位于彼此正上方并且文本内容可变时，这个属性便于打印表格信息。

GrowToBottom 属性

每个报表都有这个属性，打印报表时，设置该属性，其控件会与栏位的底部对齐。



以表格形式打印信息时，此属性是必须的。在表格的一行中，可以有多个可延伸的控件。该属性使得它可以设置所有控件的高度以适应区域的最大高度。

Anchor 属性

每个报表对象都有这个属性。它决定了当改变该控件的容器大小时，该控件如何改变其位置或大小。当使用 Anchor 属性时，该控件可以与其容器同步扩展或移动。

大多数情况下，容器是栏位。但这不是一定的-也可能是“表”或“交叉表”。

Anchor 属性可以设置为如下值，也可以将其任意组合：

Value	Description
Left	对象的左边缘。当容器的大小发生变化时，该对象不会被左右移动。
Top	对象的上边缘。当容器的高度发生变化时，该对象不会被上下移动。
Right	对象的右边缘。当容器的宽度发生变化时，该对象的右边缘与容器之间的距离是固定的。如果容器的左边缘也是固定的，那么该对象将与容器同步扩张和收缩。
Bottom	对象的底部边缘。当容器的高度发生改变时，该对象的底部边缘和容器之间的距离是固定的。如果该对象的上边缘也是固定的，那么该对象将与容器同步扩张和收缩。

默认情况下，该属性的值为 Left、Top。这意味着，当容器的大小改变时，该控件不发生改变。在下表中，列出了一些频繁使用的属性值的组合：

Value	Description
Left, Top	默认值，当容器大小改变时，对象不发生改变。
Left, Bottom	当容器的高度发生变化时，该对象上下移动。该对象相对于该容器的底部边缘的位置没有发生变化。

Left, Top, Bottom	该对象的高度与容器的高度同步改变。
Left, Top, Right, Bottom	当容器的宽度和高度发生改变时，对象将同步进行扩张或收缩。

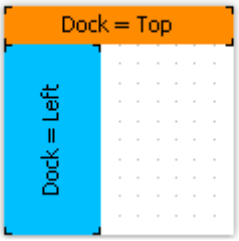
Dock 属性

每个报表对象都有这个属性。这个属性决定了对象位于容器的哪个边缘。

Dock 属性可以设置为如下值：

Value	Description
None	默认值。对象位置不固定。
Left	该对象位于容器的左边缘。对象的高度等于容器的高度。
Top	该对象位于容器的上边缘。对象的宽度等于容器的宽度。
Right	该对象位于容器的右边缘。对象的高度等于容器的高度。
Bottom	该对象位于容器的下边缘。对象的宽度等于容器的宽度。
Fill	该对象占用容器中的所有可用空间。

当多个对象同时放置在一个容器中时，结果并非完全如此。下图显示了两个对象，第一个对象位于容器的上边缘，第二个位于左边缘。



如图所示，第二个对象的高度等于除去第一个对象高度剩余空间的高度。

对接行为取决于对象的创建顺序。您可以在上下文菜单中，选择“置于顶层”或“置于底层”菜单项更改对象的顺序。

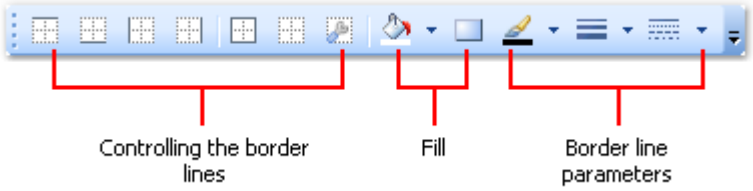
格式

在本节中，我们将着眼于以下几个问题：

- 改变一个对象的外观；
- 改变打印值的格式；
- 满足某种条件时，自动改变一个对象的外观；
- 隐藏不必要的值；
- 以不同的颜色突出显示数据行。

边框和填充

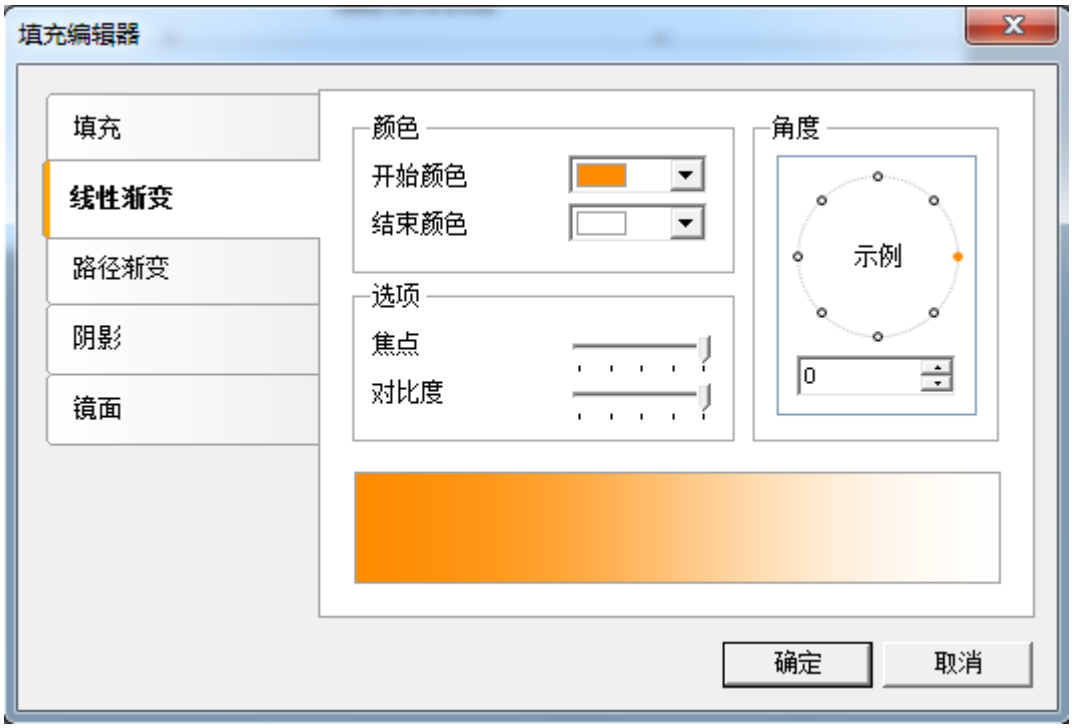
几乎所有的报表对象都具有边框和填充。你可以通过“边框和填充”工具栏使用这些属性：



对象边框包含四条线。每条线可以有不同的宽度、颜色和样式。工具栏按钮会影响框架中的所有线条。点击 按钮，显示一个对话框，该对话框允许单独设置线条：

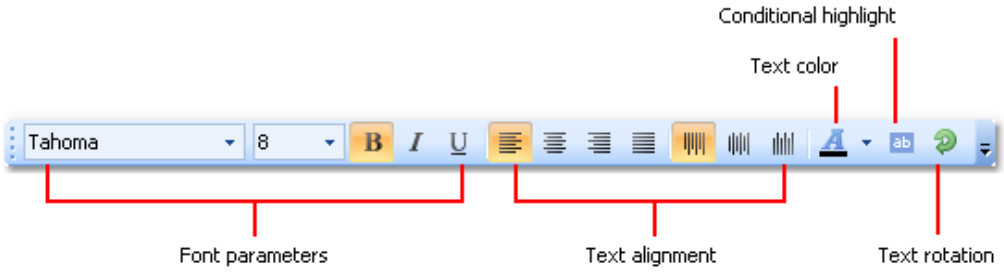


工具栏上有两个填充按钮。 按钮允许用户选择一个颜色进行实心填充。点击 按钮，将显示一个对话框，允许用户选择不同颜色之间的填充类型。



文本格式

使用“文本”工具栏，更改“文本”对象的外观。

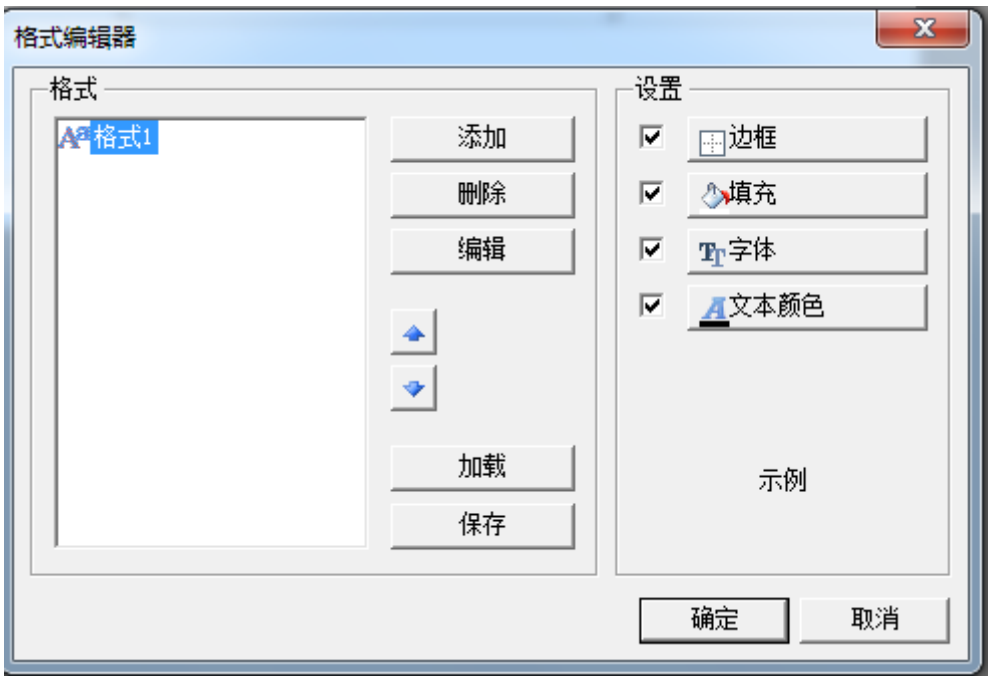


样式

您可以使用样式，来设置对象的外观。样式包含以下几个属性：

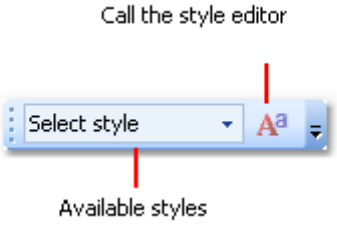
- 边框；
- 填充；
- 字体；
- 文本颜色。

样式列表存储在报表中，你可以通过“报表|样式…”菜单或者“样式”工具栏中的 Aa 按钮，实现对它的操作



您可以通过以下方式，设置对象的样式：

- 在“属性”窗口设置“样式”属性；
- 使用“样式”工具栏：



如果屏幕上没有工具栏，可通过“视图|工具栏”菜单让其显示。

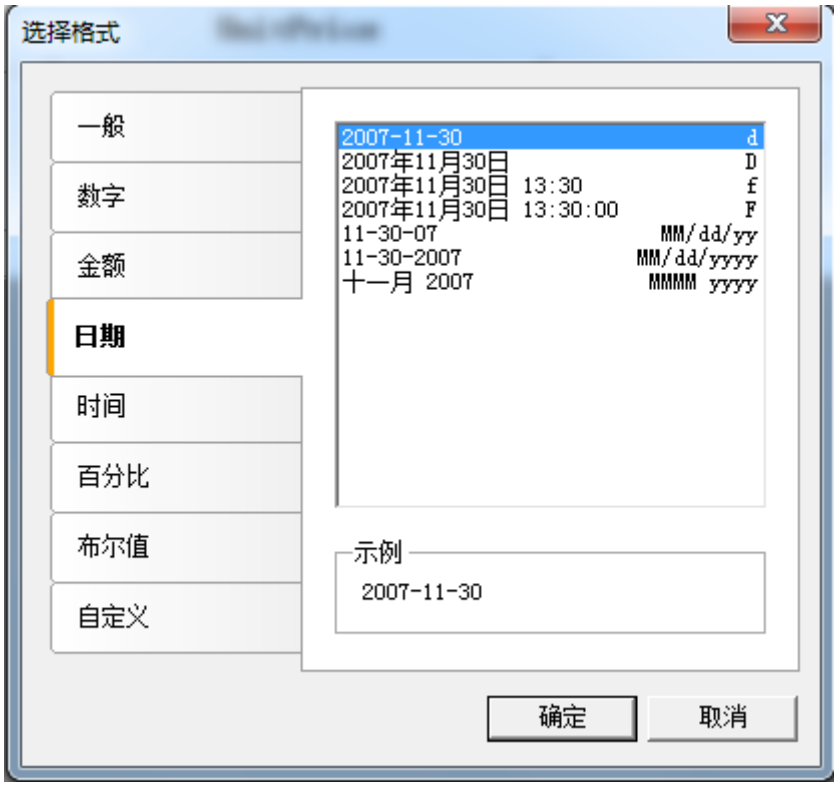
当您设置样式时，该对象的外观会根据样式的设置而改变。当您更改样式设置时，与其样式相关的对象会自动改变。

数据格式

为了打印报表中的文本数据，要使用“文本”。它默认适用于数据源的所有数据。例如，“日期”类型的数据源将以下列方式打印（它取决于你的系统区域设置）：

11.10.2008 18:04:52

如果您只需要打印日期部分，您必须设置数据格式。为了实现这一点，右键单击“文本”对象以显示其上下文菜单，选择“格式…”项。你会看到格式编辑窗口：



你可以选择一个可用的格式类型或设置自己的格式字符串。要做到这一点，需选择“自定义”格式。如果“文本”格式包含几个数据列或表达式，你可以为他们每个都选择相应的格式。为了实现这点，你可以选择窗口顶端的表达式，然后选择格式：

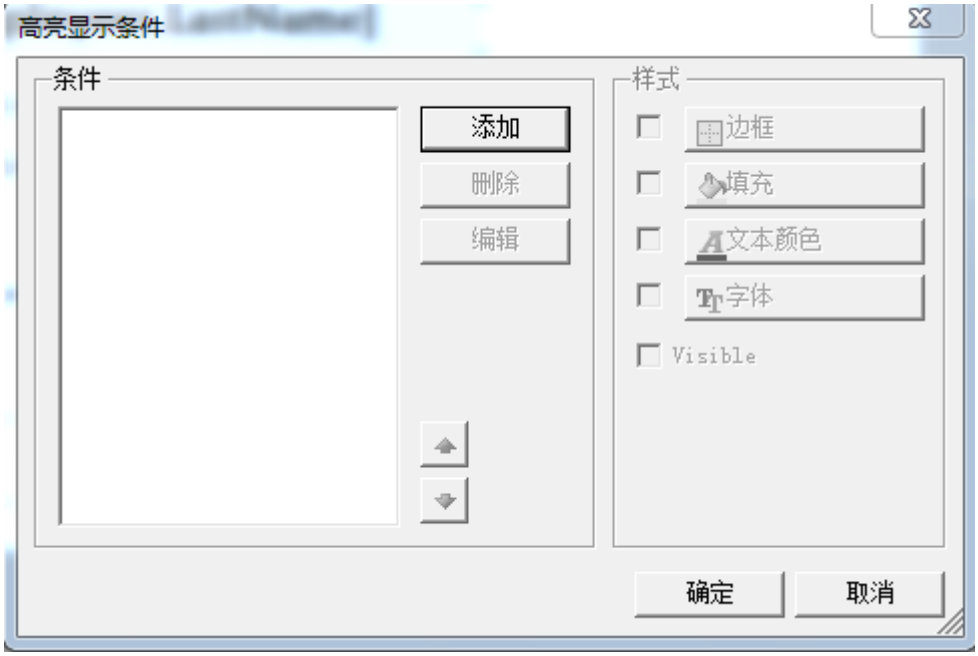


您也可以通过使用 String.Format 方法格式化数据：

```
Today is [String.Format("{0:d}", [Date])]
```

高亮显示条件

也许可以根据给定的条件更改“文本”的外观。例如，当一个对象具有负值时，可以用红色突出显示。这个功能称为”条件突出显示“。你可以通过选择“文本”对象和点击“文本”工具栏上的 ab 按钮设置此功能。你将看到以下对话框：

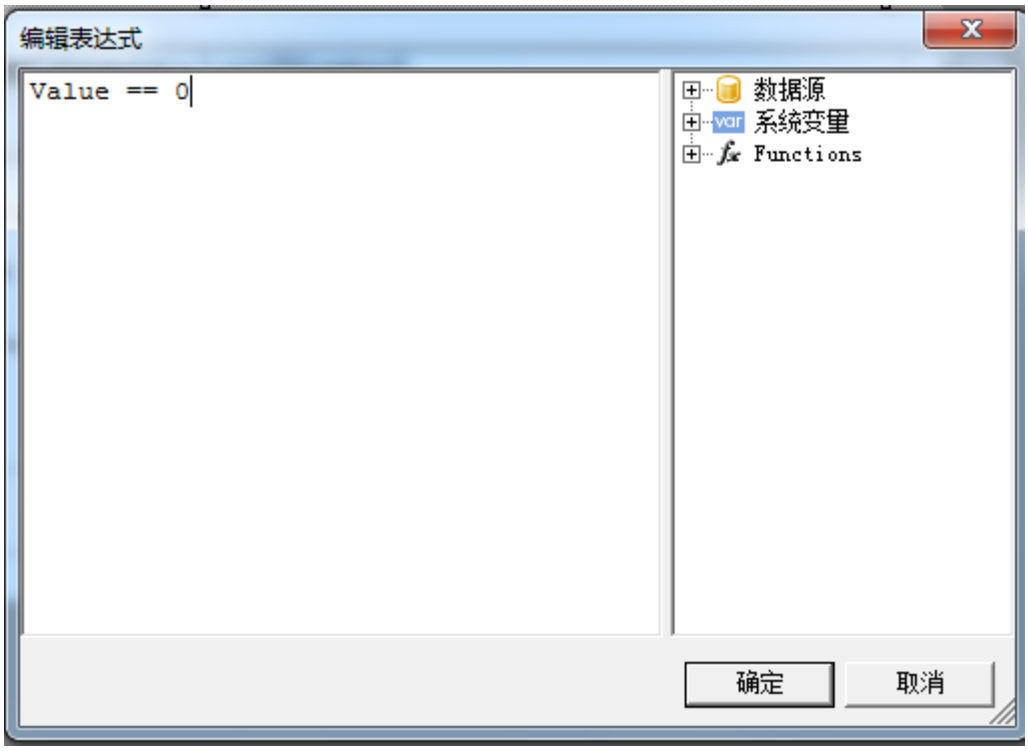


它可以定义一个或多个条件并且为每个条件设置样式。样式表可以包含一个或多个设置：

- 边框
- 填充；
- 文本颜色；
- 字体；
- 可视性

你可以标示，什么情况下需要更改设置。为了实现这一点，你可以使用复选框选择相应的设置。默认情况下，一个新样式包含一种设置-文本颜色。

为了添加一个新条件，您可以点击“添加”按钮。你将会看到一个表达式编辑器：



在此，你可以写任何返回布尔结果的表达式。大多数情况下，你会使用当前正在打印的值的“值”变量。

让我们看看下面的例子：在一个“文本”对象中，打印产品的库存量：

```
[Products.UnitsInStock]
```

当产品的数量=0 时，我们想将该对象涂成红色。则要创建下面的条件：

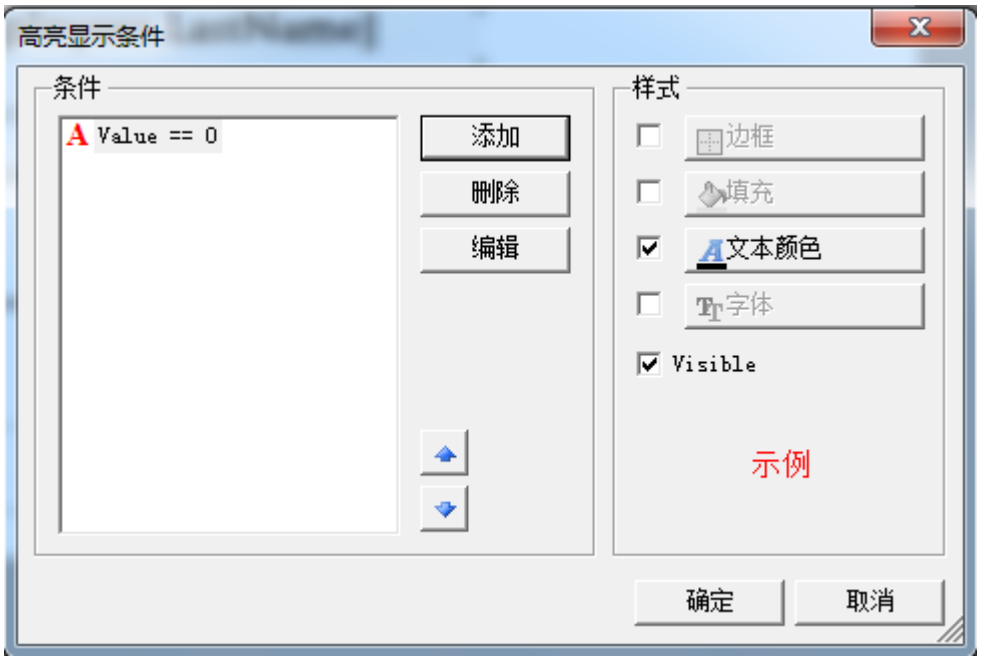
```
Value == 0
```

在给定的情况下，我们使用已经得到打印值的“值”变量。如果一个对象包含多个表达式，那么这个变量将用最后一个表达式的值。你可以使用数据字段代替“值”。

```
[Products.UnitsInStock] == 0
```

表达式以 C#的形式书写。如果选择的报表语言是 C#，对于 VB.NET 你必须使用单一的“=”符号。报表语言可以通过“报表|选项…”更改。

在给定的条件下，以下面的方式进行样式配置时，只有填充可用，并且选择了红色：



当打印的对象中，包含零值时，它将是红色的。我们将添加另一个条件，来使我们的例子更加复杂。如果库存单位小于 10，它将以黄色打印。为了做到这点，需打开条件编辑器，点击“添加”按钮。第二个条件如下所示：

```
Value < 10
```

在标明的几个条件下，Report 会从第一个条件开始检查。如果满足一定的条件，Report 将其样式设置应用到对象，并停止进程。条件的正确顺序很重要。我们在这个例子中看到的顺序是正确的：

```
1. Value == 0
2. Value < 10
```

如果我们交换条件，突出显示将工作错误：

```
1. Value < 10
2. Value == 0
```

在上面的例子中，Value==0 将不被执行，因为当其值为 0 时，第一个条件已经满足。可以通过 和 按钮，改变条件的顺序。

隐藏零值

“文本”对象有“HideZeros”属性，用于隐藏零值。让我们来看一个包含以下内容的对象：

```
Total elements: [CountOfElements]
```

如果变量 CountOfElements 的值等于 0，而 HideZeros 的属性为 true。则该对象将以下列方式打印：

```
Total elements:
```

“文本”对象还有“HideValue”的属性，它用于隐藏等于给定值的表达式的值。例如，如果属性值为“0”，那么所有 0 字段都被隐藏。该属性也可以被

用于隐藏为零的日期。作为一项规则，它就像“1.1.0001”或“1.1.1900”的日期。在这种情况下，“HideValue”属性的值必须是样的：

```
1.1.1900 0:00:00
```

如你所看到的，除了日期，你还需要标明时间。这是必须的，因为.NET 中的日期类型也包含时间。

重点注意事项：这种机制取决于你系统的区域设置，它可以在控制面板中进行设置。这是因为 Report 中使用“ToString（）”方法比较字符串。此方法将一个表达式的值转换为字符串。与此相关，在创建一个可以在使用不同区域设置的计算机上启动的报表时，需要多加注意。

最后，“文本”对象的“NullValue”属性允许打印一些文本信息。它经常被用于打印破折号，而不是空值。让我们看一个包含以下内容的对象：

```
Total elements: [CountOfElements]
```

如果变量的 CountOfElements 值为 null，而 nullvalue 属性值设置为 - ，则该对象将打印如下：

```
Total elements: --
```

隐藏重复值属性

“文本”对象具有“Duplicate”属性，它允许控制如何打印重复值。如果“文本”对象在“数据”栏位，则该属性可用。如果他们打印时与数据行相邻，则被认为是重复值。

“Duplicates”属性可以设置为以下值：

- Show-显示重复值（默认）；
- Hide-隐藏具有重复值的对象；
- Clear-清楚对象中的文本，但是显示对象；
- Merge-将具有相同值的对象合并。

这些模式之间的区别如下图所示：



突出显示奇/偶行

为了美化报表外观，你可以用不同的颜色突出数据行。这个可以通过使用栏位的“EvenStyle”属性或其对像实现。这个属性包含一个样式名称，用于突出显示栏位行。

通常我们使用“EvenStyle”代替栏位。这样可以在导出报表时，避免可能出现的问题。

为了配置突出显示，请执行以下操作：

- 可以通过“报表|样式…”为要突出显示的行定义样式。
- 在栏位或者其对象的“EvenStyle”属性中，表明新样式的名称。

默认情况下，对象只能使用“EcenStyle”属性给出的填充属性。这项操作在“EvenStylePriority”属性中定义-默认情况下是“UseFill”。如果你需要使用其它样式参数，将此属性设置为“UseAll”。

下图是使用这种的技术一个现成报表：

Product name	Unit price
Chai	18,00
Chang	19,00
Chartreuse verte	18,00
Côte de Blaye	263,50
Guaraná Fantástica	4,50
Ipoh Coffee	46,00

只有一个“数据”区域的报表

这种类型的报表被频繁使用。它允许从数据源打印行。例如，顾客表。

连接区域到数据源

要打印数据源中的数据，你需要已经连接到数据源的“数据”栏位。该栏位将根据数据源中的行数进行多次打印。

如果该“数据”栏位没有连接数据源，它只被打印一次。

当你创建一个新报表时，它已经包含了多个空栏位，包括“数据”栏位。你同样可以通过选择菜单“报表|栏位配置”选项，在弹出的“栏位配置”窗口添加栏位。

为了实现栏位与数据连接，可以双击它。在编辑器窗口选择数据源，然后点击“OK”。

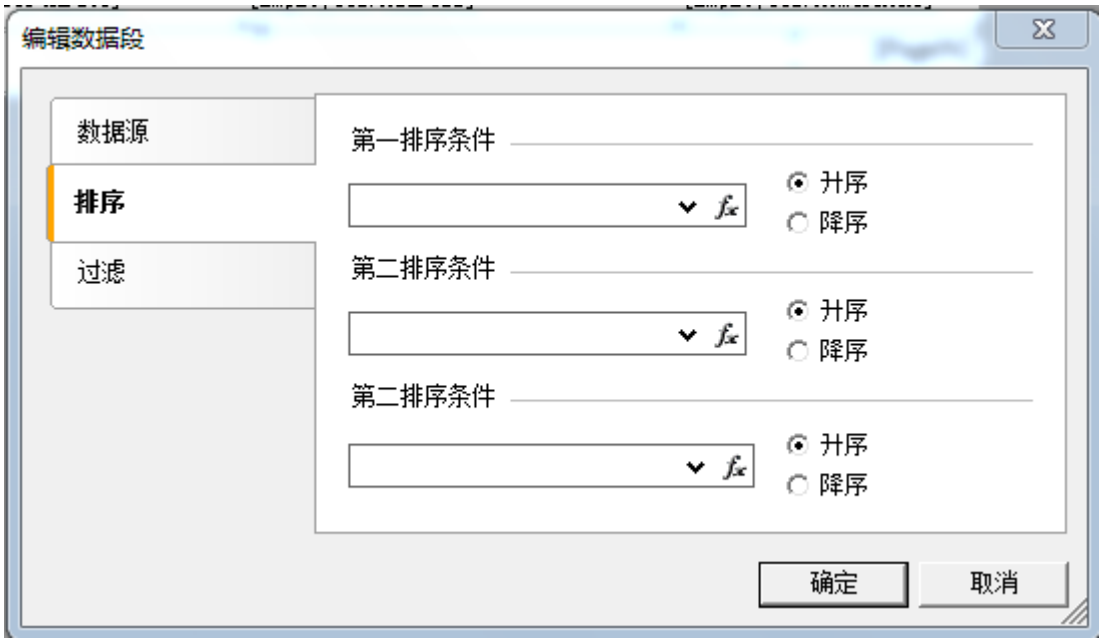
打印文本

当你将 Band 与数据连接后，你可以将“文本”对象放置在栏位中，它将显示数据字段中的信息。最快的方法是-从“数据”窗口拖拽一个数据字段放置在该栏位上。你可以在“文本对象”一章了解更多关于“文本”对象的内容。

数据分类

默认情况下，“数据”栏位按自然顺序打印数据。通常我们需要在打印前，对数据进行分类。例如，客户列表可以通过按字母排序更舒适的展示。

你可以在“数据”栏位编辑器控制分类。双击栏位的空白区域，便可调出编辑器。

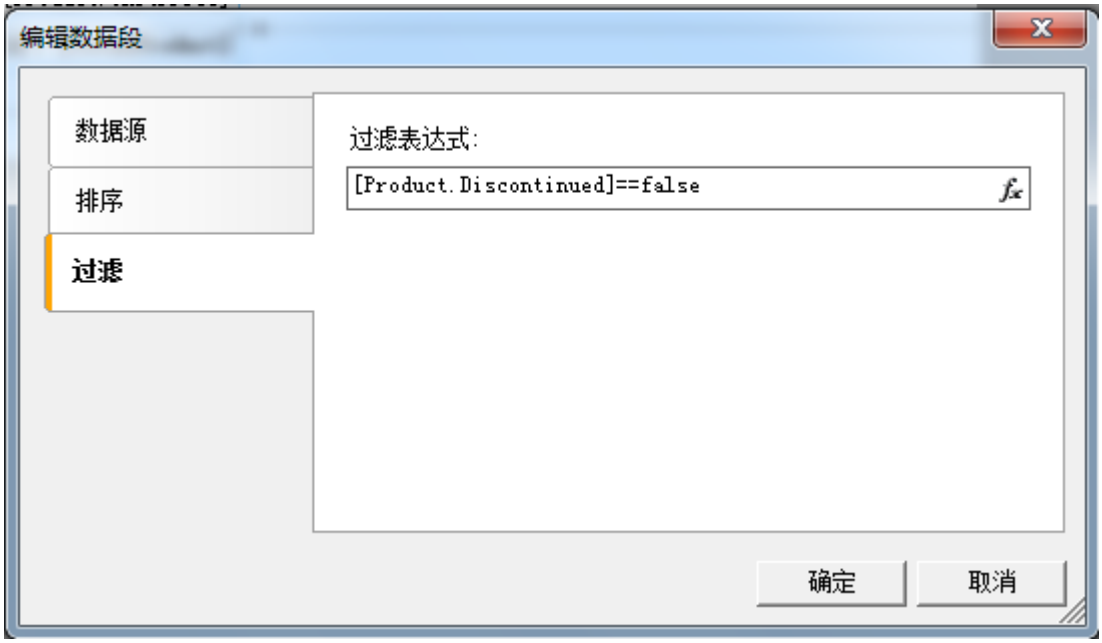


你可以选择一个数据字段或表达式，作为分类标准。您可以标示几个不同的分类（不超过三个）。这有时是必要的，比如，如果你希望客户通过他们的城市进行排序，除此之外，还有根据客户的姓名排序。对于每个条件，你可以选择分类的顺序-升序或降序。

数据分类的另一种方法-使用 SQL query 作为数据源。该查询将在数据服务器上执行，并返回一个已经排序的行。

数据过滤

为了过滤在“数据”栏位打印的行，调出它的编辑器并切换到“Filter”标签：



你可以标示任意正确的表达式作为过滤表达式。有关表达式的详细信息可以在“表达式”一章找到。

上面的例子使用了下面的表达式：

```
Products.Discontinued] == false
```

这意味着，所有“Disconnected flag”等于 false 的数据行，都将被选中。

您也可以使用复杂的过滤条件：

```
[Products.Discontinued] == false && [Products.UnitPrice] < 10
```

这意味着，所有“Disconnected flag”等于 false 并且其价格低于 10 的数据行，都将被选中。

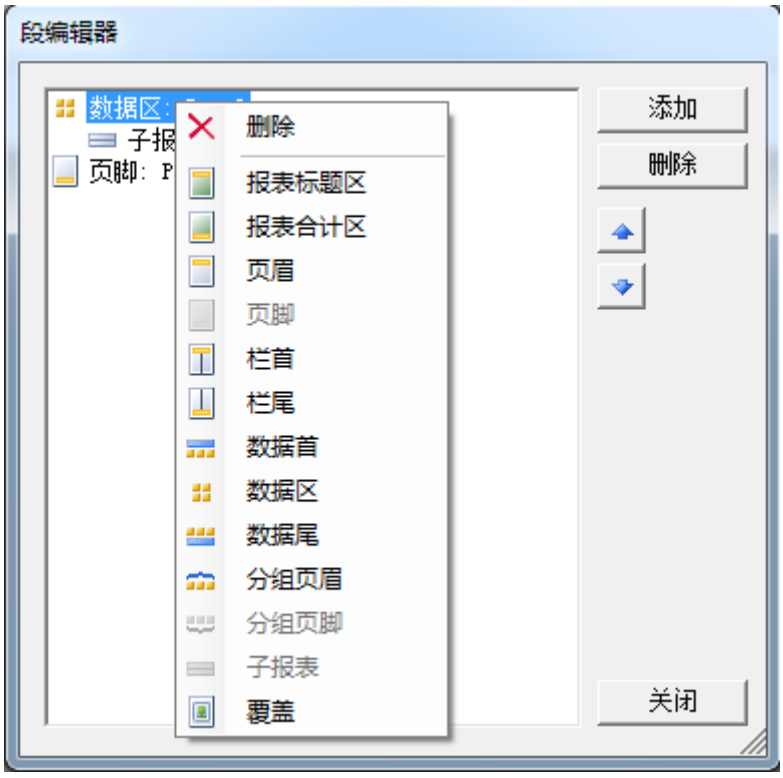
假设数据源包含所有行，而这种过滤方法将过滤其中的一部分。如果数据源包含大量的行，这会拖慢报表。在这种情况下，你可以使用 SQL 查询语句作为数据源，您可以在其中执行所需的过滤。该查询将在数据服务器上执行，在报表中只返回那些需要的行。

您也可以使用对话框的形式进行数据过滤。详情请参见“对话框形式”章节。

数据页首和页尾

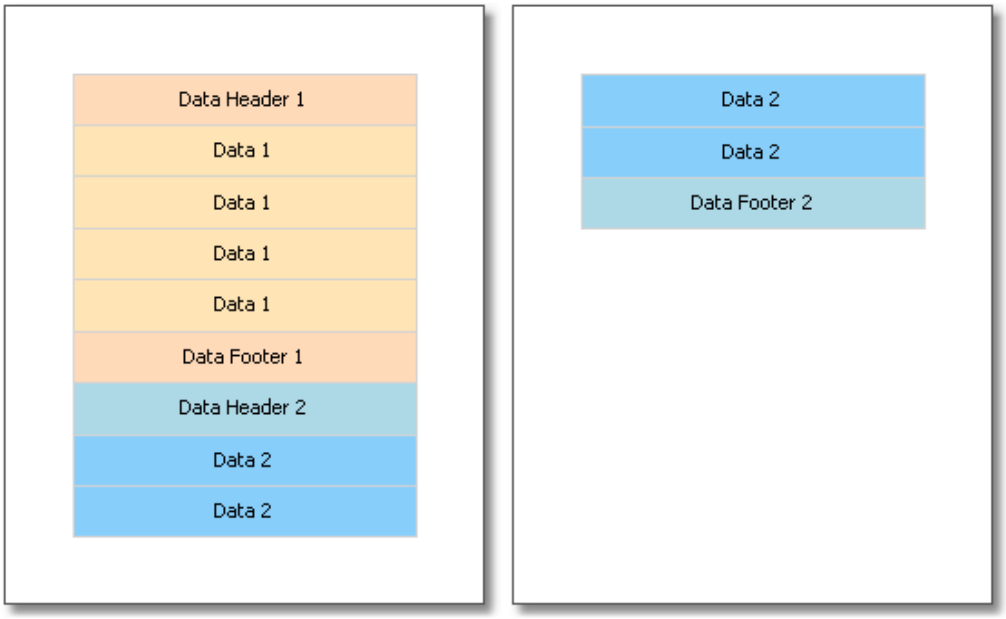
一个“数据”band 包含一个页首和一个页尾。页首在数据打印之前被打印，页尾在所有数据打印之后被打印。

您可以通过选择菜单“报表|设置报表栏…”，添加页首和页尾。在该窗口，选择“数据”band，单击鼠标右键。在下拉列表中选择“页首”或“页尾”选项。

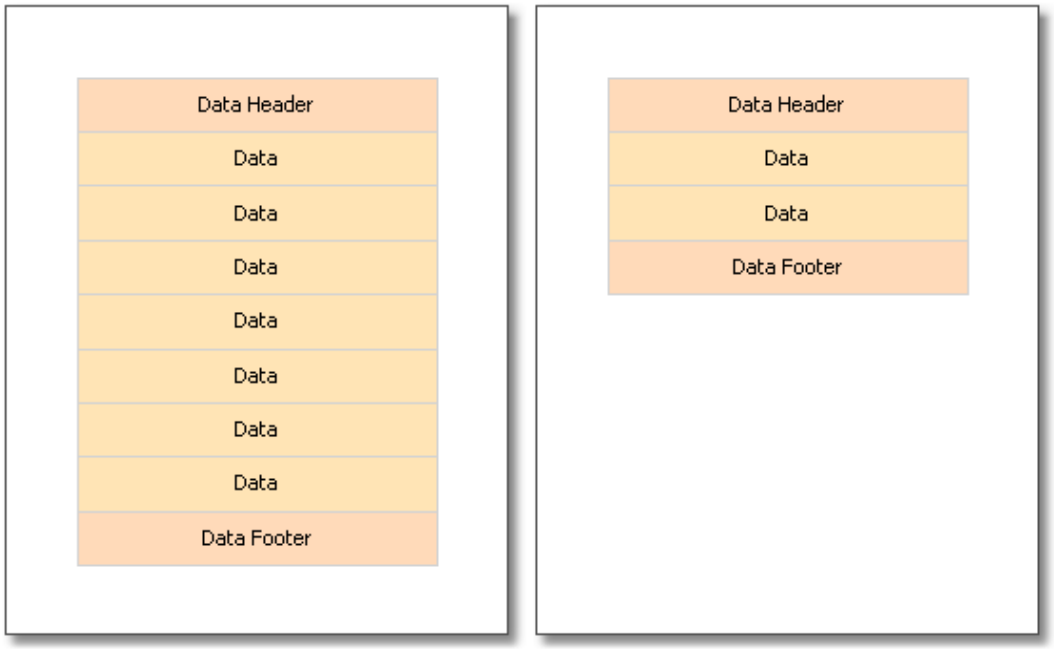


这些 band 在下面的情况中很有用：

当在一个页面上打印多张表（“主-主”报表）的情况下，每个数据 band 可以有自己的页眉和页脚：



打印一个列表时，如果报表的一页无法完全打印该列表。可以通过使用数据“页眉/页脚”的“重复在每一页”属性，在报表的每一页均打印这些栏位。



Breaking data and keeping it together

在本节中，我们将着眼于两种数据印刷的模式-“分隔”和“keep together”。

在常规 band 打印模式下，Report 会检查当前页面是否有足够的空间打印一个栏位。如果没有，该栏位将在下一页面完整打印。如果 band 的“CanBreak”

属性是 enabled，那么 Report 会尝试在当前页面的可用空间打印 band 的部分内容，即“分隔”band。

区域分隔的尝试成功与否，取决于放置在区域上的对象的类型以及它的配置。以下对象可以被分隔：

- “文本”
- “多行文本”；
- “表格”

这些对像也具有“CanBreak”属性。如果已启用，则该对象可以被分隔。不能分隔的对象总是在足够的空间上完整显示。

下图显示了一个区域如何被分隔：



分隔算法并不总是正常工作。当一个区域存在多个具有不同字体的对象时，该算法会发生异常。

区域分隔的目的是为了节省打印表的空间。数据连接的目的恰恰相反：在一张纸上完整显示一组区域。这种情况，会有大量未使用的空间，但是数据却以一种舒适视觉的方式被打印。

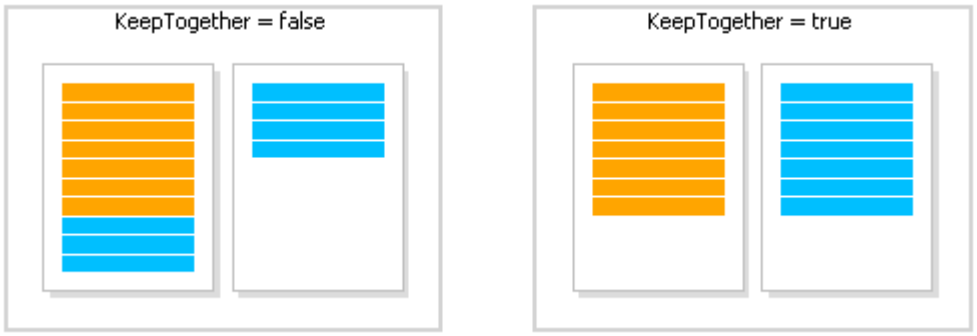
“保持在一起“允许在一个页面上保留一组区域（或列，如果报表有列）。如果打印时，数据到达页面的结尾，FatReport 会在另一个新页面上重新定位已经准备要打印的所有数据。

你可以在下列情况使用“保持在一起”：

- 同页打印“数据”band 的所有行；
- 同页打印组的所有元素（页首，数据，页尾）；
- 将主数据源的行与明细表的所有行同页打印（“主-从”报表）；
- 报表的页首或数据头至少与一行数据同页打印；
- 报表页尾或数据尾至少与一行数据同页打印；
- 主子区域同页打印

下面介绍“保持在一起”的用法：

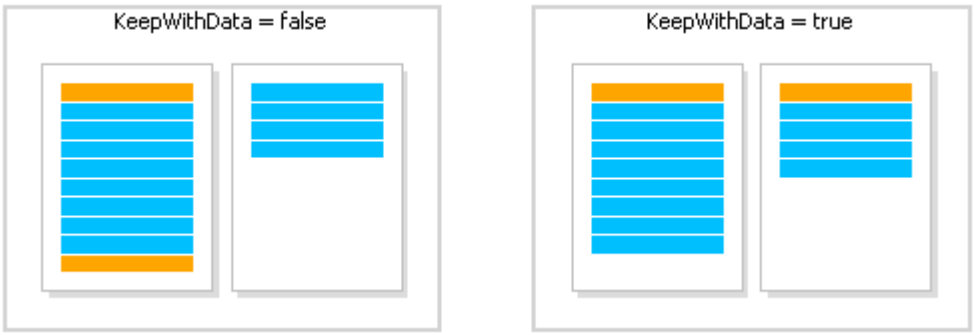
为了将所有数据行或一组元素（页首，数据，页尾）保持在一起，需要激活“Keep Together”属性。此属性用与“数据”和“分组页眉”。下图显示了没有 keep together 时，数据如何打印：



为了防止“hanging”页首和页尾，我们使用“KeepWithData”属性。下面的区域包含此属性：

- 报表抬头；
- 报表尾；
- 数据头；
- 数据尾；
- 分组页眉；
- 分组页脚

该属性要求首/尾至少和一个数据行相邻：



为了保存区域和子区域在一起，可以激活“KeepChild”属性。

打印空数据行

在预先设置好的表格上打印时，通常要打印固定量的数据行。如果实际数据行小于固定的数据行，则需要打印空行。这可以通过将它连接到“数据”区域，并在“子”区域的帮助下实现操作。

“子”区域具有“最少数据行”属性。如果该属性的值设置为大于 0，区域将添加数据行到指定量。例如，我们需要打印 15 行，但数据源中只有 8 行数据。这种情况下，“子”区域会则会打印 7 次。



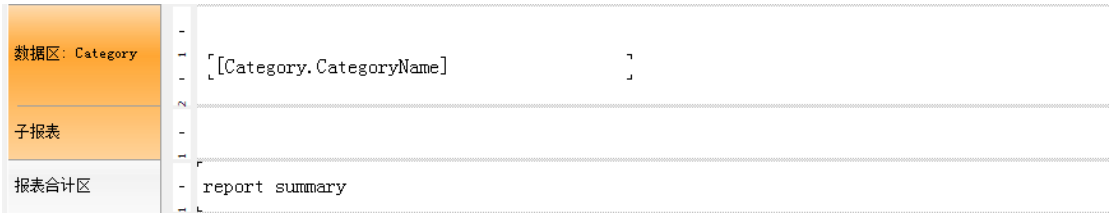
所制作的报表如下图所示：

1	Beverages
2	Condiments
3	Confections
4	Dairy Products
5	Grains/Cereals
6	Meat/Poultry
7	Produce
8	Seafood
9	
10	
11	
12	
13	
14	
15	

如果数据源中的行数超过“最少数据行”属性设置的固定行数，那么空行将不会被打印。

打印空行的另一个方法是，填充页面的可用空间。这种情况下，“子”区域连接到“数据尾”或“群组区域”类型的 band，并且填充该页面的可用空间。页尾区域将在页面的底部打印。

为了以这种方式打印一个空行，需要将“子”区域连接到页尾区域，并启用它的“填充未试用空间”属性。你将看到子区域在其绑定区域的上方显示。下图显示的是“子”区域连接到“报表合计区”区域：



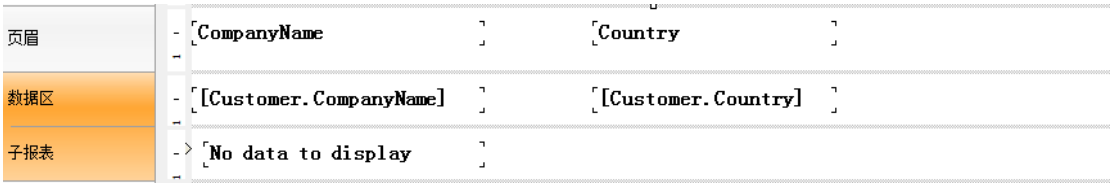
当我们运行报表时，会看到以下内容：



打印“无数据”文本

当数据区域连接一个空数据源时，它将不会被打印。有时，我们需要打印一些“无数据”的文本，而不是一个空白页面。

- 在数据区域添加子区域；
- 将子区域的 PrintIfDatabandEmpty 的属性设置为 true（也可以在“属性“窗口”设置）；
- 在子区域上添加“文本”对象，并写入“No data to display”文本。



报表将以下列方式打印：

- 如果数据源包含多行数据，则数据区域和其相连的区域将同时被打印（数据首/数据尾）；
- 如果数据源为空，则只打印“无数据显示”文本的子区域

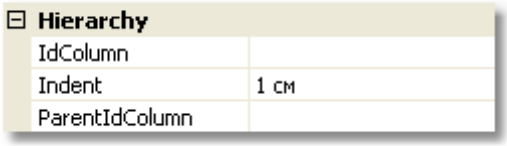
分层打印

一个“数据”区域允许打印层次列表。为此，需要一个区域和一个数据源。分层结构必须在包含两个数据字段的数据源的帮助下定义：

- 主键列。这是数据行的标志符。

- 包含此项目的主键的字段。

为了打印分层形式的报表，你需要在“属性”窗口设置以下“数据”区域的属性：

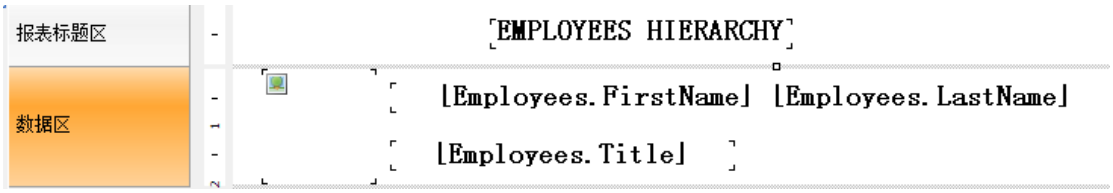


- 在“IdColumn”属性中标示主键；
- 在“ParentIdColumn”属性中，标示包含父值的列；
- 在“Indent”属性中设置分层结构的缩进。

让我们看看如何从“员工”演示表中打印员工的分层结构。该表包含我们需要的两列：

- EmployeeID 字段是主键并包含员工 ID;
- ReportsTo 字段包含“父”员工的 ID。

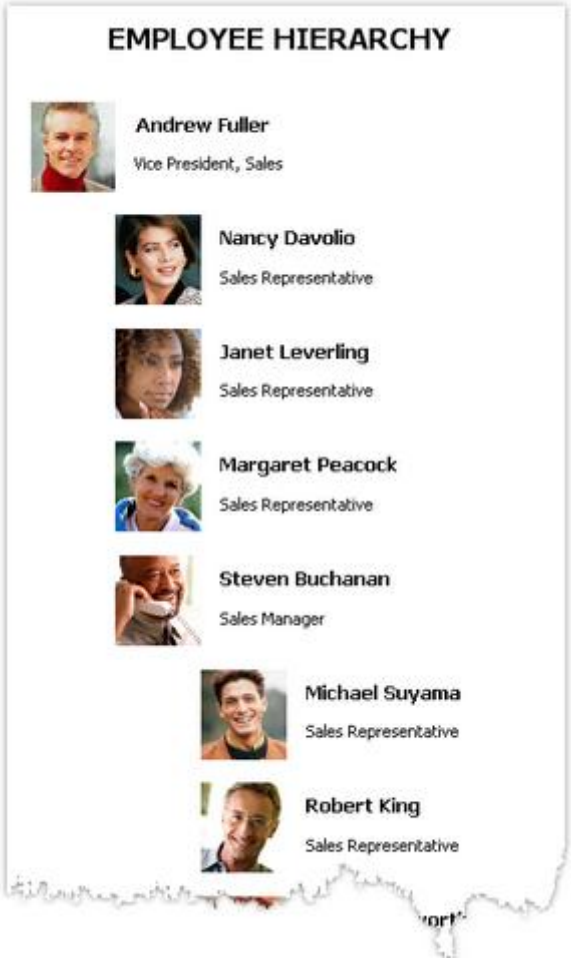
创建一个如下图所示的报表：



以下面的方式，设置负责分层结构的“数据”区域属性：



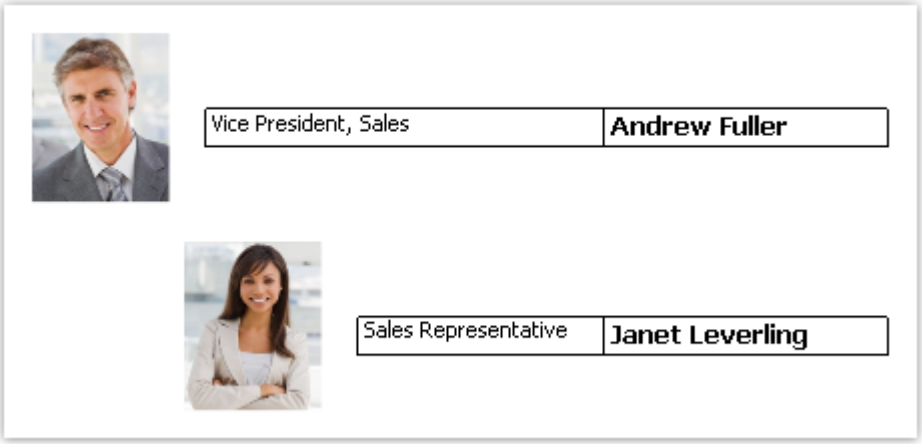
当我们运行报表时，我们会看到以下内容：



当打印层次结构时，Report 移动区域到右侧（由 Indent 属性的值标示），也可以通过相同的值减少区域的宽度。这使您可以使用区域的对象的 Anchor 属性。下面是该属性可以在这种情况下使用的可能值：

- 左，上（默认）-对象随区域移动；
- 右，上-该对象固定在它原始的位置；
- 左，右，上-对象的右侧停留在原始的位置，左侧随区域移动。

它可以让你得到一些有益的效果：



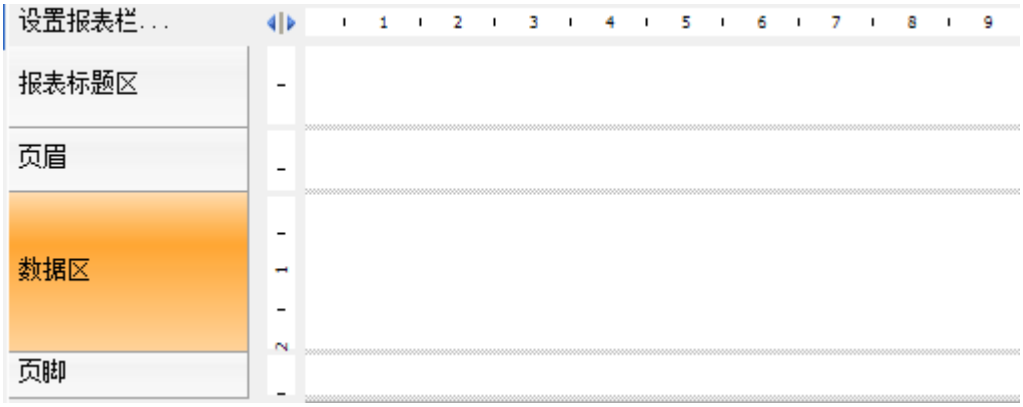
在这个例子中，图片对象的 Anchor 属性设置为左，上；职称对象设置为左，右，上；名称对象设置为右，上。

主-从报表

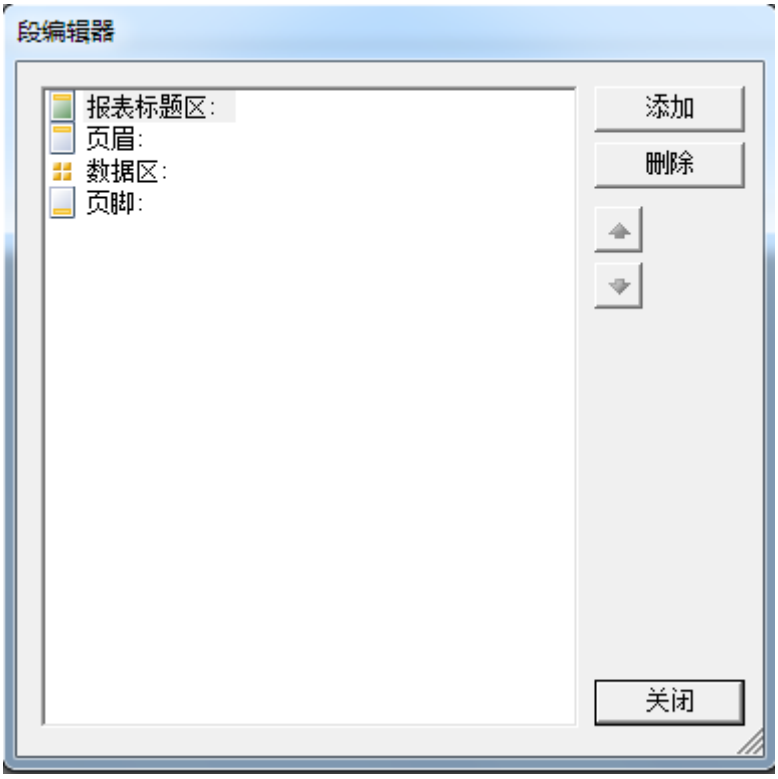
通过使用两个“数据”band，很容易创建“主-从”报表。在这份报表中，使用两个数据源，他们之间存在关系。主表的一行可以包含明细表的多行数据。更多明细表的关联将在“数据”章节找到。

我们可以用以下方式在报表中添加一个区域，明细表区域放置在主区域中。选择菜单“报表|配置区域…”，调出“配置区域”窗口。

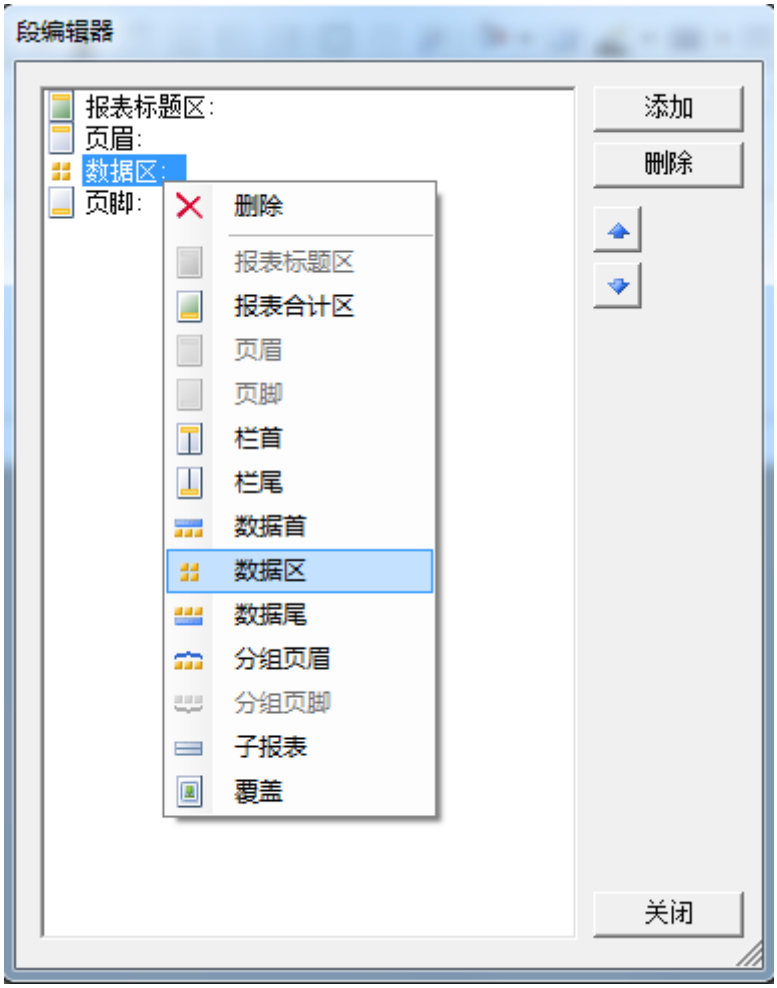
让我们从头开始看如何创建主-从报表。为此，我们将打开报表设计器和添加新报表。此时它已经包含了一个“数据”区域：



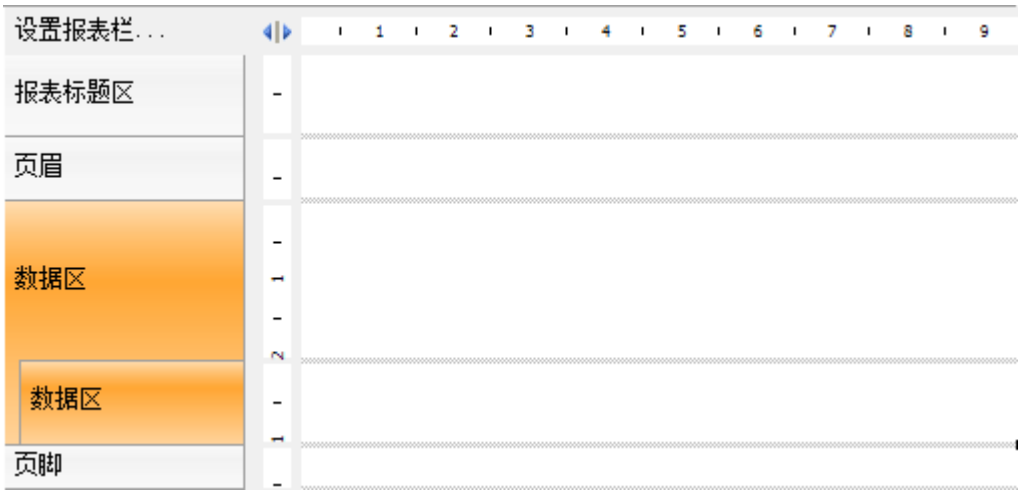
为了添加一个明细表区域，需打开“设置报表栏”窗口。我们可以通过点击“设置报表栏…”按钮，或者选择菜单“报表|设置报表栏…”实现此操作。在配置窗口，将显示区域结构：



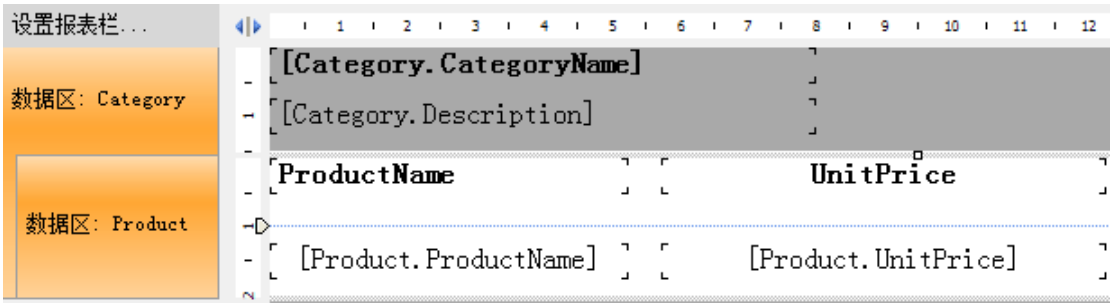
如图所示，选择“数据区”band，单击鼠标右键显示下拉菜单（或者点击窗口下方的”添加“按钮）。在打开的窗口中，选择“数据区”band：



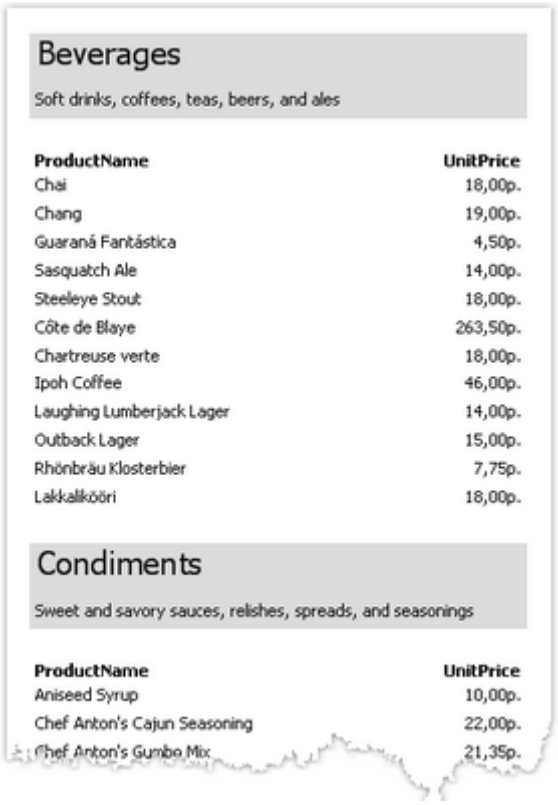
在此之后，一个嵌套的“数据”被添加到所选择的区域。点击“关闭”按钮，关闭窗口。你将看到报表模板发生以下的变化：



嵌套的数据区域可以在窗口的左侧清晰的看到。在此之后，你需要将该区域与相应的数据源连接并将数据字段放置在该区域上。我们将使用两个数据源 Categories 与 Products-从 Report 自带的样本数据库中选择。



如果我们运行该报表，将看到以下内容：


Beverages

Soft drinks, coffees, teas, beers, and ales

ProductName	UnitPrice
Chai	18,00p.
Chang	19,00p.
Guaraná Fantástica	4,50p.
Sasquatch Ale	14,00p.
Steeleye Stout	18,00p.
Côte de Blaye	263,50p.
Chartreuse verte	18,00p.
Ipoh Coffee	46,00p.
Laughing Lumberjack Lager	14,00p.
Outback Lager	15,00p.
Rhönbräu Klosterbier	7,75p.
Lakkalikööri	18,00p.

Condiments

Sweet and savory sauces, relishes, spreads, and seasonings

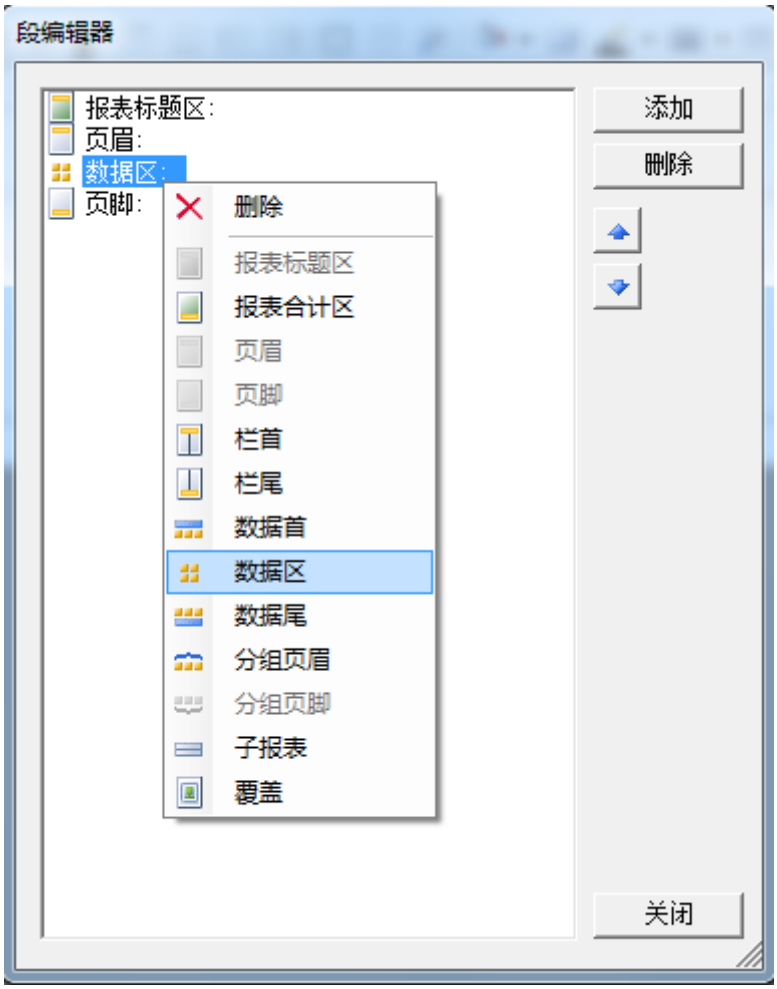
ProductName	UnitPrice
Aniseed Syrup	10,00p.
Chef Anton's Cajun Seasoning	22,00p.
Chef Anton's Gumbo Mix	21,35p.

通过这种方式，你可以创建一个无限嵌套数据的主-从报表，例如，主-细-子细报表。另一种方法，与嵌套报表连接，用于创建主-从报表类型。

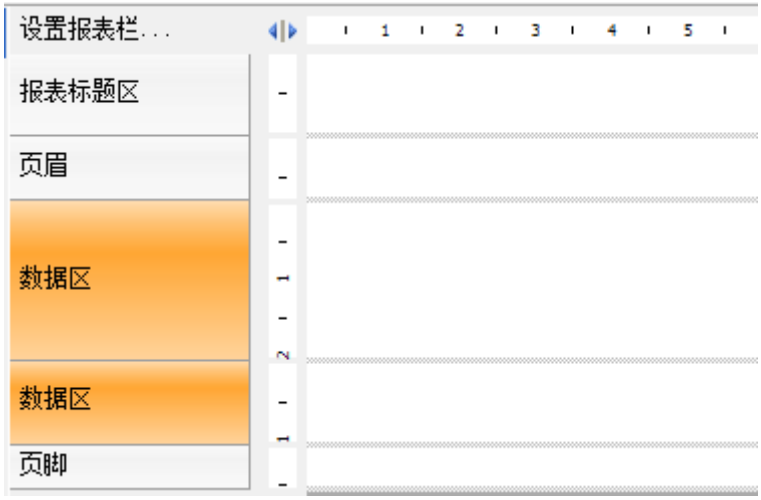
主-主表

在报表页面，你可以打印几个简单的列表。这可以通过将两个或多个“数据” bands 放置在页面上实现。主-从表中，区域互相嵌套并从相关联的数据源打印数据，而此类型与主-从表相反，它的两个区域和数据源互不依赖。

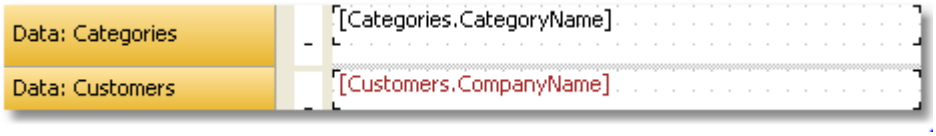
我们将通过一个例子说明，如何创建一个在一张页面上打印两列表的报表-类别表和客户表。我们将创建一个新报表，并与相应的数据源连接。为了添加第二个“数据”区域，我们可以打开“设置报表栏”窗口。



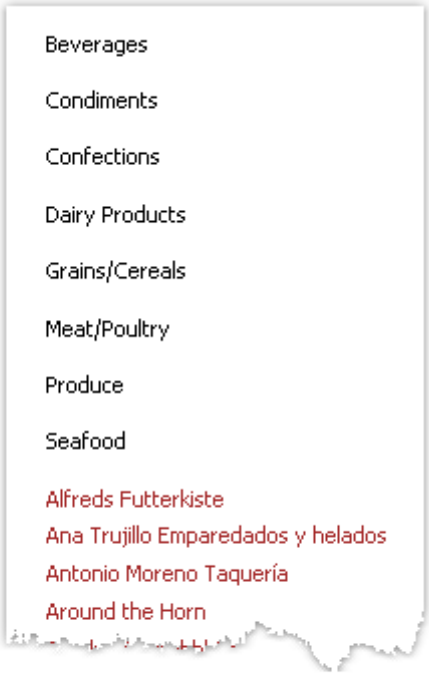
在列表的空白位置右键单击，选择下拉菜单中的“数据”区域，如下图所示。这将创建一个新的独立的“数据”区域。报表模板如下图所示：



现在，我们将连接区域到数据源，并在区域中添加几个数据字段：

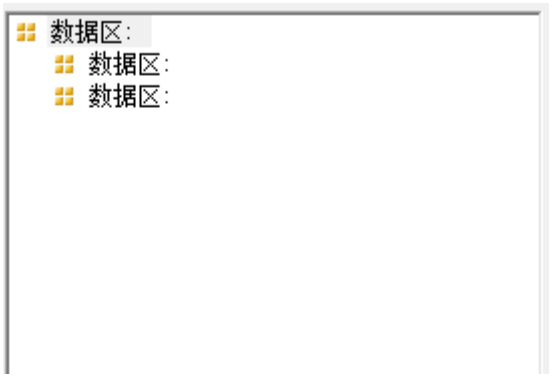


如果我们运行报表，我们将看到下面的内容：

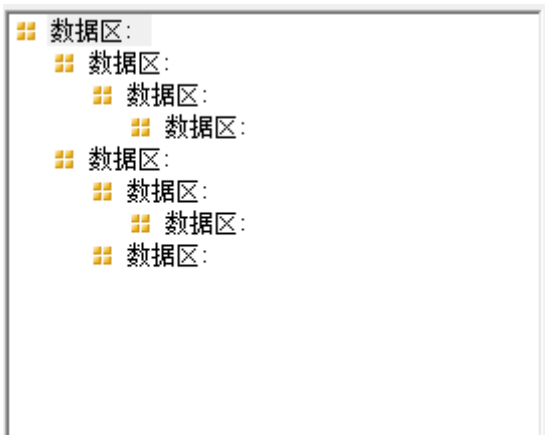


主-细-子细报表

一个“数据区”区域包含一个或多个嵌套的“数据区”区域。这样就可以创建一个主-细-子细报表。为此，需打开“设置报表栏”窗口，在主“数据”区域右键单击，在其中添加一个明细“数据”区域。重复此步骤，以添加第二个明细区域。



以这种方式，可以在主“数据区”区域添加无数个明细区域。一个报表的结构类似下图（这只是一个例子，它只是 Report 表面的功能）：



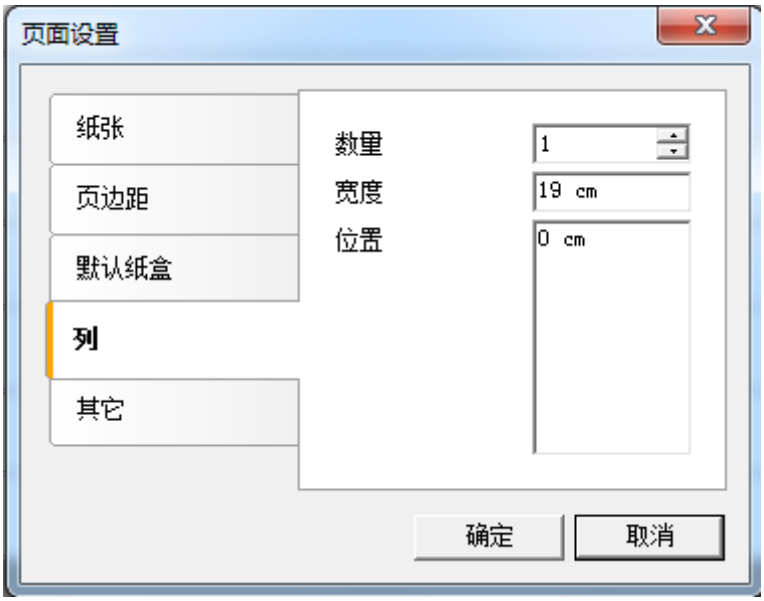
分栏报表

一个普通的报表，只要它还没有到达页的末尾，便会打印数据，在此之后，生成一个新的页面并继续打印。一个分栏报表可在不同栏内打印数据。当到达页面末尾时，将继续在同一页面的另一分栏内打印。在这个意义上，一个普通的报表可以被看作只有一个分栏的报表。

在 Report 中，有两种打印分栏的方式。

页面分栏

第一种方法是在报表页面上设置分栏的数目。你可以在“页面设置”窗口，选择“列”选项卡进行设置：



如图所示，您可以设置以下参数列：

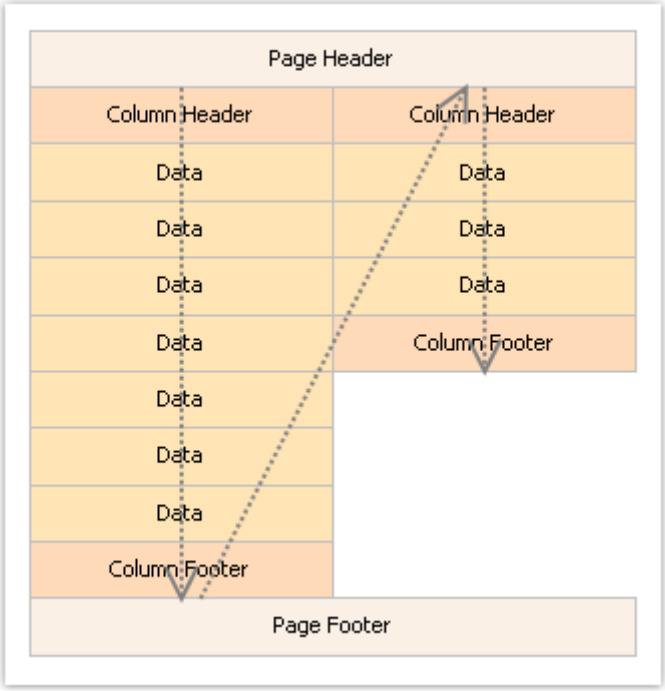
- 数量；
- 宽度；
- 位置

为了将一个普通报表转换成分栏报表，您只需要设置页面的列数，剩余的参数 Report 会自动计算。当您启用分栏时，设计页面的区域模式将发生变化：



以灰色显示的区域不允许放置对象。它是用于打印下一分栏的对象。

创建分栏报表时，“栏首”和“栏尾”可用。从他们的名字看，他们分别在每个分栏的顶部和底部打印。下图展示了分栏报表区域的打印顺序。



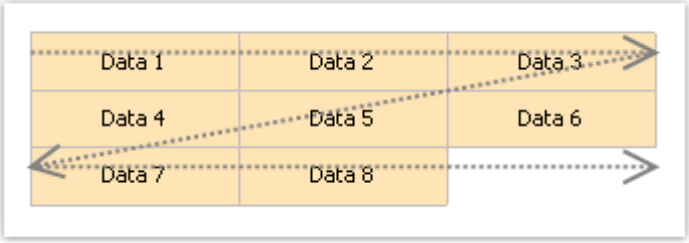
数据区域分栏

另一个打印多栏报表的方式是基于使用“数据”区域分栏。

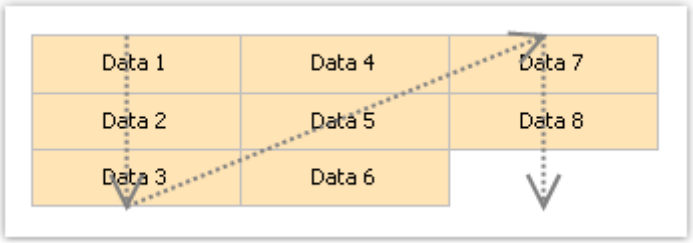
分栏参数可以通过配置“属性”窗口的“分栏”属性改变。你可以设置以下参数：

- 数量；
- 宽度；
- 栏打印模式。你可以选择两种模式：“AcrossThenDown”和“DownThenAcross”；
- 当选择“DownThenAcross”模式时，每栏的最小行数；

分栏区域可以两种模式打印：在“AcrossThenDown”模式中，分栏以下面的方式打印：



在“DownThenAcross”模式中，分栏以下面方式打印：



在这个模式中，Report 计算一个分栏中数据行的个数，分栏被平均填充。你也可以通过 “Columns.MinRowCount”属性设置分栏的最小行数。

"册子"-类型报表

在打印小册子形式的报表时，你可能会面临以下的要求：

- 独立的报表页面-封皮，目录，报表内容，封底；
- 奇偶页的页边距不同；
- 奇偶页的页眉和页脚不同；

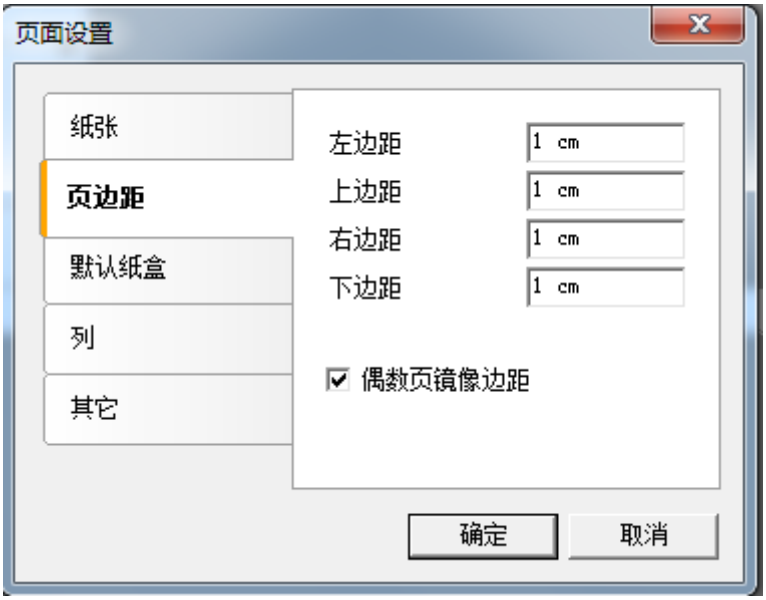
添加页面

您可以添加任意多个页面到报表模板。还可以在每个页面放置独立的报表。您可以通过点击工具栏上的 按钮，添加页面。还可以通过点击“添加…”按钮，选择窗口上的“报表页面”选项卡实现此操作。

为了创建“目录”部分，你可以查看“交叉报表”章节的专门介绍。

页面设置

在“页面设置”窗口，您可以标示 Report 的偶数页镜像边距：



如果您需要打开页面上的奇数页码，则需将“StartOnOddPage”属性设置为 true。必要时，Report 会在打印指定页之前打印空白页。

打印奇偶页

所有报表都具有“PrintOn”属性。使用这个属性，你可以奇偶也打印不同的对象。

这个属性可以在“属性”窗口设置：

这个属性决定了哪些页面可以被打印。该属性可以具有以下的某个参数或任意参数的组合：

- 首页；
- 尾页；
- 奇数页；
- 偶数页；
- 重复区域，该值需将区域的“RepeatOnEveryPage”属性设置为 true；

默认情况下，这个属性的值等于“首页、尾页、奇数页、偶数页、重复区域”。这意味着该对象将在报表的每一页进行打印。我们会给出使用此属性的几种典型的例子：

Property value	Where the object will be printed
首页	仅在第一页
尾页,奇数页，偶数页，重复区域	除第一页的所有页面
尾页,奇数页，偶数页，重复区域	除去最后一页的所有页面
重复区域	区域的"RepeatOnEveryPage" 属性设置为 true
首页，尾页,奇数页，偶数页	除重复区域的所有区域
首页，尾页,奇数页，重复区域	仅奇数页面
首页，尾页,偶数页，重复区域	仅偶数页面

例如，在奇数页和偶数页上打印不同的文本，需将两个“文本”对象放置在区域上，并通过以下方式设置：

第一 个对象在奇数页打印。设置它的“PrintOn”为“首页，尾页，奇数页，重复区域”（即除偶数页的所有值）。

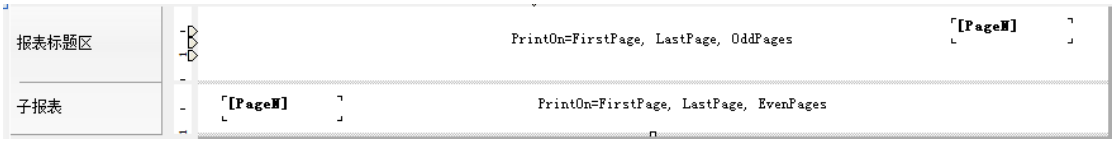
第二个对象在偶数页打印。设置它的“PrintOn”为“首页，尾页，偶数页，重复区域”（即除奇数页的所有值）。

这些对象永远不会在同一时间打印。你可以将它们放置在彼此的顶部。

所有的区域具有相同的属性。为了在奇偶页上打印不同的区域,需使用“子”区域。你可以将它附加到任意区域，这可以在“配置区域”窗口完成。主区域和子区域的配置方法如下所示：

- 主区域在奇数页打印。将它的“PrintOn”属性设置为“首页、尾页、奇数页、重复区域”（即除偶数页的所有值）。
- 子区域在偶数页打印。将它的“PrintOn”属性设置为“首页、尾页、偶数页、重复区域”（即除奇数页的所有值）。

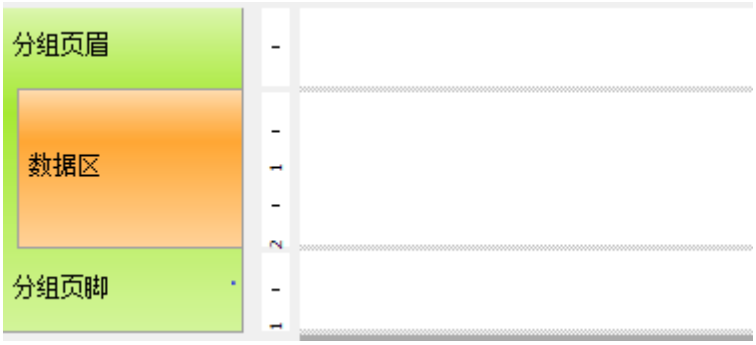
每个区域可以具有不同的高度、外观和内容。看下面打印不同页眉的例子：



分组与合计

前面我们看了“主-从”报表的类型，它打印两个相关数据源的数据。Report允许创建一个类似的报表，但是只需要一个数据源。为此，我们需要使用分组。

分组是三个区域的集合：“分组页眉”、“数据”和“分组页脚”。在设计器中，看起来像：



分组一般包含一个分组页眉和数据。分组页脚是可选择的，你可以删除它。

为了使用分组，你需要为分组页眉设置分组条件，并选择连接“数据”区域的数据源。条件可以是任意表达式，但作为一个规则，必须是数据源的一个字段。分组的打印方式如下：

1、打印分组页眉；

2、打印数据行；

3、检查分组条件是否改变；

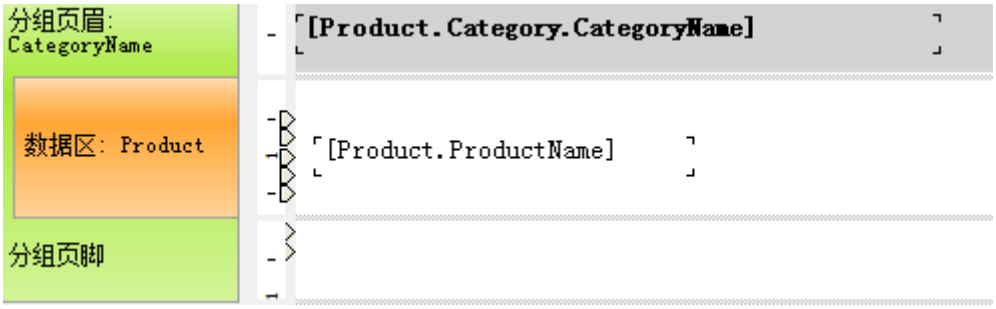
4、如果分组条件未改变，继续打印下一行数据；

5、如果分组条件改变，打印分组页脚，然后开始打印下一个分组。

假设我们有一个具有以下数据的产品表：

CategoryName	ProductName
Beverages	Côte de Blaye
Beverages	Chartreuse verte
Beverages	Steeleye Stout
Beverages	Guaraná Fantástica
Beverages	Sasquatch Ale
Beverages	Rhönbräu Klosterbier
Beverages	Lakkalikööri
Beverages	Outback Lager
Beverages	Ipoh Coffee
Beverages	Laughing Lumberjack Lager
Beverages	Chang
Beverages	Chai
Condiments	Original Frankfurter grüne Soße
Condiments	Sirop d'érable
Condiments	Chef Anton's Gumbo Mix
Condiments	Northwoods Cranberry Sauce
Condiments	Grandma's Boysenberry Spread
Condiments	Chef Anton's Cajun Seasoning
Condiments	Aniseed Syrup
Condiments	Louisiana Hot Spiced Okra
Condiments	Vegie-spread
Condiments	Louisiana Fiery Hot Pepper Sauce
Condiments	Gula Malacca
Condiments	Genen Shouyu

CategoryName 字段的数据可以被分组。这个字段将在分组页眉打印。数据显示在 ProductName 领域。该报表如下所示：



运行此报表，结果如下：

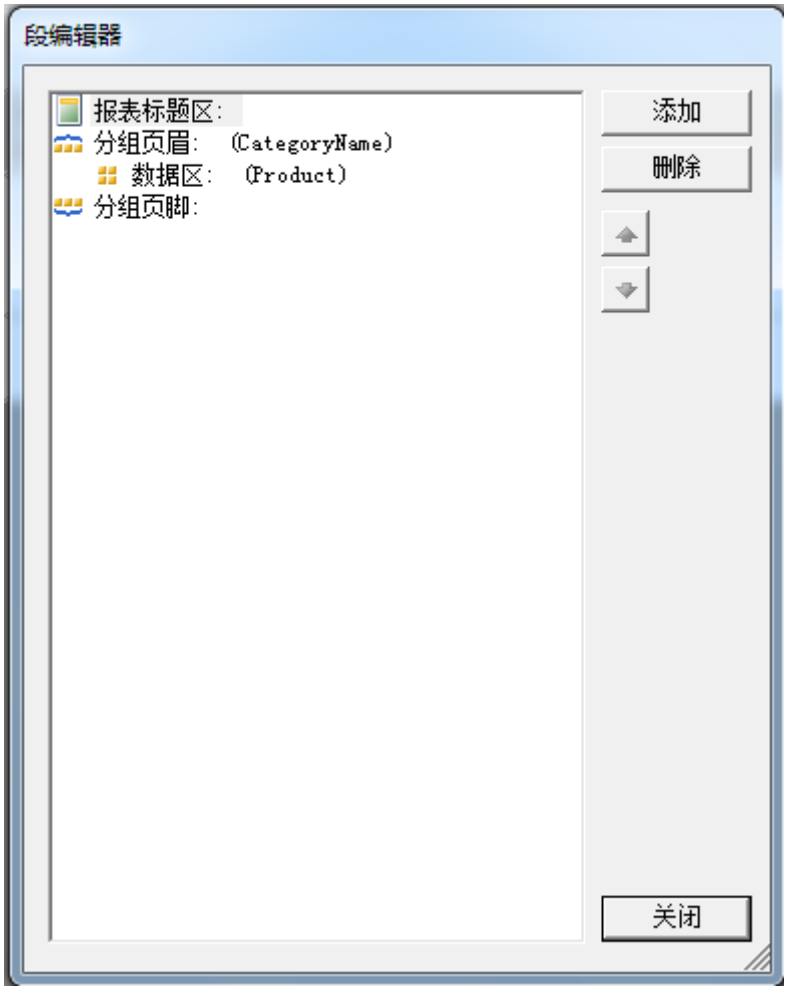
Beverages
Côte de Blaye
Chartreuse verte
Steeleye Stout
Guaraná Fantástica
Sasquatch Ale
Rhönbräu Klosterbier
Lakkalikööri
Outback Lager
Ipoh Coffee
Laughing Lumberjack Lager
Chang
Chai

Condiments
Original Frankfurter grüne Soße
Sirop d'érable
Chef Anton's Gumbo Mix
Northwoods Cranberry Sauce
Grandma's Boysenberry Spread
Chef Anton's Cajun Seasoning
Aniseed Syrup
Louisiana Hot Spiced Okra
Vegie-spread
Louisiana Fiery Hot Pepper Sauce
Gula Malacca
Genen Shouyu

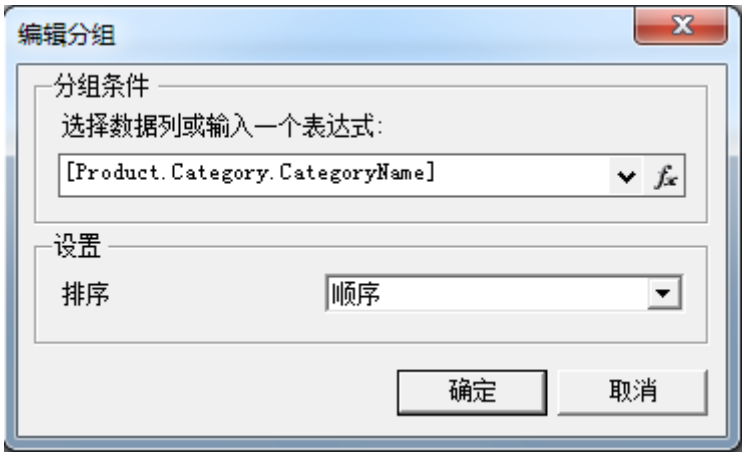
创建分组

可以使用两种方法实现添加分组到报表。

第一种方法：在“设置报表栏”窗口添加“分组页眉”。要做到这一点你需要点击“添加”按钮，然后选择“分组页眉”band。Report 将添加一个分组到可用“数据”band 或者该区域不在报表中时创建一个完整分组。

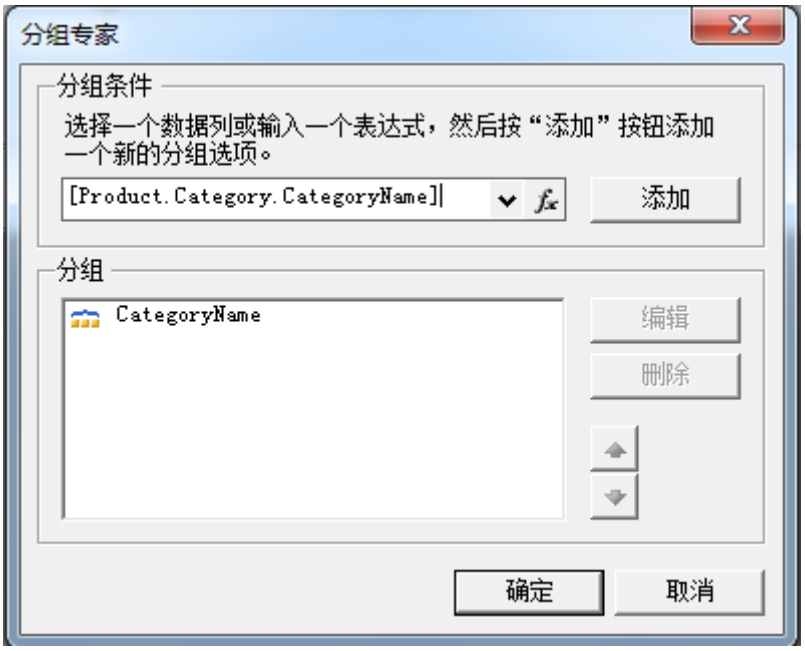


为了配置分组报表，双击“分组页眉”区域。你会看到分组页眉编辑窗口：

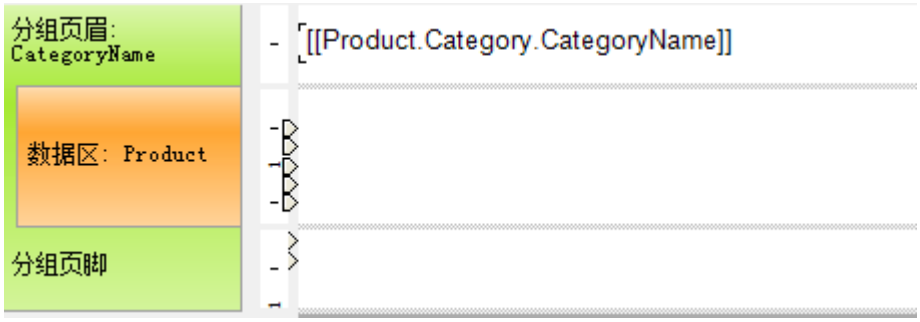


你需要设置分组条件。这可以是任意表达式或数据源字段。也可以选择排序，默认情况下，数据是按升序排序。

第二种方法：使用向导，它可以通过菜单“报表|分组专家”调出窗口。为了创建一个分组，需输入分组条件，然后点击“添加”按钮。



该向导将把分组中的所有元素添加到报表中。此外，它还会在分组页眉创建一个“文本”对象，用于打印分组条件。



数据排序

为了使分组正常工作，它需要满足以下条件：数据源必须在被用于分组条件的字段上进行排序。如果该条件不能满足，你会看到很多含有 1-2 个数据行的分组：



幸运的是，有以下两种可能对数据源进行排序的方法。

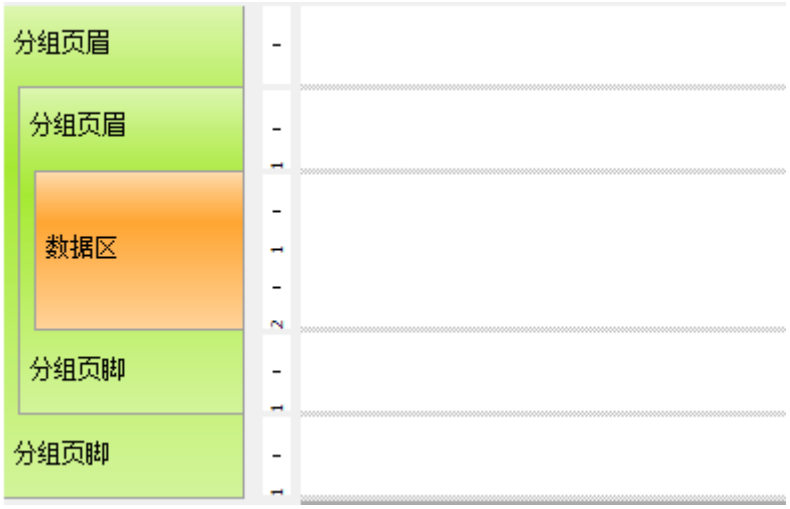
- 你可以在分组编辑器中设置数据的排序。数据源将根据分组条件自动排序；
- 你可以在“数据”band 编辑器上设置排序。

这两种方法效果是一样的。然而第一种方法便于使用。当你创建分组时，在对话框中设置分组数据和排序。

在某些情况下，不能使用第一种方法。假设，我们将在产品名称的第一个字母设置分组。在这种情况下，该产品将只以第一个字母排序，这是不能接受的排序。你应该是用第二中方法，并标明使用产品的全名排序。

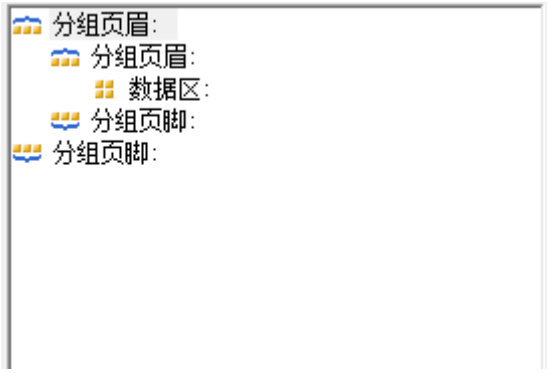
嵌套分组

一个嵌套分组包含几个“分组页眉”band。最后一个 band 包含“数据区”band：



每个分组页眉有自己的分组条件。

创建一个嵌套分组报表跟创建普通分组报表的步骤一样。在第一种情况下，你可以创建一个简单的分组报表，然后在“设置报表栏”窗口添加一个嵌套分组。为此，选择现有的“分组页眉”区域，点击“添加”按钮，添加另一个“分组页眉”区域。



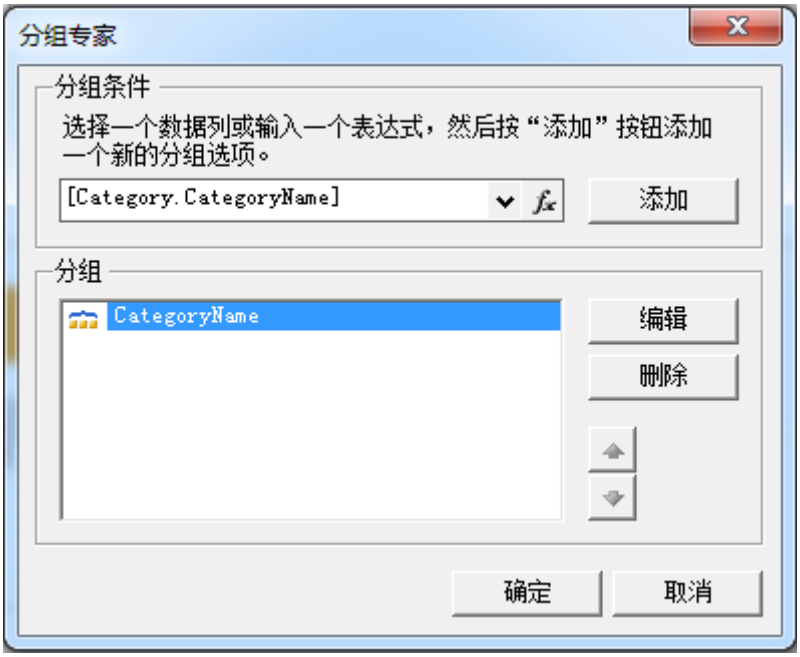
在此之后，打开添加分组的编辑器，然后设置分组条件。

第二种情况，使用“分组专家”。设置分组条件，然后点击“添加”按钮。向导会将新的分组添加到现有分组。

嵌套报表的打印和普通报表差距不大。打印数据时，Report 会检查所有分组的分组条件。如果条件改变，相应的分组结束打印，另一新组开始打印。

管理分组

可以使用分组专家管理分组。它可以通过菜单“报表|分组专家…”实现。



利用该向导，您可以选择添加或删除分组，更改分组顺序： 和 可用于改变分组顺序。利用“编辑…”按钮，你可以更改选定分组的分组条件。

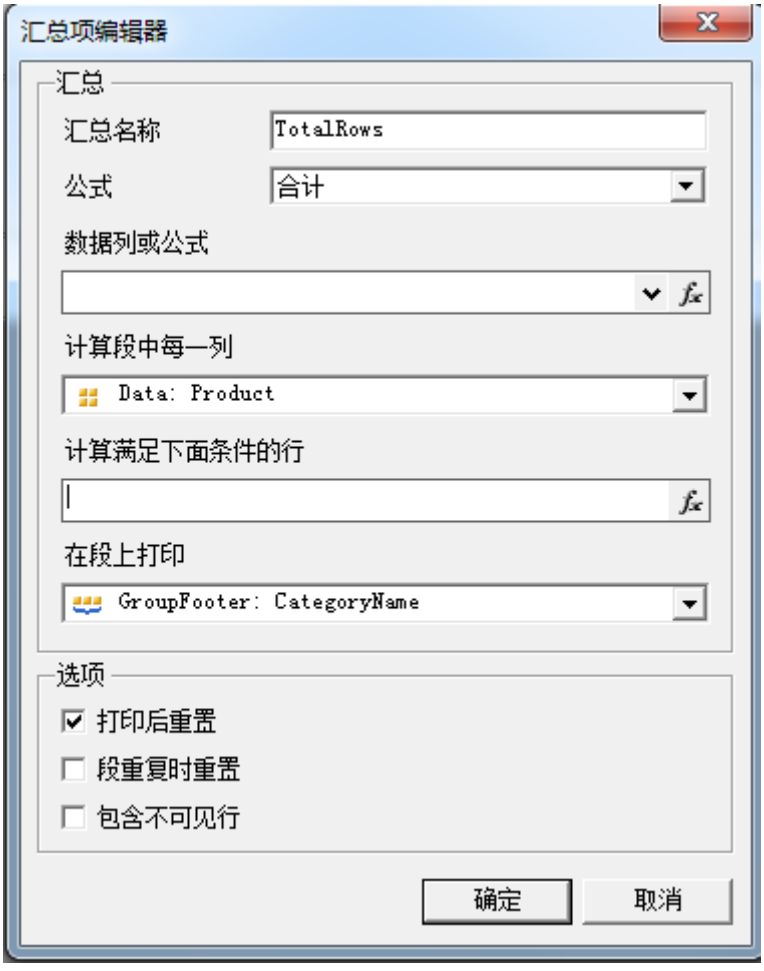
打印合计

分组经常被用来打印每个分组的部分总值。例如，这可以是该分组的行数，或者一个数据字段的合计。为了打印这些值，需使用报表合计。报表合计的使用在“数据”章节描述

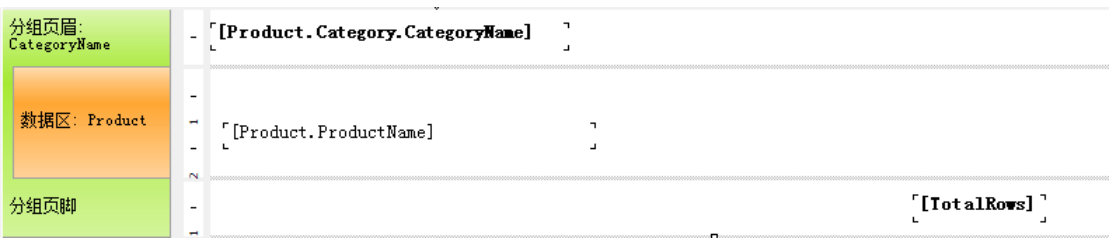
为了打印分组中的合计值，需做到以下几点：

- 在“数据”窗口选择“动作|新建合计…”选项卡，创建一个报表合计；
- 在“计算段中的每一行”组合框中选择分组数据 band；
- 在“在段上打印”组合框中选择分组页脚；
- 将打印合计值的“文本”对象，放置在分组页脚。

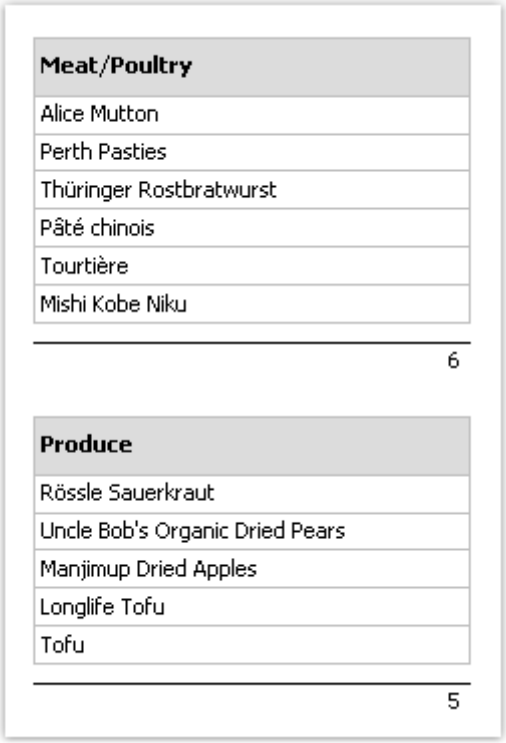
例如，要打印每组的行数，需对合计进行下列配置：



为了显示合计值，需将其拖动到分组页脚：



配置好的报表将如下图所示：



重复分组页眉和分组页脚

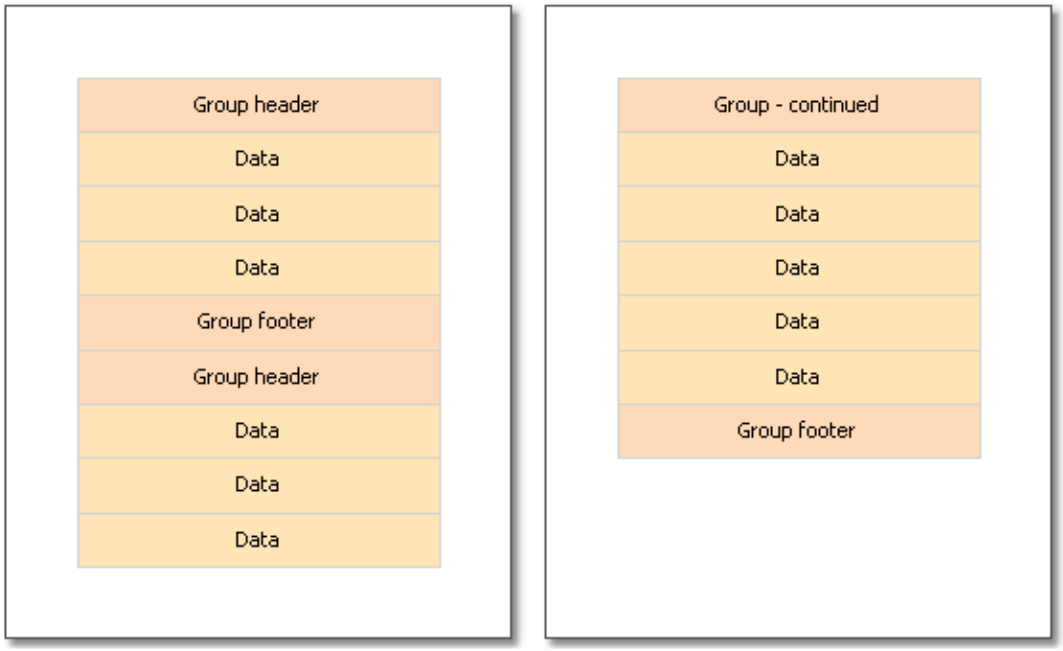
分组页眉和分组页脚具有“RepeatOnEveryPage”属性。当分组不在报表的一张页面时，它是可用的。通过这个属性，你可以在该组打印时，在每页都打印分组页眉/分组页脚。在打印这种页眉/页脚时，Report 会设置它的“Repeated”标志。这可以用于在普通的分组页眉上重复打印不同的对象。例如，在每页打印“继续…”的文字。为此，需使用“文本”对象的“PrintOn”属性。

为了打印不同的文本，需在分组页眉上放置两个文本对象，一个在另一个的上方。

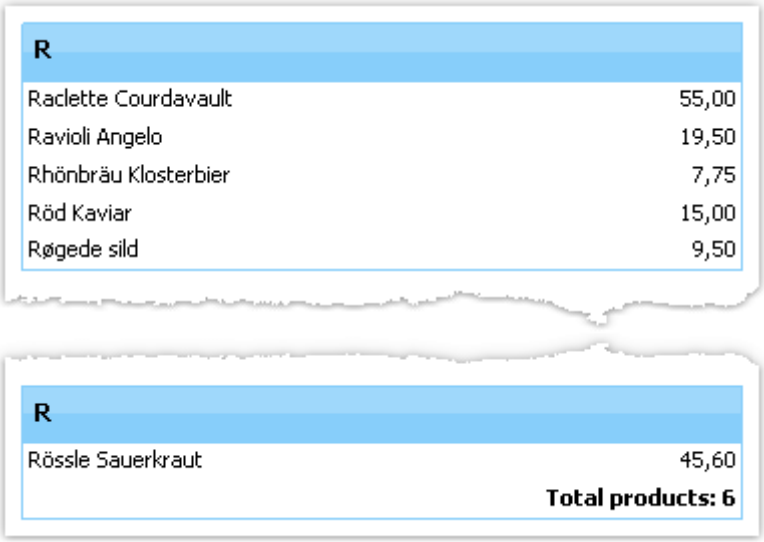
第一个对象将以普通页首打印。设置它的“PrintOn”属性值为“首页、尾页、奇数页、偶数页”。

第二个对象将只在重复的页首打印。设置它的“PrintOn”属性值为“RepeatedBand”。在页首添加具有“继续…”文本的“文本”对象。

报表将如下图所示：



分组页脚也可以在每页重复：



在这个报表中，分组页脚有两个对象，将一个对象放置在另一个上方：



分组属性

“分组页眉”区域具有一些有用的属性：

“StartNewPage”属性允许在打印分组前生成一个新页面。结果是每个分组均在新页面上打印。

新页面不会在第一个分组前添加。这是为了避免第一页为空。

“ResetPageNumber”属性允许在打印分组报表时重新设置页码。通常它和“StartNewPage”属性一起使用。因此，如果两个属性均可用，每个分组都将在新页面上打印，并具有他们自己的页码。

嵌套报表

有时，需要在主报表的某个地方显示额外的数据，这可能是一个非常复杂的独立的报表结构。你可以使用 Report 的区域集合来解决这个任务。然而，在某种情况下，优选的是“嵌套报表”对象。

“嵌套报表“是一个可以放置在某个区域的普通的报表对象。当你这样做时，Report 会在报表中添加一个新页面，并将它与嵌套报表连接。在此页面上，你可以创建任何结构的新报表。

当你打印一个包含“嵌套报表”的报表时：

1、当“嵌套报表”不符合要求时，打印主报表；

2、打印嵌套报表区域；

3、继续打印主报表。

当在主报表上添加嵌套报表后，便不能再包含以下区域：“报表抬头/尾”，“页首/页脚”，“栏首/栏尾”，“重叠”。

打印模式

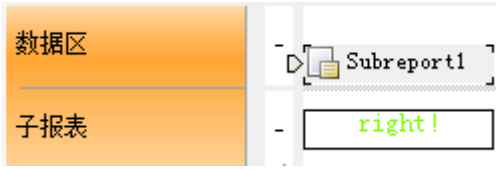
嵌套报表有两种打印模式。

第一种模式，嵌套报表的区域和对象在主报表的页面上打印。在区域上放置对象时会受到限制：

- “嵌套报表”必须放置在区域的下边缘；
- 切勿把其他对象放置在“嵌套报表”对象的下方。否则当报表运行时，这些对象会被嵌套报表的对象覆盖。



为了将其他对象放置在嵌套报表下方，可以使用“子”区域。以下面的方法放置对象：



第二种打印模式不同于嵌套报表对象在区域上的打印，它包含“嵌套报表”对。你可以在“嵌套报表”对象的下拉菜单启用此模式。要做到这点，选择“Print On Parent”选项。这个模式对对象的放置位置不加限制。除此之外，主区域还可以根据嵌套报表中的打印数据的多少进行扩张或缩小。

第二种模式唯一的问题是，嵌套报表中可能包含大量数据。当打印时，主区域将有很大的高度。为了正确打印该区域，需要分隔它的内容（“CanBreak”属性）。分隔算法不能保证百分百正确，有时，它会导致对象的位移。

边-对-边嵌套报表

通过将两个“嵌套报表”并排放置在同一区域，可以打印两个独立的数据表。下面显示了打印这种报表的方法：

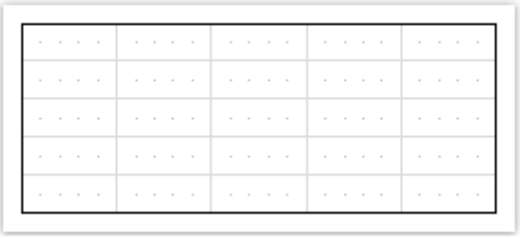
- 当“嵌套”报表不满足时，一直打印主报表；
- 打印第一个嵌套报表；
- 转到嵌套报表开始打印的页面，并打印以下嵌套报表；
- 当所有嵌套报表打印完后，从最长嵌套报表停止的地方继续打印主报表。

嵌套子报表

在一个嵌套报表的页面上，您可以放置另一个“嵌套报表”对象，所以嵌套报表被嵌套。嵌套级别数不受限制；多重嵌套很难理解。如果您有能力，可以使用数据区域打印嵌套数据。一个“数据”区域可以包含一个或多个嵌套“数据”区域。如果你需要打印一个主-明细报表或主-明细-子明细报表，则没必要使用嵌套报表。

表格类型报表

“表格”对象由行、列和单元格组成。它是微软表格的简化模拟。它的外观如下图所示：



配置列

您可以通过下拉菜单添加或删除列，如：

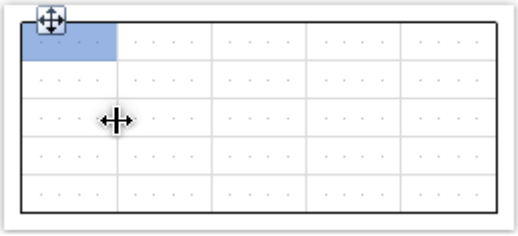
- 选择表或者表中的任意元素并把光标定位在相应的列。光标的形状变成一个黑色的小箭头。
- 单击左键，选中固定列；
- 右键单击显示列的下拉菜单；
- 如果您想选中几个相邻的列，点击左键，向左或向右移动鼠标，选中相邻列。

列的下拉菜单也可以通过“报表数”窗口调出。打开窗口，选择所需列，然后单击鼠标右键。

管理列宽

您可以通过以下方法之一设置列的宽度：

● 选择表或表中的任意元素，并把光标放置在两列之间的边界。游标的形式变为一个水平拆分：



● 选择一列，并在“宽度”属性中注明所需的宽度。这个属性可在“属性”窗口访问。

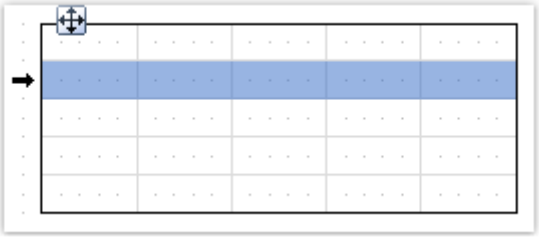
你也可以使用“自动调整大小”列属性。当运行该报表时，列的宽度会自动计算分配。为了限制列的宽度，你可以注明“MinWidth”和“MaxWidth”属性。

列的宽度永远不能超过页面的宽度。

配置行

行以同样的方式进行配置。您可以通过下面的方法选择行：

● 选择表或表中的任意元素，并把光标放置在所选行的左侧。光标的形式变成一个小的黑色箭头：



● 单击鼠标左键，以选择行；

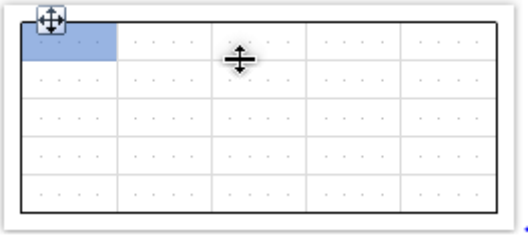
● 右键单击鼠标以显示行的下拉菜单。

如果你需要选择相邻的几行，点击鼠标左键，不松手，向左或向右移动鼠标，以选中相邻行。

管理行高

你可以通过使用以下方法之一设置行的高度：

● 选择一个表或者表中的任意元素，将光标定位于两行之间。光标的形式变成垂直拆分：



点击鼠标左键并移动鼠标，以改变行的高度。

- 选择行并在“高度”属性中注明所需高度。这个属性也可在“属性”窗口访问。

你也可以设置行的“自动调整大小”属性。当报表运行时，行的高度会进行自动分配。为了限制行高，你可以使用“MinHeight”和“MaxHeight”属性：

行的高度不能超过页面的高度。

配置单元格

单元格即文本对象。从本质上讲，单元格继承了“文本”对象。上面谈到的所有关于“文本”对象的东西同样适用于表的单元格。

编辑单元格的文本类似编辑“文本”对象的文本。此外，你还可以从“数据”窗口添加或删除一个元素到单元格。

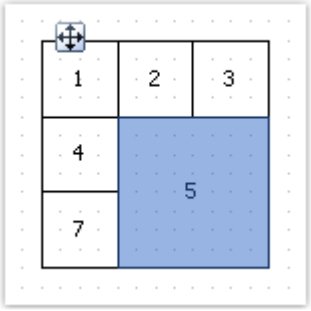
单元格的边框和填充可在“边框和填充”工具上进行配置。

右键单击单元格可调出单元格的下拉菜单。

合并拆分单元格

你可以合并表中相邻的单元格。结果，会显示一个大的单元格，步骤如下：

- 用鼠标选择第一个单元格；
- 点击鼠标左键，移动鼠标以选中单元格；
- 在选中的区域，右键单击鼠标以显示单元格下拉菜单；
- 在下拉菜单中，选择“合并单元格”选项。



拆分单元格时，打开下拉菜单选择“拆分单元格”。

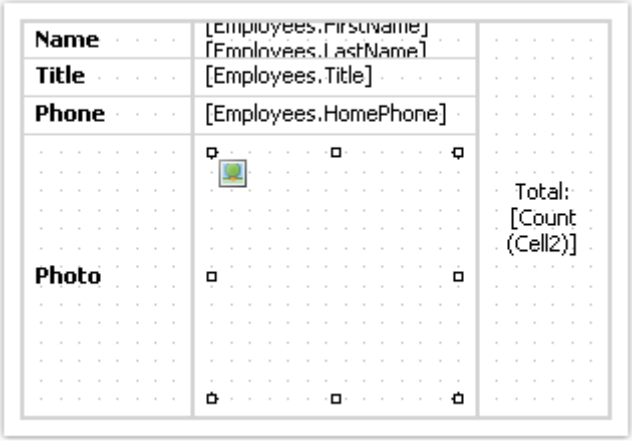
在单元格中插入对象

在单元格中，你可以插入其他对象，例如，图片。下面的对象不能插入单元格：

- “表格”；

- “交叉表”；
- “嵌套报表”；

为了在单元格中添加一个对象，只需将对象拖拽到单元格中。你可以在单元格中自由移动一个对象，也可以将它拖到表格之外。



一个单元格可以作为放置其中的对象的容器。这意味着，你可以在其单元格中使用对象的“Dock”和“Anchor”属性。当单元格的大小改变时，对象的大小也随之改变。

打印表格

一个表格可以以两种方式打印：

第一种形式，表格在它隶属的区域中打印，看起来跟设计器中一样。这种模式下，如果表格的宽度大于报表页面的宽度，表格也不会跨页。这是默认的打印形式。

第二种是动态形式。表通过脚本语言创建。最终的表格和初始表不同，就像设计的报表不同于报表模板。动态模式下，当表格与报表页面不符时，可跨页面。

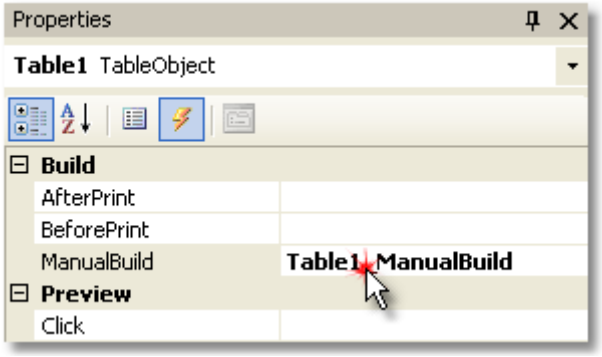
第二种模式下，该表格不在它所放置的区域上打印，而是生成包含表格部分内容一系列区域。这种模式会受到以下限制：

- 不要把其他对象放置在表的下方或旁边。可用“子”band 代替。
- 不要把两个“表”对象放置在一个区域。

让我们详细看一下动态模式。

这种模式和编程相连，它需要更高的报表设计技能。

用脚本创建表格。为了创建一个脚本，需选择“表格”对象，在其“属性”窗口点击“事件”按钮，然后双击“ManualBuild”时间：



完成这点后，一个空事件处理程序被添加到报表代码中：

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
}
```

在此模式下，源表被用作模板。在该事件代码中，你可以根据需要从源表中多次打印行和列。在此期间，可以包含无限行和列的最终表格形成。这样的表在不符合报表页面时，可跨页拆分。

打印表格，需用到下面“表格”对象的一些方法：

Method	Parameters	Description
PrintRow	int index	打印该行指定的索引，行编号从 0 开始
PrintColumn	int index	打印该列指定的索引，列编号从 0 开始
PrintRows	int[] indices	打印表中的几行
PrintRows	-	打印表中的所有行
PrintColumns	int[] indices	打印表中的几列
PrintColumns	-	打印表中的所有列
PageBreak	-	在打印下一列或行时，插入一个页面分隔

可以通过下面的方法之一打印表格：

第一种方法-从上往下打印，然后从左到右。这种方法更适合包含大量行数的表格，你必须以下列顺序调用方法：

- PrintRow(row index);
- 一次或多次调用 PrintColumn(column index)或 PrintColumns(columns indices) 方法以打印列索引；
- 调用一次 PrintColumns()方法以打印所有列；
- 重复该序列以打印表中所有需要的行。

表中的每一行都必须包含相同的列数，当使用 PrintColumn(int index)和 PrintColumns(int [] indices)方法时，这点需牢记。

第二种方法-从左到右，然后从上到下。这种方法适用于打印具有大量列数的表格。你必须以如下顺序调用方法：

- PrintColumn(column index);
- 一次或多次调用 PrintRow(row number) 或 PrintRows(rows indices) 方法以打印行索引；
- 调用一次 PrintRows() 方法打印所有行；
- 重复以上序列以打印所有需要的列。

每一列都需要包含相同数目的行数。当使用 PrintRow（int index）和 PrintRow（int[] indices）方法，尤其需要注意。

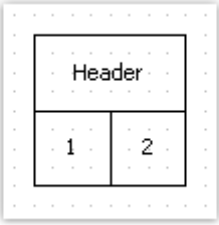
违反调用打印方法的顺序，在执行报表时，会导致错误。错误之一，试图以下面的代码打印报表：

```
Table1.PrintRows();
Table1.PrintColumns();
```

这种方法顺序是不正确的。你必须先打印 table 而不是 PrintRow 或 PrintColumn 方法。

打印复合表头

在此，我们讨论一下跨越单元格的表头。当打印表的行或列，跨越了单元格时，单元格自动增加大小。我们将在下个示例中显示：

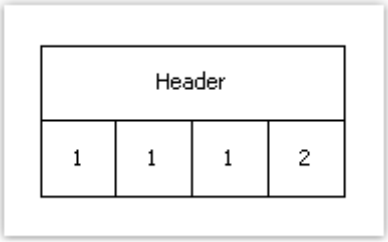


我们将创建一个 ManualBuild 事件处理程序，第一列将被打印 3 次，第二列打印 1 次：

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // printing row 1 and columns 0, 0, 0, 1
    Table1.PrintRow(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(1);
    // printing row 1 and columns 0, 0, 0, 1
    Table1.PrintRow(1);
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(1);
    }
```

注意，每行的列数必须相同，如果违反此规则，我们将得到一个不可预知的结果。

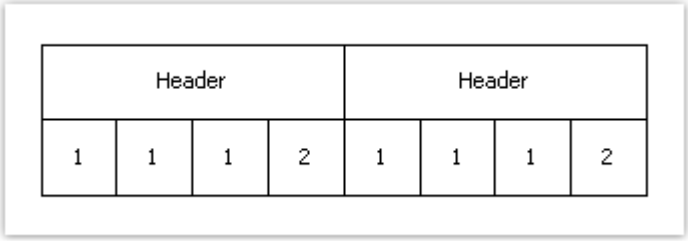
执行这段代码的结果，如下图所示：



如图所示，表头自动跨越单元格。我们将使代码更复杂一点，使两组列可以被打印：

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // print 0 row and two groups of 0, 0, 0, 1 columns
    Table1.PrintRow(0);
    // group 1
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(1);
    // group 2
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(1);
    // print 1 row and two groups of 0, 0, 0, 1 columns
    Table1.PrintRow(1);
    // group 1
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(1);
    // group 2
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(0);
    Table1.PrintColumn(1);
}
```

当我们运行报表，我们将看到下面的结果：



当用下面的代码打印第二列时：

```
Table1.PrintColumn(1);
```

表头完成并且进一步打印新表头的第一列：

```
// group 2
Table1.PrintColumn(0);
```

使用合计

“表”对象的动态模式支持以下合计功能：

Function	Parameters	Description
Sum	TableCell cell	返回包含在单元格中的值的总和
Min	TableCell cell	返回包含在单元格中值的最小值
Max	TableCell cell	返回包含在单元格中值的最大值
Avg	TableCell cell	返回单元格中值的平均值
Count	TableCell cell	返回指定单元格中行的个数

在普通打印模式下（非动态），这些功能不可用。

为了使用合计功能，需将其放置在表格的单元格中。例如，下面的函数计算包含在“Cell1”单元格中的值的总和：

```
[Sum(Cell1)]
```

在此期间，所有的单元格位于更高，并对当前单元格的左侧进行了分析。

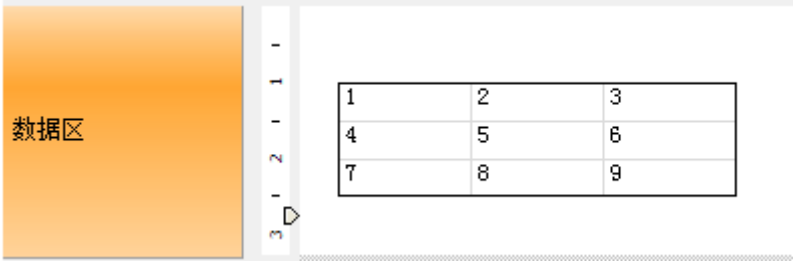
设计表格

动态创建的表格，可以自动分隔在不同的页面。这种行为由表的“布局”属性控制。您可以选择下列值之一：

Value	Description
AcrossThenDown	Table is rendered across then down.
DownThenAcross	Table is rendered down then across.
Wrapped	Wide table is wrapped and rendered on the same page.

范例

让我们看看打印表格的范例。作为一个模板，我们将使用以下报表：



开始，选择表，并创建一个“ManualBuild”事件处理程序。

例1.　从上往下打印整个表

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // printing row 0 and all of its columns
    Table1.PrintRow(0);
    Table1.PrintColumns();
    // printing row 1 and all of its columns
    Table1.PrintRow(1);
    Table1.PrintColumns();
    // Printing row 2 and all of its columns
    Table1.PrintRow(2);
    Table1.PrintColumns();
}
```

作为结果，下表将被打印，其跟模板没什么不同：

1	2	3
4	5	6
7	8	9

例 2.从下往上打印具有重复行的表格

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // Printing row 0 and all of its columns
    Table1.PrintRow(0);
    Table1.PrintColumns();
    // Printing 3 copies of row 1 and all of its columns
```

```
    for (int i = 0; i < 3; i++)
    {
    Table1.PrintRow(1);
    Table1.PrintColumns();
    }
    // printing row 2 and all of its columns
    Table1.PrintRow(2);
    Table1.PrintColumns();
}
```

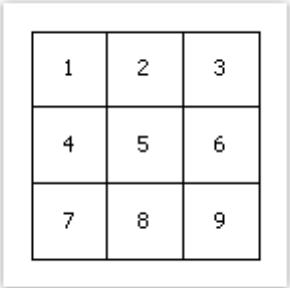
在这个例子中，中间行被打印了 3 次，因此，我们得到如下结果：

1	2	3
4	5	6
4	5	6
4	5	6
7	8	9

例 3.从左到右打印整个表

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // printing column 0 and all of its rows
    Table1.PrintColumn(0);
    Table1.PrintRows();
    // printing column 1 and all of its rows
    Table1.PrintColumn(1);
    Table1.PrintRows();
    // printing column 2 and all of its rows
    Table1.PrintColumn(2);
    Table1.PrintRows();
}
```

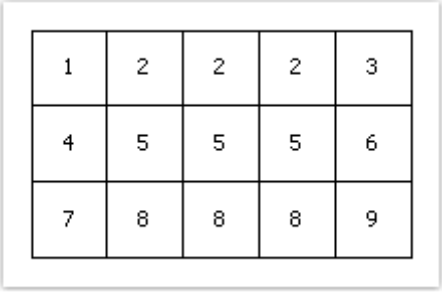
作为结果，下表将被打印，其跟模板没什么不同：



例 4.从左向右打印具有重复列的表格

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // printing column 0 and all of its rows
    Table1.PrintColumn(0);
    Table1.PrintRows();
    // printing 3 copies of column 1 and all of its rows
    for (int i = 0; i < 3; i++)
    {
    Table1.PrintColumn(1);
    Table1.PrintRows();
    }
    // printing column 2 and all of its rows
    Table1.PrintColumn(2);
    Table1.PrintRows();
}
```

在这个例子中，表格的中间列被重复打印了 3 次，其结果如下：



例 5.打印具有重复行和列的表格

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // --------- printing row 0
    Table1.PrintRow(0);
    // printing column 0
    Table1.PrintColumn(0);
```

```
    // printing 3 copies of column 1
    for (int i = 0; i < 3; i++)
    Table1.PrintColumn(1);
    // printing column 2
    Table1.PrintColumn(2);
    // --------- printing 3 copies of row 1
    for (int j = 0; j < 3; j++)
    {
    Table1.PrintRow(1);
    // printing column 0
    Table1.PrintColumn(0);
    // printing 3 copies of column 1
    for (int i = 0; i < 3; i++)
    Table1.PrintColumn(1);
    // printing column 2
    Table1.PrintColumn(2);
    }
    // --------- printing row 2
    Table1.PrintRow(2);
    // printing column 0
    Table1.PrintColumn(0);
    // printing 3 copies of column 1
    for (int i = 0; i < 3; i++)
    Table1.PrintColumn(1);
    // printing column 2
    Table1.PrintColumn(2);
}
```

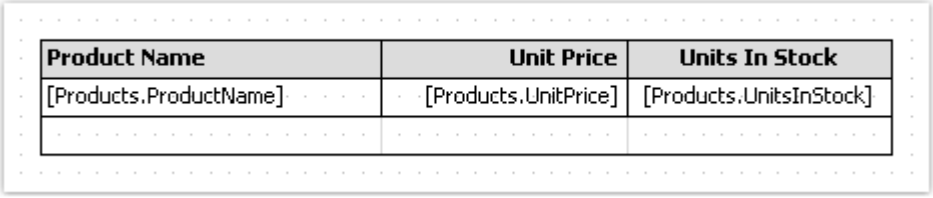
注意，每行的列数必须相同。如果违反此规则，我们将得到不可预知的结果。

在这个例子中，表的中间行金额中间列均被打印了 3 次，结果如下图所示：

1	2	2	2	3
4	5	5	5	6
4	5	5	5	6
4	5	5	5	6
7	8	8	8	9

例 6.使用数据源

考虑完所有例子后，我们要打印一个包含普通文本的表。在这个例子中，我们将展示如何使用数据源形成一个表。为此，我们将创建一个具有以下形式的表：



Product Name	Unit Price	Units In Stock
[Products.ProductName]	[Products.UnitPrice]	[Products.UnitsInStock]

我们将创建一个 ManualBuild 事件处理程序，执行以下操作：

- 获取数据源，在报表中定义；
- 初始化（用数据填充它）；
- 表中打印行的次数等于数据源中行的个数。

以下是该程序的代码：

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // get the data source by its name
    DataSourceBase rowData = Report.GetDataSource("Products");
    // initialize it
    rowData.Init();
    // printing the table header
    Table1.PrintRow(0);
    Table1.PrintColumns();
    // loop through the data source rows
    while (rowData.HasMoreRows)
    {
    // printing the table row
    Table1.PrintRow(1);
    Table1.PrintColumns();
    // select the next data row
    rowData.Next();
    }
    // printing the table footer
    Table1.PrintRow(2);
    Table1.PrintColumns();
}
```

如果我们运行此报表，结果将如下所示：

Product Name	Unit Price	Units In Stock
Chai	18,00	39
Chang	19,00	17
Aniseed Syrup	10,00	13
Chef Anton's Cajun Seasoning	22,00	53
Chef Anton's Gumbo Mix	21,35	0
Grandma's Boysenberry Spread	25,00	120
Uncle Bob's Organic Dried Pears	30,00	15
Northwoods Cranberry Sauce	40,00	6

例 7.插入页面分隔符

使用“表”的“分页符”的方法，可以在打印表格时插入一个分页符。在打印一个行/列时调用它。

我们将使用例 1 演示“分页符”方法的工作原理。让我们在新页打印第三行。

```
private void Table1_ManualBuild(object sender, EventArgs e)
{
    // print the row 0 with all its columns
    Table1.PrintRow(0);
    Table1.PrintColumns();
    // print the row 1 with all its columns
    Table1.PrintRow(1);
    Table1.PrintColumns();
    // insert page break before the row 2
    Table1.PageBreak();
    // print the row 2 with all its columns
    Table1.PrintRow(2);
    Table1.PrintColumns();
}
```

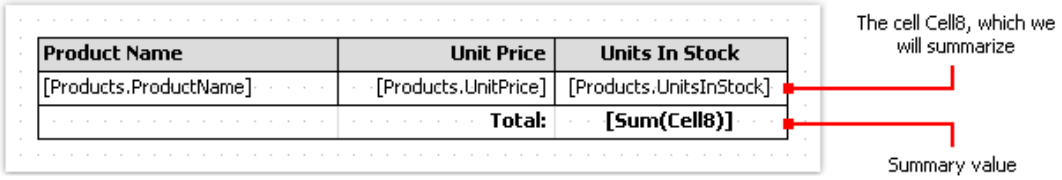
其结果如下：

1	2	3
4	5	6

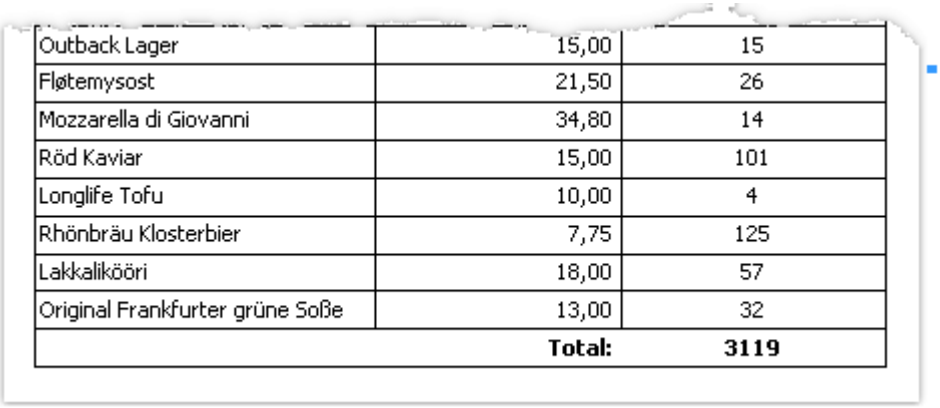
7	8	9

例 8.打印合计

我们将使用例 6 演示合计功能的使用，我们将以下列方式修改它：



报表的运行结果，如下图所示：

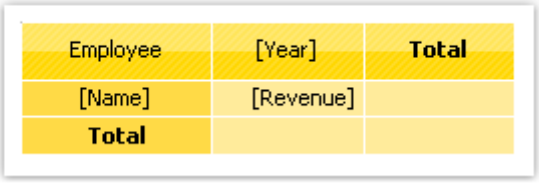


Outback Lager	15,00	15
Fløtemysost	21,50	26
Mozzarella di Giovanni	34,80	14
Röd Kaviar	15,00	101
Longlife Tofu	10,00	4
Rhönbräu Klosterbier	7,75	125
Lakkalikööri	18,00	57
Original Frankfurter grüne Soße	13,00	32
	Total:	**3119**

交叉报表

“交叉表”是一种多样式的表，它类似于“表”对象，由行、列和单元格组成。同时，我们不知道交叉表中有多少行和列-这取决于它连接的数据。

该对象看起来像：



Employee	[Year]	**Total**
[Name]	[Revenue]	
Total		

打印时，交叉表向上和向下填充表中的值。其结果如下：

Employee	1999	2000	2001	2002	**Total**
Andrew Fuller	$3,900.00	$2,100.00		$1,800.00	$7,800.00
Janet Leverling	$6,100.00	$3,200.00			$9,300.00
Nancy Davolio	$3,300.00	$2,700.00	$3,100.00	$1,700.00	$10,800.00
Steven Buchanan		$3,999.00	$8,100.00		$12,099.00
Total	$13,300.00	$11,999.00	$11,200.00	$3,500.00	**$39,999.00**

一些理论

让我们看一下交叉表的元素：

	1	2	3	4
a	a1	a2	a3	a4
b	b1	b2	b3	b4

在该图中，我们看到一个具有 2 行和 4 列的交叉表。这里 a，b-行标题，1，2，3，4-列标题，a1...a4，b1...b4-单元格。为了创建一个这样的报表，只需要一个具有 3 列并包含以下数据的数据源：

a	1	a1
a	2	a2
a	3	a3
a	4	a4
b	1	b1
b	2	b2
b	3	b3
b	4	b4

如图所示，第一列表示交叉表的行，第二-交叉表的列，第三列-指定行和列的单元格中的内容。当创建一个报表时，Report 在内存中创建一个交叉表，并填充数据。在此期间，如果给定数目的行和列还不存在，则交叉表动态增加。

一个表头可以有多个级别，让我们看看下面的例子：

	10		20	
	1	2	1	2
a	a10.1	a10.2	a20.1	a20.2
b	b10.1	b10.2	b20.1	b20.2

在这个例子中，一列被混合，即，它已经得到了两个值。这个报表需要以下数据：

a	10	1	a10.1
a	10	2	a10.2
a	20	1	a20.1
a	20	2	a20.2
b	10	1	b10.1
b	10	2	b10.2
b	20	1	b20.1
b	20	2	b20.2

这里，第一列表示行，第二和第三行表示交叉表的列。最后一个数据列包含单元格中的值。

下面的交叉表-小计和总计。如下所示：

	10			20			Total
	1	2	Total	1	2	Total	
a	a10.1	a10.2	a10.1+a10.2	a20.1	a20.2	a20.1+a20.2	sum(a)
b	b10.1	b10.2	b10.1+b10.2	b20.1	b20.2	b20.1+b20.2	sum(b)
Total	a10.1+ b10.1	a10.2+ b10.2	a10.1+b10.1+ a10.2+b10.2	a20.1+ b20.1	a20.2+ b20.2	a20.1+b20.1+ a20.2+b20.2	sum(a)+ sum(b)

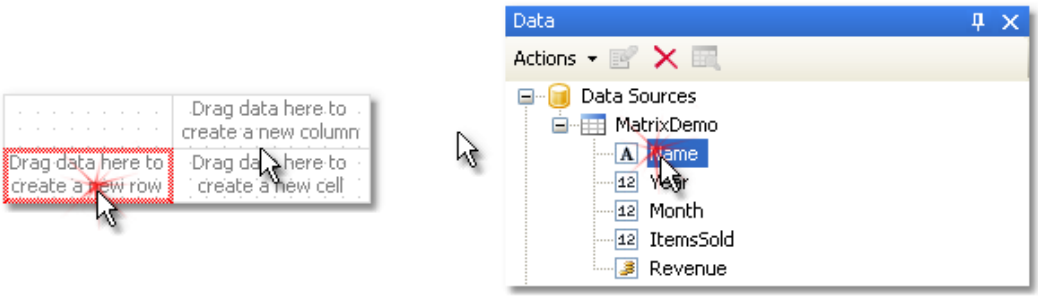
这个报表建立在前面例子所有数据的基础上。上图中灰色行中的数据，自动计算。

配置交叉报表

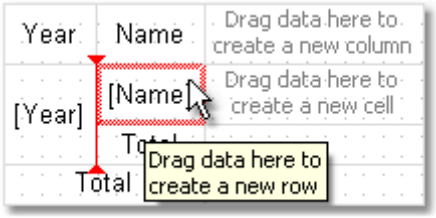
当你将一个“交叉报表”对象放置在一张表上后，它将如下图所示：



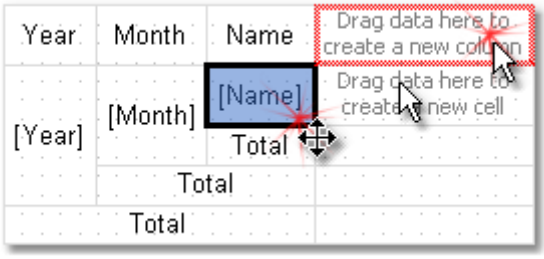
交叉表可以通过鼠标配置。为此，从“数据”窗口拖放数据列到交叉表上，创建行，列和单元格。交叉表突出红色边框显示新数据需放置的位置：



如果交叉表中已经包含了一些元素，当放置一个新元素时，会显示指示器。在给定的情况下，新数据将被放置“年”和“姓名”元素之间：



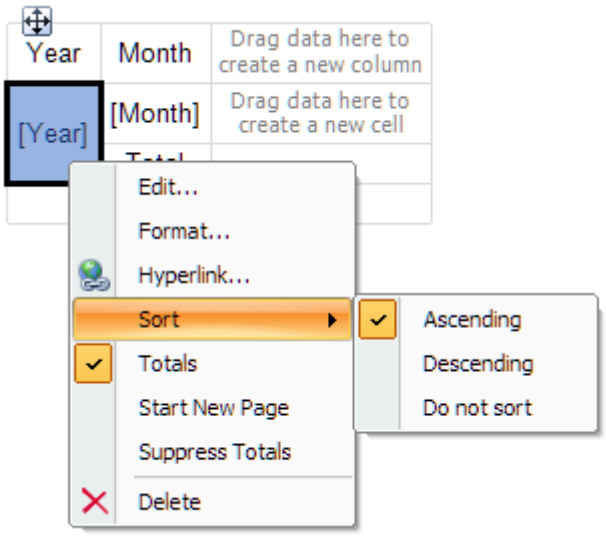
你也可以更改现有元素的顺序。要执行此操作，单击该元素的边框（黑色显示），然后将其拖拽到所需的位置：



为了删除一个元素，用鼠标选中，然后点击“删除”按钮。

配置表头

为了配置表头元素，选中它，然后右键单击显示它的上下文菜单：



默认情况下，交叉表表头的数据是以升序排序。你可以通过选择“排序”选项卡，更改排序方式。

通常，交叉表表头的每一项都有一个合计（这是一个拥有“合计“文本的单元格）。你可以删除合计，选中它然后点击“删除”键。为了再次使用合计，选择它所属的元素，并在其上下文菜单中选择“合计”选项。

“开始新页”菜单项告诉交叉表在打印完每个表头值之后插入页面分隔符。例如，如果您为“年”选项启用了页面分隔符（如上面的图所示），每年的值将在它自己的页面打印。

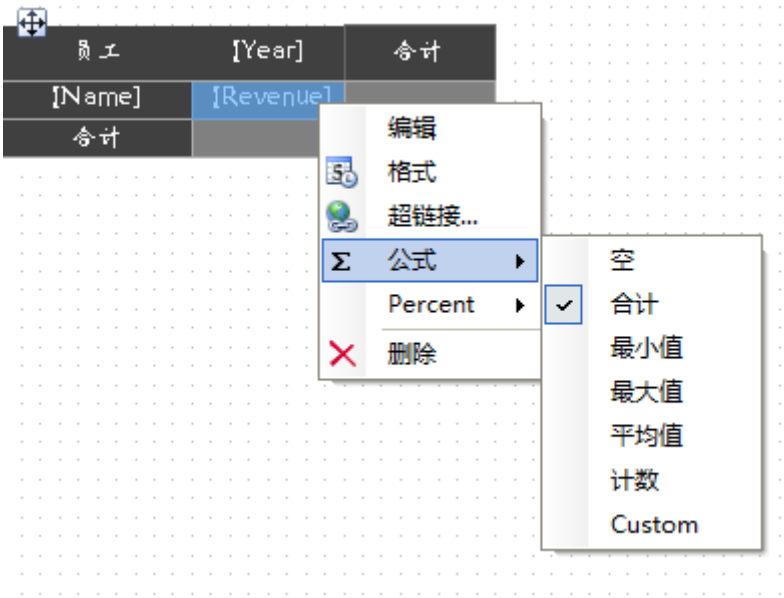
“禁止合计”选项允许当每组只包含一个数据时，禁止使用合计。

配置单元格

对于交叉报表，您可以选择合计时将要使用的函数。下面列出了可以使用的功能：

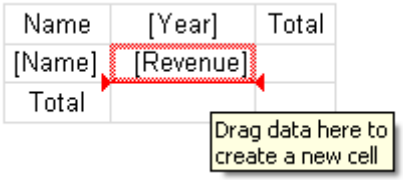
Function	Description
None	单元格中的值不被处理
Sum	返回交叉表单元格中的值的总和
Min	返回最小值
Max	返回最大值
Average	返回平均值
Count	返回非空值的个数

默认情况下，求和函数可用。你可以通过选择一个单元格并在它的上下文菜单中选择“Function”选项进行更改。



如果你不打算在给定的单元格中进行合计，选择“None”函数。

在一个交叉表中，可以包含一个或多个数据单元。在交叉表具有多个单元格的情况下，特们可以被布置成并排或在彼此的上方。交叉表的“CellsSideBySide”属性控制了如何布置单元格。它可以通过“交叉表”对象的上下文菜单更改。也可以在交叉表中添加第二个单元格时选择排序顺序。在此同时，需注意红色指示器，它指示第二个值要放置的位置：



当您添加第二个值后，该值的其余部分将以选定的顺序添加。

交叉表样式

为了改变交叉表的外观，可以单击选定的单元格。在工具栏中，您可以设置字体，粗体和填充。为了一次性更改多个单元格的外观，可选择一组单元格。为此，需选择单元格的左上方，然后移动鼠标以选中组：



你可以使用样式更改整个交叉表的外观。为此，打开“交叉表”对象的上下文菜单，然后选择样式：

管理行和列的大小

由于“交叉表”是“表”对象的一种，所以它允许以同样的方式设置行宽和列高。

默认情况下，一个交叉表的“自动调整大小”已启用。在这种模式下，交叉表自动计算列/行的大小。你也可以手动管理该对象的大小。为此，需禁用交叉表的“自动调整大小”属性。行和列具有相同的属性，你可以使用它，当交叉表的自动调整大小被禁用时。

为了限制列的最小和最大宽度，选择一列并设置它的“MinWidth”和“MaxWidth”属性。

为了限制行的最小和最大高度，选择一行并设置它的“MinHeight”和“MaxHeight”属性。

范例

让我们来看看使用“交叉表”的对象的实例。首先创建一个新报表，并把“交叉报表”对象放置在“报表抬头”区域。你也可以使用“数据”区域-这种情况下，没有必要将区域连接到数据源。在给定的情况下，你在哪两个区域放置交叉表不重要，因为当开始打印报表时，两个区域均被打印一次。该报表如下所示：



不要把“交叉报表”对象放置在将被打印在每一个新页面的区域-“页眉”，“页脚”，等等。交叉报表将在该区域每次打印时，创建一次，这将导致堆栈溢出。

大多数示例将使用“MatrixDemo”表，它是与 Report 包捆绑在一起。该表包含以下数据：

Name	Year	Month	Items Sold	Revenue
Nancy Davolio	1999	2	1	1000
Nancy Davolio	1999	11	1	1100
Nancy Davolio	1999	12	1	1200
Nancy Davolio	2000	1	1	1300
Nancy Davolio	2000	2	2	1400
Nancy Davolio	2	2	2	150

	001			0
Nancy Davolio	2001	3	2	1600
Nancy Davolio	2002	1	2	1700
Andrew Fuller	2002	1	2	1800
Andrew Fuller	1999	10	2	1900
Andrew Fuller	1999	11	2	2000
Andrew Fuller	2000	2	2	2100
Janet Leverling	1999	10	3	3000
Janet Leverling	1999	11	3	3100
Janet Leverling	2000	3	3	3200
Steven Buchanan	2001	1	3	4000
Steven Buchanan	2001	2	4	4100
Steven Buchanan	2000	1	4	3999

例1. 简单交叉报表

交叉报表将包含一行和一个列中的值，以及一个数据单元格。为了创建一个交叉报表，你需要用以下方式添加“MatrixDemo”数据列：

- 在行的表头添加“年”数据列；
- 在列的表头添加“名称“；
- 在交叉表单元格中添加“收入“数据列。

交叉报表将如下图所示：

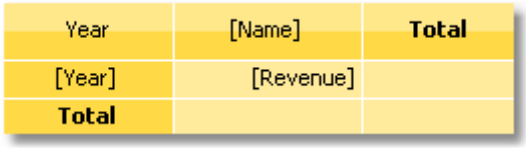
Year	[Name]	Total
[Year]	[Revenue]	
Total		

让我们提升一下交叉报表的外观：

- 为交叉表选择“橘黄色”样式；
- 为每个单元格选择“Tahoma, 8”字体；

- 将“Total”文本改变成粗体；
- 禁用交叉表的自动调整大小属性，并增加行和列的大小。

然后，交叉表将如下图所示：



运行该报表，结果如下：

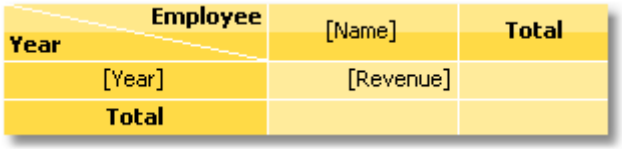
Year	Andrew Fuller	Janet Leverling	Nancy Davolio	Steven Buchanan	Total
1999	3900	6100	3300		13300
2000	2100	3200	2700	3999	11999
2001			3100	8100	11200
2002	1800		1700		3500
Total	7800	9300	10800	12099	39999

目前在交叉表中缺少下面的东西：

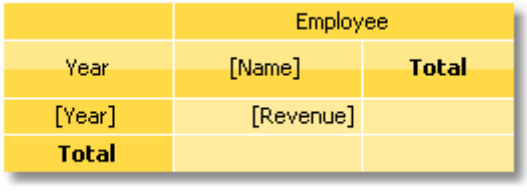
- “名称”数据列没有标题；
- 合计不以货币格式打印；

你可以用下面的方法为“名称”数据列添加标题：

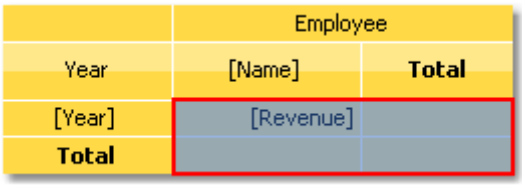
- 文字“年/员工”可放置在交叉表的左上角；
- 对角线和第二个“文本”对象可以如下所示放置在那里；



- 启用交叉表的标题，在“交叉表”对象的上下文菜单中选择“显示标题”选项，任何文本均可包含在标题中：



为了设置数据格式，选中所有单元格区域，如下图所示，然后通过在下上文菜单中选择“格式…”选项设置格式：



然后，报表将如下图所示：

	Employee				
Year	Andrew Fuller	Janet Leverling	Nancy Davolio	Steven Buchanan	**Total**
1999	$3,900.00	$6,100.00	$3,300.00		$13,300.00
2000	$2,100.00	$3,200.00	$2,700.00	$3,999.00	$11,999.00
2001			$3,100.00	$8,100.00	$11,200.00
2002	$1,800.00		$1,700.00		$3,500.00
Total	$7,800.00	$9,300.00	$10,800.00	$12,099.00	$39,999.00

例 2.多级标题

交叉表在一行中具有一个值，在一列中有 2 个值和一个数据单元格。我们将在之前例子的基础上，添加一个新项：

- 我们将把“月”这个数据列添加到行的标题，并将其放置在“年”的右边。

添加新项后，提升交叉表的外观。设置单元格的格式也是必要的。然后，交叉表将如下图所示：

		Employee	
Year	Month	[Name]	**Total**
[Year]	[Month]	[Revenue]	
	Total		
Total			

运行此报表，你将看到如下结果：

		Employee				
Year	Month	Andrew Fuller	Janet Leverling	Nancy Davolio	Steven Buchanan	**Total**
1999	2			$1,000.00		$1,000.00
	10	$1,900.00	$3,000.00			$4,900.00
	11	$2,000.00	$3,100.00	$1,100.00		$6,200.00
	12			$1,200.00		$1,200.00
	Total	$3,900.00	$6,100.00	$3,300.00	$0.00	$13,300.00
2000	1			$1,300.00	$3,999.00	$5,299.00
	2	$2,100.00		$1,400.00		$3,500.00
	3		$3,200.00			$3,200.00
	Total	$2,100.00	$3,200.00	$2,700.00	$3,999.00	$11,999.00
2001	1				$4,000.00	$4,000.00
	2			$1,500.00	$4,100.00	$5,600.00
	3			$1,600.00		$1,600.00
	Total	$0.00	$0.00	$3,100.00	$8,100.00	$11,200.00
2002	1	$1,800.00		$1,700.00		$3,500.00
	Total	$1,800.00	$0.00	$1,700.00	$0.00	$3,500.00
Total		$7,800.00	$9,300.00	$10,800.00	$12,099.00	$39,999.00

例 3.打印月份的名称

在前面的例子中，在交叉表中需打印一些月份。发生这种情况，是因为“月”数据列包含月的数目，而不是它的名称。为了打印月份的名称，请执行以下操作：

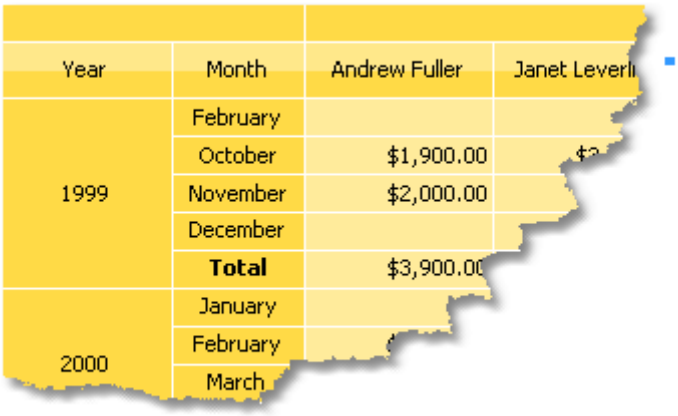
选择一个单元格，打印月份的数字。它是一个命名为“Cell8”的单元格；

在“属性”窗口单击事件按钮，双击“BeforePrint”事件；

Report 将在报表脚本语言中添加一个空事件处理程序。写下如下代码：

```
private void Cell8_BeforePrint(object sender, EventArgs e)
{
string[] month Names = new string[] {
"January", "February", "March", "April",
"May", "June", "July", "August",
"September", "October", "November", "December" };
// Cell8 is a cell that prints the month number.
// Cell8.Value is a value printed in the cell (i.e. the month number).
// This value is of System.Object type, so we need to cast it to int
Cell8.Text = monthNames[(int)Cell8.Value - 1];
}
```

当你运行该报表时，将看到以下结果：



Year	Month	Andrew Fuller	Janet Leverl
1999	February		
	October	$1,900.00	$2
	November	$2,000.00	
	December		
	Total	$3,900.00	
2000	January		
	February		
	March		

例 4.突出显示条件

你可以为交叉报表的单元格设置突出显示条件，类似于 “文本”对象。

让我们看看例 2，看看如何以红色突出显示金额超过 3000 的数据。对于此，选择“收入”单元格，然后点击“文本”工具栏上的按钮。在条件编辑器中，添加以下条件：

```
Value > 3000
```

为条件选择红色文字颜色。准备好的报表将如下：

		Employee				
Year	Month	Andrew Fuller	Janet Leverling	Nancy Davolio	Steven Buchanan	**Total**
1999	2			$1,000.00		$1,000.00
	10	$1,900.00	$3,000.00			$4,900.00
	11	$2,000.00	$3,100.00	$1,100.00		$6,200.00
	12			$1,200.00		$1,200.00
	Total	$3,900.00	$6,100.00	$3,300.00	$0.00	$13,300.00
2000	1			$1,300.00	$3,999.00	$5,299.00
	2	$2,100.00		$1,400.00		$3,500.00
	3		$3,200.00			$3,200.00
	Total	$2,100.00	$3,200.00	$2,700.00	$3,999.00	$11,999.00
2001	1				$4,000.00	$4,000.00
	2			$1,500.00	$4,100.00	$5,600.00
	3			$1,600.00		$1,600.00
	Total	$0.00	$0.00	$3,100.00	$8,100.00	$11,200.00
2002	1	$1,800.00		$1,700.00		$3,500.00
	Total	$1,800.00	$0.00	$1,700.00	$0.00	$3,500.00
Total		$7,800.00	$9,300.00	$10,800.00	$12,099.00	$39,999.00

如图所示，合计值没有突出显示。这是因为我们只为一个单元格选择了突出显示条件。为了突出显示其他值，需为整个交叉表的单元格设置突出显示。

在这个例子中，我们使用了依赖单元格自身的值突出显示的条件。除此之外，你可以根据交叉表的标题突出显示一个单元格。我们将通过下面的例子显示，如何以红色突出显示属于 2000 年的单元格。为此，如下图所示，选择交叉表的单元格：

		Employee	
Year	Month	[Name]	**Total**
[Year]	[Month]	[Revenue]	
	Total		
Total			

设置以下突出显示条件：

```
(int)Matrix1.RowValues[0] == 2000
```

在这个例子中，“Matrix1”是交叉表的一个名称。交叉表的“RowValues”属性已经获得了一个“object[]”类型，并包含了一个其值来源于当前打印行的行标题的数组。数组中值的数目等于一个标题中级别的个数。在我们的例子中，包含两个值，第一个是“年”，第二个是“月”。

不要突出显示最后一行。“RowValue”属性具有一个不确定的值，在创建报表时会导致错误。

当我们运行该报表时，其结果如下：

		Employee				
Year	Month	Andrew Fuller	Janet Leverling	Nancy Davolio	Steven Buchanan	**Total**
1999	2			$1,000.00		$1,000.00
	10	$1,900.00	$3,000.00			$4,900.00
	11	$2,000.00	$3,100.00	$1,100.00		$6,200.00
	12			$1,200.00		$1,200.00
	Total	$3,900.00	$6,100.00	$3,300.00	$0.00	$13,300.00
2000	1			$1,300.00	$3,999.00	$5,299.00
	2	$2,100.00		$1,400.00		$3,500.00
	3		$3,200.00			$3,200.00
	Total	$2,100.00	$3,200.00	$2,700.00	$3,999.00	$11,999.00
2001	1				$4,000.00	$4,000.00
	2			$1,500.00	$4,100.00	$5,600.00
	3			$1,600.00		$1,600.00
	Total	$0.00	$0.00	$3,100.00	$8,100.00	$11,200.00
2002	1	$1,800.00		$1,700.00		$3,500.00
	Total	$1,800.00	$0.00	$1,700.00	$0.00	$3,500.00
Total		$7,800.00	$9,300.00	$10,800.00	$12,099.00	$39,999.00

您也可以使用交叉表的“ColumnValues”属性作为列值的参考。

例 5.突出显示偶数行

为了提升交叉表的外观，你可以用其他颜色突出显示偶数行或列。我们将利用例 2 显示如何操作它。

选择交叉表数据的整个区域，如下图所示：

		Employee	
Year	Month	[Name]	**Total**
[Year]	[Month]	[Revenue]	
	Total		
Total			

调出突出显示编辑器。添加下面的条件：

```
Matrix1.RowIndex % 2 != 0
```

并选择背景色比前一个更暗一点。在这个例子中，“Matrix1”交叉表的一个名称。交叉表的“Rowindex”属性返回当前打印行的数目。

以同样的方法使用交叉表的“ColumnIndex”属性配置列的突出显示。

当我们运行此报表时，我们将看到如下结果：

		Employee				
Year	Month	Andrew Fuller	Janet Leverling	Nancy Davolio	Steven Buchanan	**Total**
1999	2			$1,000.00		$1,000.00
	10	$1,900.00	$3,000.00			$4,900.00
	11	$2,000.00	$3,100.00	$1,100.00		$6,200.00
	12			$1,200.00		$1,200.00
	Total	$3,900.00	$6,100.00	$3,300.00	$0.00	$13,300.00
2000	1			$1,300.00	$3,999.00	$5,299.00
	2	$2,100.00		$1,400.00		$3,500.00
	3		$3,200.00			$3,200.00
	Total	$2,100.00	$3,200.00	$2,700.00	$3,999.00	$11,999.00
2001	1				$4,000.00	$4,000.00
	2			$1,500.00	$4,100.00	$5,600.00
	3			$1,600.00		$1,600.00
	Total	$0.00	$0.00	$3,100.00	$8,100.00	$11,200.00
2002	1	$1,800.00		$1,700.00		$3,500.00
	Total	$1,800.00	$0.00	$1,700.00	$0.00	$3,500.00
Total		$7,800.00	$9,300.00	$10,800.00	$12,099.00	$39,999.00

例 6.使用表达式

在先前的例子中，我们通过从“数据”窗口拖动列创建交叉表。您还可以使用表达式实现此目的。为了在交叉表中插入一个表达式，需执行以下操作：

- 从“数据”窗口添加任何元素到交叉表。它可以是任意元素，例如，一个系统变量“Date”-我们只是用它来创建一个交叉表元素；
- 双击该元素，然后在表达式编辑器窗口选择所需的表达式。

如果你的交叉表具有表达式，而不是数据字段，你必须检查交叉表的“DataSource”属性是否配置正确。当你使用数据列工作时，这个属性会根据你拖动的字段进行自动填充。

让我们看一个如何使用表达式的例子。为此，我们将使用“订单明细”表作为个数据源，其中包含销售产品，员工分组列表。在此表中，存在几种关系，方便访问员工的名称，产品名称和它的类别。

我们的交叉表将显示每个员工的销售额，按产品分类。为了创建交叉表，需执行以下操作：

- 添加“Order Details.Products.Categories.CategoryName”数据列到列的标题；
- 添加任意项到行标题，以创建一个交叉表元素。然后为标题元素设置以下表达式：

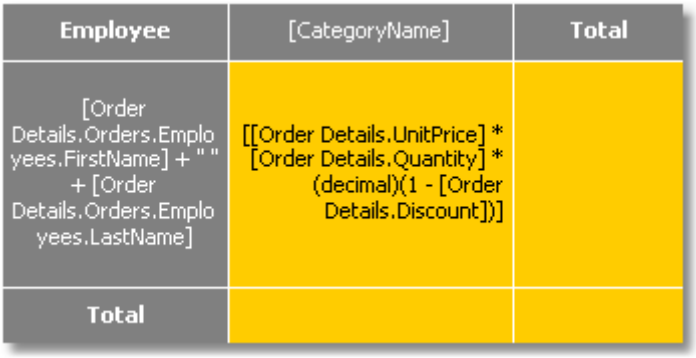
```
[Order Details.Orders.Employees.FirstName] + " " + [Order Details.Orders.Employees.LastName]
```

- 添加任意项到数据单元格，以创建一个交叉表元素。然后为单元格设置下列表达式：

```
[Order Details.UnitPrice] * [Order Details.Quantity] * (decimal)(1 - [Order Details.Discount])
```

为什么我们要将这么长的数据列作为一个员工的名字，众然我们可以从“Employees.FirstName”得到名字？我们这样做是因为交叉表连接到“订单明细”数据源。使用此数据源和其他表之间的关系，很容易就涉及到它的列。如果我们直接引用“Employees.FirstName”数据列，我们将得到表中的第一个员工的名称。

设置交叉表的外观，其将如下图所示：



当我们运行此报表时，我们将看到一个占用 2 张表的相当大的交叉报表：

例 7.单元格中的图片

交叉表的单元格继承自“文本”对象，并且可以显示文本数据。如果不够的话，你可以把任意对象添加到单元格。让我们看看如何在交叉表中显示图片。

为此，我们将采用例 6 作为基础。添加员工图片(″Employees.Photo″ 数据字段)和类别图片(″Categories.Picture″ 数据字段)。请执行以下操作：

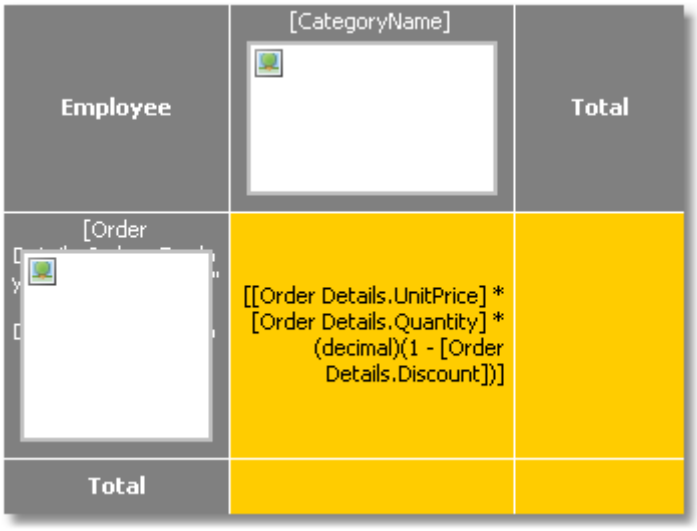
- 选择包含员工姓名的单元格，并增加它的大小；
- 将“图片”对象添加到该单元格；
- 为了显示员工的照片，将“照片”对象“连接到下面的数据列（这可以在对象编辑器中完成）：

`Order Details.Orders.Employees.Photo`

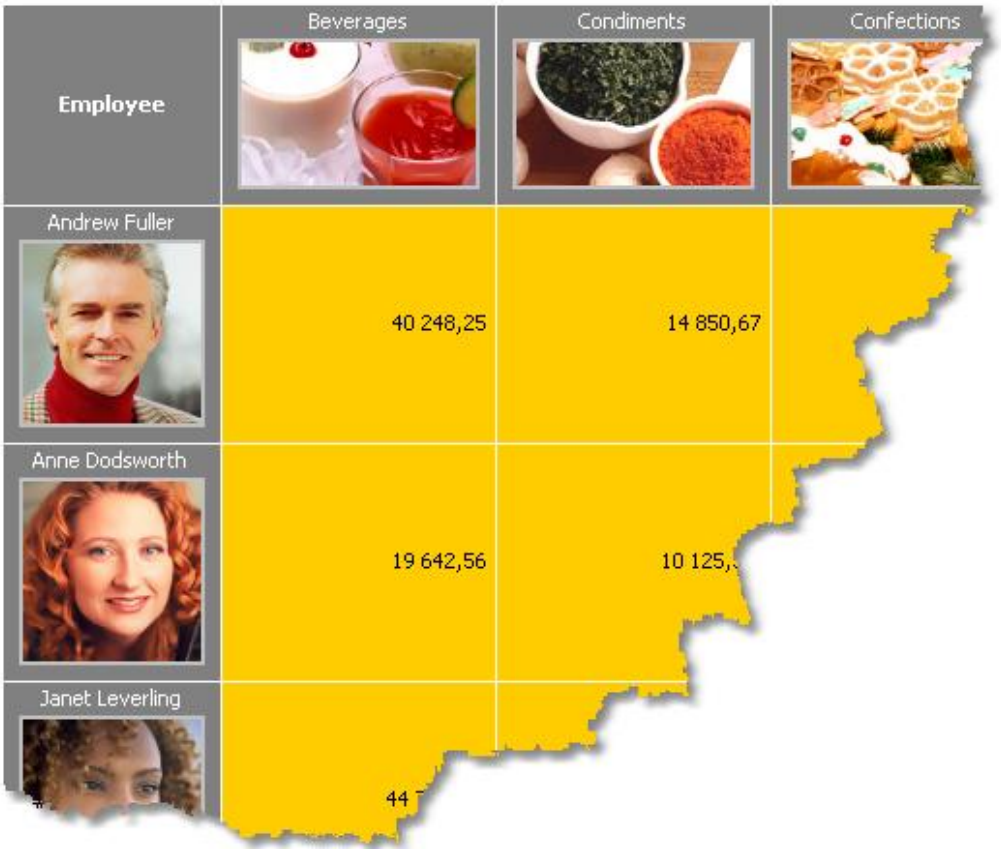
- 选择包含该类别名称的单元格，并增加它的大小；
- 将“图片”对象添加到该单元格；
- 为了显示类别的照片，将“照片”对象“连接到下面的数据列（这可以在对象编辑器中完成）：

`Order Details.Products.Categories.Picture`

然后，交叉表将如下图所示：



当我们运行该报表时，其结果如下：



例 8.单元格中的对象

将对象插入到交叉表单元格中，可以有不同的视觉效果。我们将在下面的例子中显示，如何绘制一个简单的刻度标示员工的销售水平。

该交叉表将使用“MatrixDemo”数据源。要构建一个交叉表，需以下面的方式添加数据字段：

- “年”数据列添加到行标题；
- “名称”数据列添加到列标题；

- “收入”添加到单元格。

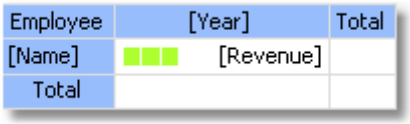
以下面的方式设置交叉表的外观：



现在，让我们添加三个“Shape”对象到“收入”单元格中。这些对象将通过以下方式作为指标：

- 如果一个单元格中的值小于 100，只有一个红色对象将被显示；
- 如果一个单元格中的值小于 3000，两个黄色对象将被显示；
- 如果一个单元格中的值大于或等于 3000，三个绿色对象将被显示。

现在，交叉表如下图所示：



为了控制交叉表单元格中的对象，我们将使用一个事件处理程序。为此，选择“收入”单元格，使用“属性”窗口创建一个“BeforePrint”事件处理程序。在程序中编写以下代码：

```
private void Cell4_BeforePrint(object sender, EventArgs e)
{
    // In our example, a cell has the Cell4 name.
    // Get cell value which is in the Cell4.Value property.
    // Some cells in our matrix will be empty. We'll take it into
account (null check).
    // The value should be cast to decimal type, because data source
column
    // [MatrixDemo.Revenue] is of System.Decimal type.
    decimal value = Cell4.Value == null ? 0 : (decimal)Cell4.Value;
    // Switch shape objects on or off depending on the value:
    // value < 100 - one object is visible;
    // value < 3000 - two objects are visible;
    // value >= 3000 - all objects are visible
    Shape1.Visible = true;
    Shape2.Visible = value >= 100;
    Shape3.Visible = value >= 3000;
    // Choose the color of objects:
    // value < 100 - red color;
    // value < 3000 - yellow color;
    // value >= 3000 - green color
    Color color = Color.Red;
```

```
        if (value >= 100)
        color = Color.Yellow;
        if (value >= 3000)
        color = Color.GreenYellow;
        // Set the objects' color
        Shape1.Fill = new SolidFill(color);
        Shape2.Fill = new SolidFill(color);
        Shape3.Fill = new SolidFill(color);
}
```

当我们运行此报表时，将看到以下结果：

Employee	1999	2000	2001	2002	Total
Andrew Fuller	3 900,00	2 100,00		1 800,00	7 800,00
Janet Leverling	6 100,00	3 200,00			9 300,00
Nancy Davolio	3 300,00	2 700,00	3 100,00	1 700,00	10 800,00
Steven Buchanan		3 999,00	8 100,00		12 099,00
Total	13 300,00	11 999,00	11 200,00	3 500,00	39 999,00

例 9.手动填充交叉表

在所有我们看过的例子中，交叉表被自动填充数据，因为它连接到数据源。交叉表的数据源在“DataSource”属性中显示。即使我们不用手动设置这个属性的值，它同样将数据字段添加到交叉表。

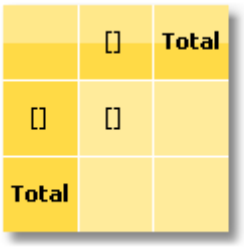
使用脚本可以手动填写交叉表。为此，需创建交叉表的“ManualBuild”事件处理程序。在处理程序代码中调用“AddValue”方法添加一个值。让我们展示如何以下面的方法创建一个将打印 10×10 的的交叉表。

	1	2	3	...
1	1			
2		2		
3			3	
...				...

操作如下：

- 在报表中添加空交叉表；
- 从“数据”窗口放置任意元素到该交叉表的行，列和单元格。然后通过双击交叉表元素，调用表达式编辑器并清除表达式；
- 删除该交叉表的“DataSource”属性。

这些步骤需要创建一个具有一个行，列和单元格的“dummy”交叉表，其结果如下所示：



现在创建一个“ManualBuild”事件处理程序。为此，选择交叉表，在“属性”窗口，单击事件按钮。双击“ManualBuild”事件，然后 Report 将创建一个空事件处理程序，编写以下代码：

```
private void Matrix1_ManualBuild(object sender, EventArgs e)
{
    // Our matrix has one level in row, column and cell.
    // Create 3 arrays of object[] type, each with one element
    // (per number of levels).
    object[] columnValues = new object[1];
    object[] rowValues = new object[1];
    object[] cellValues = new object[1];
    for (int i = 1; i <= 10; i++)
    {
    // Filling arrays
    columnValues[0] = i;
    rowValues[0] = i;
    cellValues[0] = i;
    // Adding data into the matrix
    Matrix1.AddValue(columnValues, rowValues, cellValues);
    }
}
```

在处理程序中，你应该使用“交叉表”对象的“AddValue”属性进行数据填充。这个方法有三个参数，每一个都是 System.Object 的类型的数组。参数是一个列值，第二个参数是一个行值，第三个是单元格的值。需要注意的是，每一个数组中的值的数量应符合对象的设置！在这个例子中，每个对象在列，行和单元格中只有一个级别，相应的我们为列，行和单元格均提供一个值。

当我们运行此报表时，其结果如下图所示：

	1	2	3	4	5	6	7	8	9	10	Total
1	1										1
2		2									2
3			3								3
4				4							4
5					5						5
6						6					6
7							7				7
8								8			8
9									9		9
10										10	10
Total	1	2	3	4	5	6	7	8	9	10	55

让我们展示如何将一个“21”的值添加到交叉表的第 7 列和第 3 行的交点。为此，以下面的方式改变一个代码：

```
private void Matrix1_ManualBuild(object sender, EventArgs e)
{
      object[] columnValues = new object[1];
      object[] rowValues = new object[1];
      object[] cellValues = new object[1];
      for (int i = 1; i <= 10; i++)
      {
      columnValues[0] = i;
      rowValues[0] = i;
      cellValues[0] = i;
      Matrix1.AddValue(columnValues, rowValues, cellValues);
      }
      columnValues[0] = 7;
      rowValues[0] = 3;
      cellValues[0] = 21;
      Matrix1.AddValue(columnValues, rowValues, cellValues);
}
```

如图所示，交叉表自动计算总和。

你可以使用连接到数据源的交叉表的“ManualBuild”属性。在这种情况下，事件处理程序首先被调用，然后交叉表从数据源填充数据。

报表的交互性

一个 Report 的准备报告可具有交互性。这意味着，它将在预览窗口反应用户的操作。你可以使用下面的互动：

- 单击报表对象，执行某些操作。例如，您可以运行详细的报表并在一个独立的窗口显示；
- 预览窗口可以显示报表的轮廓，它可以被用于报表的导航。

超链接

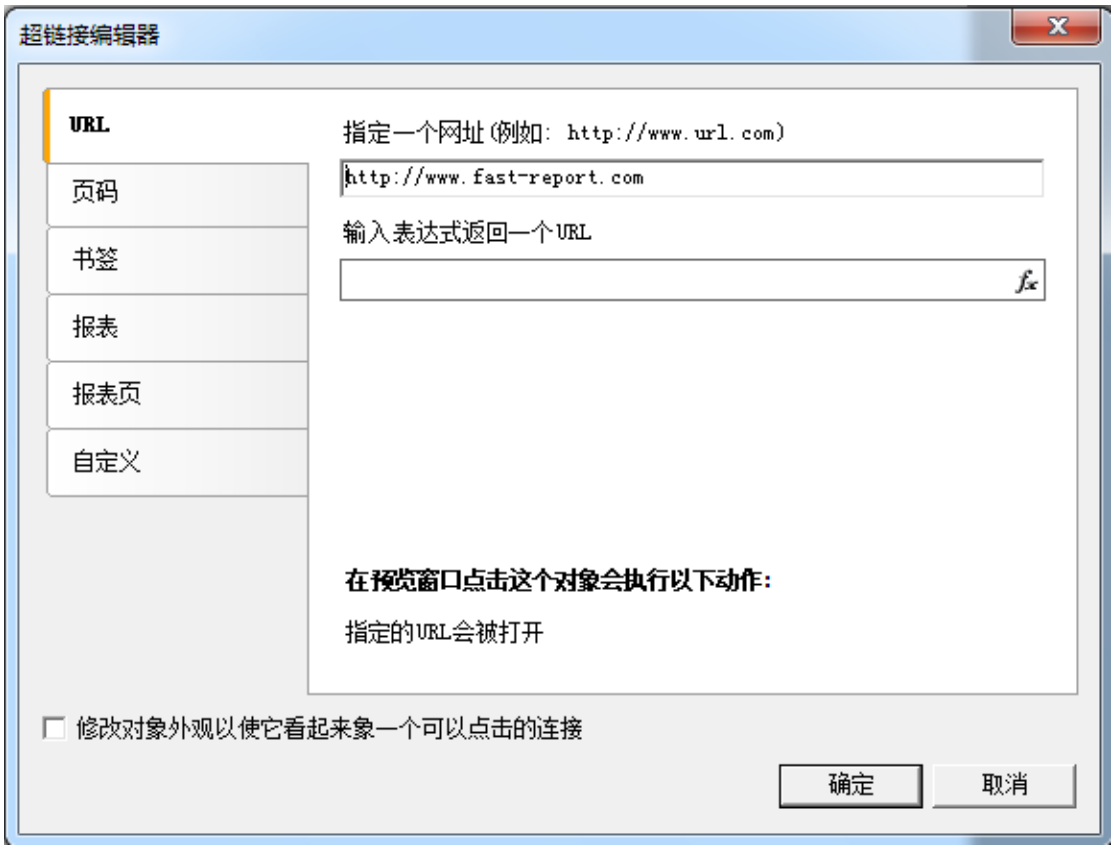
几乎所有的报表对象具有“Hyperlink”属性。使用这个属性，您可以定义当在预览窗口点击鼠标时，一个对象的反应。

当点击这样一个对象时，下列操作之一可能会发生：

- 导航到 URL 地址；
- 发送邮件；
- 执行某种系统指令；
- 导航到指定页码的报表页；
- 导航到在另一个报表对象中定义的书签；
- 在一个独立的预览窗口运行相信报表；
- 在脚本中定义的自定义操作。

配置超链接

要配置一个超链接，请选择您要进行交互的对象，然后右键单击它。在上下文菜单中，选择“超链接…”选项。超链接编辑器窗口将被打开：



通过选择窗口左侧的选项卡选择超链接的类型。然后点击“修改对象外观…”复选框。该对象的外观将以下面的方式改变：

- 文本被设置为蓝色，并带下划线；
- 一个手形光标的形式将被设置。

在某些情况下，超链接需要在预览窗口中显示，但是没有必要将其打印出来。这是很容易做到的，如果你禁用“Printable”对象属性。这可以在“属性”窗口中完成。

链接到 URL

使用这种链接类型，你可以：

- 导航到给定的互联网地址；
- 执行某些系统指令，例如，“mailto”：发生一封邮件。

当单击这种类型的链接时，执行作为参数的链接的值的 System.Diagnostics.Process.Start 方法。

你可以使用两种方法表示链接的值：

- 直接标示值，例如，http://www.ucml.com.cn；
- 标示一个表达式，它返回连接的值。当您运行这个报表时，这个表达式会被计算。

链接到页码

通过使用这个链接类型，您可以在准备报表的页面上组织一个导航。大多数情况下，使用导航到第一页。为此，标示页码（给定情况为 1）作为链接值。

您可以使用两种方法标示页码：

- 直接标示页码，例如，1；
- 标示一个表达式，返回页码。这个表达式将在你运行报表时进行计算。

链接到书签

通过使用这种类型的书签，您可以导航到一个在另一份报表中定义的书签。

对于那些知道 HTML 语言的人来说，一个书签就像一个 anchor。书签在准备好的报表中获得一个名称和一定的定位（页码和页面上的位置）。当通过它的名称移动到一个书签时，您导航到指定的位置。

为了使用这种类型的链接，您首先需要定义书签。为此，选择当你点击链接想要移动到的对象。几乎所有的报表对象都具有“书签”属性。在”属性“窗口，更改此属性。

“书签“包含一个表达式，您可以通过以下方式使用：

- 标示书签名称作为一个字符串：

“MyBookmark”

- 标示一个表达式，返回书签的名称。例如，你可以使用一个数据字段作为一个表达式。该表达式的值将在报表运行时计算。

书签被定义后，您可以在超链接配置窗口标示它的名称，这可以通过以下两种方法实现：

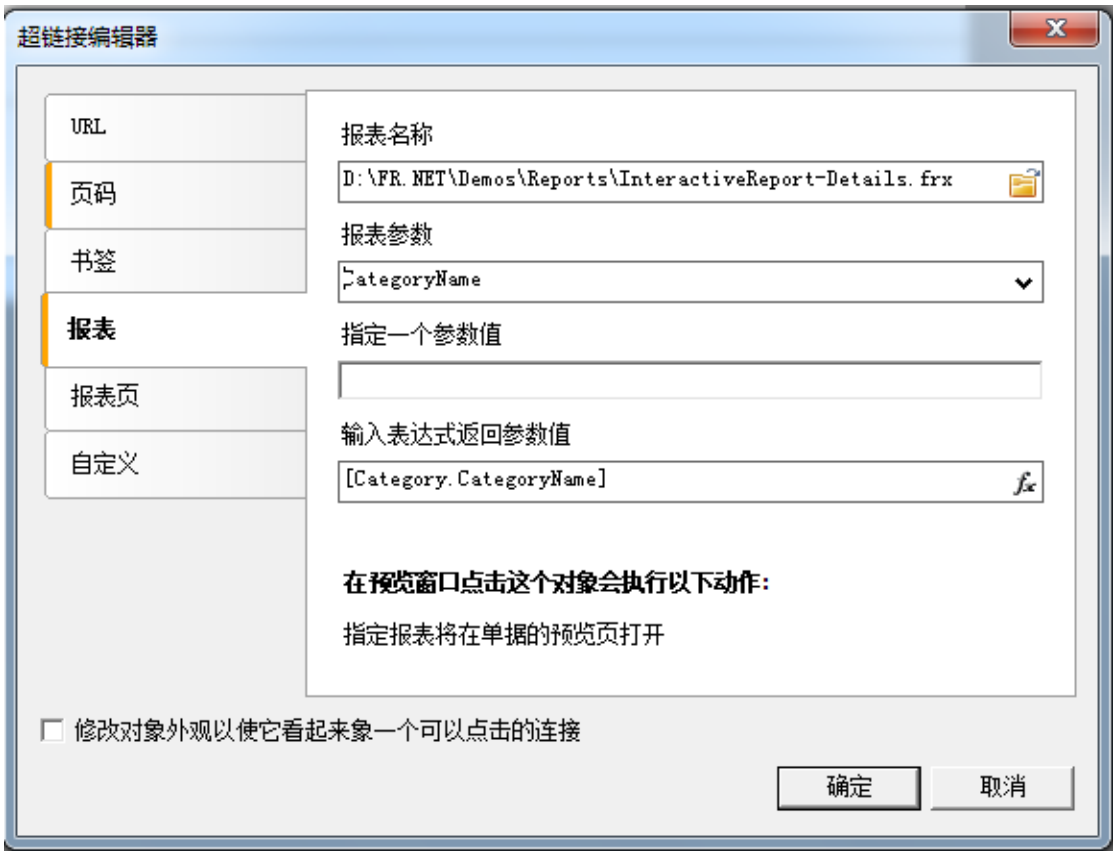
- 直接标示书签的名称；
- 标示一个表达式返回书签的名称。例如，这可以是一个数据字段，该表达式将在报表运行时计算。

链接到明细报表

使用这种类型的链接，你可以在一个独立的预览窗口显示执行另一个报表。

你必须为此类型的链接设置以下参数：

- 详细报表的名称；
- 该报表参数的名称，该名称将获取超链接的值；
- 超链接值。



当链接被点击时，将发生下面的情况：

- 标示的报表将被加载；
- 报表中的参数将被设置到超链接的值；
- 该报表将被创建，并在独立的预览窗口运行。

报表参数的值可以通过以下两种方法标示：

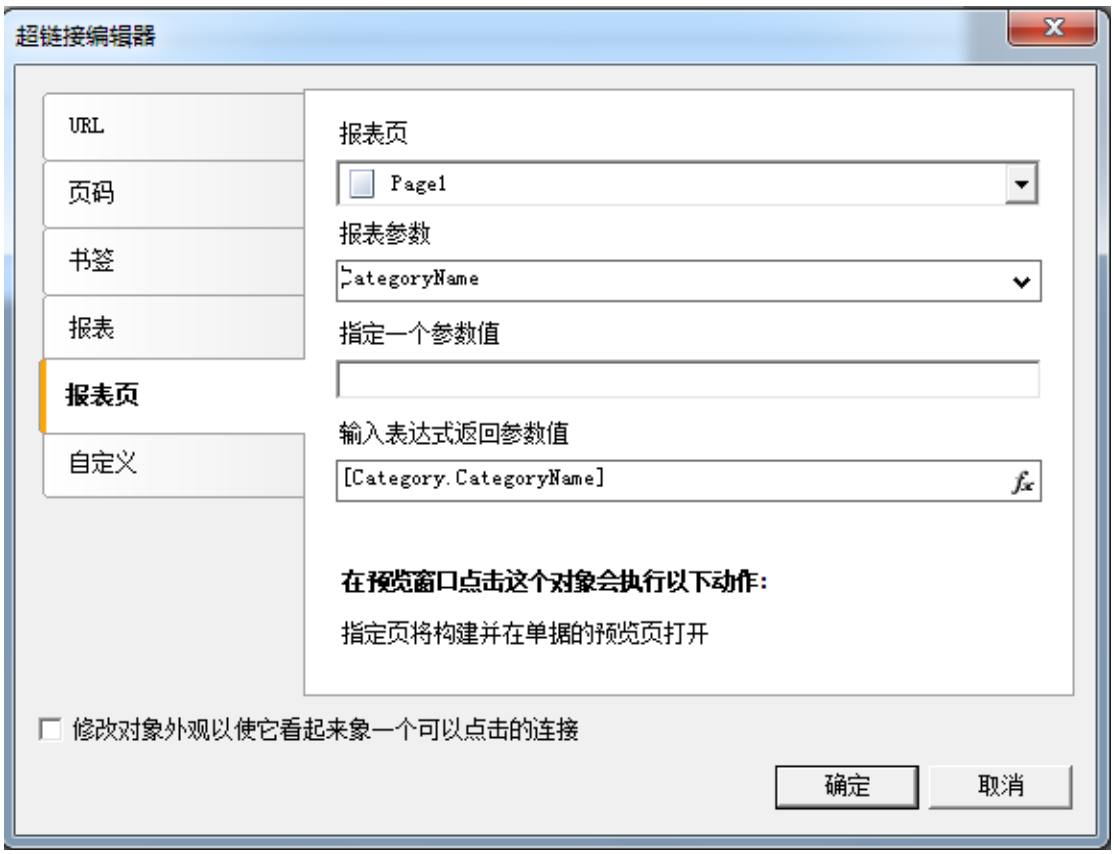
- 直接标示值；
- 标示一个表达式，返回其值。在运行报表时，这个表达式将被计算。

链接到具体页面

这种类型的链接以同样的方式运行，不同之处在于，当前报表的另一页被用作一个详细的报表。为此，你的报表必须至少包含两个页面：一个主报表，一个详细报表。

你必须设置以下参数：

- 报表中的页面名称；
- 报表参数的名称，返回超链接值；
- 超链接值。



点击超链接，则会发生：

- 报表的参数将被设置到超链接的值；
- 标示的页面将被创建并在独立的预览窗口中显示。

报表参数的值可以通过以下两种方法标示：

- 直接标示值；
- 标示一个表达式，返回其值。在运行报表时，这个表达式将被计算。

当您选择一个报表页面时，它的“Visible”属性重置为 false。这意味着，当主报表被创建时，该页面将被跳过。

自定义链接

使用这种类型的链接，您可以自定义鼠标点击时的反应。为此，需使用该对象的“Click”事件处理程序。要做到这一点：

- 选择该对象，然后打开“属性”窗口；
- 点击 按钮，显示该对象的事件；
- 双击“Click”事件。Report 切换到“Code”窗口并创建一个空事件处理程序。

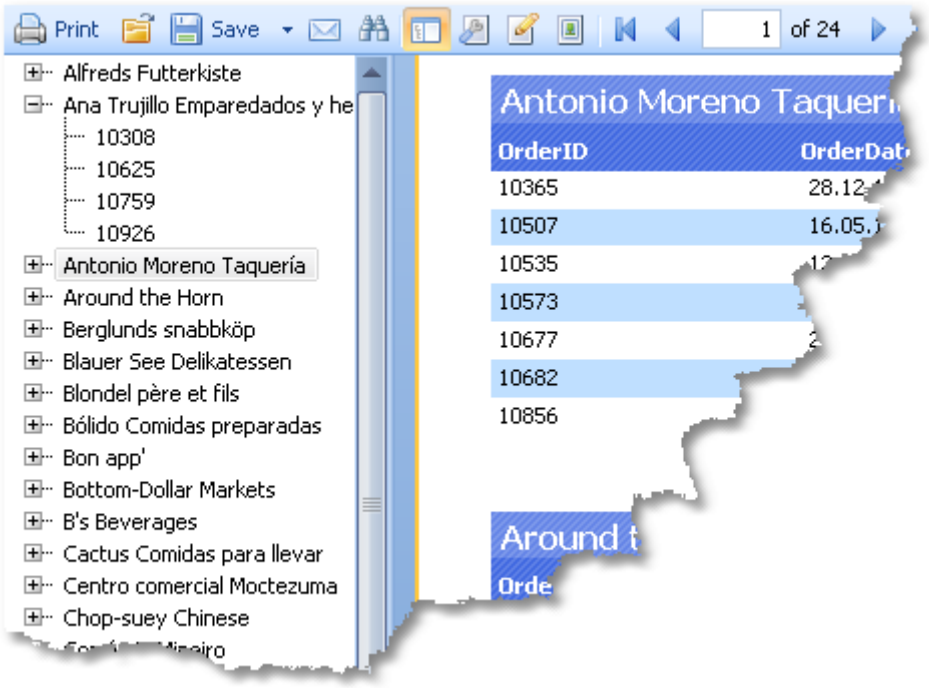
在该程序的代码中，做你所需要的一切事情。你最有可能需要的是一个处理程序调用的链接对象，和一个超链接的值。使用处理程序的参数发送者：

```
private void Text2_Click(object sender, EventArgs e)
{
    // sender - this is the object which was clicked.
```

```
    // In order to receive the value of the hyperlink, you need
    // to cast the sender to ReportComponentBase type.
    object hyperlinkValue = (sender as
ReportComponentBase).Hyperlink.Value;
    MessageBox.Show("Hyperlink value = " +
hyperlinkValue.ToString());
}
```

报表大纲

报表大纲（也被称为“文档结构图”）是在预览窗口中显示的 TreeView 控件：



该控件显示一个树状结构，这是在报表的创建过程中形成的。如果单击树中的元素，你将导航到相应的报表元素。

如果报表已经得到了一个大纲，它会自动显示出来。您可以通过单击工具栏上的按钮显示或隐藏大纲。本报表不自动创建大纲-你应该注意这点。

该报表页面和其所有区域都具有“OutlineExpression”属性。为了填充提纲，在该属性中，标示一个表达式返回该元素的文本。打印区域时，这个表达式会被计算，它的值将被添加到大纲。如果您的报表时主-明细或者分组类型，大纲的结构将类似于报表的结构。

“OutlineExpression”属性可在“属性”窗口中设置。

下面是关于如何为不同类型的报表配置大纲的一些建议：

- 如果您想在大纲中显示已经准备好的报表，设置该报表页的“OutlineExpression”属性。该表达式将返回页面的页码：

```
[PageN]
```

- 在只有一个“数据”区域的“简单列表”报表类型中，设置该区域的“OutlineExpression“属性。使用将在该区域打印的任意数据列作为一个表达式；
- 在有两个“数据“区域的主-明细报表中，在相应的区域设置“OutlineExpression”属性。例如，在“类别/产品”报表类型中，第一个区域的“OutlineExpression”将包含类别的名称，第二个-产品的名称；
- 在分组报表中，配置组头的“OutlineExpression”属性和“数据区域”。使用分组条件作为组头的一个表达式。使用将要在该区域打印的任意数据字段作为“数据”区域。

范例

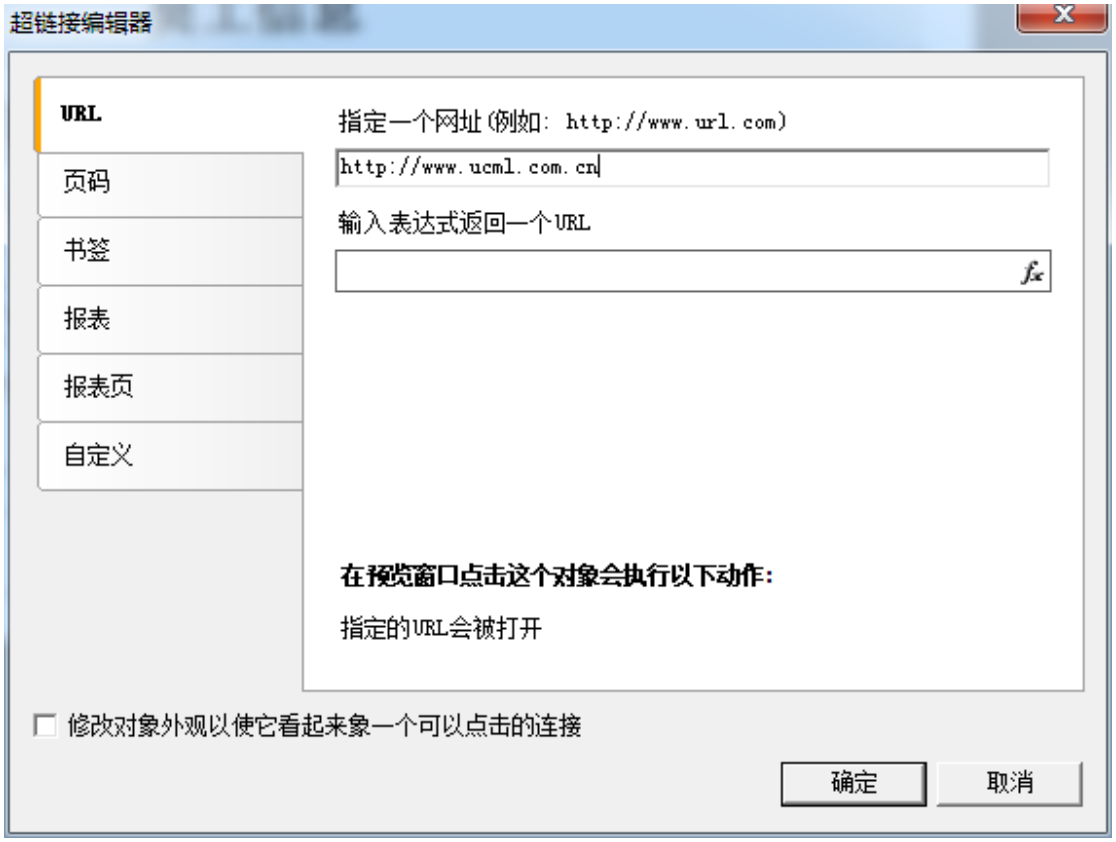
例1. 链接到 Web 页面

在这个例子中，我们将创建一个几具有“文本”对象的简单报表。当在预览窗口中的对象上单击时，我们将移动到 Report 的网页。

创建一个新报表，并添加一个“文本”对象。在其中写入下面文字：

```
Go Report home page
```

在该对象上右键单击并在上下文菜单中选择“超链接…”选项卡。以下面的方式配置链接：



然后，启用“Modify the object's appearance...” 复选框，以应用该对象的一些链接属性（蓝色文本颜色，下划线和手形光标）。

运行此报表，然后单击对象。网络浏览窗口打开，你会移动到 Report 的主页。

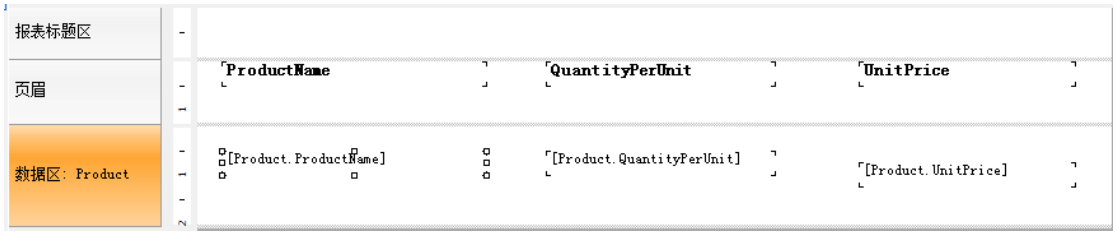
例2. 创建一个明细报表

在这个例子中，我们将创建一个显示类别列表的报表。当单击类别名称时，显示给定类别的一个包含产品列表的明细报表。

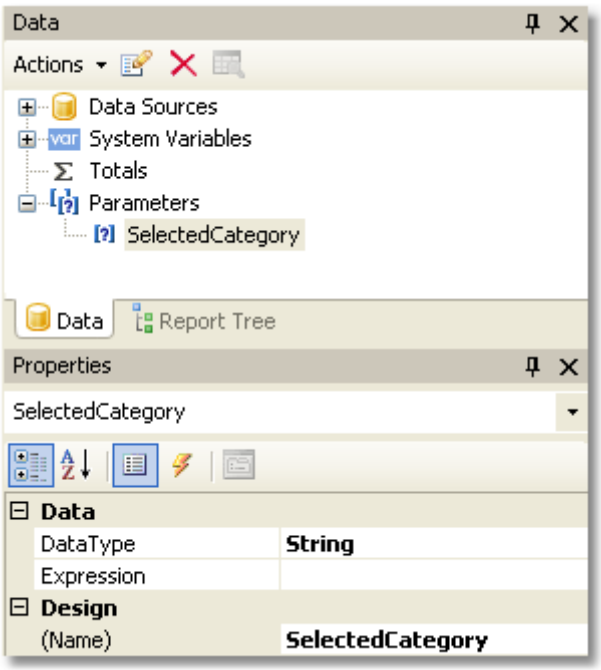
你需要做到以下几点：

- 首先创建一个明细报表；
- 定义标识类别的报表参数；
- 设置数据过滤这个参数；
- 创建主报表；
- 在主报表中，设置超链接，这样，明细报表可以在其所选择的类别参数中运行。

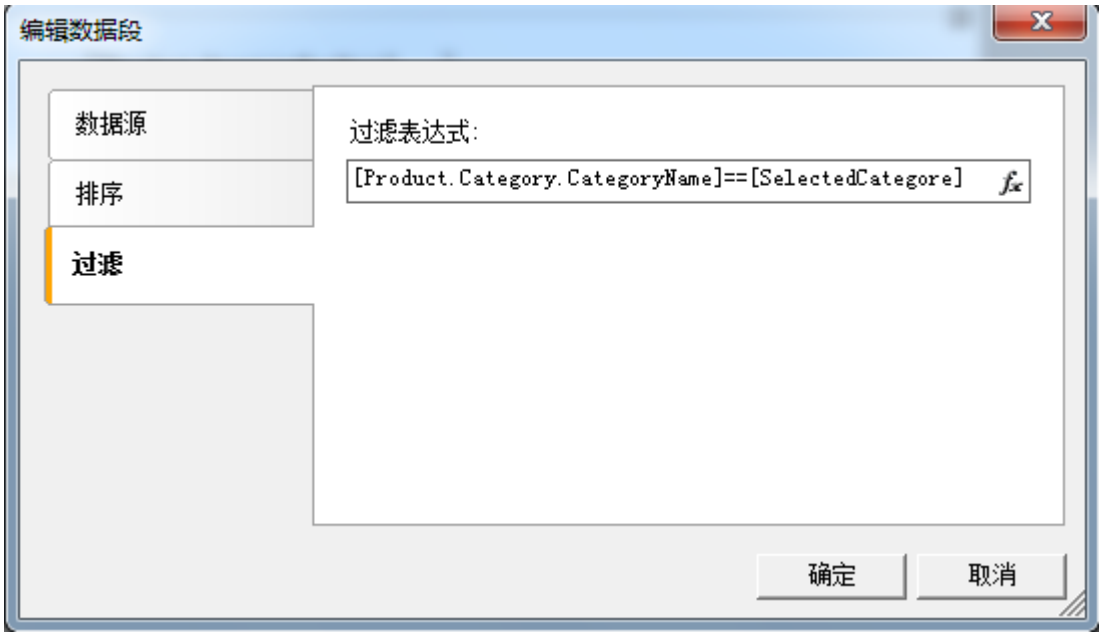
首先，我们将创建一个打印产品清单的明细报表。为此，需创建一个新报表，并选择“产品”表作为数据源。将对象以下面的方式放置：



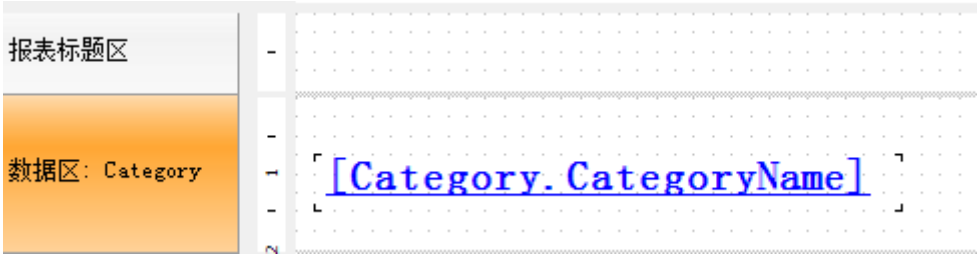
创建用于将从主报表中所选择的类别传递给明细报表的参数。对于类别识别，我们将使用同时包含在“类别”和“产品”中的“CategoyrID”字段。以下面的方式配置参数：



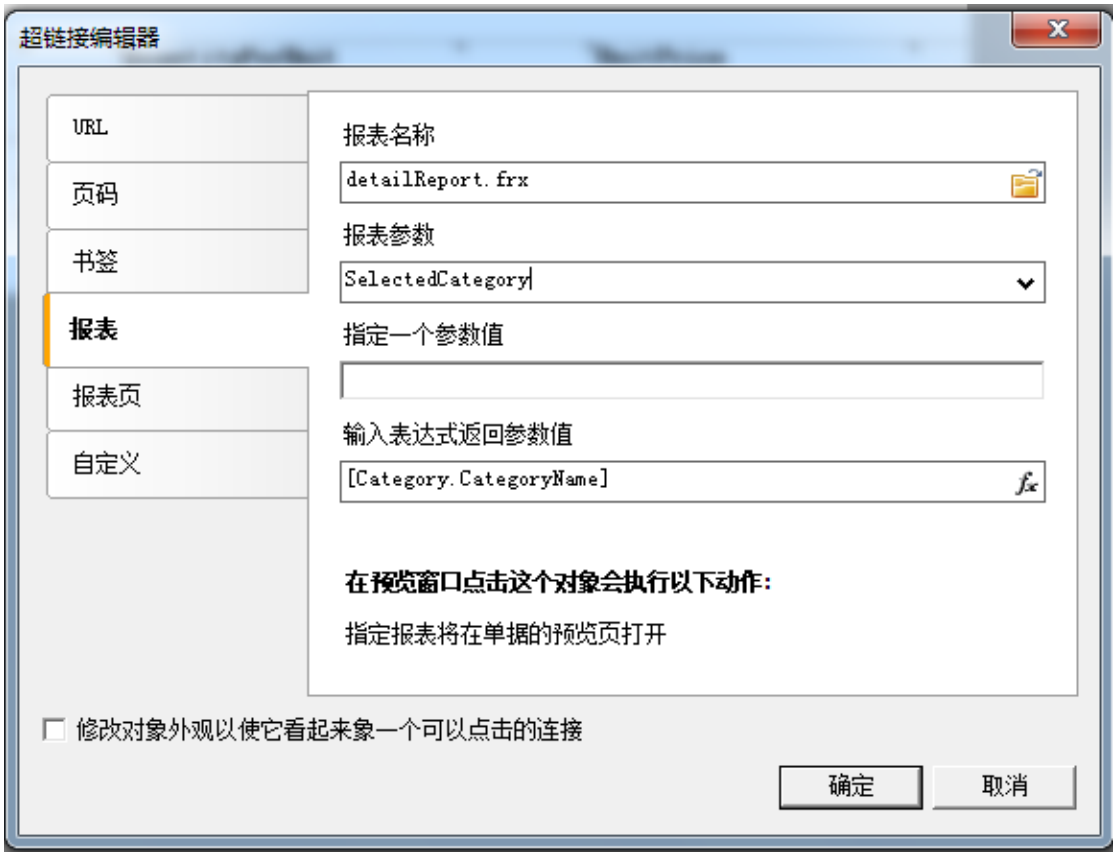
现在，我们需要设置数据过滤功能来过滤属于指定类别的所有产品。为此，双击“数据“区域。切换到“Filter”选项卡，并标示下列条件：



现在，创建主报表。创建一个新报表，并选择“类别”表作为数据源。以下面的方式放置对象：

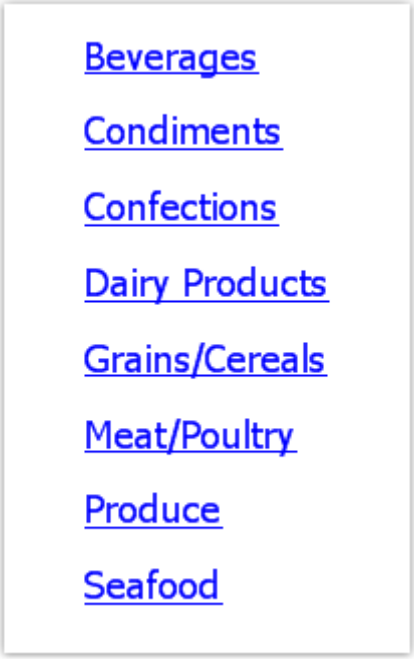


右键单击“文本”对象，并选择“超链接…”选项。以下面的方式设置链接：

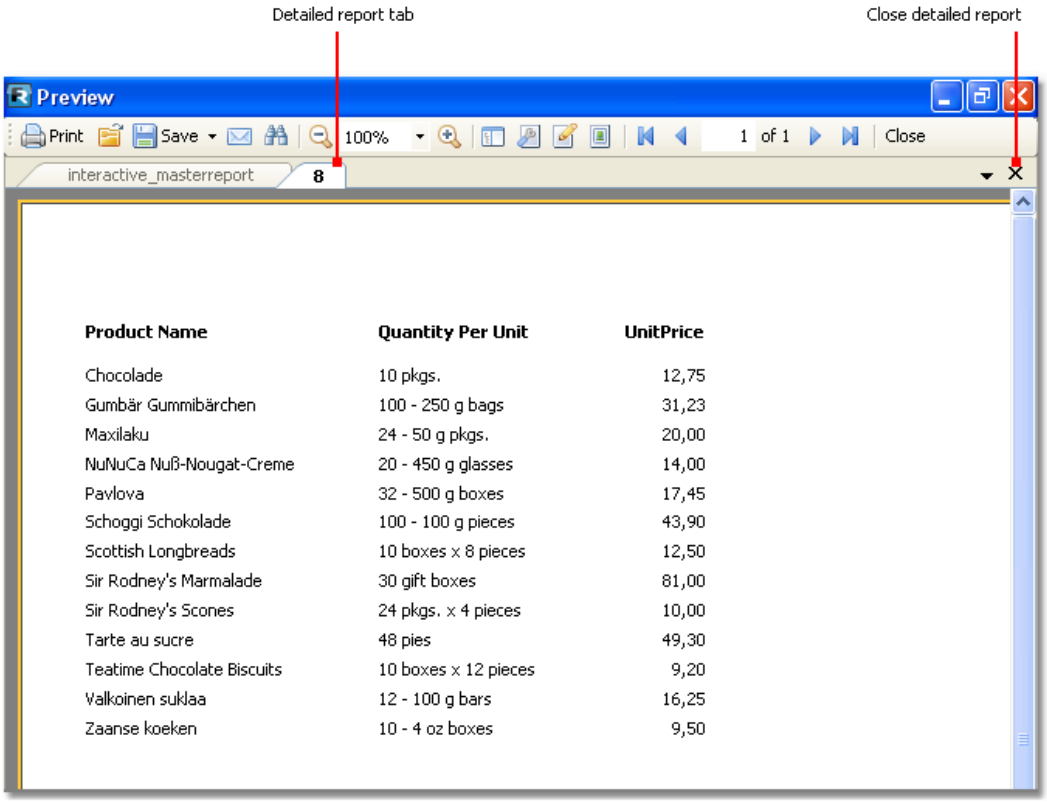


选择明细报表文件的名称作为报表的名称。报表参数可以通过点击右侧列表的按钮，从下拉列表中选择。标示“[Categories.CategoryID]”表达式作为参数值。

运行此报表，你会看到类别列表：



如果你点击其中一个类别，一份详细的报表将被创建。它会显示在预览窗口的一个独立的标签：



如图所示，该标签的标题被设置为超链接的值。在这个例子中，它是包含在“CategoryID”数据列中的数值。这似乎不够正式和美观。让我们使用类别名称而不是其数目改变此报表。为此，请执行以下操作：

在明细报表中：

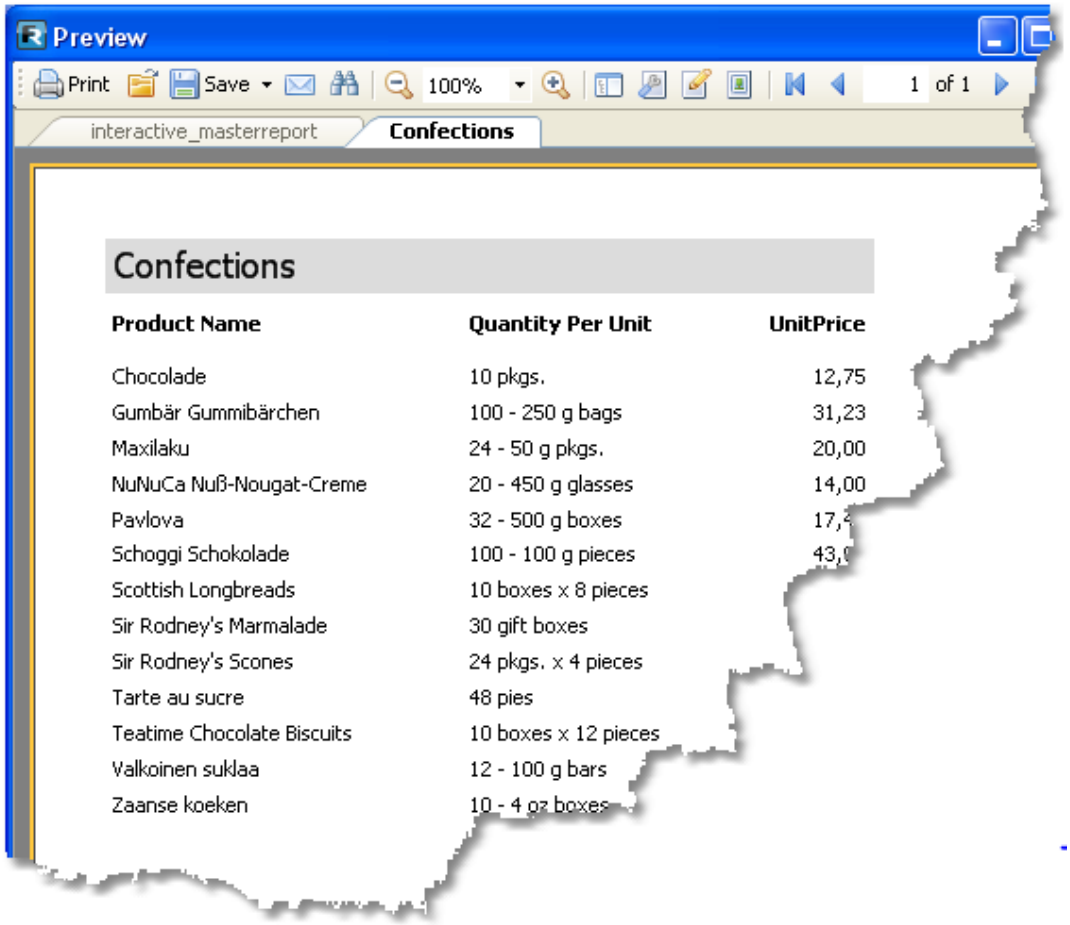
- 更改“DataType”属性的参数为“String”；
- 在报表中添加“类别”数据源。它用于过滤数据时指明“类别名称”；
- 更改“数据”区域的过滤表达式。

```
[Products.Categories.CategoryName] == [SelectedCategory]
```

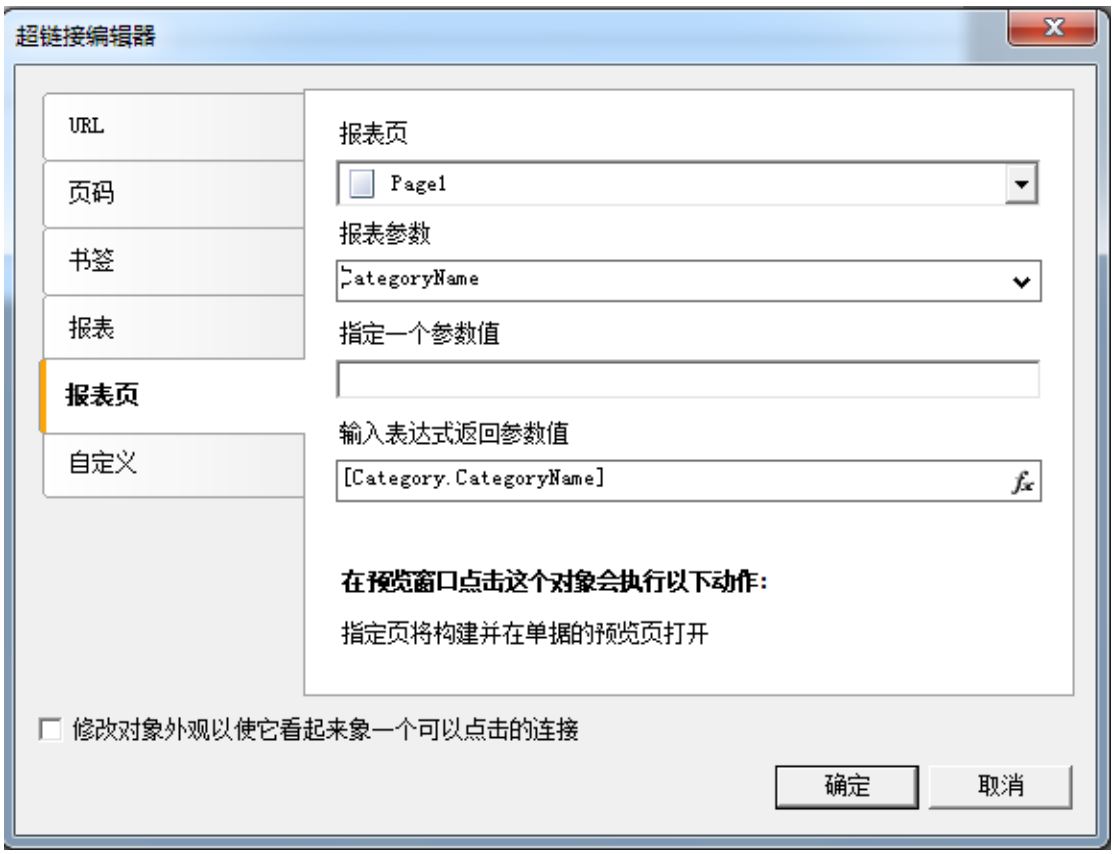
在主报表中：

- 更改超链接的设置。现在，我们将把“[Categories.CategoryName]”的值传递给报表参数。

如果我们现在运行此报表，我们将看到标签的标题设置为类别名称。我们可以改善一点明细报表。添加“对象”文本，它将用于在报表标题上打印所选类别的名称。



当我们使用这个例子时，我们创建了两个报表，并在他们之间进行切换。这样比较麻烦。为了使任务更轻松，可以将两个报表放入一个：主报表在第一页，明细报表在第二页。在这种情况下，超链接需要以如下方式进行设置：



在给定的例子中，我们需要选择 Page2 作为明细报表的页面。

例3. 交互的“交叉表”对象

在这个例子中，我们将看到如何创建一个明细报表，当我们点击“Matrix”对象时。我们将使用一个交叉表，逐年分组显示员工的销售额。

使用“MatrixDemo”表作为交叉表的数据源。它按年份和月份分组显示员工的销售额：

Name	Year	Month	Items Sold	Revenue
Nancy Davolio	1999	2	1	1000
Nancy Davolio	1999	11	1	1100
Nancy Davolio	1999	12	1	1200
Nancy Davolio	2000	1	1	1300
Nancy Davolio	2000	2	2	1400
Nancy Davolio	2001	2	2	1500
Nancy Davolio	2	3	2	160

	001			0
Nancy Davolio	2002	1	2	1700
Andrew Fuller	2002	1	2	1800
Andrew Fuller	1999	10	2	1900
Andrew Fuller	1999	11	2	2000
Andrew Fuller	2000	2	2	2100
Janet Leverling	1999	10	3	3000
Janet Leverling	1999	11	3	3100
Janet Leverling	2000	3	3	3200
Steven Buchanan	2001	1	3	4000
Steven Buchanan	2001	2	4	4100
Steven Buchanan	2000	1	4	3999

以下面的方式配置交叉表：

- 将 "MatrixDemo.Year" 数据字段放置在列标题；
- 将 "MatrixDemo.Name" 数据字段放置在行标题；
- 将 "MatrixDemo.Revenue" 数据字段放置在单元格。

一个创建好的交叉表将如下图所示：

	Year				
Employee	**1999**	**2000**	**2001**	**2002**	**Total**
Andrew Fuller	3 900,00	2 100,00		1 800,00	7 800,00
Janet Leverling	6 100,00	3 200,00			9 300,00
Nancy Davolio	3 300,00	2 700,00	3 100,00	1 700,00	10 800,00
Steven Buchanan		3 999,00	8 100,00		12 099,00
Total	13 300,00	11 999,00	11 200,00	3 500,00	39 999,00

如图所示，一个单元格中的值是员工全年的销售总和。让我们创建一个明细报表，当我们点击单元格时显示。详细报表中包含选定员工在选定一年的每个月的销售额。

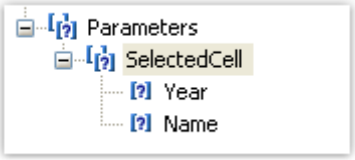
如何让单元格与数据连接，并在此基础上打印？交叉表的每个单元格有它自己的地址。这是由行和列标题中的值的组合。在这个例子中，单元格的地址是年份和员工名字的组合。正是这个值可以传递给明细报表。这怎么做呢？非常简单：

设置超链接，只显示报表名称和参数名称。参数值不需要被标示：对于交叉表的单元格，Report 自身形成其值，并将其传递给参数。

假设，我们点击左上角包含数字 3900 的单元格.这是一个名为“Andrew Fuller”的员工在 1999 年的销售额的总和。什么样的形式用于将其值传递给参数呢？Report 通过使用一个分隔符，结合列和行的值：

```
1999;Andrew Fuller
```

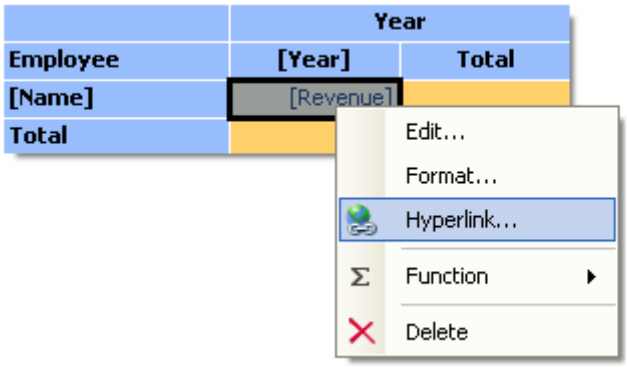
这是否意味着，我们必须从这个字符串中提取年份和员工的姓名，年份转换成整数，并使用这些值，进行数据过滤？不，这要简单得多。我们只需要创建一个具有嵌套参数的参数。您可以在“数据”一章了解这一点。在给定的情况下，父参数可以是这样的：



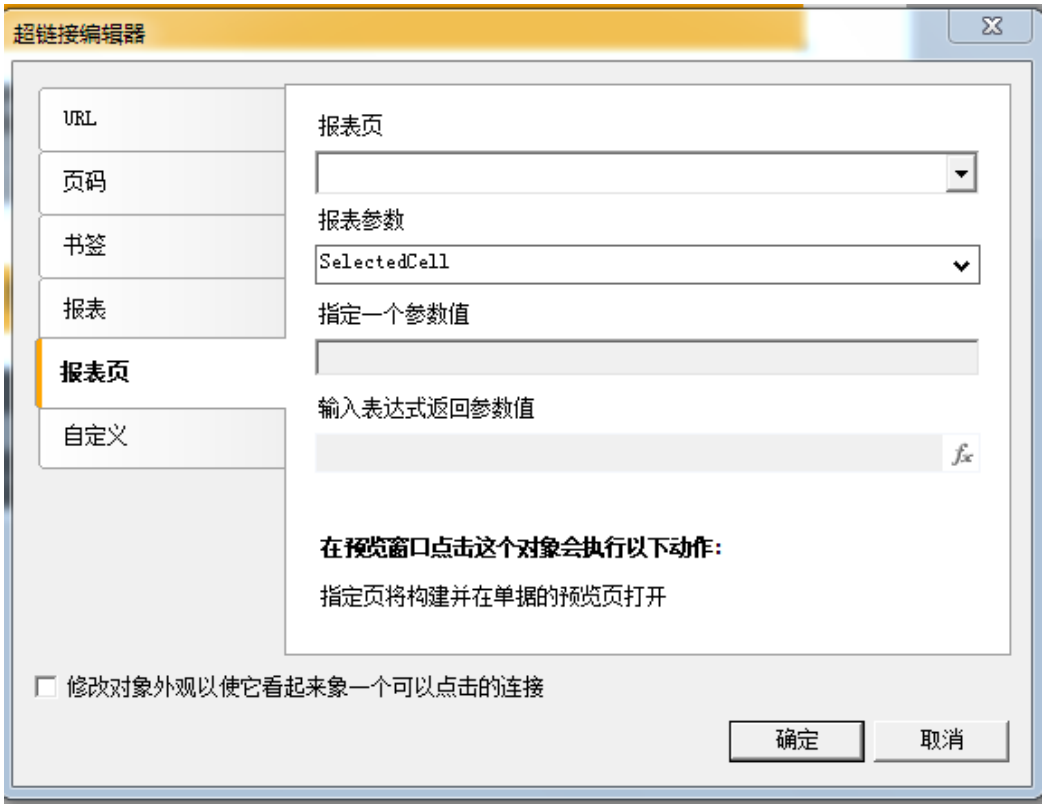
当创建参数时，考虑以下几点：

- 你不需要设置父参数，只需要给它命名；
- 父参数必须有尽可能多的嵌套参数，因为有大量交叉表传递的值。在给定的情况下，有两个值；
- 嵌套参数的顺序必须与从交叉表传递的值的顺序相对应。在这个例子中，年份将在第一个参数中传递，而员工的名字将在第二个参数中传递；
- 嵌套参数可以根据你的意愿命名，但最好给它们的命名与交叉表元素的名称相对应。
- 正确的为每一个嵌套参数设定数据类型是非常重要的。数据类型必须与其值相对应，从而传递给参数。在我们的例子中，第一个参数（年）必须是整数类型，而第二个（员工姓名）-字符串类型。

当我们清楚所有需要的东西后，我们将创建报表。选择交叉表的单元格并调用超链接编辑器：

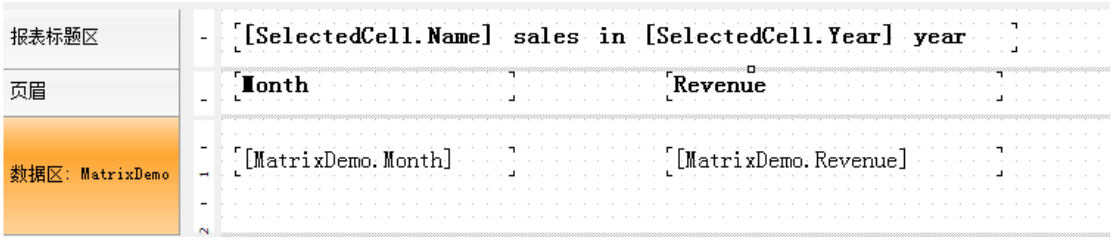


在超链接配置中，标示父参数作为报表参数（在我们的例子中-“SelectedCell）：



Report 将值传递给 SelectedCell.Year 和 SelectedCell.Name 嵌套的参数。这些值将被转换成数据类型，在参数配置中标示-这就是为什么正确配置参数的数据类型是重要的。

明细报表放置在主报表的一个独立的页面，并使用相同的数据源：



为了显示选定年份员工的销售额，设置了过滤。为此，打开“数据”区域编辑器，然后载明下列过滤条件：

```
[MatrixDemo.Year] == [SelectedCell.Year] && [MatrixDemo.Name] == [SelectedCell.Name]
```

该报表已准备就绪。运行该报表并点击左上角的单元格。将打开一份具有以下数据的明细报表：

Andrew Fuller sales in 1999 year

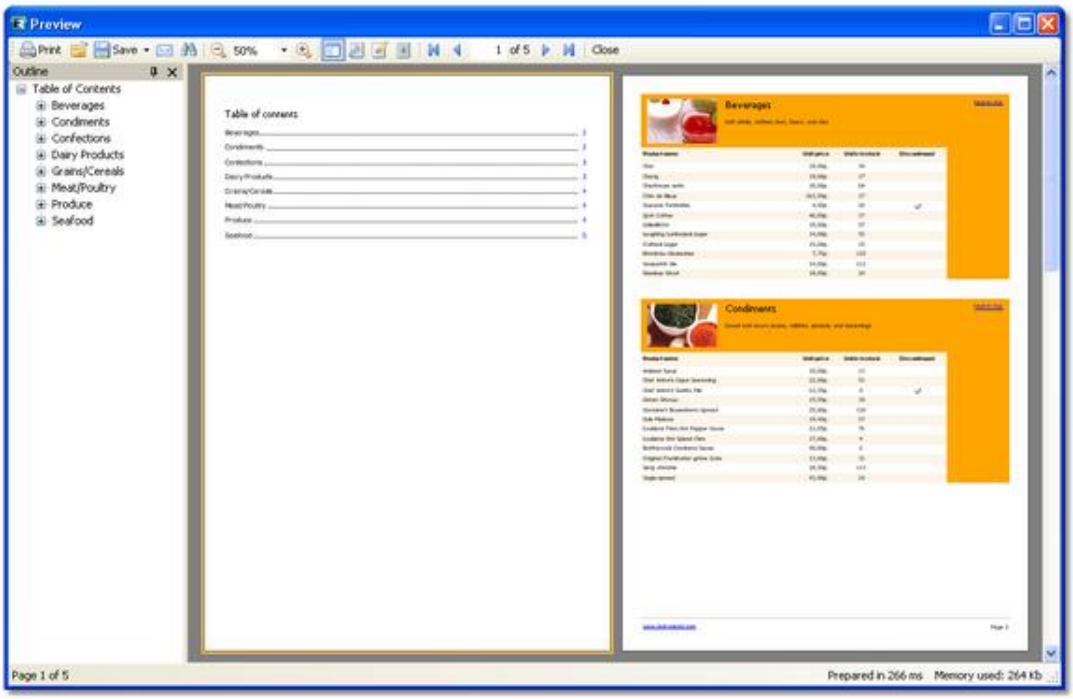
Month	Revenue
10	1 900,00
11	2 000,00

如图所示，值（1900+2000）的总和和我们点击的交叉表单元格的值相对应。

例4. 具有目录，导航和大纲的报表

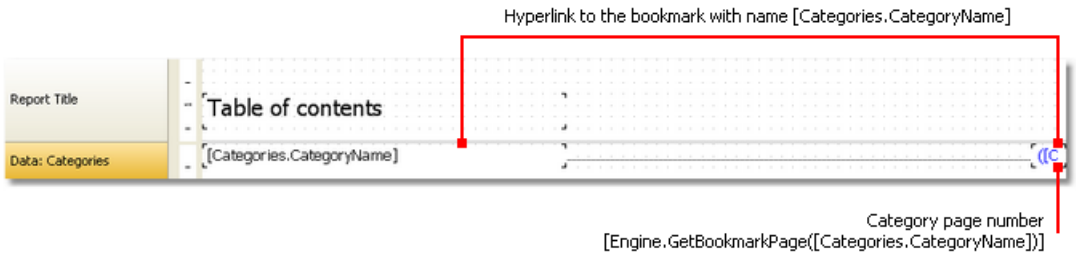
在这个例子中，我们将着眼于创建一个具有以下特点的报表：

- 在第一页，打印交互式“目录”（TOC），即你可以点击它的元素导航到相应的页面；
- 在预览窗口，显示交互式大纲；

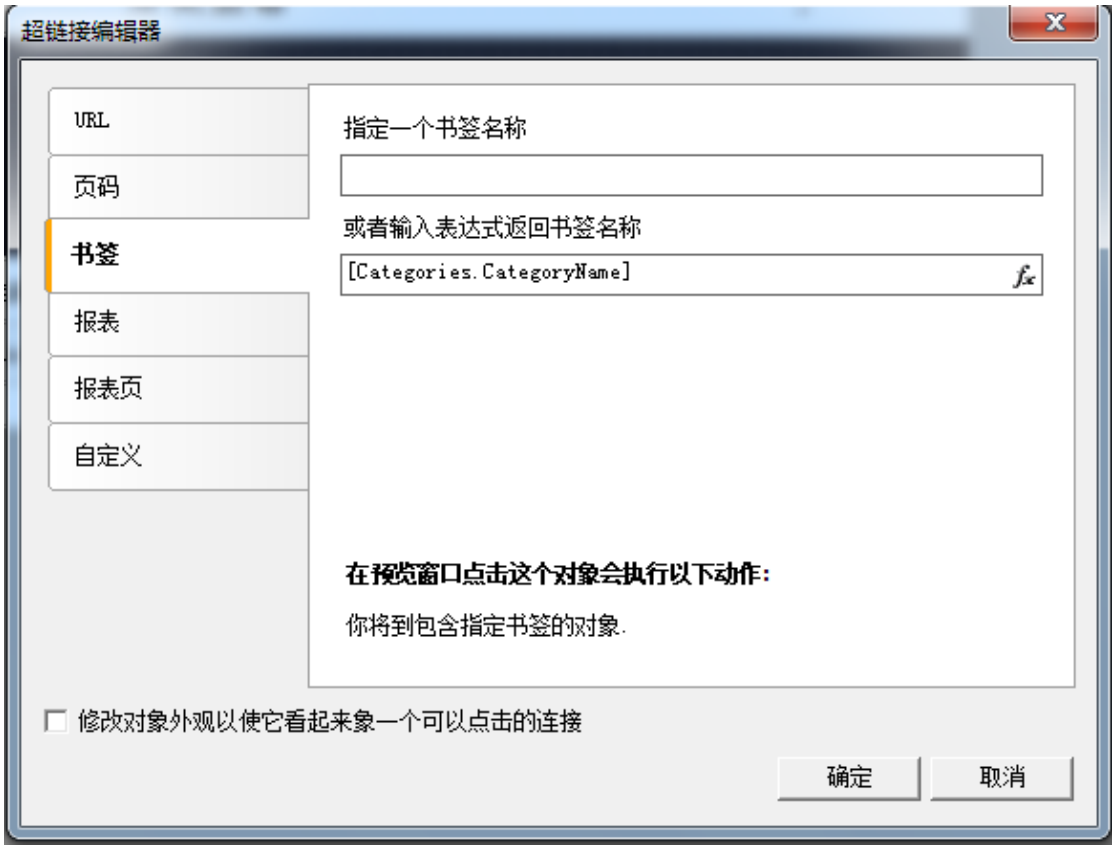


该报表使用“类别”和“产品”表。在 TOC 中，我们将打印类别列表。报告的其余部分将打印产品的分类列表。报表模板将由两部分组成：第一页用于打印 TOC，第二页用于打印该报表的主要部分。

首先，我们讨论一下 TOC。创建一个报表，并在其中添加“类别”和“产品”数据源。连接“数据”区域到“类别”表，并以下面的方式放置对象：



为了使 TOC 对象具有交互性，配置它的“超链接”属性：



标示类别名称作为书签。我们将在后面定义对象的书签。

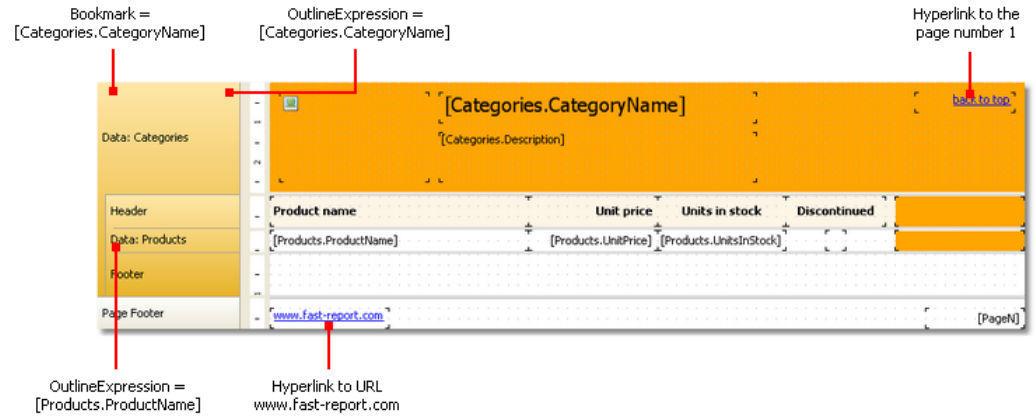
为了在目录中打印页码，你需做到以下几点：

● 启用该报表的“double pass”设置。它可以在“"Report|Options..."菜单中完成。这项必须完成，因为 TOC 页面先于其他页面打印。此时，Report 不知道将打印哪个类别。

● 使用“Engine.GetBookmarkPage”功能，它返回指定书签的页码。在我们的例子中，我们使用“[Categories.CategoryName]”表达式作为书签的名称，因此函数调用将如下：

[Engine.GetBookmarkPage([Categories.CategoryName])]

在该报表的第二页，我们将创建一个主-明细报表，如下图：



设置我们在点击 TOC 中的元素时，导航到的书签。为此，选择第一个“数据”区域并在其“Bookmark”属性中标示下面的表达式：

[Categories.CategoryName]

要设置报表大纲，需执行以下操作：

- 选择第一个报表页面。这可以通过切换页面实现；
- 在“属性”窗口，设置“OutlineExpression”属性的以下值：

"Contents"

- 切换到报表的第二页；
- 选择第一个“数据”区域并设置它的“OutlineExpression”属性：

[Categories.CategoryName]

- 选择第二个“数据”区域并设置它的“OutlineExpression”属性：

[Products.ProductName]

继承报表

我们经常会使用具有相同数据的报表-例如，相同公司标志和数据的页眉/页脚-邮件，地址等。想象一下这种情况，你需要改变一些公司的数据-例如，电子邮件。你必须在每个报表中操作！为了避免这种情况，你可以使用报表继承。这是什么呢？

例如，在每个报表中具有一些相同的元素（LOGO，公司名称，电子邮件等）。这些元素通常放置在报表的页眉/页脚。您可以创建一个只包含这些共同元素的基表。所有其他的报表将使用基本报表，从而包含这些共同的元素并定义自己的元素。

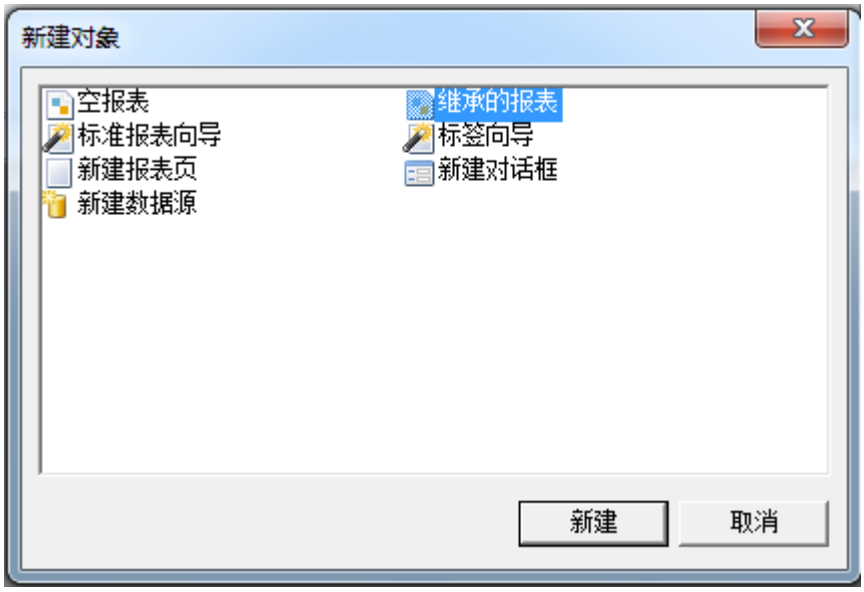
如果你需要改变一些东西（LOGO，电子邮件），你只需打开基本，并进行必要的修改。所有其他从基表继承的报表，将自动更改。事实上，当你打开一个继承的报表时，先打开的是基本，然后才是继承报表。

创建报表

要使用继承，需做到以下几点：

- 创建一个基表，并将其保存到一个文件；
- 创建一个继承基表的新报表。

要创建一个继承报表，请选择“文件|新建…”菜单项，然后在窗口中选择“继续的报表”：



你将被要求选择一个基表文件。该文件必须在此刻创建。然后，基表将加载到设计器。你可以根据自己的意愿去更改它。你看，从基表加载的对象都带有“Lock”的标志：



这意味着，你不能删除这些对象，重命名或移动到其他区域。

你可以添加新的对象或区域，更改对象的外观，大小和位置。当你完成操作后，保存报表。

更改基表

下面我们看看，如果我们改变了基表将会发生什么。我们可以：

- 从基本中删除对象。这个对象也将从继承报表中删除；
- 在基本中添加对象。该对象也将自动添加到继承报表；
- 改变对象的大小，位置，文本，外观。所有更改都将反映在继承报表，假使该对象的位置在继承报表中没有改变。

最后一点需要解释一下。让我们看看使用继承报表的两个例子。在第一个例子中，我们将做以下几点：

- 创建一个包含 Text1 对象的基表；
- 创建一个继承报表，无需做任何修改，并保存；
- 打开基表，并移动 Text1 对象；
- 打开继承报表，我们将看到 Text1 对象同样发生了移动。

在第二个例子中，我们将做以下几点：

- 创建一个包含 Text1 对象的基表；
- 创建一个继承报表；
- 在继承报表中，移动 Text1 对象到一个新的位置并保存报表；

- 打开基表并将 Text1 对象移动到不同的位置；
- 打开继承报表我们将看到 Text1 对象没有移动。

这是因为，我们在继承报表中改变了该对象。这种变化被保存在继承报表的文件中。现在，如果我们在基表中改变原来的对象，在继承报表中，它的改变将被忽视。在这种情况下，新对象的位置将被忽略。所有其他变化（如文本颜色）将仍然被反映在继承报表中。

如果我看查看继承报表文件的内容，这种行为将变得清晰。例如，这反映了原始对象在继承报表中的保存方式，假使该对象没有改变。

```
<inherited Name="Text1"/>
```

当打开继承报表时，Report 会加载所有在基表中定义的对象的属性，并保存在继承报表中。

限制

报表继承的目的是要满足以下目标：保存通用报表元素，如在独立文件中的页眉和页脚，并在继承报表中重新使用它们。不要试图使用继承报表来执行复杂的任务。尤其要避免以下操作：

- 不要从继承报表中继承报表；
- 不要在基表中使用复杂的对象，如表和交叉表；
- 不要在基表中使用脚本；
- 不要在基表中使用参数。

图表报表

该报表使用微软图表库来显示图表。这个库包含在.NET FRAMEWORK 4.0 中。可以在下面的 URL 中下载：

http://www.microsoft.com/downloads/details.aspx?FamilyID=130f7986-bf49-4fe5-9ca8-910ae6ea442c

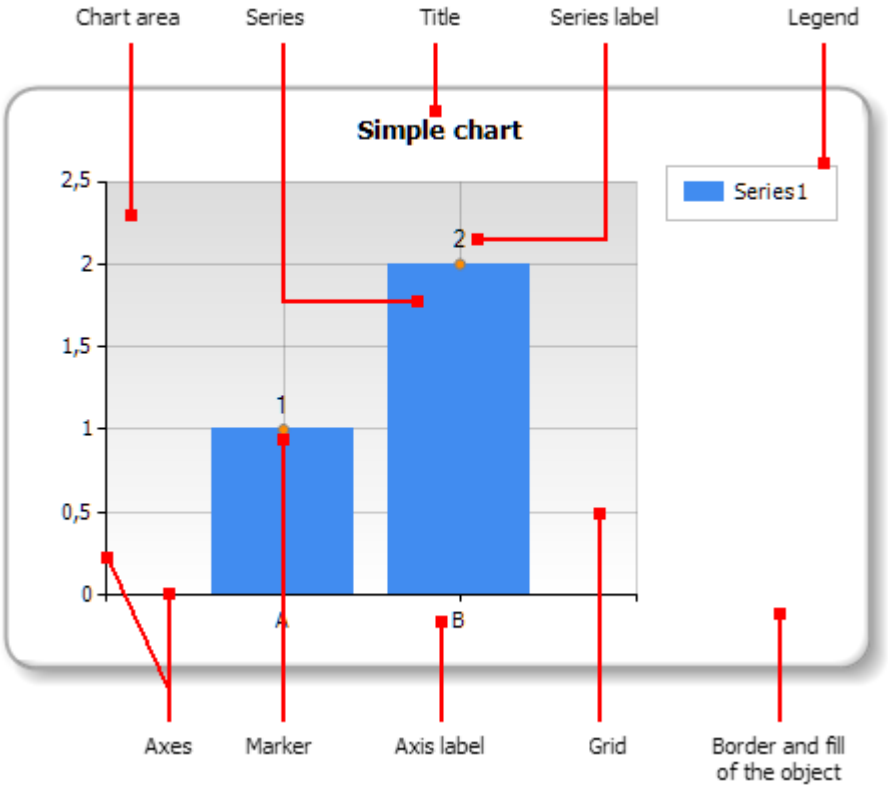
该报表使用微软图表库来显示图表。这个库包含在.NET FRAMEWORK 4.0 中。可以在下面的 URL 中下载：

http://www.microsoft.com/downloads/details.aspx?FamilyID=130f7986-bf49-4fe5-9ca8-910ae6ea442c

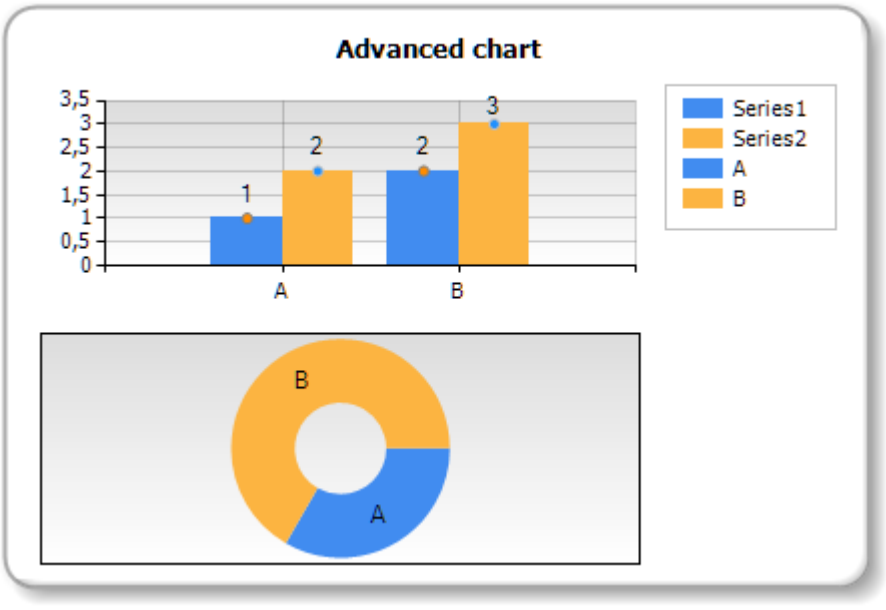
- 超过 30 种类型的图表
- 3D 支持
- 在一个图表中添加不同类型的图组
- 对每个图表元素的所有外观与动作全面操控

图表元素

微软图表由下面的元素组成：

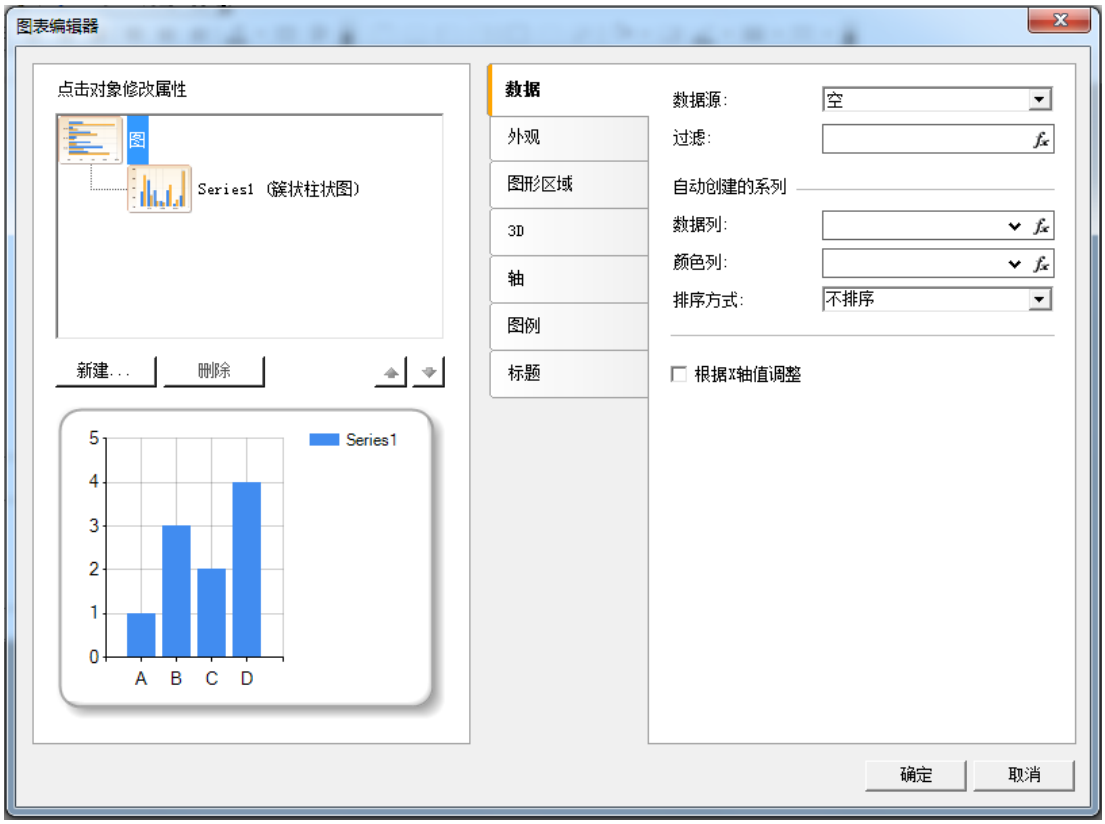


一个图表可能包含一个或多个图表区域。一个图表区域又可能包含一个或多个图组。下面的图表包含两个图组（每个区域包含两个图形，第二个区域包含一个图形）：



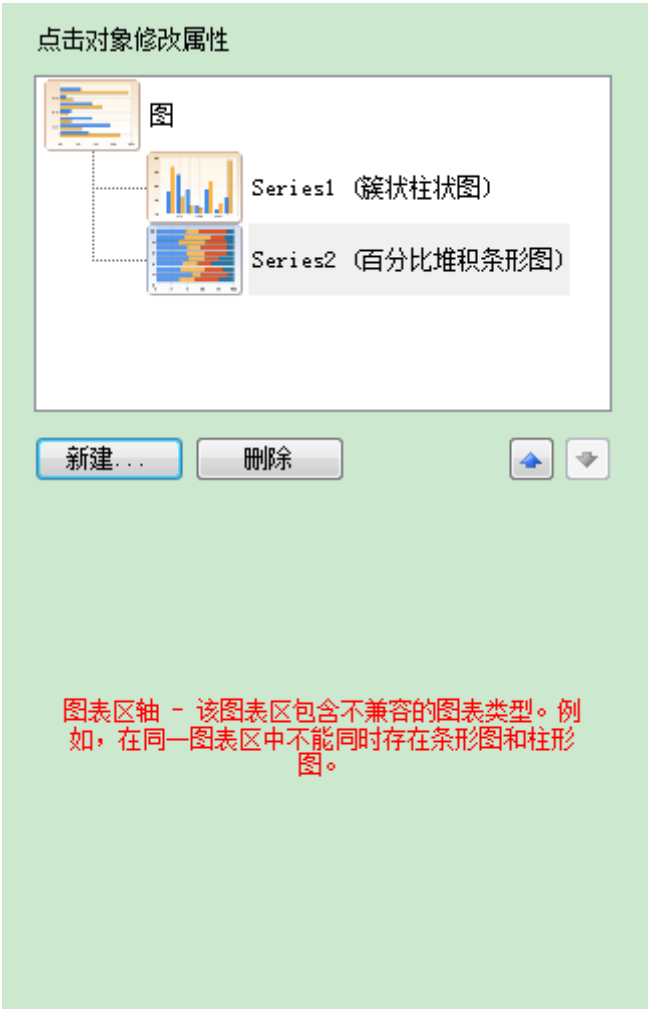
图表编辑器

图表控件在图表编辑器中有大量的设置。通过双击图表控件来打开图表编辑器：

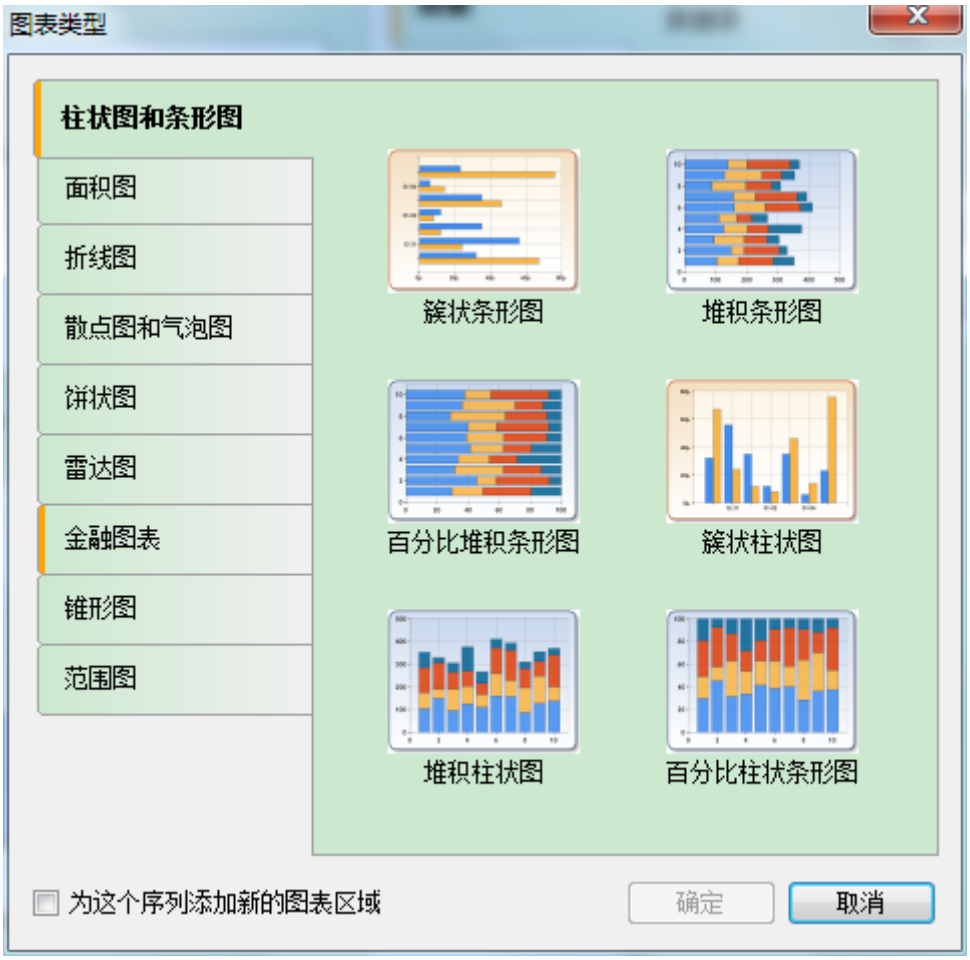


操作图形

图表控件可包含一个或多个图形，图形列表在编辑器中显示：



点击图表编辑器中的“新建”按钮，你将可以看到图表类型对话框：



选择需要的类别，然后选择序列类型。如果希望将该序列放到独立的图表区域中，那么，选中“为这个序列添加新的图表区域”这个复选框。对于一些图表序列类型，不管复选框是否选中新的图表区域都会自动添加。

删除图表序列，可以按“删除”按钮。可以点击  按钮来改变图表序列的显示顺序。

设置外观

使用图表编辑器，你可以对图表的每个元素的外观进行设置。所有的属性（超过 100 个）被分成许多类别。一些属性针对图表对象，其它的针对图表序列对象。如果你在序列列表中选择图表对象，那么可以看到如下的属性窗体：



- 外观：图表的边框与填充
- 图形区域：边框、填充与阴影
- 3D：3D 设置
- 轴：设置轴的外观，它的标题、标签、网格、标识与定制标签与脚本
- 图例：图例样式的停靠，边框、填充、阴影与字体
- 标题：样式标题的停靠、边框、填充、阴影与字体

如果你选择了序列列表中的图表控件，你将看到下面的属性页：



- 外观：一些对选择的图表类型的设置
- 填充和边框：图表的填充与边框值
- 标签：图表标答，你可以选择标签类型、字体、颜色和填充
- 标记：序列标记，你可以选择标记的类型、颜色与边框

连接图表到数据

有如下几种方法对图表进行数据填充：

1、使用数据源。使用这种方式，你需要为图表控件指定好数据源，并对每个序列中的图表进行数据字段的绑定。

2、把值设置到每个序列

3、使用脚本为控件填充数据

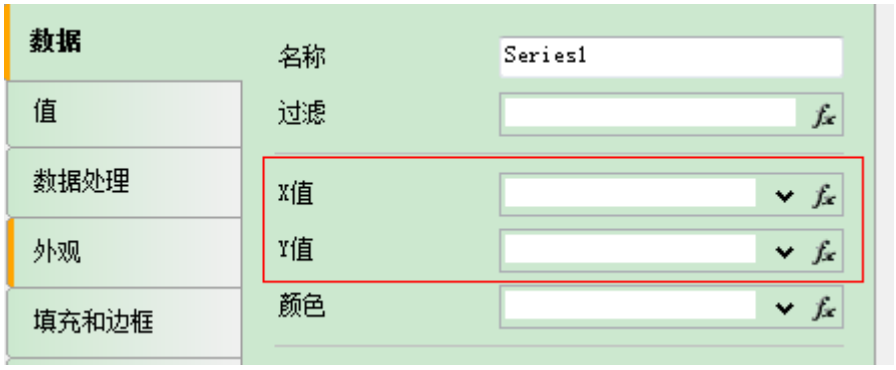
将图表绑定到数据源，有如下几种方法：

- 在序列中选择图表控件
- 切换“数据”属性标签页
- 选择数据源



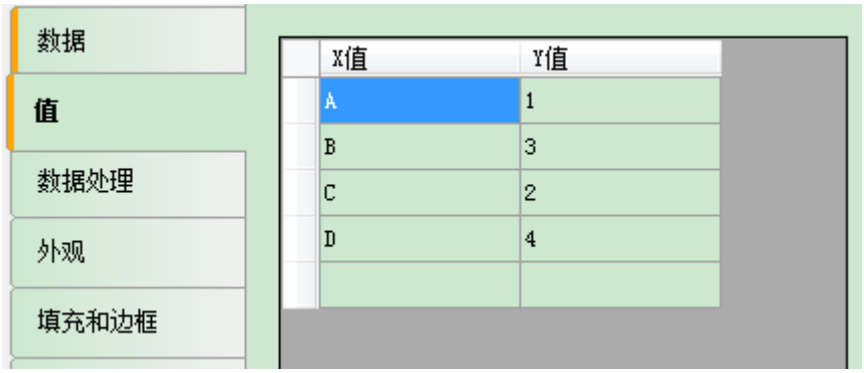
- 如果需要的话，可以设置数据过滤表达式。这个过滤将被作用于图表序列中的所有图表
- 在序列列表中选择图表

为每个图表值绑定数据字段，根据图表的类型，可能有两个或多个值供选择。大多数据图表有两个值-X 值与 Y 值。



- 如果需要，可以设置数据的过滤表达式，这个表达式仅作用于当前图表。
- 在“颜色”控件选项中，你可以指定一个数据类型为颜色值的数据字段。

你可能需要为列表设置值，这时，数据连接并不是必须的，要达到这种效果，在列表中选择一个图表，切换到“值”选项卡，在表格中填写值：



数据排序

默认情况下，图表控件按自然顺序显示数据，你可以改变这种顺序。要达到这种效果，选择图表列表中的图表，切换到“数据处理”属性页：



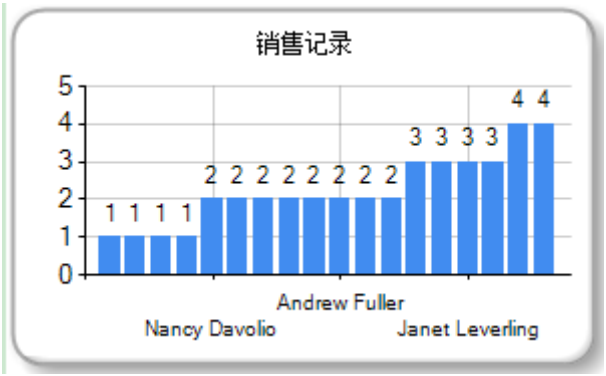
你可以在排序模式中选择一个-不排序、按 X 值排序、按 Y 值排序。

数据分组

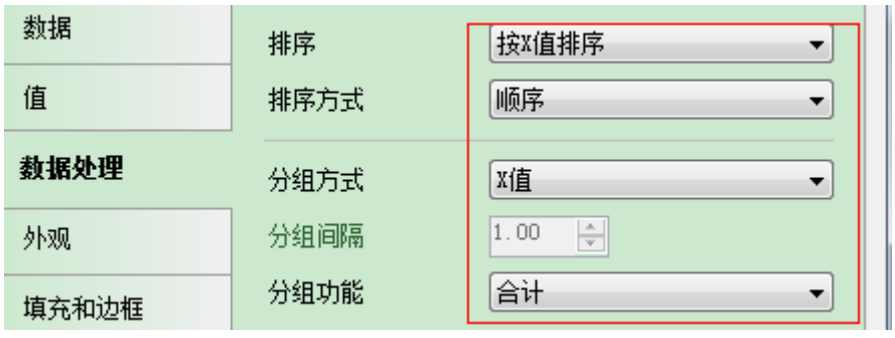
有时我们会面对这样一个问题，图表 X 轴的同样值有多条记录。例如，MatrixDemo 表，在我们的演示示例中，有如下的数据：

名称	年	月	销售数量	收入
Andrew Fuller	2002	1	2	1800
Andrew Fuller	1999	10	2	1900
Andrew Fuller	1999	11	2	2000
Andrew Fuller	2000	2	2	2100
Janet Leverling	1999	10	3	3000
Janet Leverling	1999	11	3	3100
Janet Leverling	2000	3	3	3200
...				

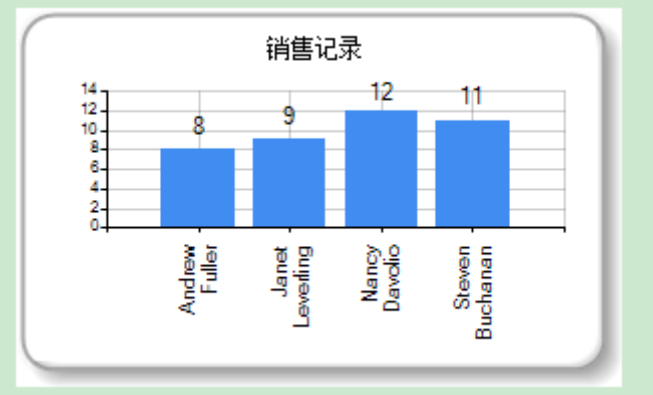
如果要基于这份数据来创建一个图表（例如：设置 X 轴的值为名称，Y 轴的值为销售数量），你将会看到下面的错误结果：



在这种情况下，我们需要的是同一名销售只有一条记录。为了达到这个目的，需要在图表序列中选择图表控件，切换到“数据处理”属性页，选择“按 X 值排序”和选择“合计”分组功能。



结果是，同一个员工的销售记录将被分组合计为一个值，你将看到如下结果：

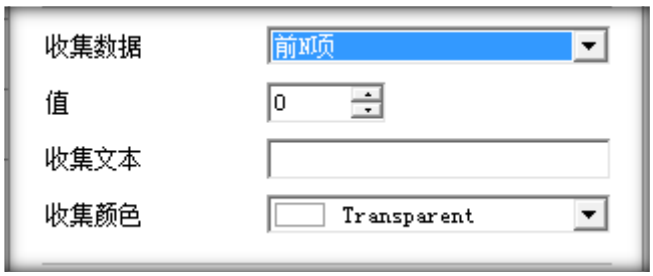


数据集合

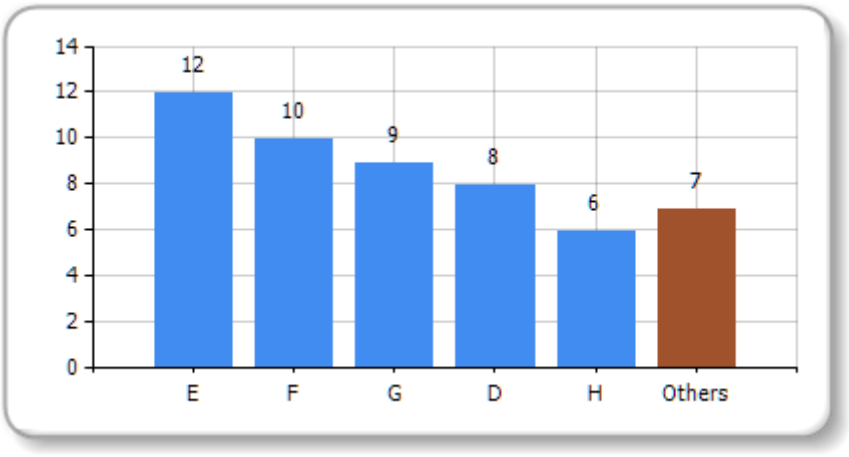
该工具用于数据处理，它允许将一系列的值集合成一个值。您可以选择以下算法之一：

Algorithm	Description
前 N 项	只显示前 N 个值。其他所有值被集合在一起并显示为“其他”值（你可以为这个值选择标签）
后 N 项	只显示后 N 个值。如果没有设置集合的值的文本，则不会显示该值。
小于	图形值小于规定值，则被收集并显示为“其他”值。
小于百分比	图形的值小于指定的百分比，则被收集并显示为“其他”值。
大于	图形值大于规定值，则被收集并显示为“其他”值。
大于百分比	图形的值大于指定的百分比，则被收集并显示为“其他”值。

例如，为了显示前 5 个值，以下列方式设置系列：



你将看到如下结果：



Exploding the values

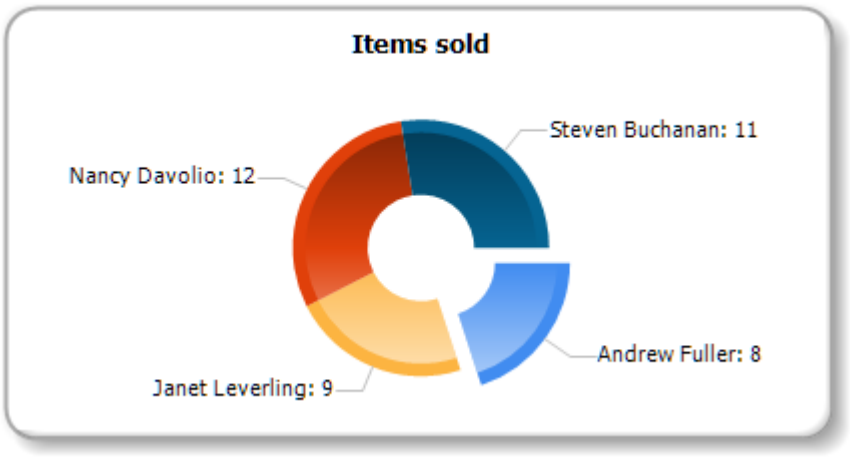
对于饼型图形，你可以分离某些值。要做到这一点，在图形列表选择图形，并切换到“数据处理”选项卡：



你可以使用下列分离类型之一：最大值，最小值和比值。如果你选择了后一个模式，你必须指定一个你想分离的值。它可以是任意表达式。例如，如果你想分离 Andrew Fuller 的值，使用以下表达式：

```
"Andrew Fuller"
```

你将得到以下结果：



配置自动-图形

您可以根据数据源的数据设置自动创建新图形的图表。要设置自动图形，需执行以下操作：

- 创建一个图形，并设置它的属性。这个图形将被用作所有新图形的样板。

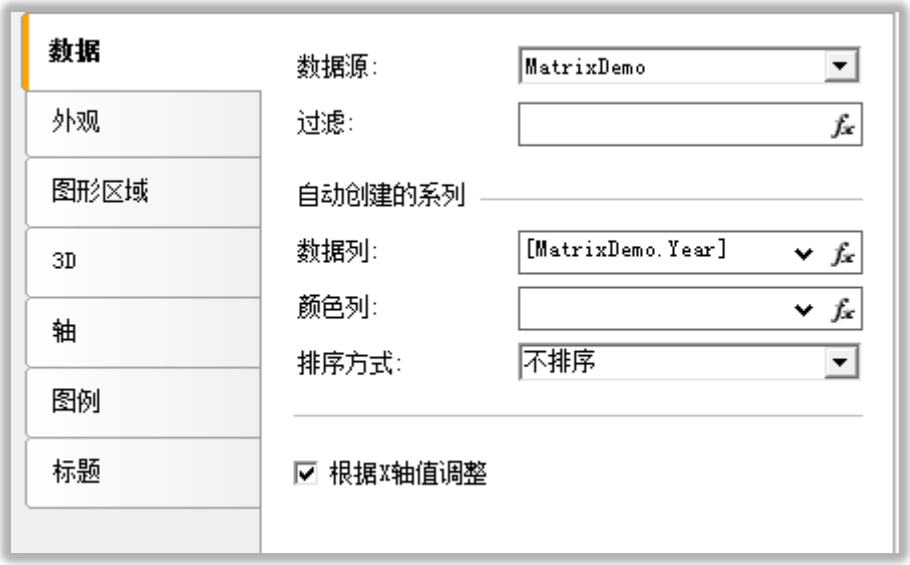
- 选择“图标”对象并设置自动-图形数据字段。这个字段的值将作为新图形的名称。如果还没有该名称的图形，则新图形会被添加。

让我们演示如何创建自动-图形。我们将使用 MatrixDemo 数据表获取一个员工每年的销售额的图表。一个图形代表一年：

- 将图表连接到 MatriDemo 数据源；
- 创建一个图形并设置它的数据：

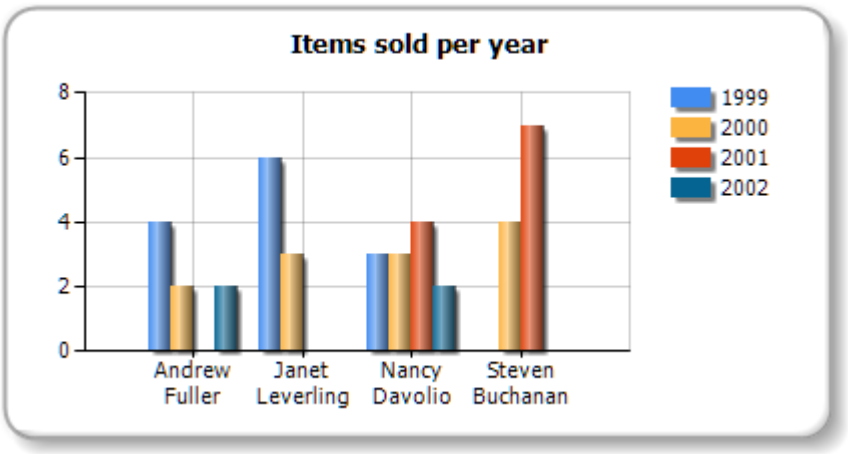


- 在“数据处理”选项卡上，选择“分组方式：X 值”复选框。这是必要的，因为我们数据源中有多个具有相同姓名的员工；
- 在图形列表选择图表并在其“数据”选项卡设置它的自动-图形字段：



我们的图形可能有不同的值（因为一些员工在指定年份没有销售额）。为了将图形的值排成一排，选择“根据 X 轴值调整”复选框。

我们将得到如下结果：



交互式图表

图表，以及其他的报表对象，可以是交互的。您可以设置图表，当你点击它的值时，另一个报表被执行并显示。为此，你需要像在“报表的交互性”一章中介绍的设置“Hyperlink”属性。在你点击图表元素时，它会自己将值传递给一个超链接。

让我们从 Report demo 程序中观察“Chart/Interactive Chart”。

创建一个具有两页的报表。第一页包含一个图表，第二页包含当您单击图表值时，将显示的详细报表。

将“Chart”对象放置在报表的第一页，并在图表编辑器中设置它的属性:

- 从图形列表选择“Chart”元素并选择 MatrixDemo 数据源;
- 从图形列表选择图形并设置 X and Y values: X = [MatrixDemo.Name], Y = [MatrixDemo.ItemsSold]:

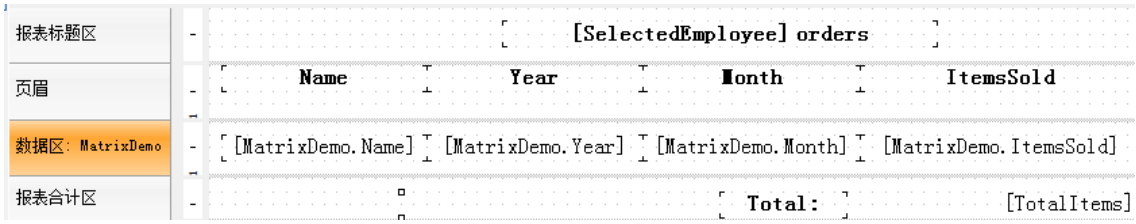


- 切换到“数据处理”选项卡并选择分组方式-“X 值”:



在报表的第二页，创建 list-type 报表：

- 在“数据“窗口，创建一个新报表参数，并命名为“SelectedEmployee”；
- 创建以下报表布局：



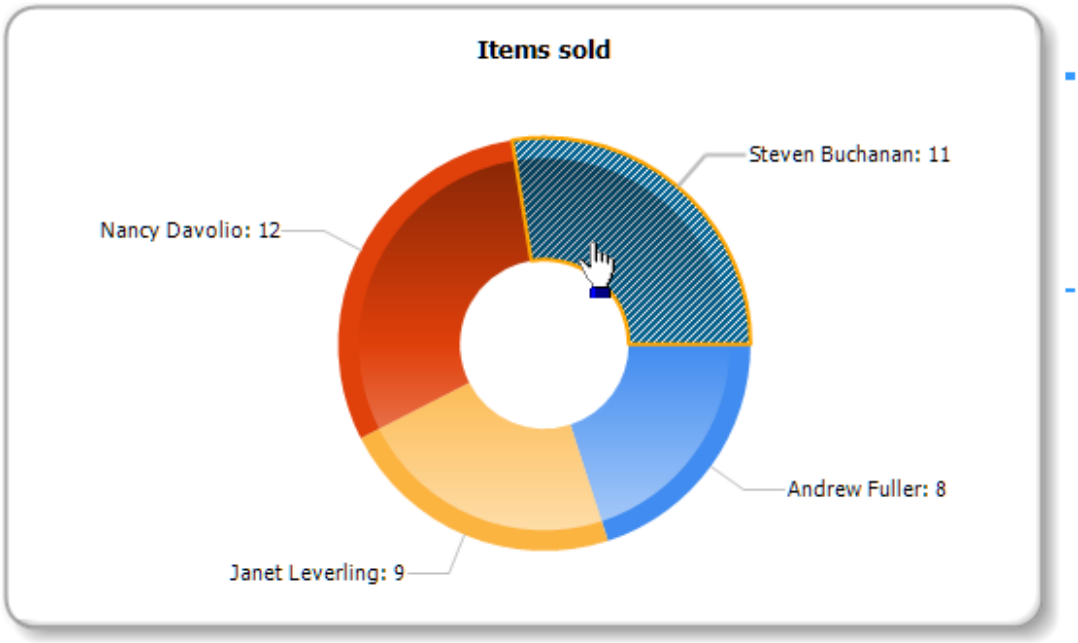
- 打开数据区域编辑器并标示以下过滤条件：

```
[MatrixDemo.Name] == [SelectedEmployee]
```

现在我们设置“Chart”对象的超链接：

- 在“Chart”对象的上下文菜单，选择“超链接…”；
- 选择超链接类型-“报表页”；
- 选择报表的第二页，并标示参数名称-SelectedEmployee。

报表完成，运行此报表，并将鼠标移动到任意报表值。这个值在视觉上被选中，并且鼠标光标更改了它的形状：



如果你点击该值，你将看到详细报表：

Steven Buchanan orders

Name	Year	Month	ItemsSold
Steven Buchanan	2001	1	3
Steven Buchanan	2001	2	4
Steven Buchanan	2000	1	4
		Total:	11

数据

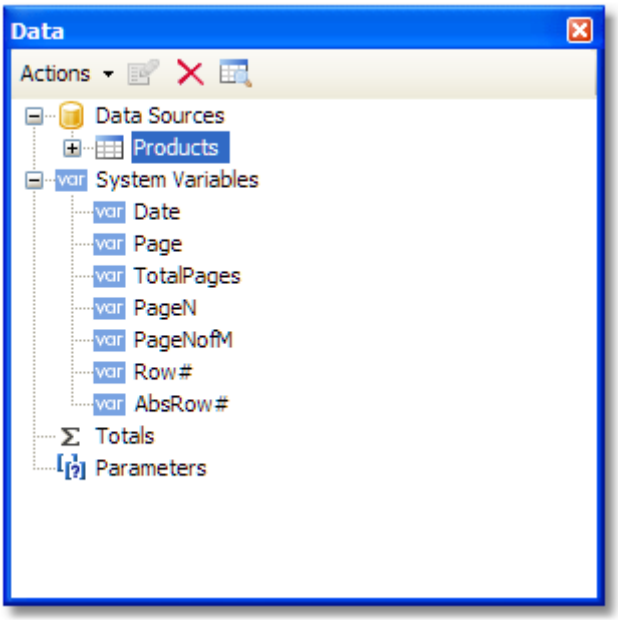
任何报表打印一些数据。在 Report 中，你可以操作以下数据：

- 数据源；
- 系统变量；
- 合计；
- 报表参数；
- 表达式，包含任意上面提到的数据；

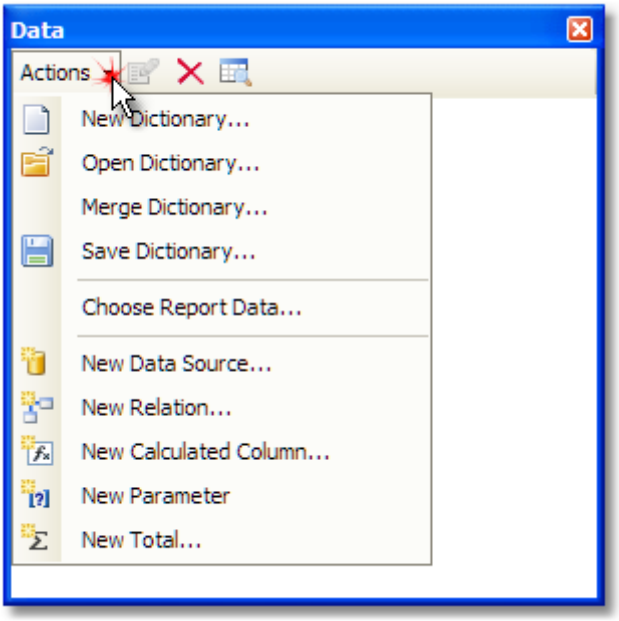
在这一章，我们将介绍如何操作这些数据。

“数据”窗口

所有的数据均来自“数据”窗口。这个窗口可以通过选择“数据|显示数据窗口”菜单显示。



“数据”窗口允许操作所有数据元素和将它们拖拽到报表页面。所有的操作都可在“Action”菜单中完成：



这些操作的一部分被复制在“数据”窗口的上下文菜单。例如，如果你选择了一个数据源，你可以使用它的上下文菜单创建一个计算列，删除数据源，或者查看其数据。

数据源

通常，数据源表示一个数据库或 SQL 查询。一个报表中可以有多个数据源。大多数报表，只需要一个数据源。像主-明细报表需要两个数据源，并且两个数据源彼此连接。

数据源包含一个或多个数据字段。每个字段有定义的数据类型。为了查看字段类型，可选择它打开“属性”窗口。字段类型在“DataType”属性中标示。这个图标在字段名称附近，也可以定义它的类型。

有两种方式可以为报表定义数据源。

第一种方法-数据源在报表的应用程序和注册时定义。这依赖于创建这个应用程序的程序员。用户应该只选择在报表中需要的数据源。它可以在“数据|选择报表数据…”菜单实现。



在一份报表中注册的所有数据都列在这个窗口中。只需勾选你报表中所需的数据。当你使用报表时，它可在任何时刻进行。

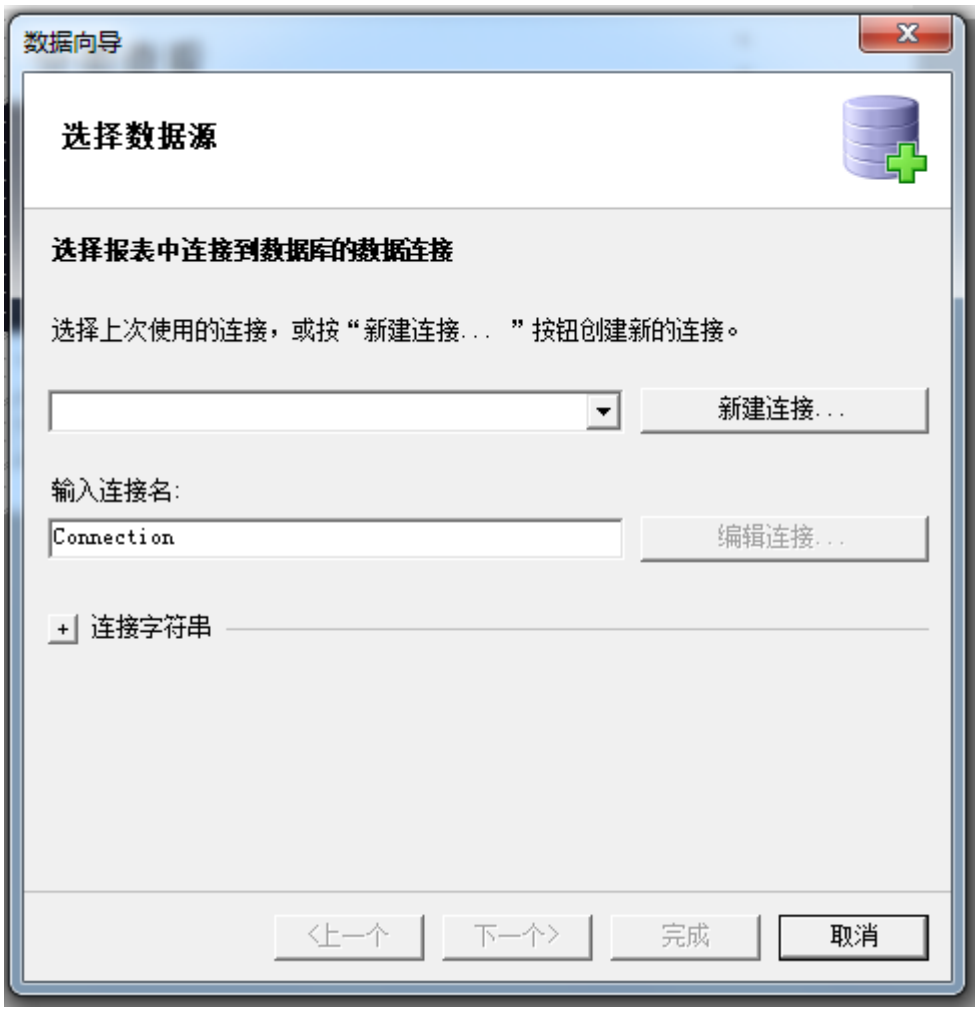
第二种方法-你自己创建一个数据源。它可以使用数据库表或者 SQL 查询。这种情况下，数据源定义被保存在报表文件中。

Report 允许连接到许多常用的 DBMS，比如 MS SQL,Oracle,Interbase,Acess。除此之外，你可以使用以 XML/XSD 保存格式的数据文件。

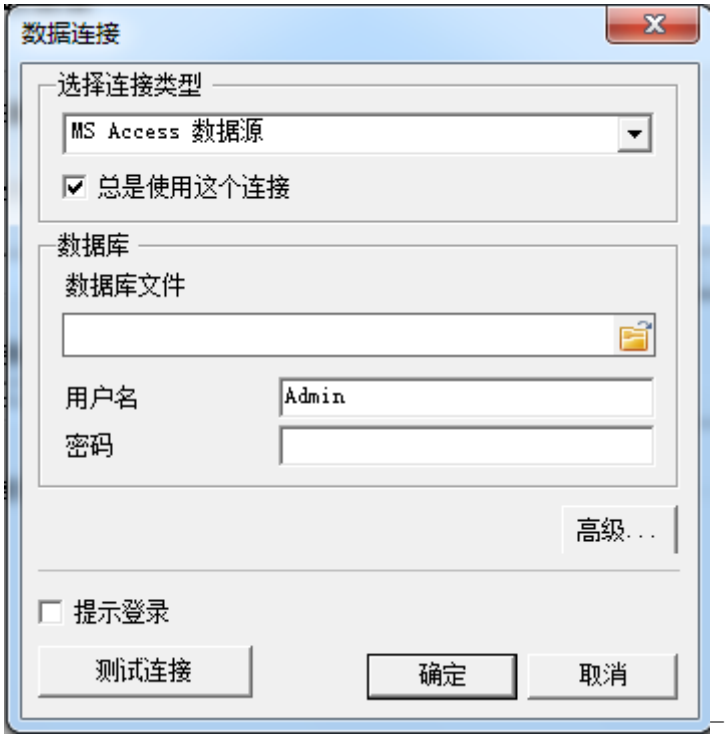
DB 表内容不会保存在报表文件中。连接字符串和数据源队列被存储。一个连接字符串可以包含这些数据如登录名和密码，这就是为什么它被加密保存在报表文件。当需要时，您可能会通过使用自己的密钥增加对数据的加密安全性。这种情况下，报表文件只能在你的程序中正常打开。

创建数据源

为了创建一个新的数据源，选择“数据|添加数据源…”菜单或者点击“数据”窗口的“动作”按钮并选择“新建数据源…”选项。你将看到“数据向导”窗口：



首先，你被要求创建连接。为此，点击“新建连接…”按钮。你将看到一个连接配置的窗口：



下列元素在图中显示：

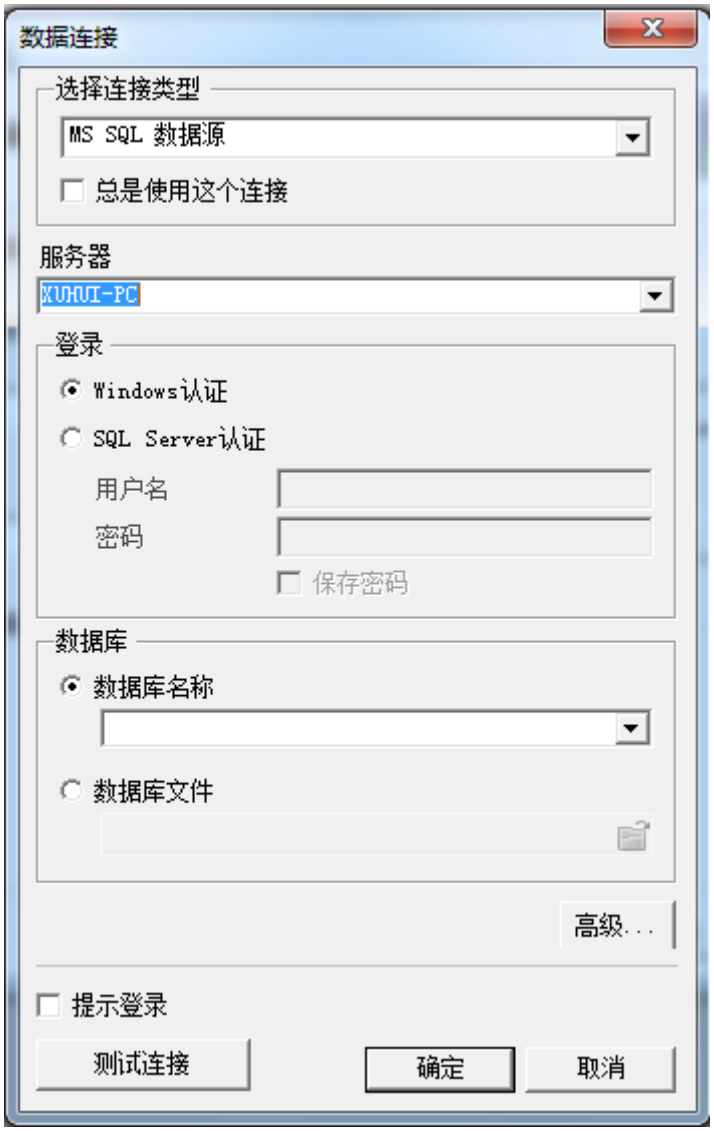
1、连接类型；

2、如果可用，选择的连接类型将以默认方式使用。

3、连接配置；

4、测试连接按钮

与 MS Access 数据库的连接显示在图片中。如果选择了另一个连接类型，连接配置区域将改变。例如，连接 MS SQL 数据库具有以下配置：



选择所需的连接类型并设置其参数。单击确定按钮后，窗口会被关闭，你将返回数据向导窗口。

下一步，你需要设置连接名称。这个名称将在“数据“窗口显示。



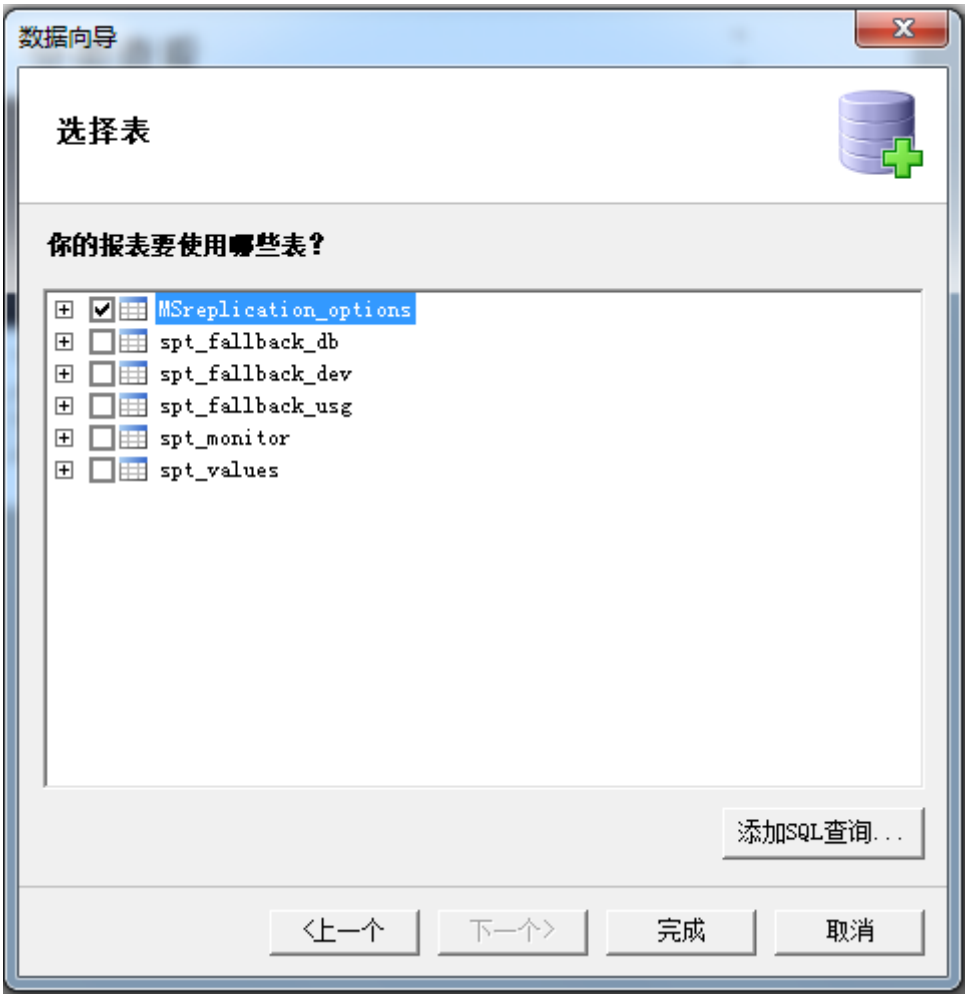
单击“下一步“按钮。这里你将被要求选择数据库中表：

勾选所需的表，然后点击“完成“按钮关闭该向导。现在，您可以在”数据“窗口中看到你创建的包含所选数据源的连接：



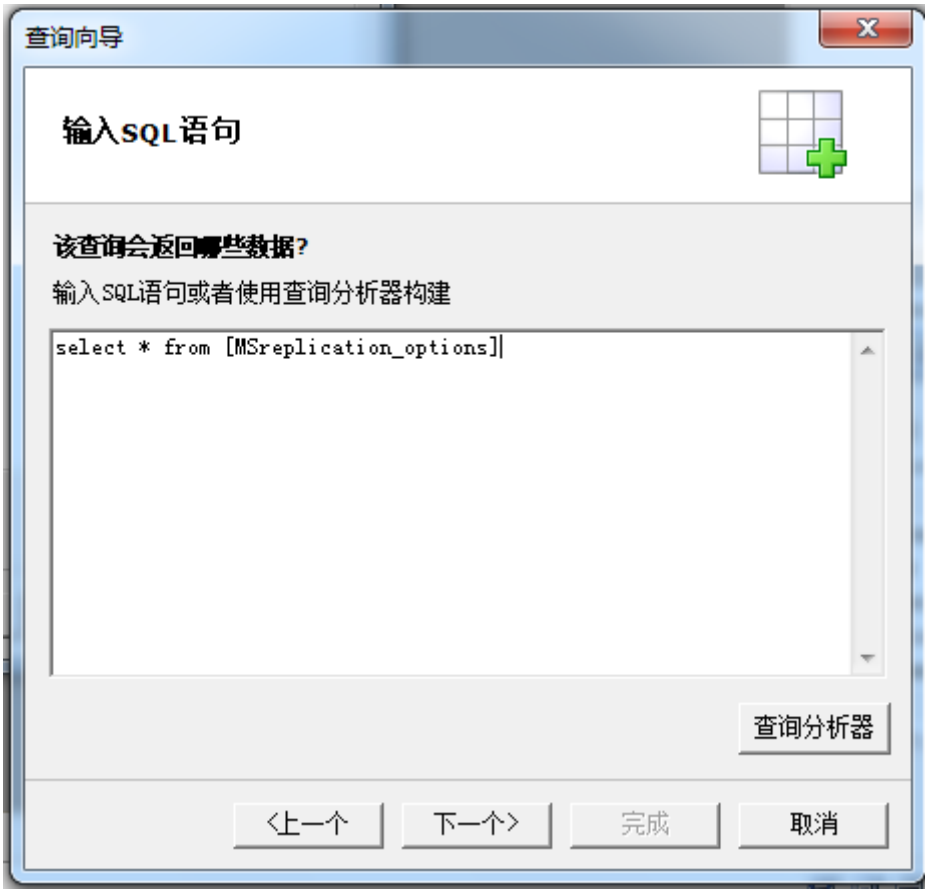
创建 SQL 查询

数据向导允许快速选择数据库中的表。创建 SQL 查询，需要额外的工作。为此，在向导的第二部点击“添加 SQL 查询…”按钮。



你会看到查询向导窗口。查询向导有四页。使用“下一个”和“上一个”切换页面。

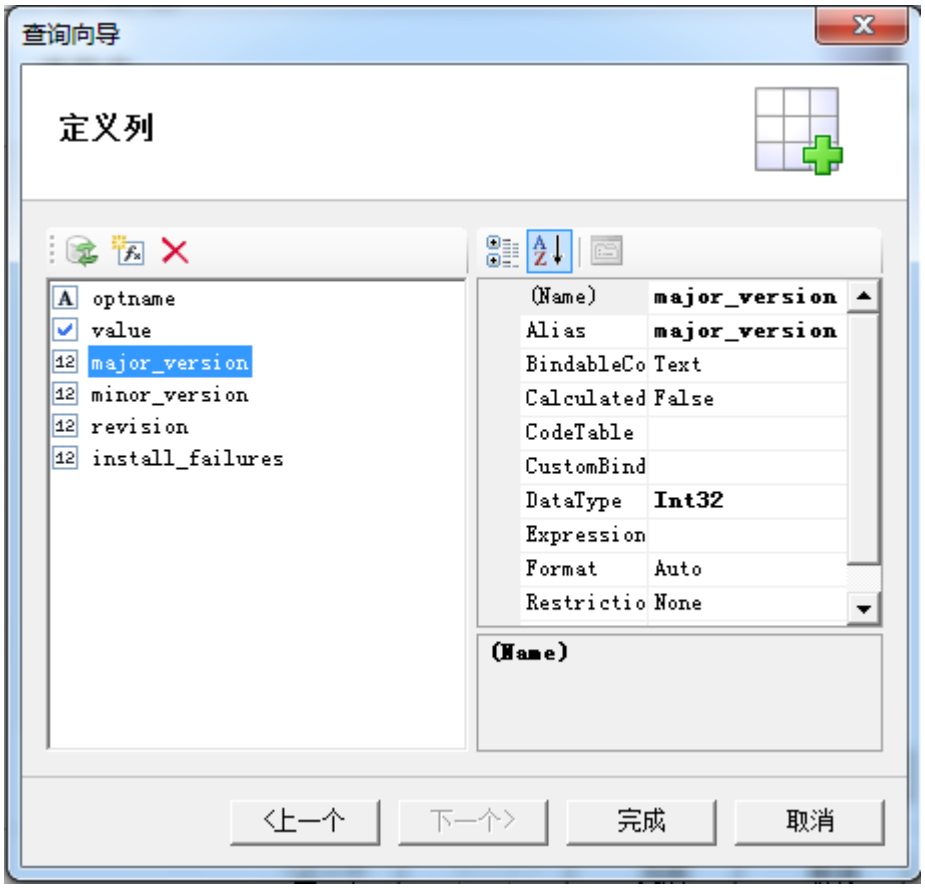
第一步，你需要为查询命名。这个名称将显示在“数据”窗口。输入唯一的名字然后点击“下一步”按钮。



在第二步中，你必须输入 SQL 查询语句。使用的是支持你的 MSDB 的语言。您可以使用查询生成器生成可视化查询。为此，单击“查询分析器”按钮。查询分析器将在后面详细描述。

当您输入查询语句后，点击“下一步”按钮。在第三步，可以定义查询参数。如果您的查询具有参数这是必需的。我们会在本章节的后面详细介绍。

向导的最后一步，您可以设置通过查询返回的字段：



如果您在查询语句或参数定义时犯了一个错误，你将会在向导的最后一页看到错误信息。

作为一项规则，它足以保证该查询返回所有需要的列。在这个步骤中，您可以执行以下操作：

- 使用“删除”按钮，删除不需要的列；
- 按“刷新”按钮重设列；
- 按“添加计算字段”按钮添加一个计算字段。对于一个新的字段，有必要设置“名称”，“数据类型”和“表达式”的属性。

按“完成”按钮，关闭向导后，您将返回到“数据向导”窗口。

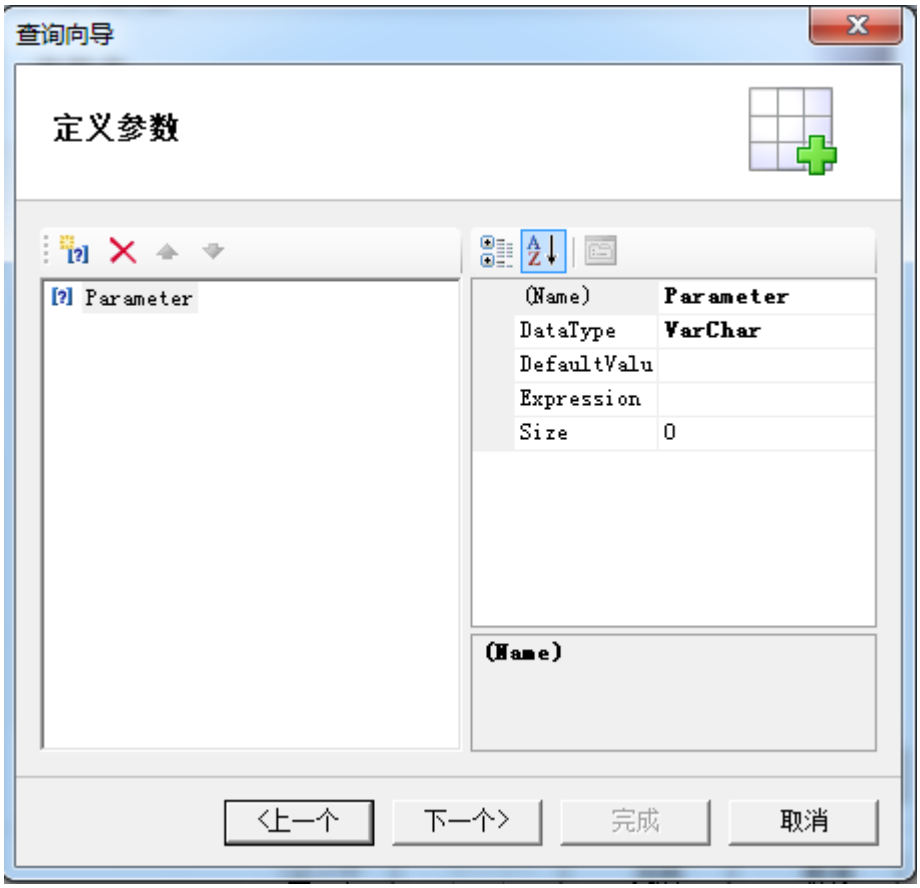
查询参数

在查询文本中可以存在参数，让我们看看下面的查询：

```
select * from DVDs
where Title = @param1
```

这是 MS SQL 演示数据库的查询语句。参数“param1”在查询中定义。这里需指出：对于不同的数据库管理系统，描述一个参数的方式也不同。对于 MS SQL 参数的特点是一个“@”的符号，MS ACCESS 参数没有名字，并于“？”符号标示。

如果您的 SQL 查询中包含参数，你必须声明它们。它可以在“查询向导”的第三个步骤完成。要创建一个参数，按下“添加参数”按钮。新参数将被创建：



下列参数的属性应在属性窗口中进行设置：

Property	Description
Name	参数名称。在这里，你需要标示与你在查询文本中使用的相同的名称。一些 DBMS（例如，MS 访问）不支持命名参数。在这种情况下，不要更改此属性。
DataType	参数数据类型
DefaultValue	该值在没有指定“表达式”属性，或者它无法进行计算（例如，在报表设计模式中执行查询操作时）时使用。
Expression	表达式返回参数的值。当您运行报表时处理这个表达式。您可以在该属性中标示任何表达式（参见“表达式”一章详细说明）。
Size	参数数据大小。如果该参数为“字符串”数据类型，则此属性应标明。

如果错误地设置参数属性，你会在转向向导的最后一页时得到一个错误提示。

将值传递给参数

参数通常被用来从用户请求一个值。让我们看看一个值传递给查询参数的两种方式。

第一种方式，传递一个值编程。由于没有简单的方法可以直接将值传递给查询参数，你需要使用报表参数，这个可以通过代码来轻松设置。你应该做到以下几点：

- 创建报表参数（我们将在本章的后面讨论报表参数）。设置相同数据类型的报表参数，因为它是在查询中使用的参数。
- 在查询参数的“表达式”属性中，关联一个报表参数，例如：

```
[MyReportParameter]
```

- 将值传递给报表参数：

```
report1.SetParameterValue("MyReportParameter", 10);
```

第二种方式中，您可以使用对话的形式来做到这一点（对话的形式将在后面讨论）。举例来说，如果你要问访问一个字符串类型的值，请执行以下操作：

- 添加一个对话框到您的报表;
- 在其上方放置一个“TextBoxControl”。这个控制将被用于输入字符串值;
- 设置参数，如下所示：

```
Name=param1
DataType=VarChar
DefaultValue= (empty string)
Expression=TextBox1.Text
Size=255
```

其中 TextBox1 的是包含由用户输入值的控件。

编辑连接

在“数据向导”的帮助下创建的数据连接，可以进行编辑。为了做到这一点，在“数据”窗口选择数据连接，然后点击工具栏上的“编辑”按钮。你会看到一个数据向导窗口。在此窗口中，您可以按“编辑连接...”按钮更改连接设置。连接的类型不能改变。

在向导的第二页，你可以选择报表中的表。当你完成后，单击“完成”按钮。

编辑数据源

在“数据向导”的帮助下创建的数据源，可以被编辑。为此，在“数据”窗口选择数据源，然后点击工具栏上的“编辑”按钮。你会看到一个“查询向导”窗口。在此窗口中，您可以更改 SQL 查询文本，设置查询参数和数据字段。

为了删除数据源，选择它，然后点击工具栏上的“删除”按钮。这种物理删除数据源并没有实现，它只是将其变成无法访问。您可以进入“数据|选择报告数据...”菜单，启用这样的数据源。然而，这绝不应该麻烦你，因为删除的数据源永远不会保存在报表文件中，因此，当下次查看报表时，它不会得到恢复。

别名

每一个数据元素（数据源和字段）都有它自己的名字。默认情况下，这是在数据库中定义的名称。在某些情况下，可能很难理解什么是隐藏在其背后的名称，例如，ProdID。

数据元素得到了第二个名字 - 别名。通过使用别名，你可以重命名一个元素。例如，如果我们有一个数据源 CATEGORY_TABLE，它具有一个名为“PROD_ID”的字段，你可以给以下别名：

```
CATEGORY_TABLE --> Categories
PROD_ID --> Product ID
```

你可以以下面的方式关联一个数据字段：

```
[Categories.Product ID]
```

当关联数据元素后，你必须使用别名，如果它已被定义。在这种情况下切勿使用其原来的名称来关联一个元素。

在这种情况下为了重新给数据元素命名，在“数据”窗口选择它，然后按 F2 键。此外，您还可以在该对象的上下文菜单中选择“重命名”选项。在此之后，输入所需的名称，然后按 Enter 键。

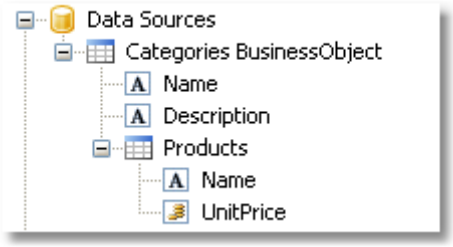
您也可以通过使用“属性”窗口重命名一个元素。选择“数据”窗口中的元素，切换到“属性”窗口，并更改“别名”属性的值。

为了删除一个别名（重置为原始名称），选择一个元素，并在其上下文菜单中选择“删除别名”选项卡。

数据源层次

我们看到的数据源，是相关联的，也就是说，他们来自一个关系数据库管理系统（通常称为 RDBMS）。 Report 中也支持其他类型的数据 - 分层数据源。这样的数据源来自所谓的业务对象，它经常被用来在应用程序中代表一个关系数据源的.NET 类。

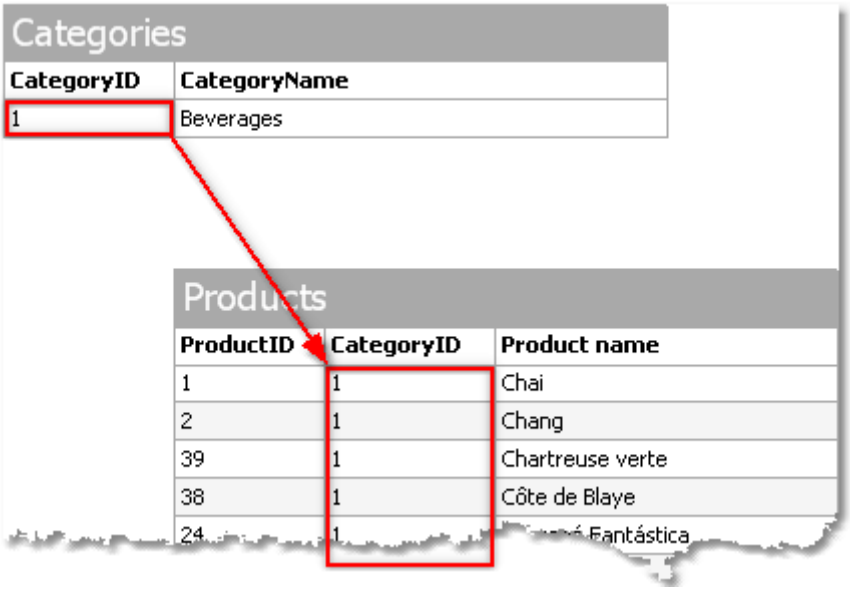
在报表中添加一个分层数据源的唯一方法是通过编程进行注册。它会在“程序员手册”进行讨论。现在，我们将着眼于序和分层数据源之间的一些差异。在下图中您可以看到两个数据源，“Categories BusinessObject”和“Products”。正如你看到的，“Products”数据源包含在其父“Categories BusinessObject”内：



这意味着，这两个数据源彼此相关的，并且可以在“主 - 从”报表类型使用。您也可以单独在一个“简单列表”报表类型使用这些数据源。

关系

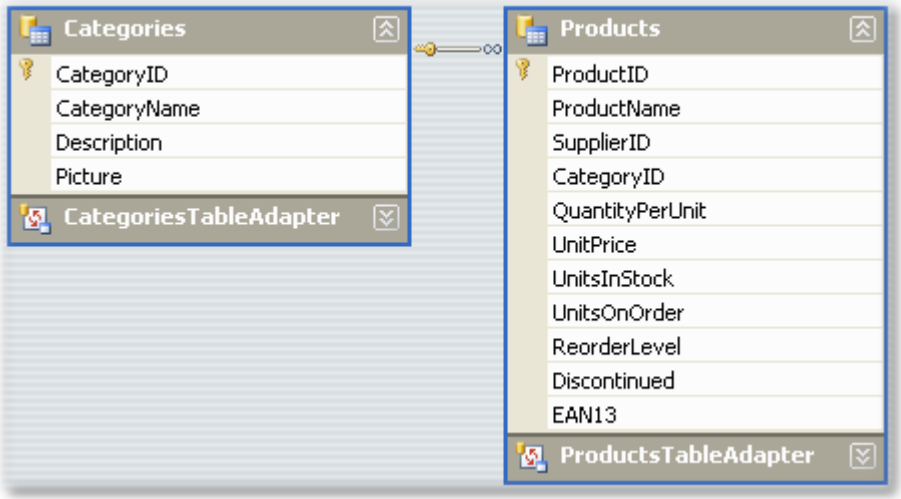
两个数据源之间的关系可任意设置。关系用来定义“主 - 从”的关系。例如，在“类别”表中的一个记录可以在“产品”表中有多条记录：



为了创建一个关系，你需要指出以下几点：

- 父表；
- 子表；
- 在主表中设置主键；
- 在子表中设置主键。

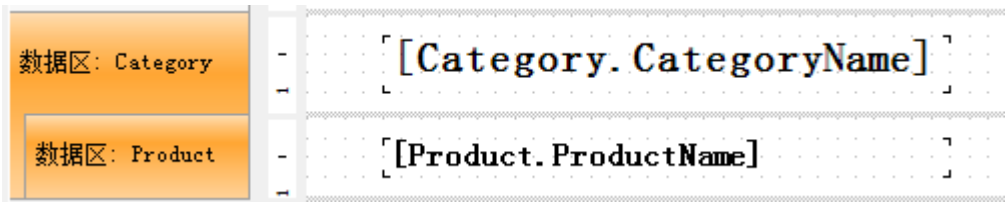
作为一个例子，我们将着眼于演示数据库中的“Categories”和“Products”表。它们具有以下结构：



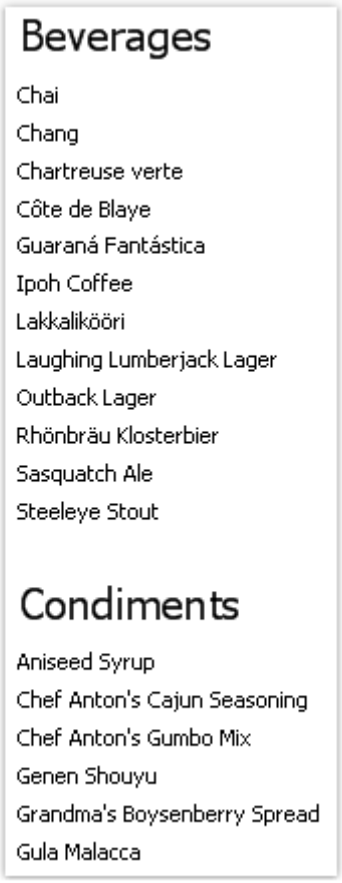
两个表都具有 CategoryID 字段，可以对其中的关系进行设置。因此，一个类别可以包含多个产品。

在 Report 中如何使用数据源？有两种方法：

第一种方法使人们可以建立“主 - 从”式报表。要做到这一点，需使用两个“数据”区域。主区域连接到主数据源，详细信息区域- 明细数据源。我们的例子是这样的：



这样的报表，如果你运行它，将打印每一个类别的产品列表：



第二种方法指的是主从明细数据源。我们将通过一个例子说明这一点。比如说，我们要打印所有产品的列表。为此，我们需要一个“数据”区域，连接到“产品”表：



这样的报表会打印所有类别的所有产品。如果，我们要在每个产品旁边，打印它所属的类别名称。如果不使用关系，这将更难。我们都知道有关产品的类别是它的 ID（在“Products”表中“CategoryID”列表示）。我们想打印的类别名称存储在“Categories”表的“CategoryName”字段。在关系的帮助下，我们可以以下列方式关联一个类别的名称：

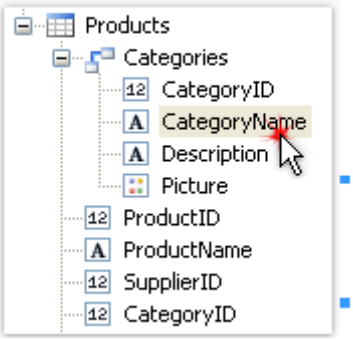
```
[Products.Categories.CategoryName]
```

对于“产品”表的当前行，Report 会在“Categories”表中找到相应的父行，并返回“类别名称”字段中的一个值。

在一般情况下，这种引用父表字段的方式允许无限次表的祖先：

```
[Child_table.Its_parent.Parent_of_a_parent.And_so_on.Column_name]
```

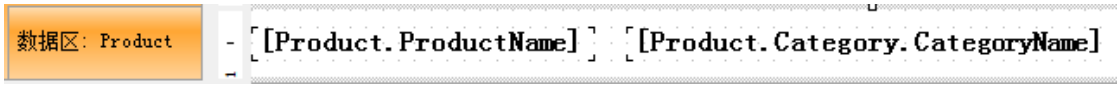
为了在报表中添加这样的数据字段，在“数据”窗口中打开“产品”表。你会看到在其字段中，有一个可以链接到“类别”表：



如果我们拖动在上面报表中显示的字段，那么我们将得到一个“Text”对象的文本：

```
[Products.Categories.CategoryName]
```

我们的报表将如下图所示：

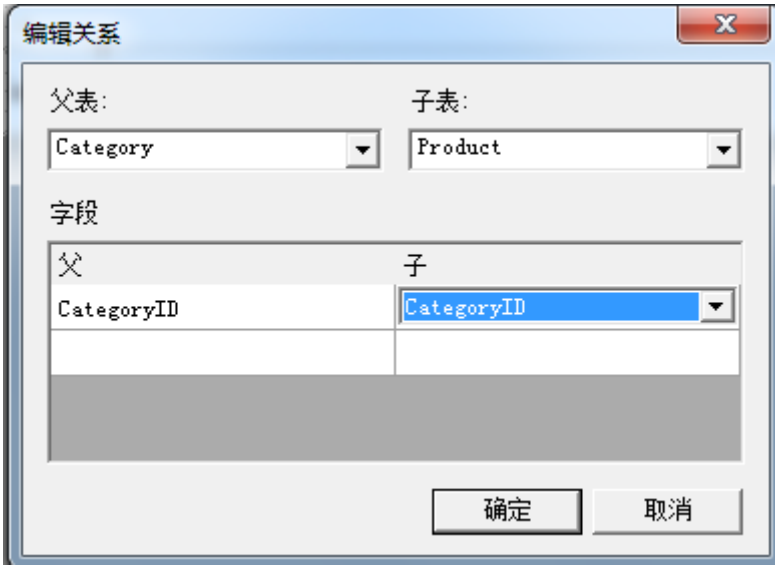


如果我们运行此报表，结果如下：

Alice Mutton	Meat/Poultry
Aniseed Syrup	Condiments
Boston Crab Meat	Seafood
Camembert Pierrot	Dairy Products
Carnarvon Tigers	Seafood
Chai	Beverages
Chang	Beverages

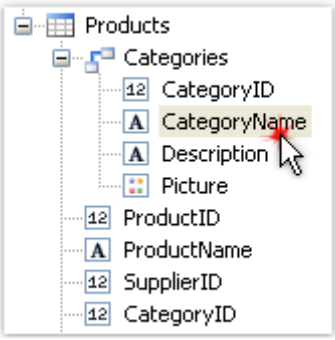
创建关系

为了创建一个关系，在“数据”窗口单击“动作”按钮，选择“新建关系...”项。您将看到关系编辑器：



首先，你需要选择父表和子表。在此之后，在窗口的下半部分，您需要选择相关的数据字段。该表可以用关联一个或多个数据字段。各字段设置完成后，按下确定按钮关闭关系编辑器。

你已经创建的关系可以在“数据”窗口看到，如果你选择子数据源，并打开其字段的列表。在其字段中，你会看到与父数据源的关系：



父源的数据字段可以使用拖放方法插入到报表。所以，如果我们选择如图所示的字段，并将其拖到报表页面，我们将得到一个包含以下内容的“文本”对象：

```
[Products.Categories.CategoryName]
```

编辑关系

为了编辑关系，打开子数据源的字段列表，找到需要的关系，然后单击位于工具栏上的“编辑...”按钮。这将调出我们前面介绍过的关系编辑器。

系统变量

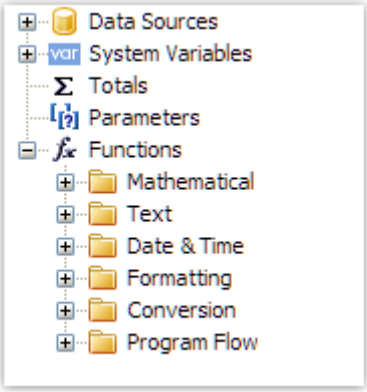
下面是可以在一个报表中使用的系统变量的列表：

Variable	Description
Date	报表的开始日期和时间
Page	当前页码
TotalPages	该报表的总页数。要使用此变量，您需要启用该报告的 double pass。你可以通过“Report|Properties...”菜单完

	成此操作。
PageN	页码形式：Page N
PageNofM	页码形式："Page N of M".
Row#	组内的数据行数。这个值在一个新组的开始被重新设置。
AbsRow#	数据行的绝对数量。该值不会在一个新组的开始被重置。
Page#	当前页码。如果你将几个报表编制成一个包，这个变量将返回在这个包中的当前页码。 这个变量实际上是一个宏。组件在预览窗口中查看其值将被替换。这意味着你不能在表达式中使用它。
TotalPages#	该报表的总页数。如果你将几个报表编制成一个包，这个变量将返回一个封装的页数。你并不需要使用双通，以获得正确的值。
HierarchyLevel	分层报表中的当前层次水平。顶层值为 1
HierarchyRow#	分组报表的全行号如“1.2.1”

函数

Report.Net 含有大量的内置函数（60 以上）。所有功能都被分裂为几个类别，可以通过“数据”窗口访问：



你可以在任何表达式中使用函数，在脚本（请参阅“脚本”一章）或在“文本”对象中打印其值。例如，在“文本”对象中的以下文字：

```
[Sqrt(4)]
```

将被打印成 2（4 的平方根）；

下面的表达式将返回 4：

```
Sqrt(4) + 2
```

让我们来看看如何在报表中插入函数：

- 你可以从“数据”窗口拖放一个功能到报表页面。 然后包含一个函数调用的“文本”对象将被创建。你必须编辑文本以添加参数给函数调用；
- 你可以拖放一个函数到脚本代码；

● 在表达式编辑器，你可以看到作用方式相同的“数据”窗口的副本- 你可以从中拖动项目放在表达式文本。

下面我们将详细介绍每个功能。

数学

绝对值

Function	Parameters	Return value
Abs	sbyte value	sbyte
Abs	short value	short
Abs	int value	int
Abs	long value	long
Abs	float value	float
Abs	double value	double
Abs	decimal value	decimal

返回绝对值。

例子：

```
Abs(-2.2) = 2.2
```

反余弦

Function	Parameters	Return value
Acos	double d	double

返回余弦为 d 的角度（弧度）。d 必须在范围-1 到 1 之间。

180/ Math.PI 乘返回值将弧度转换为度。

例子：

```
Acos(0) * 180 / Math.PI = 90
```

反正弦

Function	Parameters	Return value
Asin	double d	double

返回正弦为 d 的角度（弧度）。d 必须在-1 到 1 之间。

180/ Math.PI 乘返回值转换弧度为度。

例子：

```
Asin(0) = 0
```

反正切

Function	Parameters	Return value
Atan	double d	double

返回正切为 d 的角度（弧度）。

180/ Math.PI 乘返回值转换弧度为度。

例子：

```
Atan(1) * 180 / Math.PI = 45
```

Ceiling

Function	Parameters	Return value
Ceiling	double d	double
Ceiling	decimal d	decimal

返回大于或等于 d 的最小整数。

例子：

```
Ceiling(1.7) = 2
```

余弦

Function	Parameters	Return value
Cos	double d	double

返回指定角（D）的余弦值。角度必须为弧度。

使用 Math.PI/180 乘以度数转换为弧度。

例子：

```
Cos(90 * Math.PI / 180) = 0
```

Exp

Function	Parameters	Return value
Exp	double d	double

返回 e（2.71828），募集到指定的功率 D。

例子：

```
Exp(1) = 2.71828
```

Floor

Function	Parameters	Return value
Floor	double d	double
Floor	decimal d	decimal

返回小于或等于 d 的最大整数。

例子：

```
Floor(1.7) = 1
```

对数

Function	Parameters	Return value
Log	double d	double

返回指定数字为 d 的对数。

例子：

```
Log(2.71828) = 1
```

最大值

Function	Parameters	Return value
Maximum	int val1, int val2	int
Maximum	long val1, long val2	long
Maximum	float val1, float val2	float
Maximum	double val1, double val2	double
Maximum	decimal val1, decimal val2	decimal

返回 val1，val2 中的最大值。

例子：

Maximum(1,2) = 2

最小值

Function	Parameters	Return value
Minimum	int val1, int val2	int
Minimum	long val1, long val2	long
Minimum	float val1, float val2	float
Minimum	double val1, double val2	double
Minimum	decimal val1, decimal val2	decimal

返回 val1，val2 中的较小值。

例子：

Minimum(1,2) = 1

Round

Function	Parameters	Return value
Round	double d	double
Round	decimal d	decimal

d 最接近的整数。

例子：

Round(1.47) = 1

Function	Parameters	Return value

Round	double d, int digits	double
Round	decimal d, int digits	decimal

D 在“数字”参数中指定精度的值。
例子：
Round(1.478, 2) = 1.48

正弦

Function	Parameters	Return value
Sin	double d	double

返回指定角（D）的正弦值。角度必须为弧度。
使用 Math.PI/180 乘以度数转换为弧度。
例子：
Sin(90 * Math.PI / 180) = 1

平方根

Function	Parameters	Return value
Sqrt	double d	double

返回 D 的平方根。
例子：Sqrt(4) = 2

正切

Function	Parameters	Return value
Tan	double d	double

返回指定角（D）的正切值。角度必须为弧度。
使用 Math.PI/180 乘以度转换为弧度。
例子：
Tan(45 * Math.PI / 180) = 1

Truncate

Function	Parameters	Return value
Truncate	double d	double
Truncate	decimal d	decimal

计算 D 的整数部分。
例子：
Truncate(1.7) = 1

文本

注意：

● 这些函数不修改传入的字符串值。相反，他们返回一个新的修改后的字符串；

● 在一个字符串中第一个字符的索引为 0。在使用函数获取字符索引时需牢记，如插入函数。

Asc

Function	Parameters	Return value
Asc	char c	int

返回一个 Integer 值，表示对应的字符的字符代码。

例子：

Asc('A') = 65

Chr

Function	Parameters	Return value
Chr	int i	char

返回与指定的字符代码相关联的字符。

例子：

Chr(65) = 'A'

Insert

Function	Parameters	Return value
Insert	string s, int startIndex, string value	string

在“s”字符串中的指定索引位置插入一个“值”，并返回一个新的字符串。

例子：

Insert("ABC", 1, "12") = "A12BC"

Length

Function	Parameters	Return value
Length	string s	int

返回“s”的长度。

例子：

Length("ABC") = 3

LowerCase

Function	Parameters	Return value
LowerCase	string s	string

将“s”中的所有字符转换成小写并返回值。

例子：

LowerCase("ABC") = "abc"

PadLeft

Function	Parameters	Return value
PadLeft	string s, int totalWidth	string

将“s”字符串中的字符右对齐，在左边用空格填充总宽度在“totalwidth”中指定的参数。

例子：

PadLeft("ABC", 5) = " ABC"

Function	Parameters	Return value
PadLeft	string s, int totalWidth, char paddingChar	string

右对齐的“S”字符的字符串，用 “paddingChar”字在左边填充“totalWidth”参数指定的总宽度。

例子：

PadLeft("ABC", 5, '0') = "00ABC"

PadRight

Function	Parameters	Return value
PadRight	string s, int totalWidth	string

左对齐的“S”字符的字符串，在右边用空格填充在“totalWidth”参数指定的总宽度。

例子：

PadRight("ABC", 5) = "ABC "

Function	Parameters	Return value
PadRight	string s, int totalWidth, char paddingChar	string

左对齐“S”字符的字符串,用 “paddingChar”字在右边填充“totalWidth”参数指定的总宽度。

例子：

PadRight("ABC", 5, '0') = "ABC00"

Remove

Function	Parameters	Return value
Remove	string s, int startIndex	string

删除“s”字符串中以“startindex”位置开始直至最后位置的所有字符。

例如：

Remove("ABCD", 3) = "ABC"

Function	Parameters	Return value
Remove	string s, int startIndex, int count	string

删除“s”字符串中的字符从“startindex”位置开始到在”count“参数中指定的参数为止。

例子：

Remove("A00BC", 1, 2) = "ABC"

Replace

Function	Parameters	Return value
Replace	string s, string oldValue, string newValue	string

返回一个字符串“s”，其中的子字符串已经被另一个新子字符串代替。

例子：

Replace("A00", "00", "BC") = "ABC"

Substring

Function	Parameters	Return value
Substring	string s, int startIndex	string

检索“s”字符串的子字符。该子字符串从“startindex”参数指定的字符位置开始。

例子：

Substring("ABCDEF", 4) = "EF"

Function	Parameters	Return value
Substring	string s, int startIndex, int length	string

检索“s”字符串的子字符。该子字符串从“startindex”参数指定的字符位置开始，并具有在“length”参数中指定的长度。

例子：

Substring("ABCDEF", 1, 3) = "BCD"

TitleCase

Function	Parameters	Return value
TitleCase	string s	string

将指定的字符串转换为标题字符。
例子：
TitleCase("john smith") = "John Smith"

Trim

Function	Parameters	Return value
Trim	string s	string

删除“S”字符串中的所有空白字符。
例子：
Trim(" ABC ") = "ABC"

UpperCase

Function	Parameters	Return value
UpperCase	string s	string

转换 “S”的所有字符为大写并返回结果。
例子：
UpperCase("abc") = "ABC"

日期 & 时间

AddDays

Function	Parameters	Return value
AddDays	DateTime date, double value	DateTime

添加指定的天数（“值”）到“日”的日期，并返回一个新的日期。
例子：
AddDays(#7/29/2009#, 1) = #7/30/2009#

AddHours

Function	Parameters	Return value
AddHours	DateTime date, double value	DateTime

添加指定的小时数（“值”）到“日”的日期，并返回一个新的日期。
例子：
AddHours(#7/29/2009 1:30#, 1) = #7/29/2009 2:30#

AddMinutes

Function	Parameters	Return value
AddMinutes	DateTime date,	DateTime

	double value	

添加指定的分钟数（“值”）到“日”的日期，并返回一个新的日期。

例子：

AddMinutes(#7/29/2009 1:30#, 1) = #7/29/2009 1:31#

AddMonths

Function	Parameters	Return value
AddMonths	DateTime date, int value	DateTime

添加月的指定数目（“值”）到“日”的日期，并返回一个新的日期。

例子：

AddMonths(#7/29/2009#, 1) = #8/29/2009#

AddSeconds

Function	Parameters	Return value
AddSeconds	DateTime date, double value	DateTime

增加指定的秒数（“值”）到“日”的日期，并返回一个新的日期。

例子：

AddSeconds(#7/29/2009 1:30:01#, 1) = #7/29/2009 1:30:02#

AddYears

Function	Parameters	Return value
AddYears	DateTime date, int value	DateTime

增加了年的指定数目（“值”）到“日”的日期，并返回一个新的日期。

例子：

AddYears(#7/29/2009#, 1) = #7/29/2010#

DateDiff

Function	Parameters	Return value
DateDiff	DateTime date1, DateTime date2	TimeSpan

返回两个日期之间的时间间隔（天，小时，分钟，秒钟）。

例子：

DateDiff(#1/2/2009#, #1/1/2009#) = 1.00:00:00

DateSerial

Function	Parameters	Return value
DateSerial	int year, int month, int day	DateTime

从指定的年，月，日创建一个新的 DateTime 值。

例子：

DateSerial(2009, 7, 29) = #7/29/2009#

Day

Function	Parameters	Return value
Day	DateTime date	int

由指定的日期获取当月天数。

例子：

Day(#7/29/2009#) = 29

DayOfWeek

Function	Parameters	Return value
DayOfWeek	DateTime date	string

获取指定日期表示的星期几的本地化名称。

例子：

DayOfWeek(#7/29/2009#) = "wednesday"

DayOfYear

Function	Parameters	Return value
DayOfYear	DateTime date	int

由指定的日期获取当前日期在一年中的天数（1-365）。

例子：

获取一年中的天数（1-365）

DaysInMonth

Function	Parameters	Return value
DaysInMonth	int year, int month	int

返回指定的月份和年份的天数。

例子：

DaysInMonth(2009, 7) = 31

Hour

Function	Parameters	Return value
Hour	DateTime date	int

在指定日期获取小时部分（0-23）。
例子：
Hour(#7/29/2009 1:30#) = 1

Minute

Function	Parameters	Return value
Minute	DateTime date	int

获取指定日期的分钟部分（0-59）。
例子：
Minute(#7/29/2009 1:30#) = 30

Month

Function	Parameters	Return value
Month	DateTime date	int

获取指定日期的月份部分。
例子：
Month(#7/29/2009#) = 7

MonthName

Function	Parameters	Return value
MonthName	int month	string

获取指定月份的当地名称。
例子：
MonthName(1) = "January"

Second

Function	Parameters	Return value
Second	DateTime date	int

获取指定日期的秒部分（0-59）。
例子：
Second(#7/29/2009 1:30:05#) = 5

Year

Function	Parameters	Return value
Year	DateTime date	int

获取指定日期的年份。
例子：
Year(#7/29/2009#) = 2009

格式

Format

Function	Parameters	Return value
Format	string format, params object[] args	string

以指定的“格式“字符代替指定的“args“数组中格式项目的值。

例如，下列函数的调用：

Format("Name = {0}, hours = {1:hh}", myName, DateTime.Now)

包含以下格式："{0}" and "{1:hh}"。他们将被 myname 和 DataTime.Now 的值所代替。结果可能如下：

Name = Alex, hours = 12

每个格式项采取以下格式：

{index[,alignment][:formatString]}

- 索引- 一个从零开始的整数，指示对象格式列表的元素；
- 调整 - 可选的整数，指示区域的最小宽度，包含格式化值。如果格式化值的长度小于 alignment，那么该区域用空格填充。如果 alignment 为负，则格式化的值是左对齐的区域;如果 alignment 为正，则格式化的值是右对齐；
- 格式字符串-一个格式说明符的可选字符串。

下表描述了标准数字格式字符串。

Format Specifier	Name	Description
C or c	Currency	数被转换为表示货币金额的字符串。Format("{0:C}", 10) = "$10.00"
D or d	Decimal	这种格式只支持整数类型。数字转换为字符串的十进制数字（0-9）。Format("{0:D}", 10) = "10"
E or e	Scientific	数被转换为如下形式的字符串"-d.ddd…E+ddd" or "-d.ddd…e+ddd", where each 'd' indicates a digit (0-9). Format("{0:E}", 10) = "1,000000E+001"
F or f	Fixed-point	该数字转换为以下形式的字符串“-ddd.ddd...”，其中每个“d”表示一个数字（0-9）。Format("{0:F}", 10) = "10.00"
G or g	General	数被转换为最紧凑的表示法。 Format("{0:G}", 10) = "10"
N or n	Number	数转换为字符串的形式为“-D，DDD，ddd.ddd...”，其中每个“d”表示一个数字（0-9）。Format("{0:N}", 1234.56) = "1,234.56"
P or p	Percent	数被转换为一个表示百分比的字符串。转换后的数字是为了呈现为百分比乘以 100。Format("{0:P}", 0.15) = "15.00%"

X or x	Hexadecimal	该数字转换为十六进制数字的字符串。格式说明符的大小写指示是否使用大写或小写字符的十六进制数字大于 9。例如，使用“X”产生“ABCDEF”和“X”产生“abcdef”。Format("{0:X}", 26) = "1A"

如果格式化浮点值，则可能表示小数位放置在格式字符串之后：

Format("{0:C1}", 12.23) = "$12.2"

如果标准数字格式说明符不提供您所需要的格式类型，可以使用自定义格式字符串：

Format character	Description
0	零占位符
#	数字占位符
.	小数点
,	千位分隔符
%	百分比占位符
;	部分分隔符

使用范例：

Format("{0:$#,##0.00}", 1024.25) = "$1,024.25"

Format("{0:00%}", 0.25) = "25%"

Format("{0:$#,##0.00;($#,##0.00);Zero}", 1024.25) = "$1,024.25"

Format("{0:$#,##0.00;($#,##0.00);Zero}", -1024.25) = "($1,024.25)"

Format("{0:$#,##0.00;($#,##0.00);Zero}", 0) = "Zero"

下表描述了标准格式说明符格式化 DateTime 值：

Format Specifier	Name	Example
d	Short date pattern	"8/9/2009"
D	Long date pattern	"Sunday, August 09, 2009"
f	Full date/time pattern (short time)	"Sunday, August 09, 2009 2:44 PM"
F	Full date/time pattern (long time)	"Sunday, August 09, 2009 2:44:01 PM"
g	General date/time pattern (short time)	"8/9/2009 2:44 PM"
G	General date/time pattern (long time)	"8/9/2009 2:44:01 PM"
t	Short time pattern	"2:44 PM"
T	Long time pattern	"2:44:01 PM"

下表描述了自定义日期/时间格式说明符和它们所产生的结果。

Format Specifier	Name	Example
d	Short date pattern	"8/9/2009"
D	Long date pattern	"Sunday, August 09, 2009"
f	Full date/time pattern (short time)	"Sunday, August 09, 2009 2:44 PM"
F	Full date/time pattern (long time)	"Sunday, August 09, 2009 2:44:01 PM"
g	General date/time pattern (short time)	"8/9/2009 2:44 PM"
G	General date/time pattern (long time)	"8/9/2009 2:44:01 PM"
t	Short time pattern	"2:44 PM"
T	Long time pattern	"2:44:01 PM"

The following table describes the custom date/time format specifiers and the results they produce.

Format Specifier	Description
d	显示月份的当前日，范围介于 1 和 31 之间（含）。如果该天只有一个单一的数字（1-9），则它显示为单个数字。
dd	显示月份的当前日，范围介于 1 和 31 之间（含）。如果该天只有一个数字（1-9），则将其格式化为前面带 0（01-09）。
ddd	显示当天的缩写名称。
dddd	显示当天的全名。
f or F	显示秒部分的最显著位数。
h	显示范围 1-12 的小时数。如果该小时是单个数字（1-9），则它显示为一个单一的数字。
hh	显示范围 1-12 的小时。如果该小时是单个数字（1-9），则将其格式化为前面带 0（01-09）。
H	显示范围 0-23 的小时数。如果该小时是单个数字（1-9），则它显示为一个单一的数字。
HH	显示范围 0-23 的小时。如果该小时是单个数字（1-9），则将其格式化为前面带 0（01-09）。
m	显示范围 0-59 的分钟数。如果分钟是一位数字（0-9），则它显示为一个单一的数字。

mm	显示范围 0-59 的分钟数。如果分钟是一位数字（0-9），则将其格式化为前面带 0（01-09）。
M	显示数字 1 至 12(含)之间的月份。如果月份是一位数字(1-9)，则它显示为一个单一的数字。
MM	显示月份，数字 1 至 12(含)之间。如果月份是一位数字(1-9)，则将其格式化为前面加 0（01-09）。
MMM	显示月份的缩写名称。
MMMM	显示月份的全名。
s	显示范围 0-59 的秒数。如果第二个是单个数字（0-9），则它显示为只有一个单一的数字。
ss	显示范围 0-59 的秒数。如果第二个是单个数字（0-9），则将其格式化为前面加 0（01-09）。
t	显示 A.M/ P.M 的第一个字符。
tt	显示 A.M./P.M. .
y	显示全年的最高两位数字。在今年的前两个数字被省略。如果年份是一位数字（1-9），则它显示为一个单一的数字。
yy	显示全年的最高两位数字。在今年的前两个数字被省略。如果年份是一位数字（1-9），则将其格式化为带有前导 0（01-09）。
yyyy	显示年份，包括世纪。如果年份少于 4 位数字，那么前面的零被追加为必要使显示的年份四位数字。
z	显示系统当前时区偏移的整小时数。偏移量始终显示为带有前导符号（零显示为“0”），表示在格林威治时间平均时间（+）或迟于格林威治时间平均时间（ - ）。该值的范围是-12 到 13。如果偏移量为单个数字（0-9），则它显示为带有适当的前导符号的一位数。
zz	显示系统当前时区偏移的整小时数。在始终显示为带有前导或尾随符号（零显示为“+00”），指示早于格林威治时间（+）或迟于格林威治时间平均时间偏差（ - ）。该值的范围是-12 到 13。如果偏移量为单个数字（0-9），则将其格式化为带有前导 0（01-09）并带有适当的前导符号。
zzz	显示系统当前时区偏移的小时和分钟。在始终显示为带有前导或尾随符号（零显示为“+00:00”），指示早于格林威治时间（+）或迟于格林威治时间平均时间偏差（ - ）。值的范围为-12:00 到 +13:00。如果偏移量为单个数字（0-9），则将其格式化为带有前导 0（01-09）并带有适当的前导符号。
:	时间分隔符
/	日期分隔符

使用范例：

```
Format("{0:d MMM yyyy}", DateTime.Now) = "9 Aug 2009"
Format("{0:MM/dd/yyyy}", DateTime.Now) = "08/09/2009"
Format("{0:MMMM, d}", DateTime.Now) = "August, 9"
Format("{0:HH:mm}", DateTime.Now) = "16:07"
```

Format("{0:MM/dd/yyyy hh:mm tt}", DateTime.Now) = "08/09/2009 04:07 PM"

FormatCurrency

Function	Parameters	Return value
FormatCurrency	object value	string

使用设置报表栏窗口格式化指定的值作为货币。

例子：

FormatCurrency(1.25) = "$1.25"

Function	Parameters	Return value
FormatCurrency	object value, int decimalDigits	string

格式化指定的值作为货币。在“十进制数”参数标示小数点右边有多少值。

例子：

FormatCurrency(1.25, 1) = "$1.3"

FormatDateTime

Function	Parameters	Return value
FormatDateTime	DateTime value	string

格式指定的值作为日期/时间，使用 Windows 区域设置。此功能不包括在结果字符串中的中性值。

例子：

FormatDateTime(#1/1/2009#) = "01/01/2009"

FormatDateTime(#1/1/2009 1:30#) = "01/01/2009 1:30:00 AM"

FormatDateTime(#1:30#) = "1:30:00 AM"

Function	Parameters	Return value
FormatDateTime	DateTime value, string format	string

格式指定的值作为日期/时间，使用“格式”参数所指定的命名格式。此参数的有效值为：

"Long Date"

"Short Date"

"Long Time"

"Short Time"

例子：

FormatDateTime(#1/1/2009 1:30#, "Long Date") = "Thursday, January 01, 2009"

FormatDateTime(#1/1/2009#, "Short Date") = "01/01/2009"

FormatDateTime(#1:30#, "Short Time") = "01:30 AM"

FormatDateTime(#1:30#, "Long Time") = "1:30:00 AM"

FormatNumber

Function	Parameters	Return value
FormatNumber	object value	string

格式化指定的值作为数，使用 Windows 的区域设置。

例子：

FormatNumber(1234.56) = "1,234.56"

Function	Parameters	Return value
FormatNumber	object value, int decimalDigits	string

格式化指定的值作为一个数字。在“十进制数”参数标示取到小数点右边多少位。

例子：

FormatNumber(1234.56, 1) = "1,234.6"

FormatPercent

Function	Parameters	Return value
FormatPercent	object value	string

使用 Windows 设置报表栏，将指定值格式化为百分比。

例子：

FormatPercent(0.15) = "15.00%"

Function	Parameters	Return value
FormatPercent	object value, int decimalDigits	string

格式化指定的值为百分数。在“十进制数”参数标示取到小数点右边多少位。

例子：

FormatPercent(0.15, 0) = "15%"

转换

ToBoolean

Function	Parameters	Return value
ToBoolean	object value	bool

将指定的值转换为布尔类型。

例子：

ToBoolean(1) = true

ToBoolean(0) = false

ToByte

Function	Parameters	Return value
ToByte	object value	byte

将指定的值转换为字节。

例子：

ToByte("55") = 55

ToChar

Function	Parameters	Return value
ToChar	object value	char

将指定值转换为 char。

例子：

ToChar(65) = 'A'

ToDateTime

Function	Parameters	Return value
ToDateTime	object value	DateTime

将指定值转换为日期类型。

例子：

ToDateTime("1/1/2009") = #1/1/2009#

ToDecimal

Function	Parameters	Return value
ToDecimal	object value	decimal

将指定值转换成十进制数。

例子：

ToDecimal(1) = 1m

ToDecimal("1") = 1m

ToDouble

Function	Parameters	Return value
ToDouble	object value	double

将指定的值转换为双精度型。

例子：

ToDouble(1) = 1

ToDouble("1") = 1

ToInt32

Function	Parameters	Return value

ToInt32	object value	int

将指定值转换成整型。

例子：

ToInt32(1f) = 1

ToInt32("1") = 1

ToRoman

Function	Parameters	Return value
ToRoman	object value	string

将指定的值转换成罗马形式。该值必须在 1-3998 之间。

例子：

ToRoman(9) = "IX"

ToSingle

Function	Parameters	Return value
ToSingle	object value	float

将指定的值转换为浮点型。

例子：

ToSingle(1m) = 1f

ToSingle("1") = 1f

ToString

Function	Parameters	Return value
ToString	object value	string

将指定值转换成字符型。

例子：

ToString(false) = "False"

ToString(DateTime.Now) = "08/09/2009 4:45:00 PM"

ToWords

Function	Parameters	Return value
ToWords	object value	string

指定的货币值转换成文字。

例子：

ToWords(1024.25) = "One thousand and twenty-four dollars and 25 cents"

Function	Parameters	Return value
ToWords	object value,	string

	string currencyName	

将指定的货币值转换成文字。在“currencyName”参数表示货币。此参数的有效值为：

"USD"

"EUR"

"GBP"

例子：

ToWords(1024.25, "EUR") = "One thousand and twenty-four euros and 25 cents"

Function	Parameters	Return value
ToWords	object value, string one, string many	string

将指定的整数值转换为文字。 “One”参数包含单数形式的名称; “Many”参数包含复数形式的名称。

例子：

ToWords(124, "page", "pages") = "One hundred and twenty-four pages"

ToWords(1, "page", "pages") = "One page"

ToWordsEnGb

Function	Parameters	Return value
ToWordsEnGb	object value	string

将指定的货币值转换成大不列颠英语单词。此函数和 ToWords 之间的具有以下区别：

- 默认情况下使用英镑货币；
- 在转换 milliards 和 trilliards 时，使用不同的词语。

例子：

ToWordsEnGb(121) = "One hundred and twenty-one pounds and 00 pence"

Function	Parameters	Return value
ToWordsEnGb	object value, string currencyName	string

将指定的货币值转换为大不列颠英语单词。“currencyName”参数表示货币。此参数的有效值为：

"USD"

"EUR"

"GBP"

例子：

ToWordsEnGb(1024.25, "EUR") = "One thousand and twenty-four euros and 25 cents"

Function	Parameters	Return value
ToWordsEnG	object value,	string

b	string one, string many	

将指定的整数值转换为英国英语单词。 “one”参数包含单数形式的名称; “many”参数包含复数形式的名称。

例子：

ToWordsEnGb(124, "page", "pages") = "One hundred and twenty-four pages"

ToWordsEnGb(1, "page", "pages") = "One page"

ToWordsRu

Function	Parameters	Return value
ToWordsRu	object value	string

指定的货币值转换成俄语单词。

例子：

ToWordsRu(1024.25) = "Одна тысяча двадцать четыре рубля 25 копеек"

Function	Parameters	Return value
ToWordsRu	object value, string currencyName	string

将指定的货币值转换成俄语单词。 “currencyName”参数表示货币。此参数的有效值为：

"RUR"

"UAH"

"USD"

"EUR"

例子：

ToWordsRu(1024.25, "EUR") = "Одна тысяча двадцать четыре евро 25 евроцентов"

Function	Parameters	Return value
ToWordsRu	object value, bool male, string one, string two, string many	string

将指定的整数值转换成俄语单词。“male”参数表示性别的名称。“one”，“two”和“five”参数包含用“1”，“2“和”5“所使用的名称的格式。

例子：

// the "страница" word is of female gender, male = false

ToWordsRu(122, false, "страница", "страницы", "страниц") =

"Сто двадцать две страницы"

// the "лист" word is of male gender, male = true
ToWordsRu(122, true, "лист", "листа", "листов") = "Сто двадцать два листа"

Program Flow

Choose

Function	Parameters	Return value
Choose	double index, params object[] choice	object

返回“选择”数组中的在“索引”参数指定的索引的元素。第一个数组元素索引值为 1。

例子：
Choose(2, "one", "two", "three") = "two"

IIf

Function	Parameters	Return value
IIf	bool expression, object truePart, object falsePart	object

返回“truePart”值，如果该“表达式”为 true。否则，将返回“falsePart”的值。

例子：
IIf(2 > 5, "true", "false") = "false"

Switch

Function	Parameters	Return value
Switch	params object[] expressions	object

提供给表达式的参数是由表达式和值组成。 Switch 函数计算奇数表达式从最低到最高索引，并返回与该计算结果相关联的第一个表达式为 true 时的偶数值。

例子：
// returns one of the following values - "a greater than 0",
// "a less than 0", "a equals to 0", depending on "a" value
Switch(
a > 0, "a greater than 0",
a < 0, "a less than 0",

a == 0, "a equals to 0")

合计

在许多报表中，我们可能需要显示一些信息总量：组的和，列表中的行数等等。 Report 中使用合计来执行这项任务。对于合计，你需要指明以下参数：

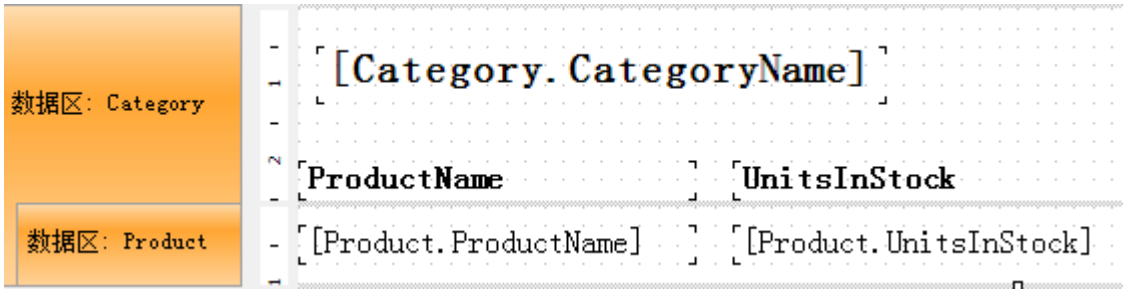
- 合计函数类型；
- 表达式，其应该被计算。对于“Count”函数，你不需要标示表达式；
- 条件。如果满足该条件，函数将被计算。这不是强制性的配置条件；
- 数据区域，该函数处理的数据区域；
- 合计值被打印的区域。

合计函数列表如下：

Function	Description
Sum	计算表达式的和。
Min	计算表达式的最小值。
Max	计算表达式的最大值。
Average	计算表达式的评价值。
Count	返回行数

创建合计

我们将着眼于利用合计函数作为一个例子。让我们创建“主 - 从”报表，其使用两个表 - “类别”和“产品”：



准备好的报表将如下所示：

Beverages

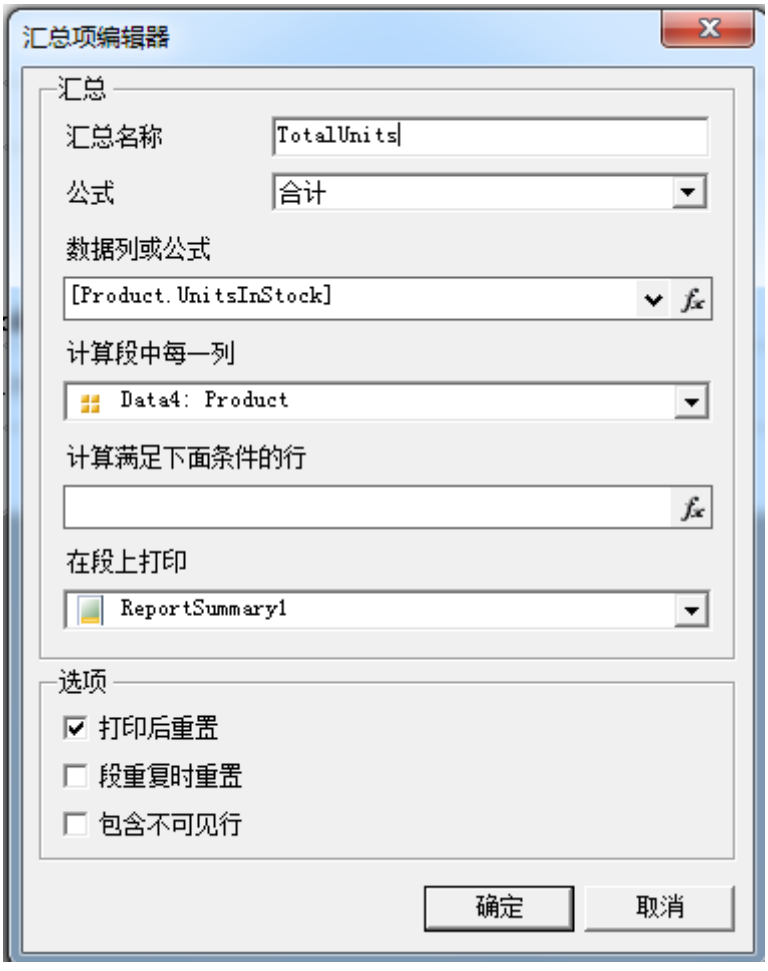
Product Name	Units In Stock
Chai	39
Chang	17
Chartreuse verte	69
Côte de Blaye	17
Guaraná Fantástica	20
Ipoh Coffee	17
Lakkalikööri	57
Laughing Lumberjack Lager	52
Outback Lager	15
Rhönbräu Klosterbier	125
Sasquatch Ale	111
Steeleye Stout	20

Condiments

Product Name	Units In Stock
Aniseed Syrup	13
Chef Anton's Cajun Seasoning	53
Chef Anton's Gumbo Mix	0
Genen Shouyu	39
Grandma's Boysenberry Spread	120
Gula Malacca	27
Louisiana Fiery Hot Pepper Sauce	76
Louisiana Hot Spiced Okra	4
Northwoods Cranberry Sauce	6
Original Frankfurter grüne Soße	32
Sirop d'érable	113
Vegie-spread	24

让我们在这份报表中添加合计函数，用于打印每个类别单元的总库存数量-“UnitsInStock“数据字段的总和。合计将被打印在”数据页脚“区域。

要打印合计值，首先要创建它。为此，在“数据“窗口点击“动作”按钮，选择“新建合计”选项。另一种方法-在数据树中右键单击“合计”元素选择“新建合计”选项。你将看到合计编辑器窗口。

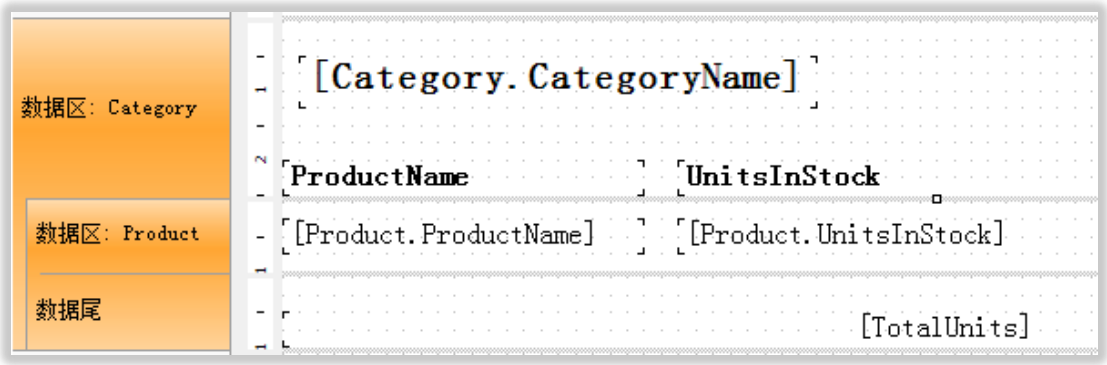


首先，你会被要求标示合计的名字。你会将其与合计的名字相关联，所以以这样一种方式命名，当它被计算时很容易理解。让我们命名我们的合计为“TotalUnits”。

然后我们选择“合计”函数。

现在，我们需要表明该合计计算的数据范围。为此在“计算段中每一列”字段中，我们选择其中的产品清单打印在“数据”区域。在“在段上打印”字段中，我们选择合计打印的区域 – 也就是“数据尾”区域。

单击确定按钮关闭编辑器。你会在“数据”窗口看到新的合计。现在你可以将它拖动到报表：



当我们运行此报表时，将看到以下结果：

Beverages

Product Name	Units In Stock
Chai	39
Chang	17
Chartreuse verte	69
Côte de Blaye	17
Guaraná Fantástica	20
Ipoh Coffee	17
Lakkalikööri	57
Laughing Lumberjack Lager	52
Outback Lager	15
Rhönbräu Klosterbier	125
Sasquatch Ale	111
Steeleye Stout	20
	559

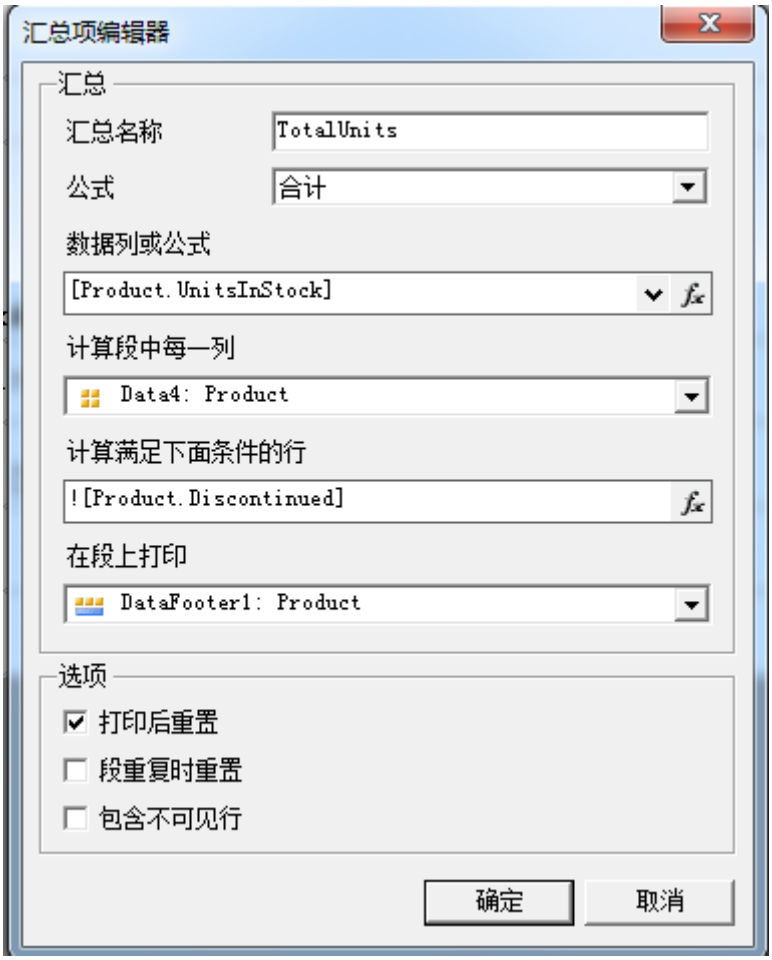
Condiments

Product Name	Units In Stock
Aniseed Syrup	13
Chef Anton's Cajun Seasoning	53
Chef Anton's Gumbo Mix	0
Genen Shouyu	39
Grandma's Boysenberry Spread	120
Gula Malacca	27
Louisiana Fiery Hot Pepper Sauce	76
Louisiana Hot Spiced Okra	4
Northwoods Cranberry Sauce	6
Original Frankfurter grüne Soße	32
Sirop d'érable	113
Vegie-spread	24
	507

合计条件

在前面的例子中，合计计算所有数据行。我们可以通过在合计编辑器上标明条件来限制这个范围。其条件返回 true 时，合计将只计算这些行。

例如，我们可以设置以下条件：



这将意味着，合计只计算其“Discontinued”标签没有被设置的这些产品。

运行合计

在我们的例子中，打印“数据页脚”后合计会进行重新设置。发生这种情况是因为我们在合计编辑器标示了它在合计打印出来后必须进行重置。其结果是，每个类别印刷自己的合计值。

如果取消选中“打印后重置”复选框，则合计被打印后不会重置。这就是所谓的累加值。

如果你需要同时打印两种类型的总计 - 普通合计和运行合计 - 创建一个具有类似设置和取消选中“打印后重置”的合计标签。

页面合计

为了创建一个打印在页面页脚的合计函数，你将在"Print on the band:"字段具有指定页面。

在页首打印合计

通常你会在页脚区域（如数据页脚，组页脚等）打印合计值。这是一种自然的打印顺序，因为当你打印合计时，它的值会进行正确计算并准备使用。然而，

在某些情况下，您将需要在标题打印合计（例如，组头）。如果您尝试这样做，你会看到一个零值。在这一刻，当你打印合计时，它还没有进行计算。

为了解决这个问题，Report 中有一个被称为“延迟打印”功能。 “文本”对象有一个名为“ProcessAt”的属性，它可以有下列值之一：

Value	Description
Default	默认的打印模式。这是默认值。
ReportFinished	该对象的值将在报表的末尾进行计算。
ReportPageFinished	当在页面中的所有区域对象将完成时值将被计算。
PageFinished	该对象的值将在页面的末尾进行计算。
ColumnFinished	该对象的值将在该列的末尾进行计算。
DataFinished	该对象的值将在数据区域结束时计算（当其页脚打印时）。
GroupFinished	该对象的值将在该组的结尾来计算（当其页脚打印时）。

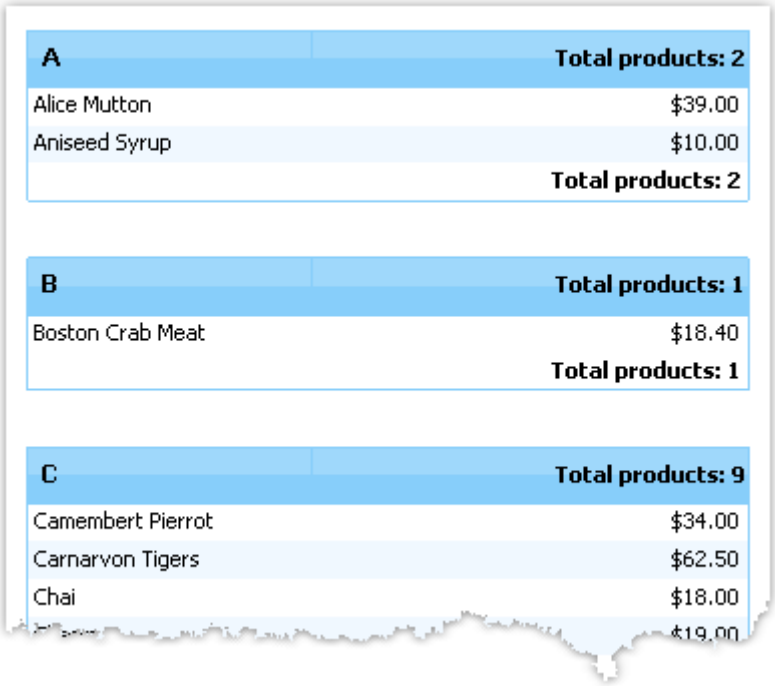
让我们来看看它是如何工作的。把它打印合计的“文本”对象放置在分组页眉。设置“文本”对象“GroupFinished”的“ProcessAt”属性：



当您运行报表，Report 的将做到以下几点：

- 它会打印分组页眉。合计值将被打印为 0（错误），但 Report 中会记住这个对象以后再处理它；
- 它会打印所有数据行；
- 它会打印组页脚。在这一刻，Report 的将获取在分组页眉打印的对象，并再次对其进行处理，以打印正确的合计值。

报表将如下图所示：



使用“ProcessAt”属性的其他值，您可以在报表标题打印报表合计（设置 ProcessAt= ReportFinished），或在页眉打印页面合计值（设置 ProcessAt= PageFinished）。

如果你打开报告文件缓存（“报告/选项...”菜单，“使用文件缓存”复选框）延迟打印功能将无法正常工作。

报表参数

你可以在报表中定义参数。参数是一个变量，它的值可通过报表本身和外面定义（一个程序，调用报表可以将参数值传送给它。请参阅“程序员手册”的详细说明）。参数可以在表达式中使用，并像“文本”对象显示在报表对象中。

最常见的使用参数的方法：

- 在参数中设置条件来进行数据过滤；
- 在报表中打印参数值。

一个参数具有以下属性：

Property	Description
Name	参数的名称可以包含任何符号除了“.”
DataType	参数数据类型
Expression	该表达式返回参数的值。调用参数时，该表达式将被处理。
Value	参数值。这个属性在设计器中不可用，并且可以通过编程来填补。

你必须设置“名称”和“数据类型”属性。 “表达式”属性可以为空。在这种情况下，参数的值应通过编程方式传递。

创建一个参数

若要创建参数，选择“数据”窗口中的“参数”元素，右键单击它并在其上下文菜单中选择“新建参数”项：



按 F2 键，给参数命名，然后进入“属性”窗口，设置参数的“数据类型”属性。

参数可以嵌套。要创建嵌套的参数，选择父参数，右键单击它并在其上下文菜单中选择“新建参数”项：



您可以关联两个参数，父参数和嵌套参数。嵌套级别不受限制。

在报表中使用参数

您可以使用方括号从一个表达式中获取参数：

```
[Parameter name]
```

对于嵌套参数，您需要使用这种方法获取：

```
[Parent parameter.Child parameter]
```

由于参数已经得到了明确的类型（它被赋予的 DataType 属性），然后用参数，您可以执行那些被允许的数据类型的操作。因此，字符串类型的参数可以在一个表达式如下方式使用：

如果一个参数已经得到了明确的类型（它在数据类型属性中设置），您可以使用此参数执行那些符合此数据类型的操作。 所以，字符串类型的参数可以通过以下方式在表达式中使用：

```
[StringParameter].Substring(0, 2)
```

让我们来看一个使用参数的例子。假设我们具有打印“员工”表的报表。我们要修改报表，以打印指定编号的员工的信息。要做到这一点，我们需要过滤“EmployeeID”数据字段中的数据。创建一个“EmployeeID”名称的参数。标示参数的类型 - Int32。要筛选具有指定 ID 的员工，我们需要进入“数据”区域编辑器，并在“过滤器”选项卡中标示下面的表达式：

```
[Employees.EmployeeID] == [EmployeeID]
```

为了从你的程序中将参数值传递给报表，使用下面的代码：

```
report1.SetParameterValue("EmployeeID", 2);
```

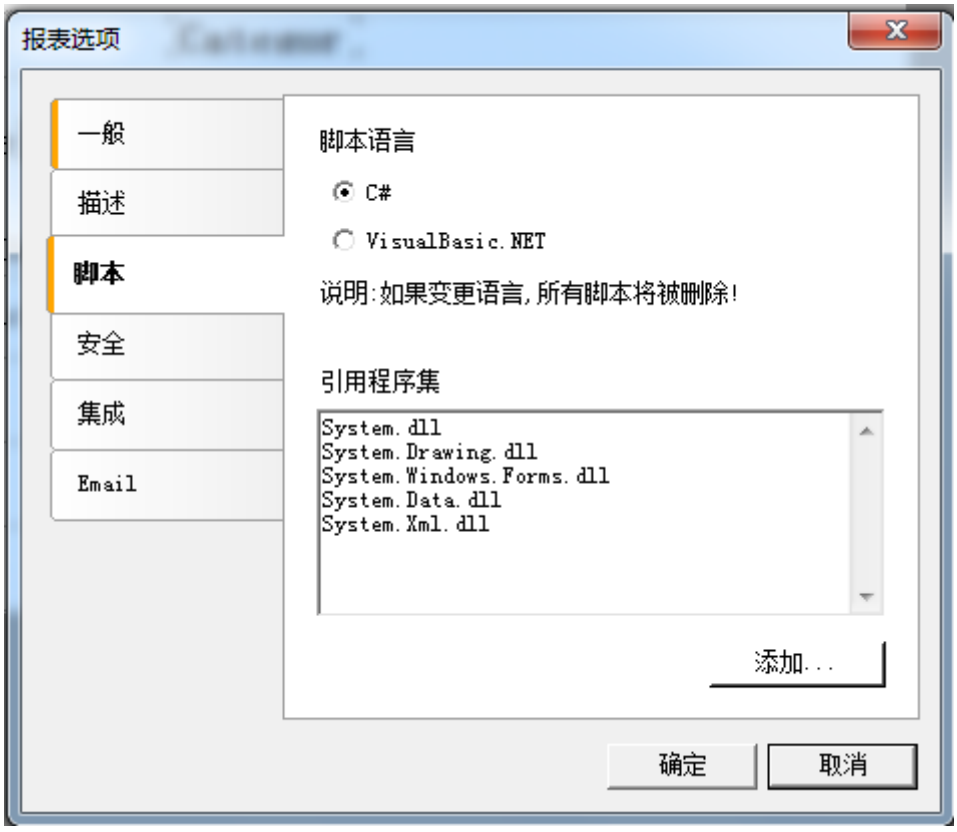
表达式

在 Report 中，许多地方需使用表达式。例如，“文本”对象可以包含在方括号中的表达式。

一个表达式是在 C＃或 VB.Net 语言的一段代码，它返回任何值。例如：

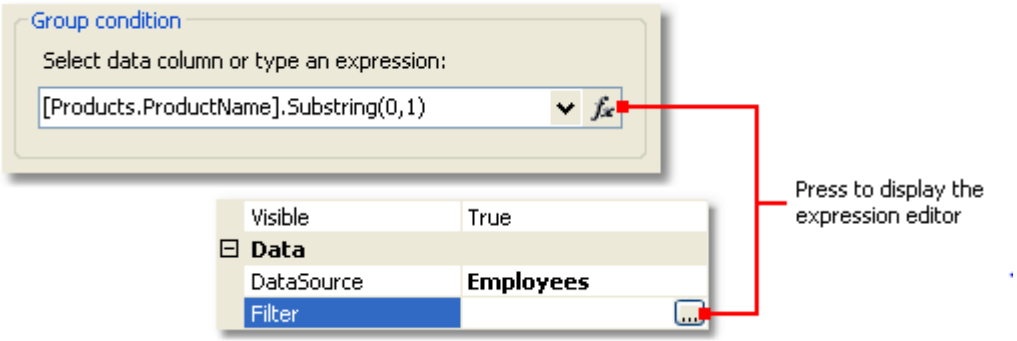
```
2 + 2
```

在一份报表中，表达式应用选择的脚本语言书写。默认情况下，它是 C＃。你可以通过在“报表|选项…”菜单窗口选择“脚本”元素更改语言。

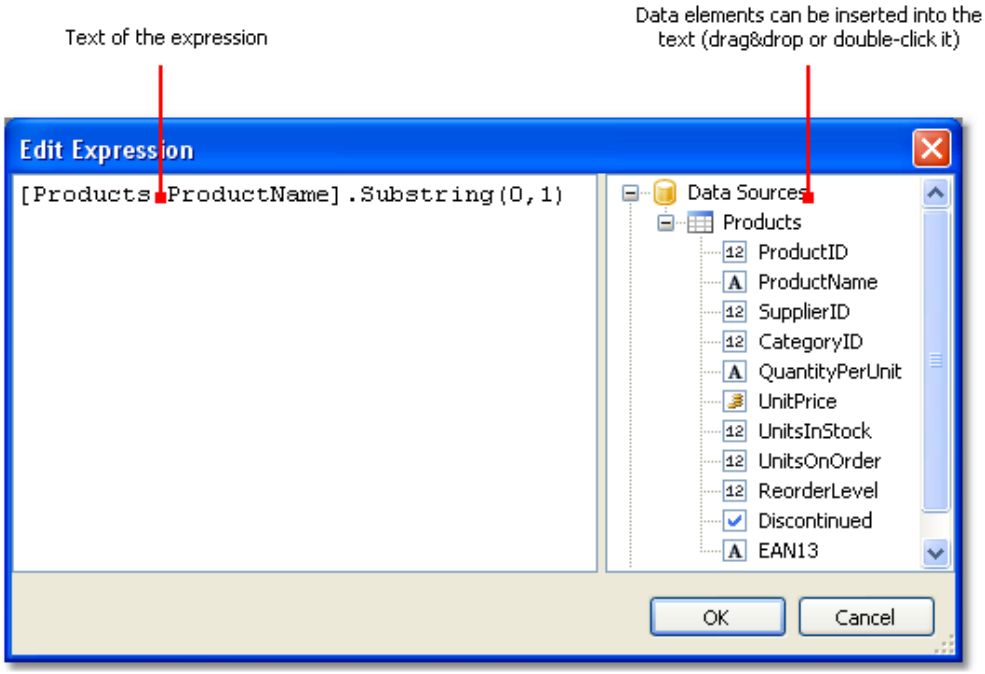


表达式编辑器

为了快速地写出表达式，使用表达式编辑器。它可以调用在 Report 的用户界面，在这里你可以输入一个表达式：



表达式编辑器呈现一个窗口，在那里你可以写一个表达式，并插入一些数据元素：



引用报表对象

为了引用报表对象，需使用该对象的名称。下面的例子将返回 Text1 对象的高度：

```
Text1.Height
```

为了引用报表属性，使用报表变量。下面的示例返回该报表被加载的文件名。

```
Report.FileName
```

此外，你可以引用嵌套对象的属性。下面的例子将返回一个报表名称：

```
Report.ReportInfo.Name
```

引用 .Net 函数

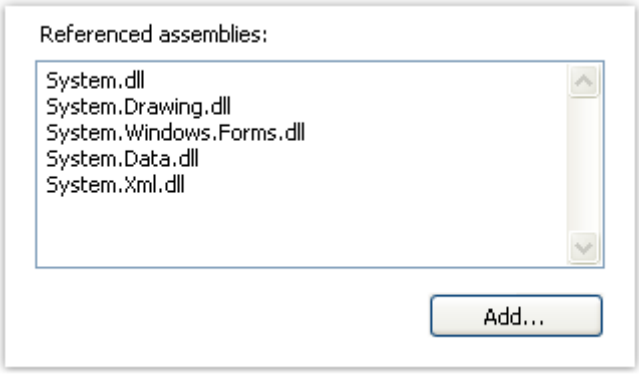
您可以在表达式中使用任意.Net 对象。下面的示例演示 Max 函数的使用：

```
Math.Max(5, 10)
```

默认情况下，报表中使用下列.Net 组件：

```
System.dll
System.Drawing.dll
System.Windows.Forms.dll
System.Data.dll
System.Xml.dll
```

您可以访问所有.Net 对象在这些组件中的声明。如果你需要访问另一个程序集，在报表的组件列表中添加它的名字。你可以在“Report|Option...”菜单，通过在一个窗口中选择“脚本”元素完成此操作：



例如，如果你想使用在你报表的应用程序中声明的函数，在一份报表的组件列表添加应用程序集（.EXE 或.DLL）。然后，你可以使用你的应用程序的命名空间调用该函数。例如，如果在应用程序中定义了下面的函数：

```
namespace Demo
{
public static class MyFunctions
{
public static string Func1()
{
return "Hello!";
}
}
}
```

你可以以下面的方式在你的报表中使用：

```
Demo.MyFunctions.Func1()
```

如果您在报表脚本的顶部添加了“using Demo”行，它会允许你缩短语法：

```
MyFunctions.Func1()
```

要引用在脚本中定义的函数或变量，只需使用它的名字：

```
myPrivateVariableThatIHaveDeclaredInScript
MyScriptFunction()
```

您可以在表达式中使用那些只返回一个值的函数。

引用数据元素

除了标准的语言元素，你可以在表达式中使用下面的报表元素：

- 数据源字段；
- 系统变量；
- 合计值；
- 报表参数。

所有这些元素都包含在“数据”窗口。请参阅“数据”一章了解更多细节。这些元素可以在表达式中使用，通过将其包括在方括号中。例如：

```
[Page] + 1
```

该表达式返回下一个打印的页面数。系统变量“Page”，返回当前报表页码，它被用于表达式。它括在方括号中。

引用数据源

为了引用数据源中的字段，使用以下格式：

```
[DataSource.Column]
```

源名称通过一个句号与字段分开，例如：

```
[Employees.FirstName]
```

源名称在某种情况下可以是复合词，如果我们通过使用一个关系引用一个数据源。请参阅“数据”部分中的更多细节。例如，这是你如何引用相关的数据源字段：

```
[Products.Categories.CategoryName]
```

让我们来看看在一个表达式中使用字段的例子：

```
[Employees.FirstName] + " " + [Employees.LastName]
```

这里要注意的是：每一个字段都有一个明确的数据类型，它是在“数据类型”属性中（你可以在“属性”窗口中看到它，如果事先在“数据”窗口选择了数据字段）设置的。一个字段如何在表达式中使用取决于其类型。例如，在上述例子中，两个字段 - 姓氏和名字 - 已经得到了一个字符串类型，这就是为什么他们能够以这样的方式被使用。在下面的例子中，我们将尝试使用数字类型的“Employees.Age”字段，这将导致一个错误：

```
[Employees.FirstName] + " " + [Employees.Age]
```

在这种情况下，我们引用“Employees.Age”字段，就好像它是一个整数变量。而且它确是如此。我们知道，所有的表达式被编译。从一个编译器的角度来看所有非标准化的东西（如引用数据字段）转换为另一种类型，这是可以理解的编译器。所以，最后一个表达式将变成如下形式：

```
(string)(Report.GetColumnValue("Employees.FirstName")) + " " +
(int)(Report.GetColumnValue("Employees.Age")).ToString()
```

正如所见，Report 以下面的方式更改数据字段的引用：

```
[Employees.FirstName]                                    -->
(string)(Report.GetColumnValue("Employees.FirstName"))
[Employees.Age] --> (int)(Report.GetColumnValue("Employees.Age"))
```

也就是说，我们可以在表达式中使用数据字段就好像它是有一个明确的类型的变量。例如，下面的表达式将返回一个员工的姓名的第一个符号：

```
[Employees.FirstName].Substring(0, 1)
```

引用系统变量

您可以在表达式中使用下面的系统变量（它们都可以在“数据”窗口访问）：

Variable	.Net data type	Description
Date	DateTime	报表的开始时间和日期

Page	int	当前页码
TotalPages	int	该报表的总页数。要使用此变量，您需要启用该报告的双通。你可以在 Report|Properties 属性中做到这一点。
PageN	string	页码格式: "Page N".
PageNofM	string	页码格式: "Page N of M".
Row#	int	组内的数据行数。这个值在一个新组的开始被复位。
AbsRow#	int	数据行的绝对数量。该值不会在一个新组的开始被重置。

每个变量都有一个明确的数据类型。而这一点，取决于它如何在表达式中使用。下面是表达式中如何使用日期的例子：

```
[Date].Year
```

该表达式返回当前年份。“日期”变量具有 DateTime 类型，大家可以引用它的“年”的属性。我们同样可以得到当前月份（[Date].Month）。

Report 将引用系统变量转换为以下形式（例如，“日期”变量）：

```
((DateTime)Report.GetVariableValue("Date"))
```

引用合计值

为了引用合计值，需使用它的名称：

```
[TotalSales]
```

Report 将引用合计转换成如下形式：

```
Report.GetTotalValue("TotalSales")
```

正如你所看到的，这里不使用数据类型。因为合计值为 Report.Variant 类型。它可以在任何表达式中直接使用，因为它可以自动转换为任何类型。例如：

```
[TotalSales] * 0.2f
```

引用报表参数

为了引用报表参数，你应该使用它的名字：

```
[Parameter1]
```

参数可以嵌套。在这种情况下，你应该使用两种父和子参数名称，格式如下：

```
[ParentParameter.ChildParameter]
```

参数有定义的数据类型。它参数的“数据类型”属性设置。它在表达式中使用的方法取决于参数的数据类型。

Report 将引用报表参数转换为以下方式：

```
((string)Report.GetParameterValue("Parameter1"))
```

脚本

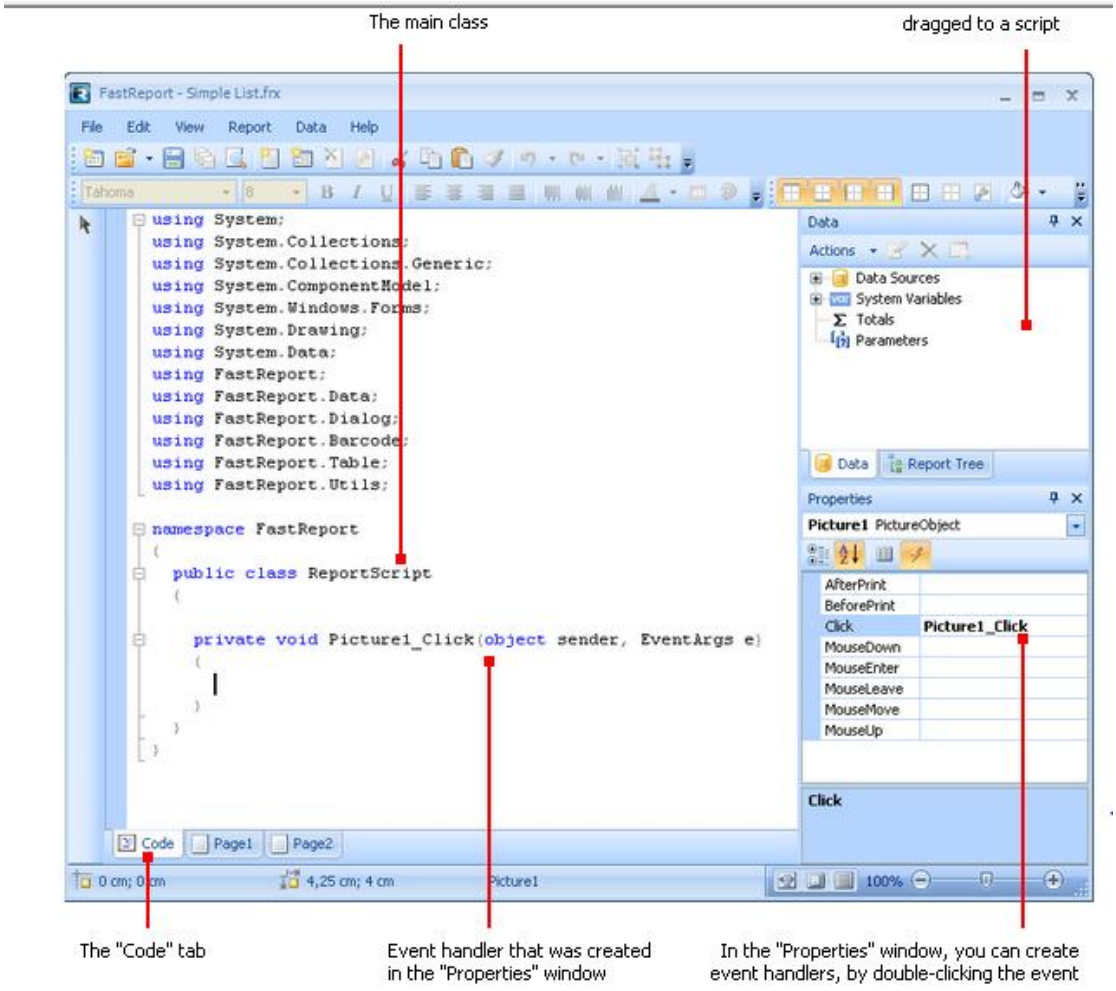
脚本是一个更高层次的编程语言，这是报表的一部分。脚本可以以下面.Net 语言之一书写：

- C#;
- VisualBasic.Net

脚本适用于很多地方。使用该脚本，您可以执行以下操作：

- 执行数据处理，这无法通过 Report 引擎的常规方法完成;
- 控制报表页和页面上的区域的印刷;
- 控制对话形式的元素之间的相互作用;
- 控制动态“表格”对象的形成;
- 等待。

为了看到报表的脚本，切换到设计器中的“代码”选项卡：



脚本语言可以在“Report|Options”中进行设置。这是在你已经创建了一个新的报表时才支持操作，因为更改语言时，现有的脚本会被删除。



总说明

相对其他的报表生成器，在 Report 中只包含你所编写的脚本。在脚本中，您可以：

- 添加您的变量，方法和属性到主脚本类；
- 创建一个报表对象的事件处理程序；
- 添加新类到脚本，如果需要的话。一个类可以在 ReportScript 主类之前或之后添加。

你不可以：

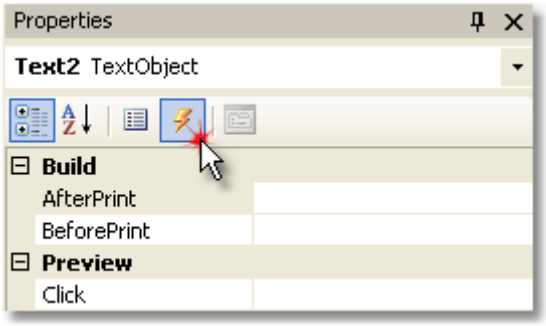
- 删除，重命名或更改 ReportScript 主类的可见性区域；
- 在主类中重命名一个命名空间。

当报表运行时，会发生以下情况：

- Report 在脚本中添加变量列表，其名称与报表对象的名称相对应。这在编译脚本之前完成，并允许你通过它们的名字来引用报表对象；
- 一个表达式处理程序被添加到脚本，它处理在报表中发现的所有表达式；
- 一个脚本被编译，如果它不为空；
- 脚本类被初始化；
- 运行报表。

事件处理程序

脚本主要用于创建对象的事件处理程序。选择所需的对象，创建事件处理程序。在“属性”窗口中，点击 按钮，为了在事件的列表上切换：



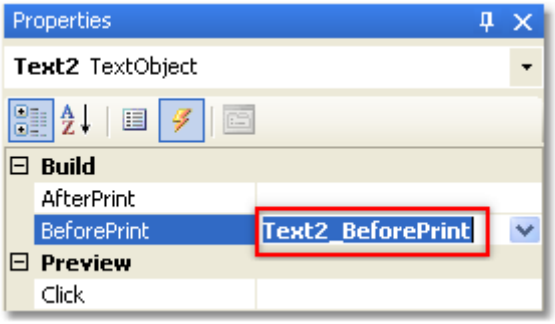
选择您需要的事件并双击它。 Report 会在报表的代码中添加一个空事件处理程序：

```
private void Text2_BeforePrint(object sender, EventArgs e)
{
}
```

该“报表”对象同样获得事件。这个对象可以通过下面的方法选择：

- 在 ″Report Tree″ 窗口选择″Report″ ；
- 在“属性”窗口的下列列表选择″Report″。

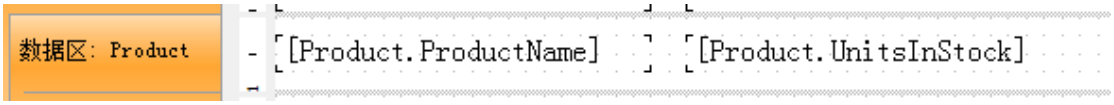
为了删除事件处理程序，选择“属性”窗口选择一个事件，请选择文本，然后按 Delete 键：



报表事件

为了控制具有最大灵活性的报表，每个报表对象得到了几个事件。例如，在一个处理程序，连接到“数据”区域，你可以筛选记录，也就是根据某些条件隐藏或显示区域。

让我们看看它在报表生成过程中引发的事件。作为一个例子，我们将采取一个简单的报表，包含一个页面，一个“数据”区域，并在区域中放置两个“文本”对象：



在报表的一开始，“报表”对象引发 StartReport 事件。在形成报表页面之前，触发 StartReport 事件。该事件在每个模板页面触发（不要和准备好的报表页面混淆！）。在我们的例子，不管有多少页准备好的报表- 事件只被触发一次，因为该报表已经得到了一个模板页。

此外，开始打印“数据”区域行。这种情况以下面的方法发送：

1、 BeforePrint 区域事件被触发;

2、 所以区域上的对象的 BeforePrint 事件被触发;

3、 所有对象都充满了日期;

4、 区域上所有对象的 AfterData 事件呗触发；

5、 BeforeLayout 区域事件被触发;

6、 对象被放置在区域上，该区域的高度被计算以及区域被拉伸（如果可以）;

7、 AfterLayout 区域事件呗触发；

8、 如果页面上的可用空间不适合区域，一个新的页面形成;

9、 其区域和所有对象都显示在一个准备好的报表页面上;

10、 AfterPrint 区域事件呗触发;

11、 所有区域对象的 AFTERPRINT 事件被触发。

只要在源代码中有日期，则会发生区域行的打印。在此之后，形成报表。页面的 FinishPage 事件被触发，最终 - “报表”对象的 FinishReport 事件。

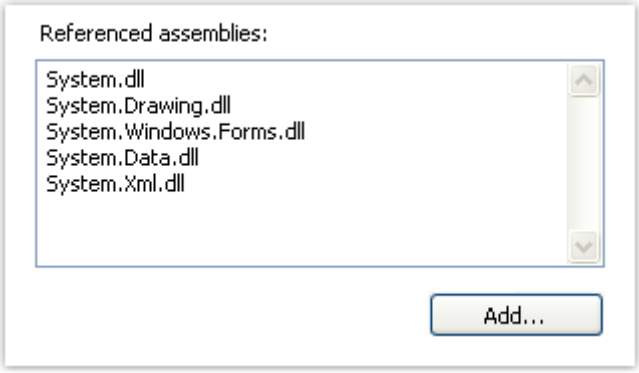
因此，通过使用不同对象的事件，您可以控制生成报表的每一步。要正确使用事件的关键 - 全面理解区域的打印过程，阐述在 11 步以上。所以，很多操作都可以完成，只用了 BeforePrint 区域 - 对象的任何改变和操作，也将被显示。但在这种情况下，不可能分析出区域将打印在哪个页面上，如果它伸展，因为区域的高度将在第 6 步进行计算，这可以在第 7 步的 AfterLayout 事件的帮助下或在步骤 10 中的 AFTERPRINT，但在后一种情况下，区域已经印刷及操作对象不会放弃任何东西。总之，你必须清楚说明，在什么时候每一个事件被触发并使用，这些，与给定的任务相对应。

使用 .Net 对象

在脚本中，您可以使用任何.Net 对象，在下面的程序集中定义：

```
System.dll
System.Drawing.dll
System.Windows.Forms.dll
System.Data.dll
System.Xml.dll
```

此外，您可以使用在 Report 的程序集中定义的任何对象。如果你需要访问另一个组件，将其添加到程序集列表。这可以在“Report|Option...”菜单中，选择“脚本”选项卡完成：



例如，如果你想在报表中使用一个在你的应用程序中声明的函数，在一份报表中的组件列表添加应用程序组件（.EXE 或.DLL）。之后，你可以使用你的应用程序的命名空间调用该函数。例如，如果下面的函数是在应用中定义：

```
Namespace Demo
{
Public static class MyFunctions
{
Public static string Func1()
{
Return "Hello!";
}
}
}
```

可以通过以下方式在脚本中调用它：

```
String hello=Demo.MyFunctions.Func1();
```

如果您添加了“Using Demo”线在报表脚本的顶部，这将允许你缩短语法：

```
String hello=MyFunctions.Func1();
```

引用报表对象

为了引用报表对象（例如，“文本”对象）使用对象的名称。下面的例子返回 Text1 对象的高度：

```
Float height =Text1.Height;
```

需要注意的是测量报表的原生单位是像素。使用像左，上，宽度和高度，这种对象的属性时需牢记它。要将像素转换成厘米，使用在“Units”类中定义的常量：

```
Float heightInPixels=Text1.Height;
Float heightInCM =heightInPixels/Units.Centimeters;
Text1.Height=Units.Centimeters*5;
```

报表和引擎对象

报表中，除了对象，还有在脚本中定义的两个变量：报表和引擎。

报表变量是指当前报表。在下面的列表中，给出了报表对象的方法的列表：

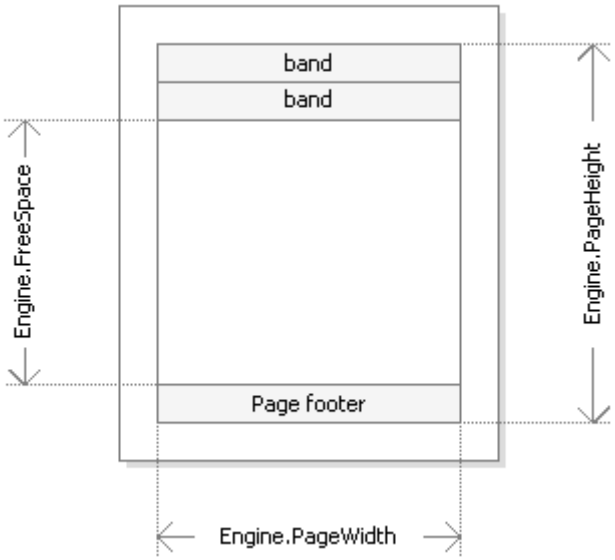
Method	Description
Object alc(String xpression)	计算表达式并返回值。
Object etColumnValue(String omplexName)	返回的数据字段的值。该名称以“DataSource.Column”的形式提交。如果该列已经获取了空值，则默认情况下（0，空字符，）转换成一个值。
objectGetColumnValueNullable(string complexName)	返回的数据字段的值。与上一个方法相反，它并不会转换成默认值，可以为null。
Parameter GetParameter(string complexName)	返回具有指定名称的报表参数。“MainParam.NestedParam”：名称可以指的是复杂的嵌套的参数
object GetParameterValue(string complexName)	返回具有指定名称的报表参数的值。
void SetParameterValue(string complexName, object value)	以指定的名称设置报表参数的值。
object GetVariableValue(string complexName)	返回系统变量
object GetTotalValue(string name)	返回在“数据”窗口定义的合计值的名称
DataSourceBase GetDataSource(string alias)	返回在报表中定义的数据源的名称

引擎对象是控制报表创建的发动机。使用引擎的方法和属性，您可以管理区域放置到页面的过程。您可以使用引擎对象的以下属性：

Property	Description
Float Curx	当前坐标上的 X 轴。该属性可以被分配一个值，从而移动打印对象。
Float CurY	当前打印位置在 Y 轴上的坐标。这个属性，值可以被分配，从而移动打印对象
Int CurColumn	在一个多列报表中的当前列数。第一列的编号为 0。
Int CurPage	正在打印的页面数。这个值可以从“页面”系统变量来接收。
Float PageWidth	页面的宽度减去左右边距的大小。
Float PageHeight	页面减去顶部和底部边距的大小的高度。

Float PageFooterHeight	页脚的高度。
float ColumnFooterHeight	字段尾的高度
float FreeSpace	页面上自由空间的大小
bool FirstPass	返回 true，如果正在执行的第一个（或唯一）报表通过。可以从 Report.DoublePass 属性获得通过的数量。
bool FinalPass	返回 true，如果正在执行的最后一个（或唯一）报表通过。

在下面的图中，你可以看到上面列出的一些属性的意思。



Engine.PageHeight 和 Engine.PageWidth 决定了打印区域的大小，这几乎总是小于页面的实际尺寸。印刷面积的大小由页边距决定，这是由 LEFTMARGIN，TOPMARGIN，RightMargin 和 BottomMargin 页面属性决定的。

Engine.FreeSpace 属性决定在页面上可用空间的高度。如果在页面上包含“页脚”区域，其高度在计算自由空间决定。需要注意的是，打印区域后，可用空间减少。

一个准备好的报表页面是如何形成的呢？ Report 引擎在页面上显示区域，直到有足够的空间用于输出区域。当没有可用空间，“报表页脚”区域被打印，并形成一个新的空白页面。从当前位置显示一个区域，这是通过 X 和 Y 坐标来确定。这个位置是由 Engine.CurX 和 Engine.CurY 属性返回。打印区域后，CURY 通过印刷区域的高度自动增加。形成一个新的页面后，CURY 的位置被设置为 0。CURX 的位置发生改变当打印多字段报表时。

Engine.CurX 和 Engine.CurY 属性的访问性不仅可读，还可写。这意味着，你可以通过一个合适的事件手动移动区域。使用这些属性的例子可以在“示例”部分看出。

当操作这些返回大小或位置的属性时，请记住，这些属性是以像素衡量的。

在引擎对象中，下面的方法被定义：

Method	Description
void AddOutline(string text)	将一个元素添加到报表大纲，并设置当前位置设置到添加的元素。
void OutlineRoot()	设置其在大纲根中的当前位置。
void OutlineUp()	将当前位置移动到一个更高级别的大纲元素。
void AddBookmark(string name)	添加书签
int GetBookmarkPage(string name)	返回书签被放置的页码编号。
void StartNewPage()	开始一个新页面

通过使用 AddOutline，OutlineRoot，OutlineUp 方法，您可以手动形成报表大纲。通常情况下，这是在每一个区域和报表页面都具有 OutlineExpression 属性的帮助下自动完成的。

该 AddOutline 方法添加一个子元素到当前大纲元素，并使其获取焦点。当前报表页面，以及在页面上的当前位置与新的元素相关联。如果多次调用 AddOutline 方法，那么你将具有以下结构：

Item1
Item2
Item3

该 OutlineRoot 方法移动当前元素到大纲的根源。例如，下面的脚本：

```
Engine.AddOutline("Item1");
Engine.AddOutline("Item2");
Engine.AddOutline("Item3");
Engine.OutlineRoot();
Engine.AddOutline("Item4");
```

将创建下列提纲：

Item1
Item2
Item3
Item4

使用书签时，引擎对象的 AddBookmark 和 GetBookmarkPage 方法被使用。通常所有报表对象都使用书签属性，会自动添加书签。

通过使用添加书签的方法，可以通过编程方式添加一个书签。这种方法在当前打印位置创建当前页面的书签。

该 GetBookmarkPage 方法返回放置书签的页码。该方法被经常用于创建表的内容时，显示页码。在这种情况下，报表必须有一个双通。

引用数据源

与 Report 表达式不同的是，它在脚本中从来不使用方括号来引用数据源。而是使用报表对象的 GetColumnValue 方法，返回的字段的值：

```
string productName = (string)Report.GetColumnValue("Products.Name");
```

如看到的那样，需要标示数据源的名称和其字段。源的名称可以是复合的，如果我们通过使用关系引用数据源。有关关系的详细信息可在“数据”一章中找到。例如，你可以用这种方式引用相关数据源的字段：

```
string categoryName = (string)Report.GetColumnValue("Products.Categories.CategoryName");
```

为了使工作更容易，使用“数据”窗口。从这里你可以拖动数据元素到脚本，在这个 Report 中自动创建的引用元素的代码。

为了引用数据源本身，使用报表对象的 GetDataSource 方法：

```
DataSourceBase ds = Report.GetDataSource("Products");
```

DataSourceBase 类的属性和方法可以从 Report.Net 类参考帮助系统接收。作为一项规则，该对象是在脚本中的使用方法如下：

```
// get a reference to the data source
DataSourceBase ds = Report.GetDataSource("Products");
// initialize it
ds.Init();
// enum all rows
while (ds.HasMoreRows)
{
// get the data column value from the current row
string productName = (string)Report.GetColumnValue("Products.Name");
// do something with it...
// ...
// go next data row
ds.Next();
}
```

引用系统变量

为了引用系统变量，使用报表对象的 GetVariableValue 方法：

```
DateTime date = (DateTime)Report.GetVariableValue("Date");
```

系统变量的列表可以在“数据”窗口中看到。从中，你可以拖动一个变量到一个脚本，在此 Report 中自动创建引用变量的代码。

引用合计值

为了引用合计值，需使用报表对象的 GetTotalValue 方法：

```
float sales = Report.GetTotalValue("TotalSales");
```

合计函数的列表可以在“数据”窗口中看到。从中，你可以拖动一个合计函数到脚本，在此 Report 中自动创建引用合计函数的代码。

合计值已经得到了 Report.Variant 类型。它可以直接用于任何表达式，因为 Report.Variant 类型可以自动转换为任何类型。例如：

```
float tax = Report.GetTotalValue("TotalSales") * 0.2f;
```

引用合计值可以在它正在处理的时间来完成。通常合计值在打印它所处的报表区域时，已经“准备好使用”。

引用报表参数

为了引用报表参数，使用报表对象的 GetParameterValue 方法：

```
int myParam = (int)Report.GetParameterValue("MyParameter");
```

参数可以被嵌套。在这种情况下，标示父参数的名称并在句号后标示其子参数的名称：

```
Report.GetParameterValue("ParentParameter.ChildParameter")
```

参数已经获取了一个明确的数据类型。它是在该参数的数据类型属性中定义的。在引用参数时，您必须考虑到这一点。你可以在“数据”窗口看到参数列表。从中，你可以拖动参数到脚本中，在此 Report 中自动创建引用参数的代码。

为了改变参数的值，可以使用报表对象的 SetParameterValue 方法：

```
Report.SetParameterValue("MyParameter", 10);
```

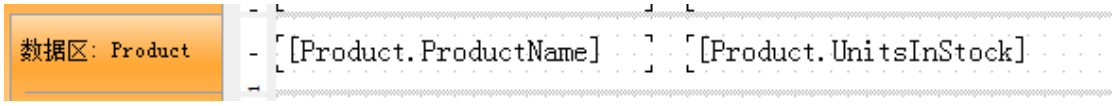
范例

例 1 更改对象的外观

在这个例子中，我们将展示如何根据对象中打印的值改变文本的颜色。我们将使用：

- BeforePrint 事件；
- 在脚本中引用数据字段；

创建具有以下外观的简单的报告：



选择打印“单价”字段的对象，并创建一个 BeforePrint 事件处理程序：

```
private void Text2_BeforePrint(object sender, EventArgs e)
{
if (((Decimal)Report.GetColumnValue("Products.UnitPrice")) > 20)
Text2.TextColor = Color.Red;
}
```

从“数据”窗口中拖动“Products.UnitPrice”数据字段添加到脚本。在此期间，以下字符串将被添加到脚本中：

```
((Decimal)Report.GetColumnValue("Products.UnitPrice"))
```

如果我们运行报表，我们将看到价格>20 的所有产品，以红色突出显示：

Chai	18,00
Chang	19,00
Aniseed Syrup	10,00
Chef Anton's Cajun Seasoning	22,00
Chef Anton's Gumbo Mix	21,35
Grandma's Boysenberry Spread	25,00

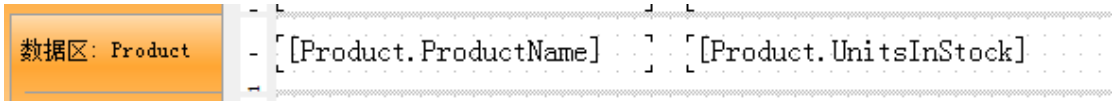
可以用条件突出显示（你可以阅读“创建报表”一章的“条件突出”一节，了解更多内容）达到相同的效果。

例 2.突出显示区域的偶数行

在这个例子中，我们将展示如何改变“数据”区域的偶数行的填充颜色。我们将使用：

- BeforePrint 区域事件;
- 在脚本中引用“Row#”系统变量。

创建具有以下外观的简单的报告：



为该区域创建 BeforePrint 事件处理程序：

```
private void Data1_BeforePrint(object sender, EventArgs e)
{
if (((Int32)Report.GetVariableValue("Row#")) % 2 == 0)
Data1.FillColor = Color.Gainsboro;
}
```

“Row＃”系统变量返回打印区域的行的数目。为了在脚本中插入引用变量，从“数据”窗口中拖动它。在此期间，字符串将被插入到脚本中：

```
((Int32)Report.GetVariableValue("Row#"))
```

如果我们运行报告中，我们将看到，偶数行会以浅灰色的颜色突出显示：

Chai	18,00
Chang	19,00
Aniseed Syrup	10,00
Chef Anton's Cajun Seasoning	22,00
Chef Anton's Gumbo Mix	21,35

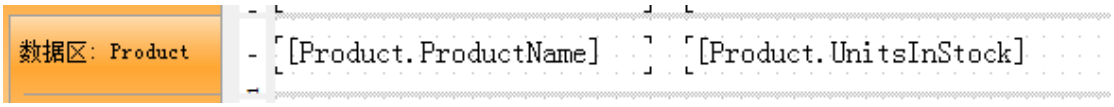
可以用“数据”区域的“EvenStyle”属性达到相同的效果。您可以在“创建报表”一章中的“突出显示奇数/偶数行的数据”一节中阅读更多内容。

例 3.数据过滤

在这个例子中，我们将展示如何根据给定的条件隐藏“数据”区域行。我们将使用：

- BeforePrint 区域事件；
- 从脚本中引用数据源。

创建具有以下外观的简单的报告：



创建 BeforePrint 事件处理程序：

```
private void Data1_BeforePrint(object sender, EventArgs e)
{
if (((Decimal)Report.GetColumnValue("Products.UnitPrice")) > 20)
Data1.Visible = false;
}
```

在给定的情况下，单价>20 的区域行会被隐藏起来：

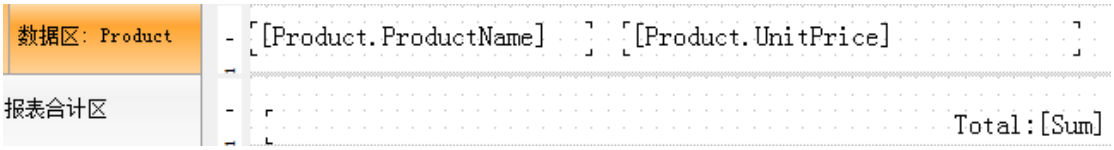
Chai	18,00
Chang	19,00
Aniseed Syrup	10,00
Konbu	6,00
Genen Shouyu	15,50
Pavlova	17,45

例 4.计算合计

在这个例子中，我们将展示如何通过使用编程的方法来计算总和。我们将使用以下命令：

- BeforePrint 区域事件；
- 从脚本中引用数据字段；
- 局部变量，其值将在报表中打印。

创建以下格式的报表：

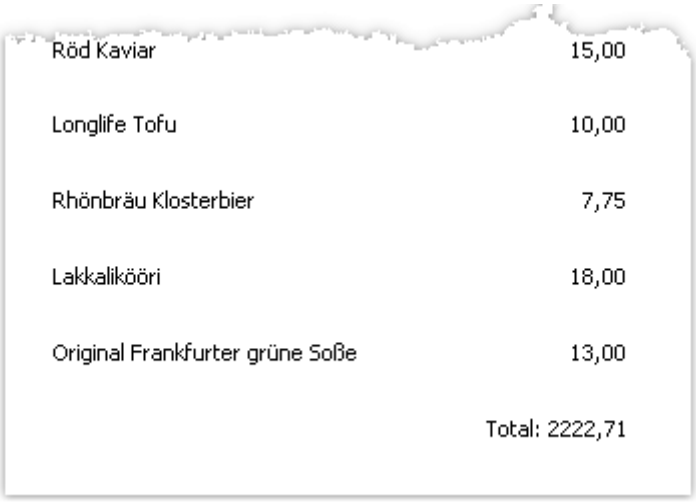


在脚本中，声明“Sum”变量，并创建属于区域的一个 BeforePrint 事件处理程序：

```
public class ReportScript
{
private decimal sum;
private void Data1_BeforePrint(object sender, EventArgs e)
{
sum += (Decimal)Report.GetColumnValue("Products.UnitPrice");
}
}
```

可以从“数据”窗口拖动“Products.UnitPrice”数据字段放置到脚本。

如果您运行报表，你会看到以下内容：



可以通过使用合计来实现同样的效果。

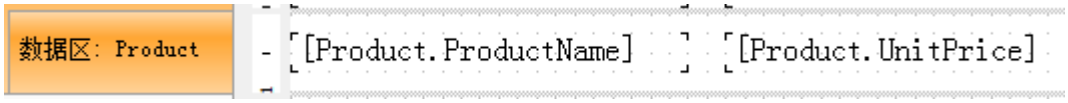
例 5.移动打印位置

在这个例子中，我们将展示如何使用 Engine 对象手工移动区域的位置。我们将使用：

- BeforePrint 区域事件；

- 引擎对象。

创建一个具有下面外观的简单的报表：



创建一个 BeforePrint 事件处理程序：

```
private void Data1_BeforePrint(object sender, EventArgs e)
{
Engine.CurX = ((Int32)Report.GetVariableValue("Row#")) * 10;
}
```

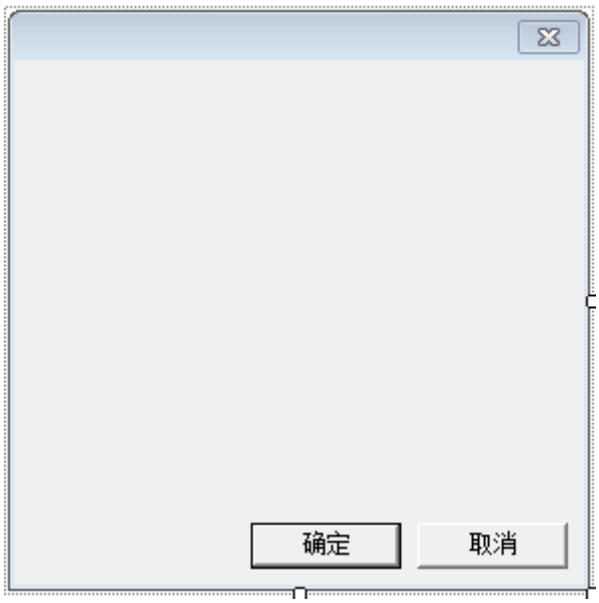
运行报表，你会看到以下内容：



对话框形式

除了普通的页面，一个报表可以包含一个或几个对话框。当启动报表时，这种对话框的控件将被显示。在对话框中，你可以输入所需的任何类型的信息来创建报表。此外，对话框可以被用于过滤显示在报表中的数据。

为了在报表中添加对话框，单击设计工具栏上的  按钮。一个新的对话框如下所示：



一个报表，有一个或几个对话框，它是这样工作的：

- 开始时，该报表显示第一个对话框;
- 如果对话框是在单击“ＯＫ”按钮的帮助下关闭，那么下一个对话框被显示;否则，该报表停止工作;
- 所有对话框都被显示后，该报表被建立。

控件

在一个对话框中，可使用以下控件：

Icon	Name	Description
	ButtonControl	表示 Windows 按钮控件。
	CheckBoxControl	表示 Windows 复选框。
	CheckedListBoxControl	显示一个 ListBox 在其每一项的左边放置复选框。
	ComboBoxControl	表示 Windows 组合框控件。
	DataGridViewControl	在一个自定义的网格显示数据。
	DataSelectorControl	显示两个列表，并允许将项目从一个列表移动到另一个。
	DateTimePickerControl	表示一个 Windows 控件，该控件允许用户选择一个日期和一个时间和显示的日期和时间与指定的格式。
	GroupBoxControl	表示一个 Windows 控件，该控件显示围绕一组控件的框架具有可选标题。

	LabelControl	表示标准 Windows 标签。
	ListBoxControl	表示一个 Windows 控件显示项目列表。
	ListViewControl	表示 Windows 列表视图控件，该控件可用四种不同视图之一显示项的集合。
	MaskedTextBoxControl	使用掩码进行区分用户的时如是否正确。
	MonthCalendarControl	表示一个 Windows 控件，该控件使用户能够通过可视月历选择一个日期。
	NumericUpDownControl	表示 Windows 显示框（也称为 up-down 控件）显示数值。
	PanelControl	使用控件集合进行分组。
	PictureBoxControl	表示 Windows 图片框控件用于显示图像。
	RadioButtonControl	使用户能够从一组 Radioutton 控件选项中选择一个选项。
	RichTextBoxControl	表示 Windows 富文本框控件。
	TextBoxControl	表示 Windows 文本框控件。
	TreeViewControl	显示标记项的分层集合。

所有控件，除了 DataSelectorControl，是标准的 Windows.Forms 控件的一个完整的模拟。该元素的名称已经得到了控件后缀，以避免重名。因此，Report 的 ButtonControl 与标准 Button 控件相对应。

在代码中引用控件

可以通过使用其名称引用控件：

```
TextBoxControl1.Text = "my text";
```

事实上，Report 的控件只是标准.Net 控件的一个封装。它封装了标准控件的很多属性，但不是全部。如果你需要一些不是右 Report 控件实现的属性，您可以按以下方式访问一个封装好的控件：

- 使用 stem.Windows.Forms.Control 类型的“Control”属性：

```
(TextBox1.Control as TextBox).ShortcutsEnabled = false;
```

- 使用具有相同名称的属性作为控件本身，但没有“Control”后缀。例如，TextBoxControl 已经获取了“TextBox”属性，这是 System.Windows.Forms.TextBox 类型，并返回封装好的 TextBox 控件：

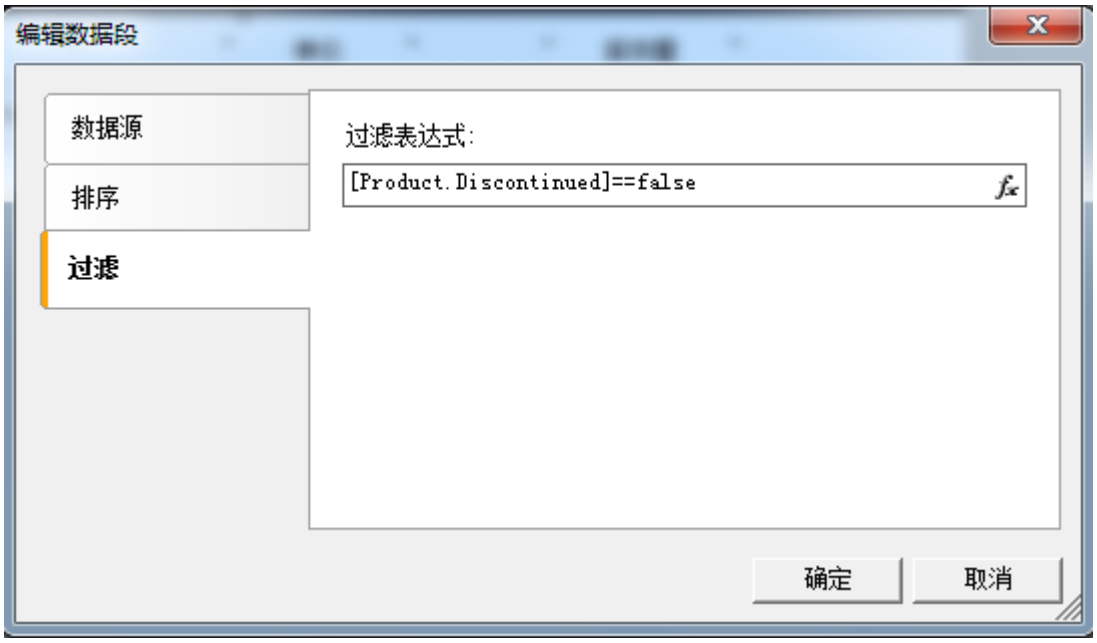
```
TextBox1.TextBox.ShortcutsEnabled = false;
```

数据过滤

对话框可以用来过滤在报表中打印的数据。例如，你有一个报表，该报表打印所有员工的列表。通过使用一个对话框，可以从中选择一个或多个员工，然后建立该报表时，数据被过滤，以便只显示所选择的员工。

为了使用数据过滤，必要的是在初始报表中包含所有数据。命名为“过滤”，假定，建立报表时不打印不必要的数据。

组织数据过滤最简单的方法是使用“数据”区域上的过滤属性。在编辑数据段中，你可以标示过滤表达式，例如：



通过使用对话框，你可以从一个用户获取一个值，并在过滤表达式中使用它。“示例”部分中看看“简单过滤器”的例子。

如果需要一个简单的值，可以使用这个方法。如果该任务是要显示值列表和查询其中的一个或多个，实施此变得困难。你可能会认为，这是一个简单的任务 - 在 ListBoxControl 控件元素中显示员工列表，并选择一个或多个值。为了实现这一点，你需要使用脚本，该脚本将执行以下操作：

- 通过名称获取数据源；
- 初始化数据；
- 用数据源中的数据填充 ListBoxControl；
- 选择员工后，创建将在“数据”区域中使用的过滤器表达式。

自动过滤-它是如何工作的

控件通过使用“DataColumn”属性连接数据字段。如果该控件可以显示值（例如，ListBoxControl）的列表，它从指定的数据字段填充值。这在对话框显示时是自动发生的。此外，用户使用对话框，在控件中选择一个或多个选项，然后关闭对话框。在这个时候，在“DataColumn”属性中标示的数据源将被自动过滤。

这种方法的优点是，你可以在任何报表中使用它，无需编写任何代码。

以下控件支持自动过滤：

Icon	Name
	CheckBoxControl
	CheckedListBoxControl
	ComboBoxControl
	DataSelectorControl
	DateTimePickerControl
	ListBoxControl
	MaskedTextBoxControl
	MonthCalendarControl
	NumericUpDownControl
	RadioButtonControl
	TextBoxControl

过滤器的操作

默认情况下 Report 筛选包含等于控件值的数据行。这种行为是在控件的“FilterOperation”属性中定义。您可以使用以下操作：

Operation	Equivalent	Effect
Equal	=	过滤值，如果它等于控件的值。
NotEqual	<>	过滤值，如果它不等于控件的值。
LessThan	<	过滤值，如果它小于控件的值。
LessThanOrEqual	<=	过滤值，如果它小于或等于该控制项的值。
GreaterThan	>	过滤值，如果它大于控件的值。
GreaterThanOrEqual	>=	过滤值，如果它大于或等于该控制项的值。

例如，如果控件的“FilterOperation”属性设置为“LessThanOrEqual”你在控件中输入值 5，那么所有相应字段的值小于或等于 5 的数据行都将被选择。

对于“字符串”类型的数据，可以使用额外的操作：

Operation	Effect
Contains	过滤值，如果它包含控件的值。
NotContains	过滤值，如果它不包含控件的值。

StartsWith	如果它从控件的值开始则筛选值。
NotStartsWith	如果它不从控件的值开始则筛选值。
EndsWith	过滤值，如果它以控件的值结束。
NotEndsWith	过滤值，如果它不以控件的值结束。

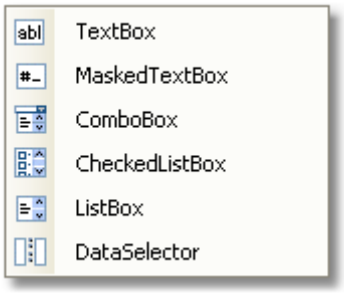
例如，如果控件的“FilterOperation”属性设置为“StartsWith”并输入“A”，那么，所有的相应数据字段中以“A”开始的值的数据行，将被选择。

在报表中添加过滤

Report 中的对话框设计器已经得到了非常舒适的设施，添加支持数据过滤的控件。对于这一点，从“数据”窗口拖放数据字段到对话框。在此期间，Report 创建的报头（LabelControl 控制）和实际用于数据过滤的控件。



控件类型取决于数据字段的类型。如果字段是字符串类型，将其插入后，您将被要求选择控件类型：



如果你已经插入了两个类似的控件，并连接到相同的数据字段，Report 在“FilterOperation”属性的帮助下自动配置数据范围。第一控件将有 FilterOperation=GreaterThanOrEqual，第二个 - LessThanOrEqual。这将在你插入的字段不是字符串类型时完成。

因此，添加数据过滤到任何报表，你需要做到以下几点：

- 在报表中添加对话框；
- 将要过滤报表的字段拖放到对话框。

过滤数据范围

这种过滤方法便于操作具有定量特性的值，例如成本。你可以过滤成本小于或大于给定值的不同的商品。使用“FilterOperation”属性标示在元素中输入的值。

使用两个控件，连接到相同的数据字段，并在“FilterOperation”属性中进行不同的设置，您可以标示开始和结束的数据范围。对于第一个控件，你需要指明 FilterOperation= GreaterThanOrEqual，第二个- LessThanOrEqual。

关联数据字段的过滤

我们知道，两个数据源之间可以设置关联。请参阅“数据”一章了解更多细节。在关联的帮助下，就可以通过从不同的数据源使用一个数据字段，在数据源中过滤数据，。

假设，你已经在对话框中放置了一个 ListBoxControl，并在“DataColumn”属性中标示下列数据字段：

Products.Categories.CategoryName

如何筛选工作？

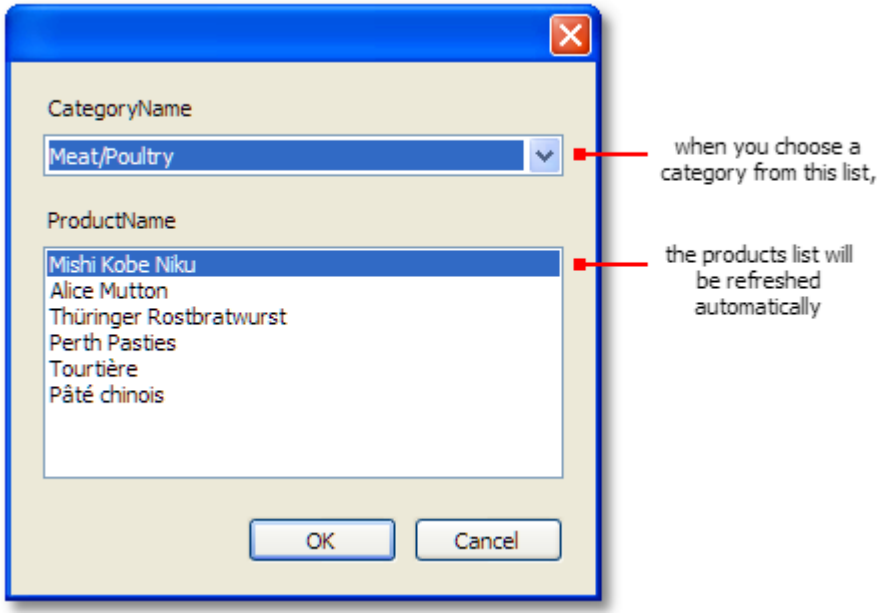
- 用值填充控件时，“类别”数据源的“类别名称”字段将被使用；
- 该过滤器将被应用到“产品”表。如果下列条件正确这些数据行会被过滤：

the [Products.Categories.CategoryName] contains one of the values, selected by the user

-

使用级联列表过滤

级联列表的选择的更给基于用户在另一个列表中选择的值。例如，你有两个列表在表单上- 一个是类别，另一个是产品。当您在第一个列表中选择类别，你会在第二个列表看到所选类别的产品。



要创建级联列表，你需要使用两个具有主-从关系的数据源。将主列表附加到主数据源中的一个字段，详细列表附加到详细数据源中的字段。设置主列表的“DetailControl”属性到详细列表。

以代码控制过滤

即使大多数情况下自动过滤已经足够了，你有也可以手动管理它。为此，使用下面的方法和属性。

在“自动填充”属性控制控件与数据填充。它用于控制显示值的列表，例如，ListBoxControl。显示对话框之前，Report 用数据填充这种控件。默认情况下，该属性设置为 true。如果它被禁用，控件将不会被填充，你必须自己做，通过调用“FillData”的方法：

```
ListBox1.FillData();
```

在“自动筛选”属性控制数据过滤。它被用于所有控件。对话框通过单击“OK”按钮关闭后，Report 自动应用数据过滤。默认情况下，该属性设置为 true。如果它被禁用，不会发生过滤，此时你必须自己做，通过调用“筛选数据”的方法：

```
ListBox1.FilterData();
```

范例

例 1."Hello, Report!"

在这个例子中，我们将会做到- 创建一个对话框，展示问候语。创建一个新的报表，并添加一个对话框。为此，点击工具栏上的  按钮：



在该对话框中，放置一个 LabelControl 并在“属性”窗口设置它的“Text”属性：



如果您运行的报表，你会看到对话框。点击“确定”按钮关闭，该报表将被创建。如果对话框通过点击“取消”按钮或“X”按钮关闭，该报表将停止工作，并返回到设计器。

例2.　请求用户输入一个文本

在这个例子中，我们将创建一个对话框，它会请求用户输入任意文本，然后在报表中打印输入的值。

创建一个新的报表，并添加一个对话框。在对话框中，放置 LabelControl 和 TextBoxControl 控件：



在给定的情况下，我们输入的值已经包含在 TextBoxControl 的“文本”属性。为了在报表中打印此值，在“报表标题”区域添加一个新的“文本”对象，并在其上执行以下操作：

```
You have entered: [TextBox1.Text]
```

其中 TextBox1 是 TextBoxControl 控件的名称。

例 3.处理对话框控件

通过使用控件的脚本和事件，您可以像在 Visual Studio 中一样处理控件。我们将通过实例表明，该 CheckboxControl 如何处理 TextBoxControl 可访问性。

创建一个新的报表，并在报表中添加一个对话框。在对话框中，放置 CheckBoxControl 和 TextBoxControl 控件，如下面的图：



现在选择 CheckBoxControl，打开“属性”窗口，然后单击按钮。双击“CheckedChanged”事件，这在改变复选框的状态时触发。 Report 中会为该事件创建一个空的处理程序。在其中编写以下代码：

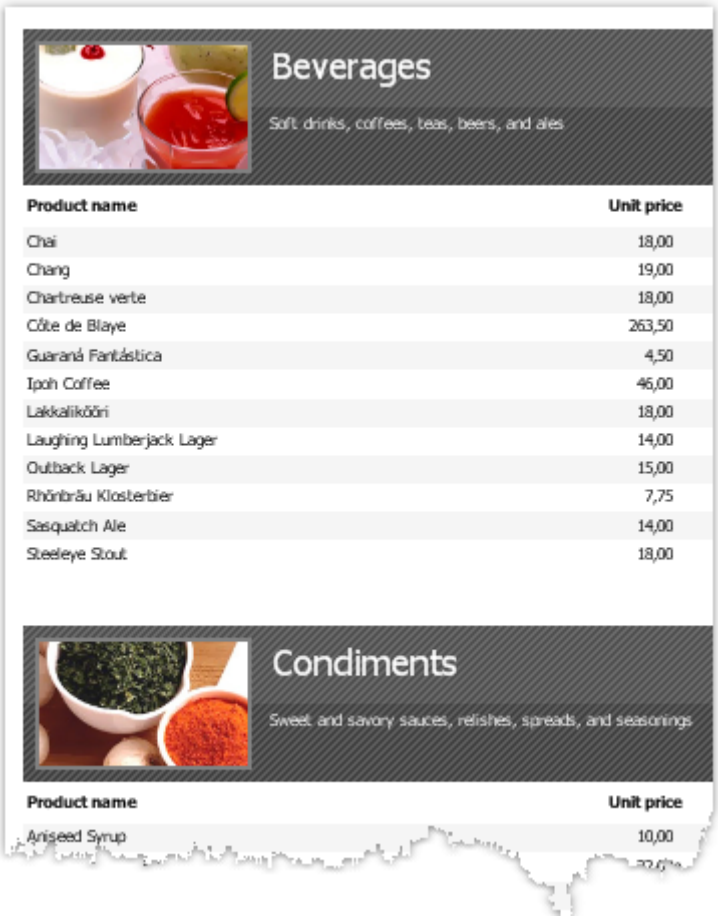
```
private void CheckBox1_CheckedChanged(object sender, EventArgs e)
{
TextBox1.Enabled = CheckBox1.Checked;
```

}

如果我们运行报表，我们可以通过复选框启用或禁用 TextBoxControl。

例 4.处理报表对象

让我们看一个报表，它打印类别和每个类别的产品列表的一个例子：



我们将展示如何在对话框的帮助下停止打印产品和只打印类别。对于这一点，在报表中添加一个对话框：



双击“OK”按钮。 Report 创建一个空的“单击”事件的事件处理程序，编写以下代码：

```
private void btnOk_Click(object sender, EventArgs e)
{
```

```
Data2.Visible = CheckBox1.Checked;
}
```

我们将控制打印产品清单区域的可见性。在我们的例子，这是一个名为“Data2”的区域。如果运行报表，并且该复选框未选中，我们将有以下结果：



例5. 简单过滤

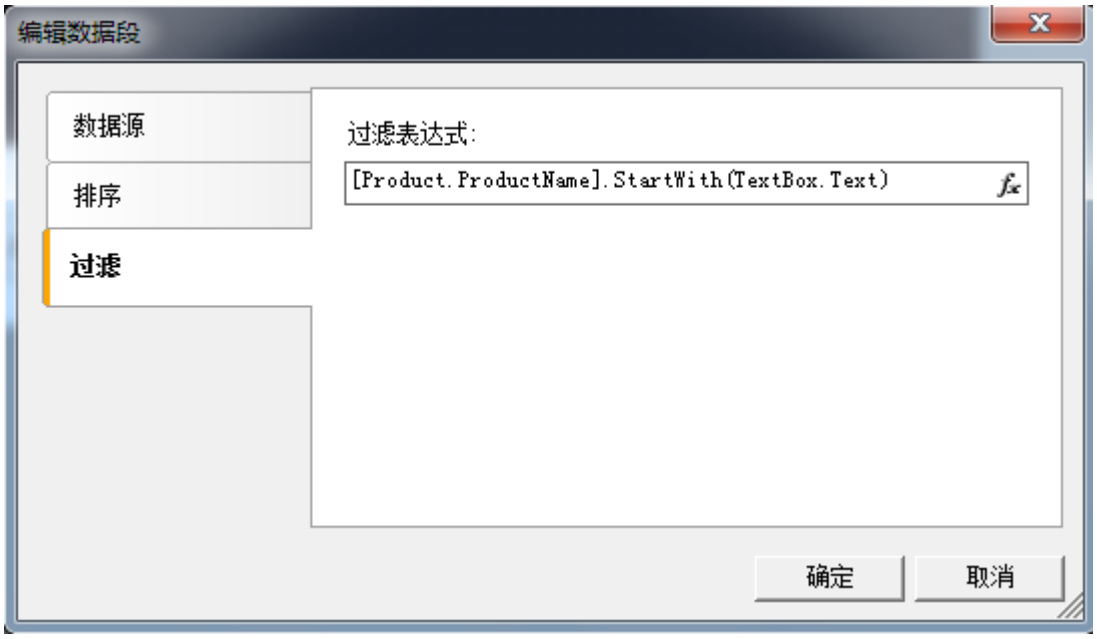
让我们看看下面的报表，它打印一个产品列表：



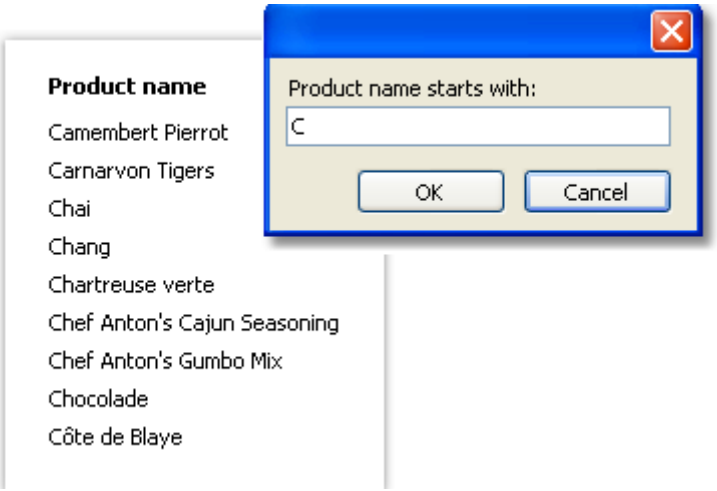
我们将在这个例子展示如何按产品名称的第一个字母筛选产品列表。在此期间，我们将不使用自动的数据过滤设施。为此，在报表中添加一个新的对话框，并放置两个控件 – LabelControl 和 TextBoxControl：



现在，打开“数据”区域编辑器和标示以下过滤器表达式：



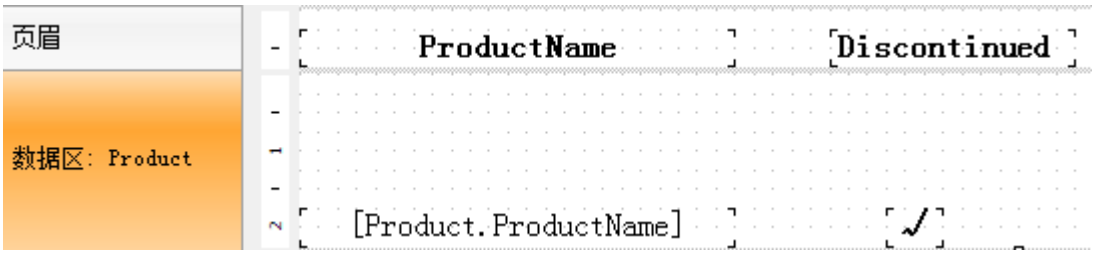
运行报表，并确保一切正常：



例6. 自动过滤

在这个例子中，我们将展示如何在报表中添加一个从“产品”表打印产品清单的过滤器。我们将根据“Products.Discontinued”字段筛选数据。

报表如下：



按工具栏上的 按钮在报表中添加一个新的对话框，然后从“数据”窗口中拖放“Products.Discontinued”字段到对话框：



这是我们需要做的 - 我们只需两次点击。 Report 自动将控件连接到数据列。

运行报表，并启用 Discontinued 标签。在此之后，点击“ＯＫ”按钮，你会看到只包含产品与 Discontinued 标签的报表：

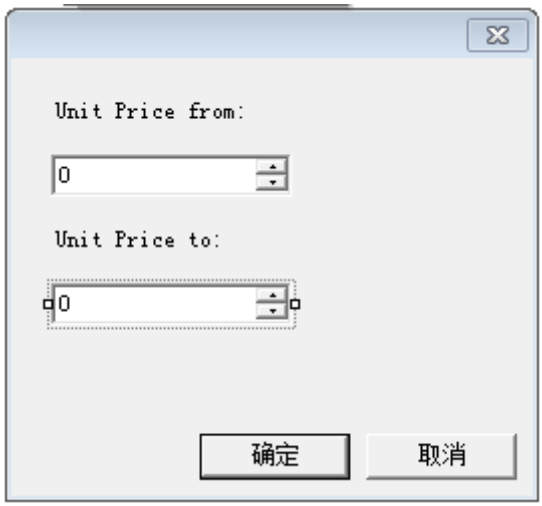
Product name	Discontinued
Alice Mutton	✓
Chef Anton's Gumbo Mix	✓
Guaraná Fantástica	✓
Mishi Kobe Niku	✓
Perth Pasties	✓
Rössle Sauerkraut	✓
Singaporean Hokkien Fried Mee	✓
Thüringer Rostbratwurst	✓

例 7.通过范围自动过滤

我们将展示从前面的例子报表中，如何打印其在指定的范围内的低成本产品。为此，我们将添加一个对话框到报表，并拖动“Products.UnitPrice”数据字段到对话框。在那之后我们将校正标签文字：



现在，以同样的方式，增加一个“Products.UnitPrice”字段，并校正其标题：



这就是我们需要做的，Report 会完成的工作的其余部分：连接控件到数据字段，并设置其 FilterOperation 属性。第一控件设置 FilterOperation=GreaterThanOrEqual，第二个 - LessThanOrEqual。

运行报表，并标示值，例如从 20 上升到 30。按下“ＯＫ”按钮时，报表将被创建。它包含指定范围内的值的产品。

Product name	UnitPrice
Chef Anton's Cajun Seasoning	22,00
Chef Anton's Gumbo Mix	21,35
Fløtemysost	21,50
Grandma's Boysenberry Spread	25,00
Gravad lax	26,00
Gustaf's Knäckebröd	21,00
Louisiana Fiery Hot Pepper Sauce	21,05
Maxilaku	20,00
Nord-Ost Matjeshering	25,89
Pâté chinois	24,00
Queso Cabrales	21,00
Sirop d'érable	28,50
Tofu	23,25
Uncle Bob's Organic Dried Pears	30,00

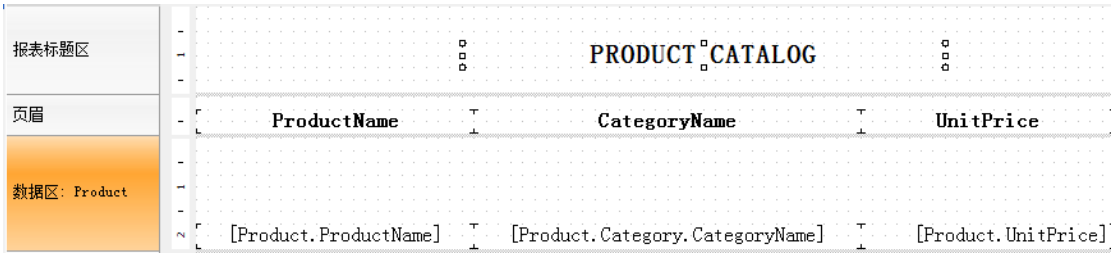
例 8 通过关联数据字段过滤

在这个例子中，我们将使用与相关数据源关联的字段进行数据过滤。

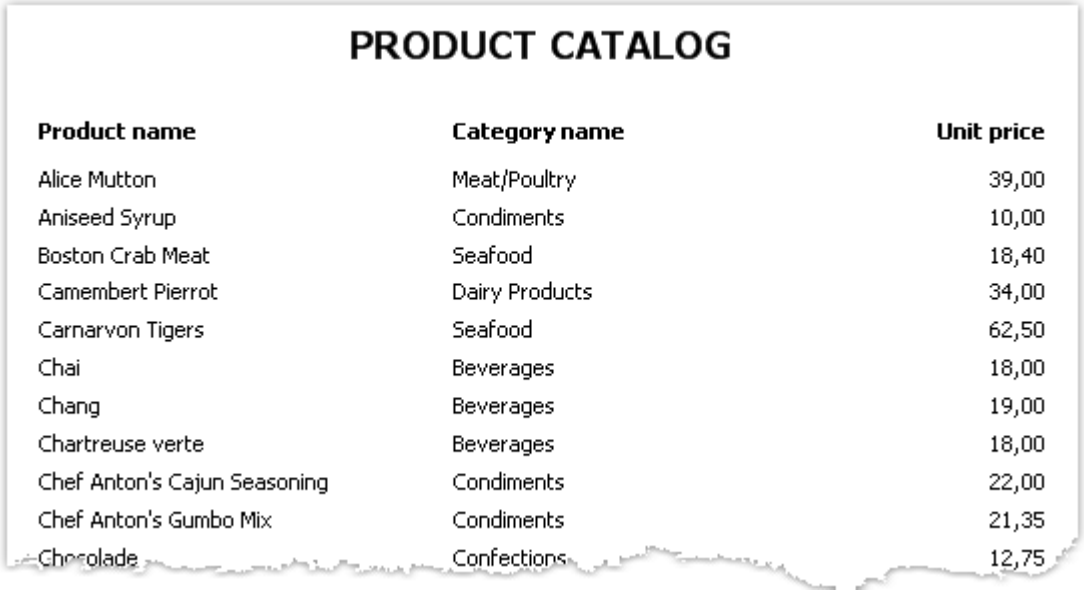
我们将着眼于一个“简单列表”报表类型，它打印产品清单。分类名称打印在每个产品旁边。这在关联的帮助下完成：

```
[Products.Categories.CategoryName]
```

该报表将如下：



当我们运行报表，我们将看到以下内容：

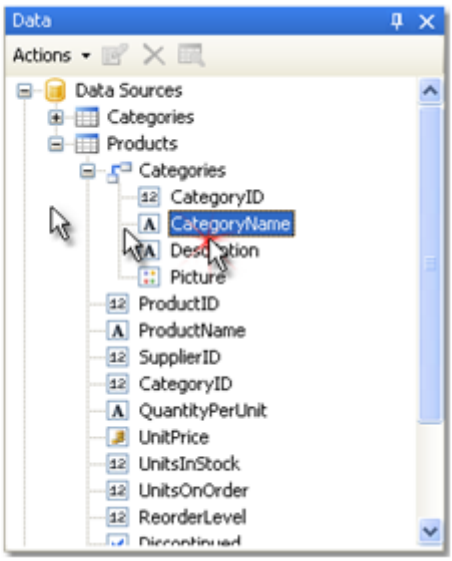


PRODUCT CATALOG

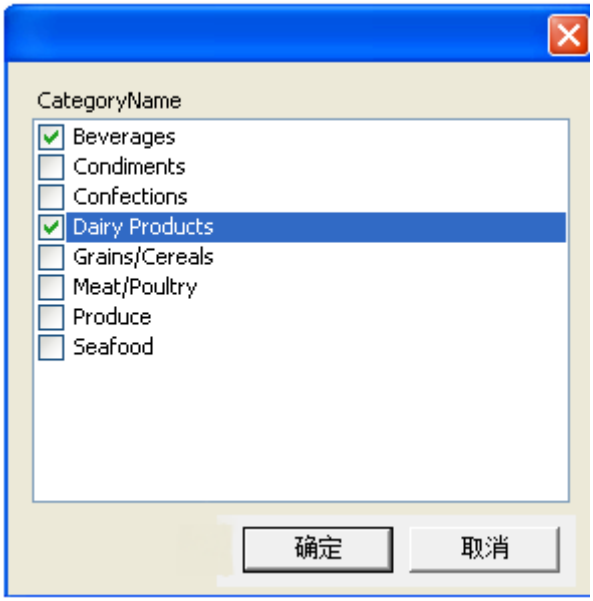
Product name	Category name	Unit price
Alice Mutton	Meat/Poultry	39,00
Aniseed Syrup	Condiments	10,00
Boston Crab Meat	Seafood	18,40
Camembert Pierrot	Dairy Products	34,00
Carnarvon Tigers	Seafood	62,50
Chai	Beverages	18,00
Chang	Beverages	19,00
Chartreuse verte	Beverages	18,00
Chef Anton's Cajun Seasoning	Condiments	22,00
Chef Anton's Gumbo Mix	Condiments	21,35
Chocolade	Confections	12,75

让我们按类别名称添加过滤。对于这一点，添加一个新的对话框，并在其中放置“Products.Categories.CategoryName”字段：

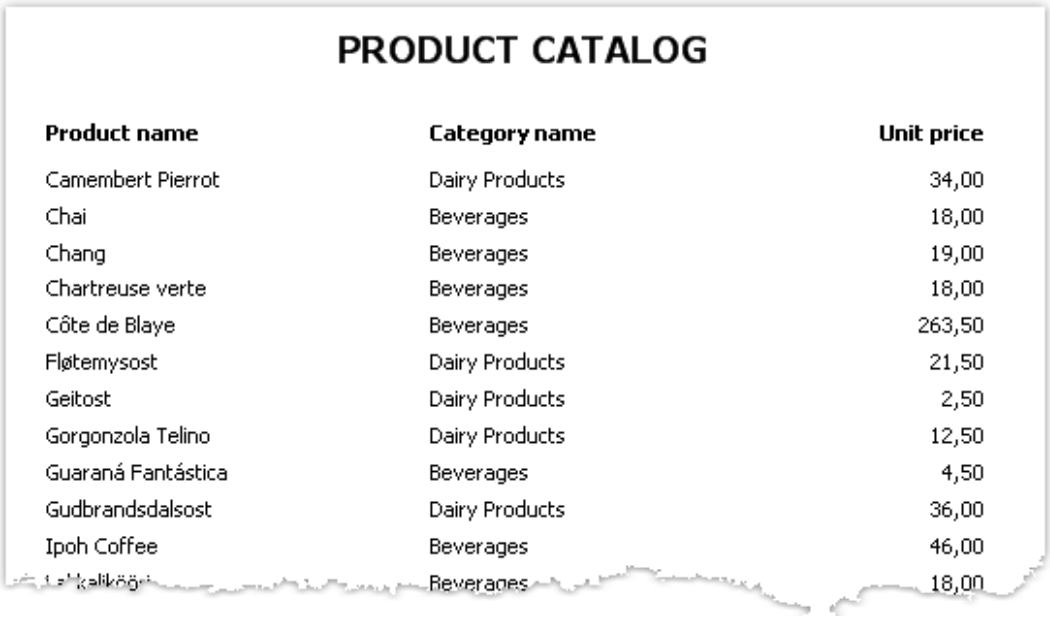




当创建控件，您将被要求选择其类型。选择 CheckedListBoxControl。如果我们运行报表，我们会看到如下的对话框：



选择几个类别，然后单击“ＯＫ”按钮。在此之后，数据将被过滤，您将看到以下报表：



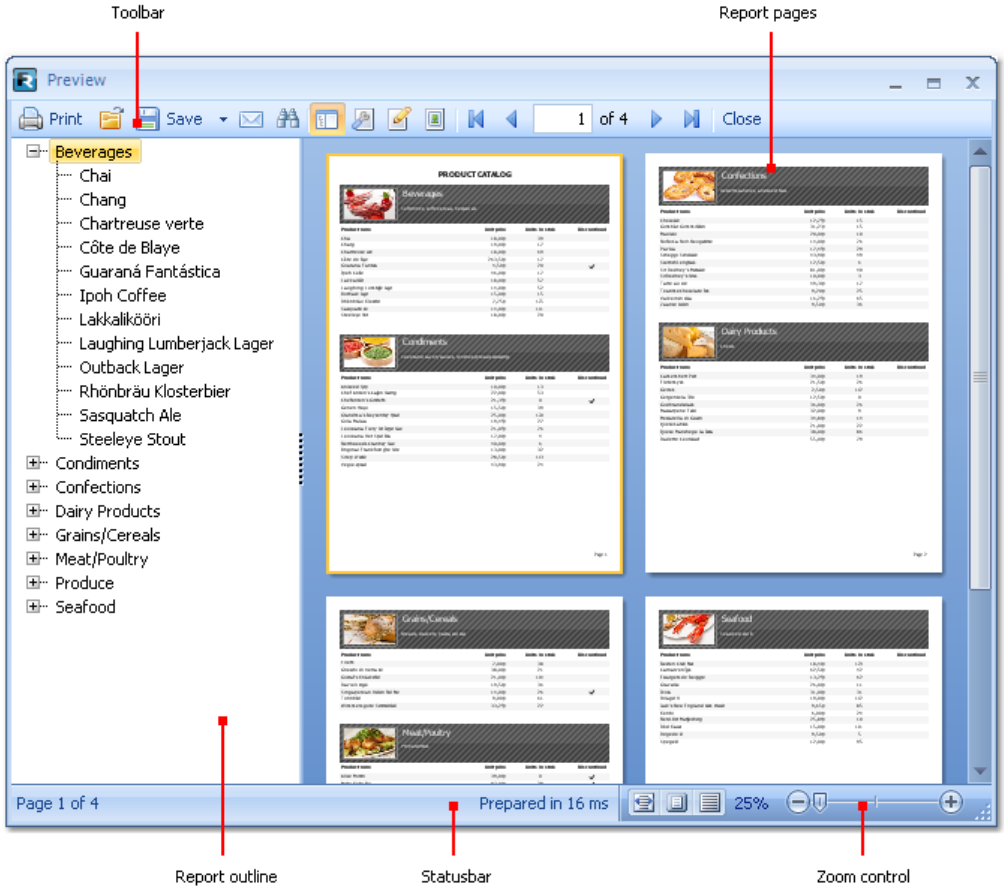
PRODUCT CATALOG

Product name	Category name	Unit price
Camembert Pierrot	Dairy Products	34,00
Chai	Beverages	18,00
Chang	Beverages	19,00
Chartreuse verte	Beverages	18,00
Côte de Blaye	Beverages	263,50
Fløtemysost	Dairy Products	21,50
Geitost	Dairy Products	2,50
Gorgonzola Telino	Dairy Products	12,50
Guaraná Fantástica	Beverages	4,50
Gudbrandsdalsost	Dairy Products	36,00
Ipoh Coffee	Beverages	46,00
Lakkalikööri	Beverages	18,00

可以看出，只显示选定类别的产品。

预览，打印，输出

内置报表可以显示在屏幕上，在打印机上打印或以支持的格式之一导出。所有这些都可以在预览窗口中完成：



在工具栏上，你可以找到以下按钮：



Button	Description
	打印报表
	以 PDF 格式打开报表
	以某种支持的格式保存报表
	通过邮件发送报表
	在报表中查询文本
	显示或隐藏大纲
	页面配置

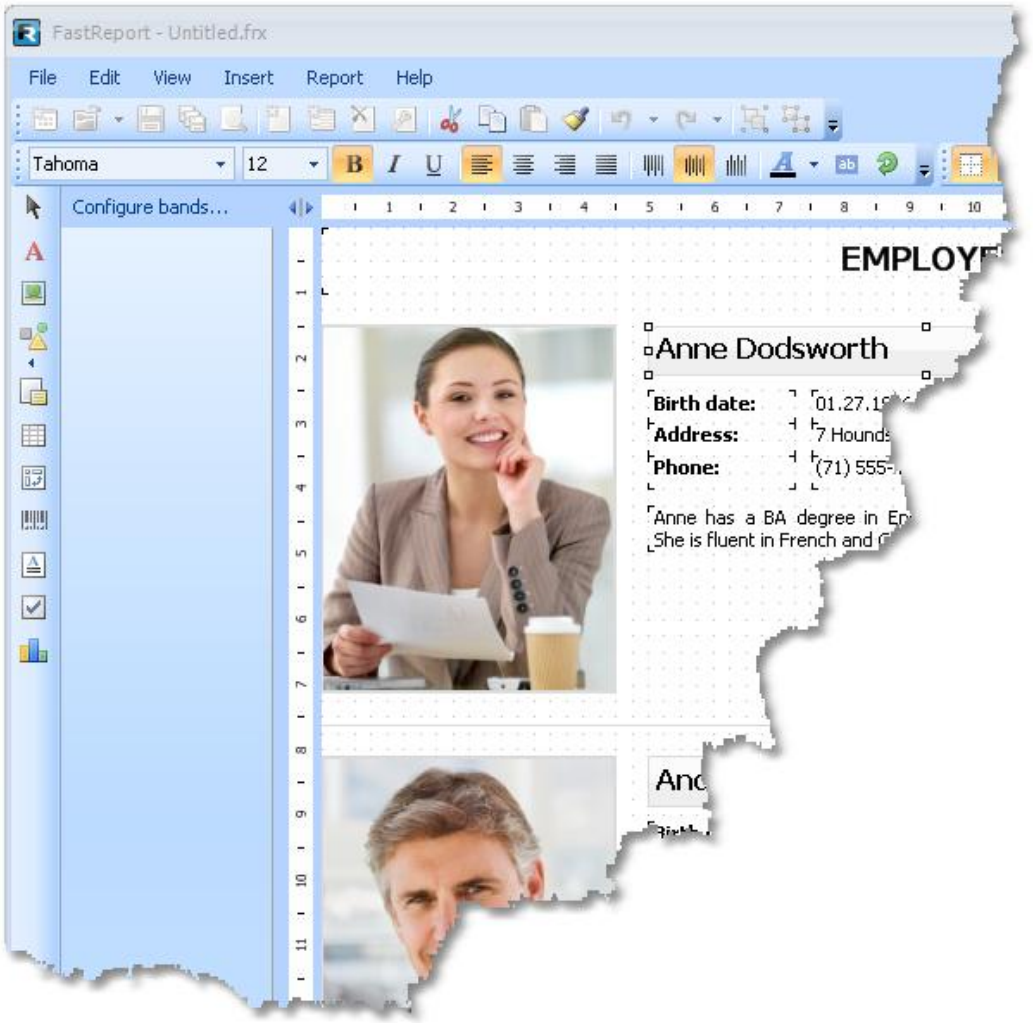
	编辑当前页面
	配置水印
	导航到首页
	导航到上一页
1	导航到指定页面。输入页码按 Enter 键.
	导航到下一页
	导航到最后一页

您可以使用下面的键盘控制：

Key	Description
Ctrl+P	打印报表
Ctrl+F	查询
Arrows	预览
PageUp, PageDown	Page up/down.
Home	导航到首页
End	导航到尾页
Esc	关闭预览窗口

编辑报表

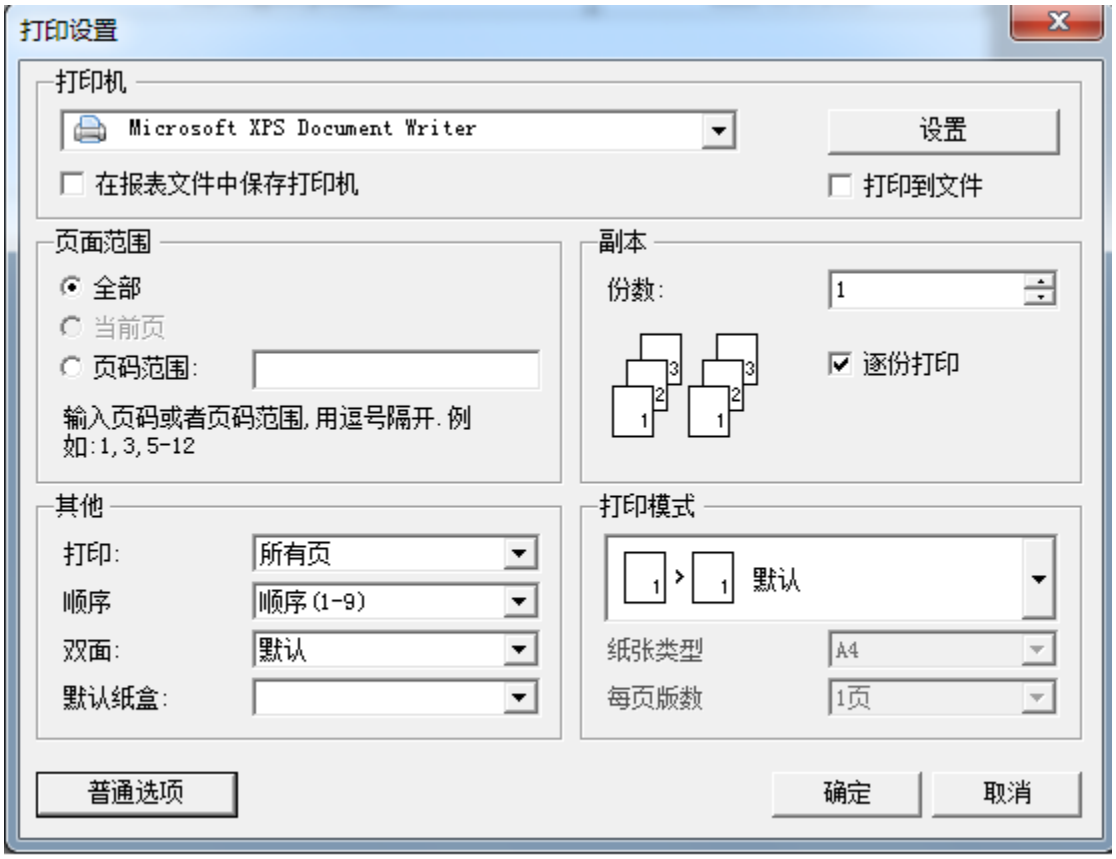
编辑准备好的报表的页面，单击预览窗口中的按钮。在此期间，当前页面将假造到报表设计器，在这里你可以做任何你想做的事情：



编辑完成后，关闭设计器。此时，你将被要求保存在报表页面所做的修改。

打印报表

为了打印一份报报表，单击按钮（或按 Ctrl + P 的组合）。你会看到打印的对话框：



我们将着眼于在该对话框访问的设置。按钮“更多/更少”允许显示整个对话框或只是基于设置的大小。默认情况下，将会显示一个简单形式的对话框。

“打印机”组：在这里，你可以选择打印机，更改其设置（“设置...”按钮），然后选择打印到文件。

“页”组：在这里你可以选择要打印的页面（全部，当前或指定的页码）。

“复制”组：在这里你可以设置打印份数，然后选择页面顺序的副本（“分页”）：

“其他”组：在这里你可以选择打印页面（所有，偶，奇），选择打印顺序（直接，反向），设置双面打印（如果您的打印机支持的话），然后选择纸张来源页面。

“打印模式”组允许选择打印模式之一：

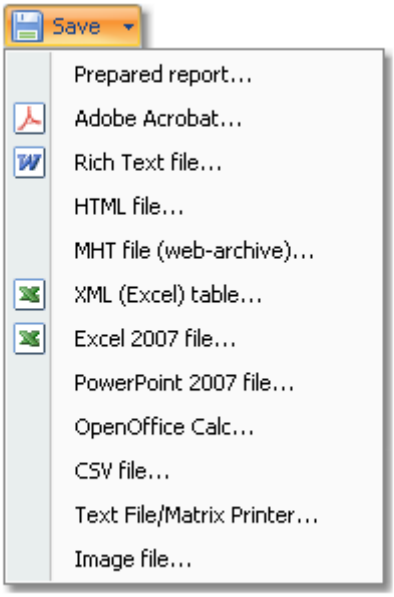
Mode	Description
	The printer prints on a paper, indicated in the report. One report page corresponds with one printed sheet.
	Use this mode, if you need to print A3 report on a A 4 format paper. One report page will produce two printed sheets. When using this mode, you have to choose the paper format from the "Print on sheet" list.

	Use this mode, if you need to print A4 report having on a A 3 format paper. On one printing sheet, you can print 1, 2, 4 or 8 report pages. When using this mode, you need to choose the format of the paper on which you want to print, from the "Printing on sheet" list, and also indicate the number of pages in the "Pages on sheet" list.

按下“打印”按钮后，该报表开始打印。如果“打印到文件”标志被选中，那么该文件的名称将被要求，该报表将保存在该文件（扩展名为 PRN 扩展名）。

输出报表

Report 允许以不同的格式输出报表。目前，支持 11 种格式：PDF，RTF，HTML，MHT，Excel（XML），Excel 2007，PowerPoint 2007，OpenOffice Calc 中，CSV，TXT，image file。为了选择导出，在预览窗口中，点击“Save”按钮，选择导出：



Saving in FPX format

FPX 格式是 Report 中的“native”格式。这种格式的优点如下：

- 保存报表不失品质。打开一个已保存的文件，你可以做所有的操作，就像打印，导出，编辑;
- 基于 XML 的压缩格式;
- 必要时，报表文件可以通过支持 ZIP 格式，并在任何文本编辑器中手动修改任何归档解压。

该格式唯一欠缺的是，预览时，你需要有 Report.Net。

为了以 FPX 格式保存，请在预览窗口单击“保存”按钮，选择“Prepared report“文件类型。对于已经打开保存的文件，点击“打开”按钮。

Export to Adobe Acrobat (PDF)

PDF（可移植文档格式）是由 Adobe Systems 创建的电子文档的独立平台格式。免费的 Acrobat Reader 包用于查看。这种格式相当灵活- 它允许包含必要的字体，矢量和位图图像;它允许传输和存储的查看和进一步印刷的文档。

导出方法是分层的。

当导出到 Excel，会出现一个对话窗口，设置输出文件的参数：



输出参数：

- “压缩”- 输出文件被压缩。它可以减少文件大小，但会增加输出时间;
- “包含字体文件”- 在一份报表中使用的所有字体将收录到 PDF 文件。这将显著增加文件的大小;
- “后台”- 页面水印将被导出为图像。这将显著增加文件的大小;
- “打印选项”- 所有的图形对象（如图片，图表），以高分辨率进一步打印输出。

在“信息”选项卡上，您可以填写文档信息字段：



在“浏览器”选项卡上，您可以设置与 Adobe Acrobat 文件浏览器相关的一些选项：

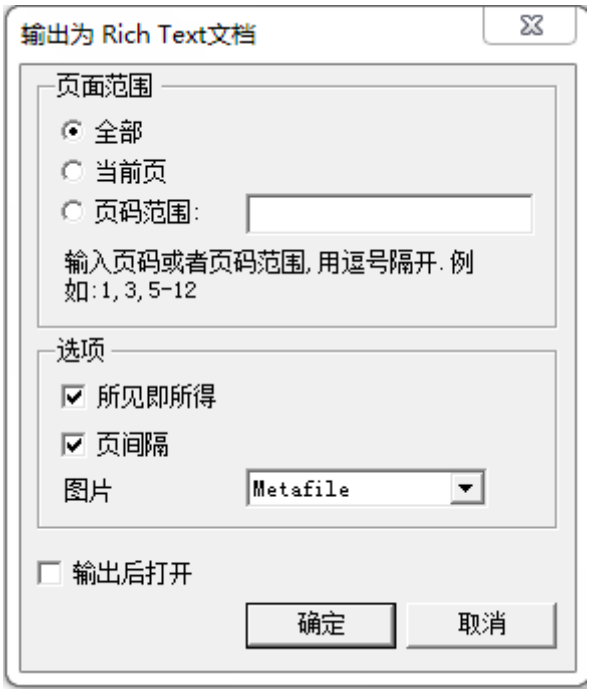


Export to Word (RTF)

RTF（Rich Text 格式）是由微软公司开发的，作为交换文本信息的标准格式。目前的 RTF 文档兼容许多新的文本编辑器和操作系统。

输出方法：表格。

当导出为 RTF，会出现一个对话窗口，设置输出文件的参数：



导出参数：

- “所见即所得”- 结果将尽量靠近报表。如果该选项被禁用，Report 的将减少由此而来的行和列的数目;
- “页间隔”- 在 RTF 文件中允许分页;
- “图片”- 在 RTF 文件中选择照片的格式。需要注意的是“图元文件”格式是最适合显示该报表的对象作为 MSChartObject 和 ShapeObject 的。

导出文件的外观和大小取决于报表模板。

Export to HTML

HTML（超文本标记语言）是 Web 页面的主要的标记语言。它提供了描述在文档中基于文本信息的结构的一种方法- 通过标示某些文本链接，标题，段落，列表，等等- 并以补充文字与互动的形式，嵌入图像和其他对象。

导出方法：表格。

当导出为 HTML，对话窗口将提示设置参数：



导出参数：

- “所见即所得”-导出的结果将尽可能接近该报表;
- “图片” - 允许导出图片;
- “子文件夹”- 所有多余的文件都保存在一个名为“文件”的单独的文件夹;
- “导航”- 创建用于页面导航的一个特殊的导航仪;
- “单页”- 所有的页面将被保存在一个文件中。

生成文件的外观和大小取决于报表模板。

Export to MHT (web archive)

MHT，MIME HTML 的简称，是用来绑定由外部链接的数据源（如图片，Flash 动画，Java 小程序，音频文件），连同 HTML 代码到一个文件中的资源的网页存档格式。一个 MHT 文件的内容进行编码，就好像它是一个 HTML 电子邮件，使用 MIME 类型多部分/相关。

导出方法：表格。

当导出为 MHT，对话窗口将提示设置参数：



导出参数：

- “所见即所得”-导出的结果将尽可能接近该报表；
- “图片” - 允许导出图片；

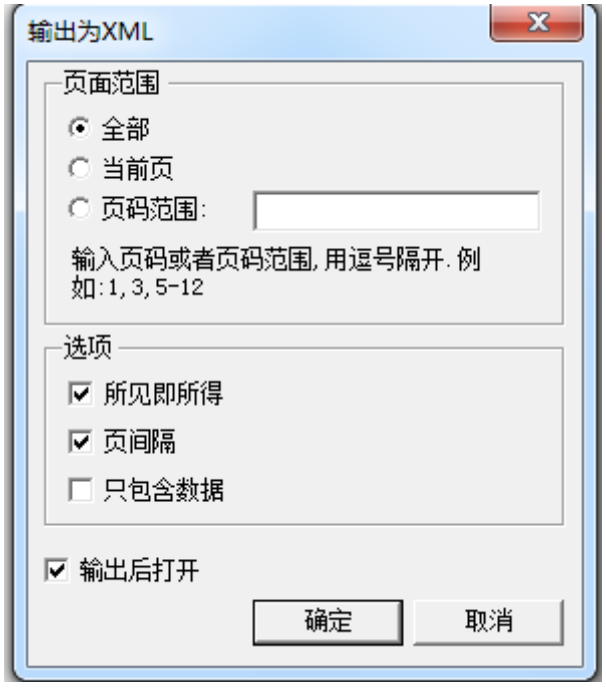
生成文件的外观和大小取决于报表模板。

Export to Excel (XML)

Excel 是用于电子工作表工作的应用程序。它包含于 Microsoft Office。

导出方法：表格。

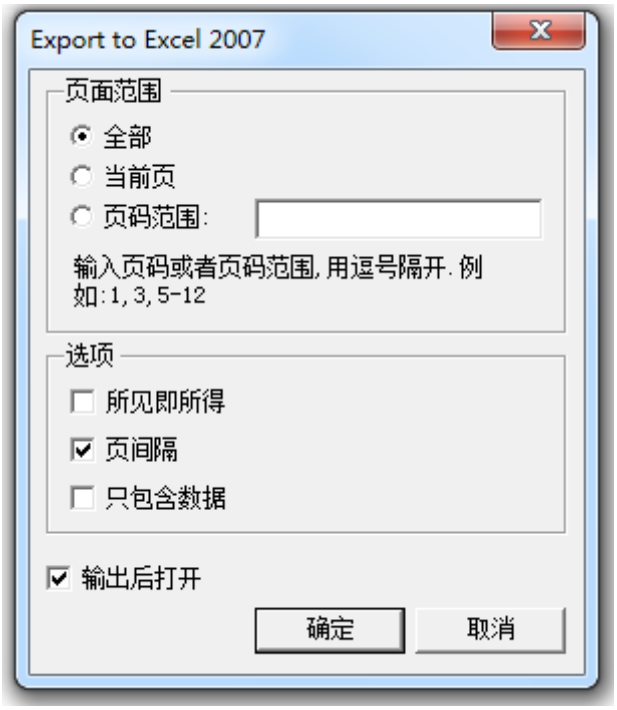
当导出到 Excel，会出现一个对话窗口，设置输出文件的参数：



- “所见即所得”- 结果将尽量靠近报表。如果该选项被禁用，Report 的将减少由此而来的行和列的数目;
- “分页符”- 在 RTF 文件中允许分页;

Export to Excel 2007

Excel 2007 是用于电子工作表工作的应用程序。它包含于微软 Office 2007。
导出方法：表格。
当导出到 Excel，会出现一个对话窗口，设置输出文件的参数：



- “所见即所得”- 结果将尽量靠近报表。如果该选项被禁用，Report 的将减少由此而来的行和列的数目;
- “页间隔”- 在 RTF 文件中允许分页;

Export to PowerPoint 2007

PowerPoint 2007 是电子演示工作的应用程序。它包含于微软 Office 2007。
导出方法是分层的。
当导出到 PowerPoint 时，会有一个对话窗口，设置输出文件的参数：



输出参数：

“图片”- 在生成的文件中选择图片的格式。

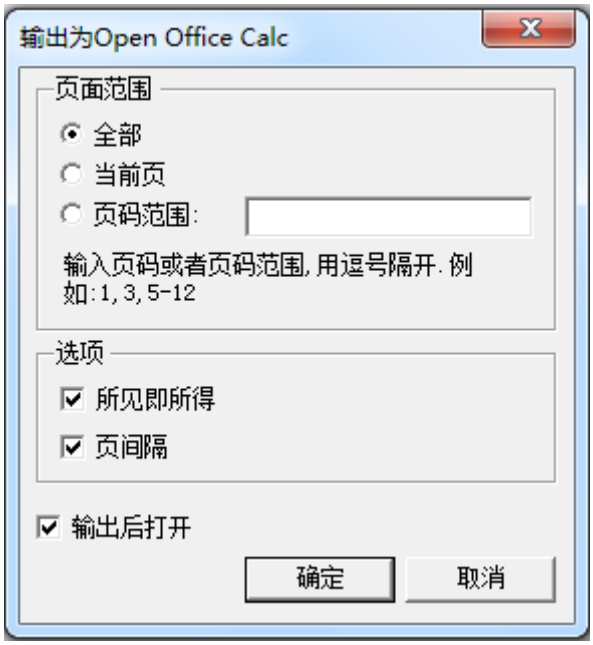
Export to OpenOffice Calc

OpenDocument 格式（ODF，OASIS 开放文档格式的 Office 应用程序）是由 OASIS 设计，基于在 OpenOffice 中使用的 XML 格式。

Report 支持导出表（ODS 文件）。这些文件可以在 OpenOffice 打开。

输出方法：表格。

当导出到 OpenOffice Calc 时，会有一个对话窗口，设置输出文件的参数：



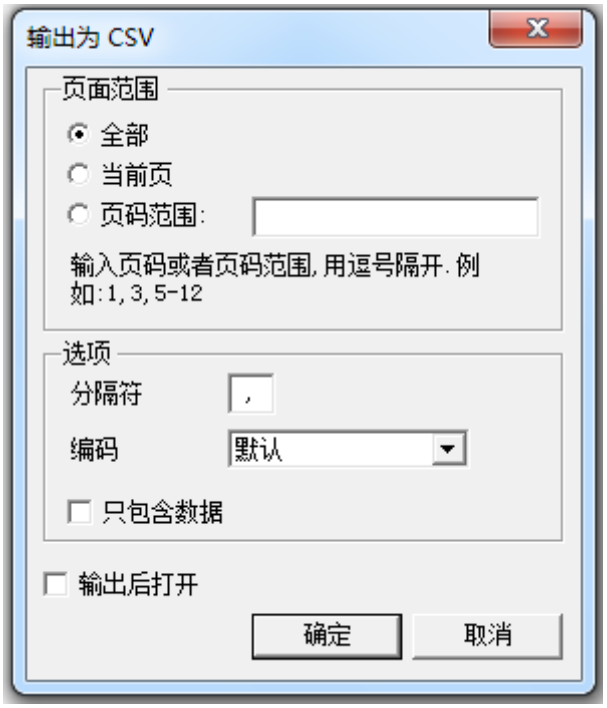
- “所见即所得”- 结果将尽量靠近报表。如果该选项被禁用，Report 的将减少由此而来的行和列的数目；
- “页间隔”- 在 RTF 文件中允许分页；

Export to CSV

CSV 文件是用于列表中的数据结构的数字存储。在 CSV 文件中的每一行对应一个表中的行。字符之间用逗号隔开，每个字符属于一个表字段。 CSV 文件常常被用于两个不同的计算机程序之间移动表格式的数据，例如数据库程序和电子表格程序。

输出方法：表格。

当导出 CSV 时，会出现一个对话窗口，设置输出文件的参数：



导出参数：

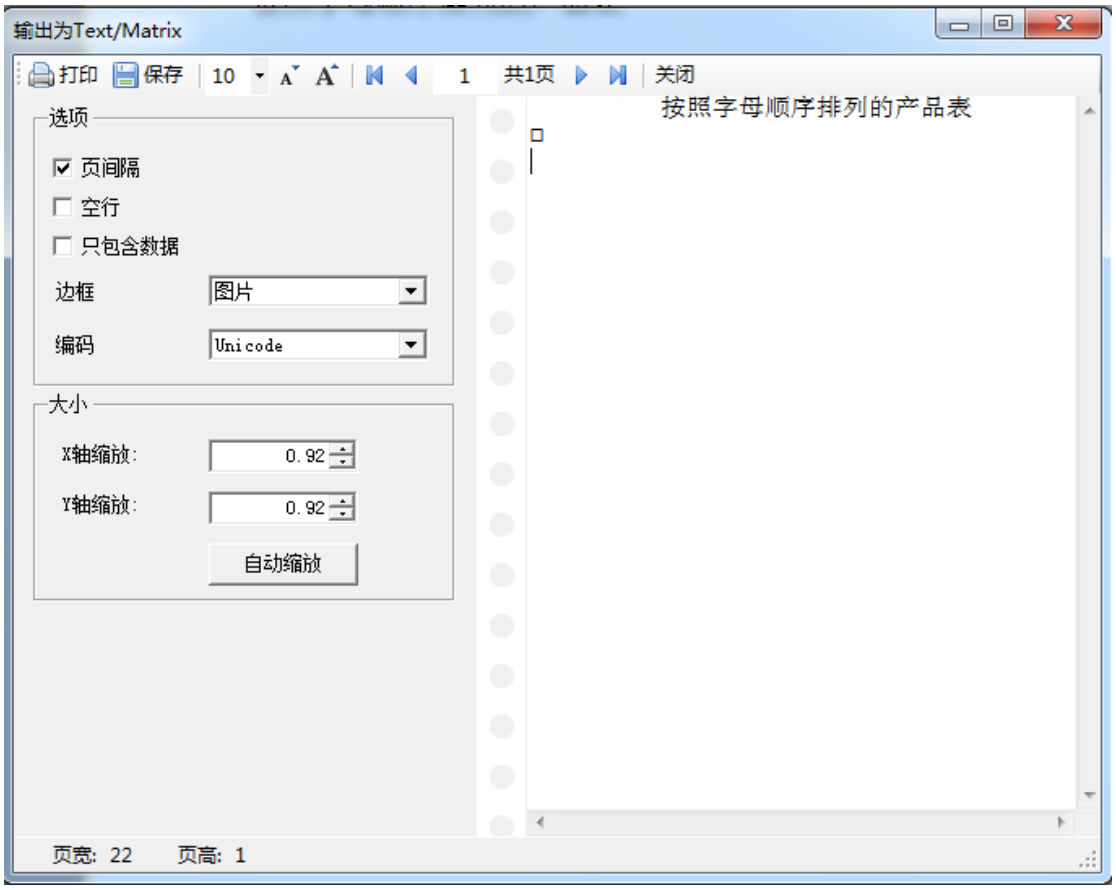
- “分隔符”- 字段分隔符;
- “编码”- 用于在生成的文件中的文本进行编码的代码页。 “默认”代码页是指 Windows 的默认代码页。需要注意的是 Excel 中不支持 Unicode 代码页;
- “只包含数据”-启用此复选框来导出数据区域上的数据。

Export to TXT

TXT 是可以在任何文本编辑器打开，或者打印 dot-matrix 的一个普通的文本文件。

输出方法：表格。

当导出为 TXT 时，会出现一个对话窗口，设置输出文件的参数：



导出参数：

- “分页符”- 在文件中允许分页；
- “空行”- 允许生成的文件中存在空行；
- “只包含数据”-启用此复选框来导出数据区域上的数据。
- “边框”- 对象的边框的类型。选择“无”，如果你不希望导出边界；
- “编码”- 用于在生成的文件中的文本进行编码的代码页；
- “X 轴缩放”- 水平刻度；
- “Y 轴缩放”- 垂直刻度；
- “自动缩放”- 自动计算 X 和 Y 比例以避免数据丢失。

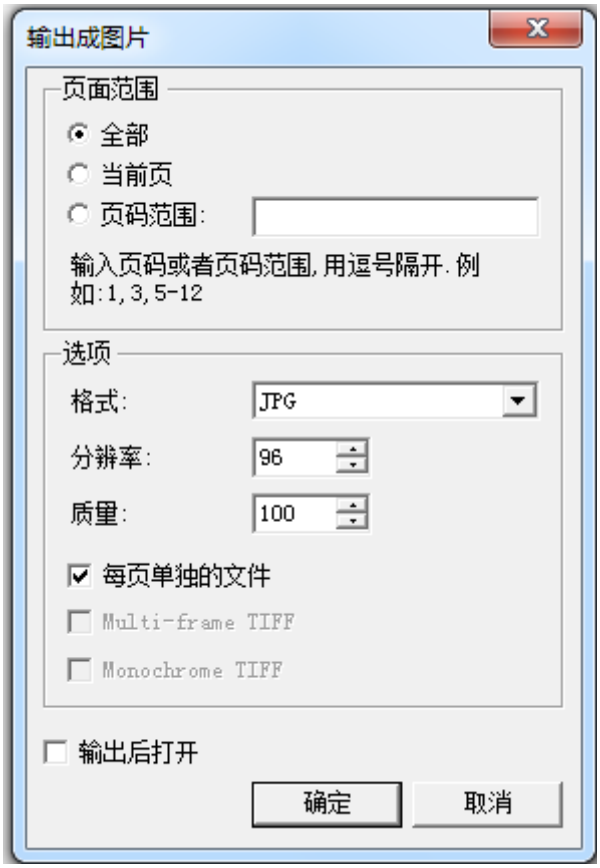
Export to picture

Report 中允许输出以下图形格式：

- BMP
- PNG
- JPG
- GIF
- TIFF
- Windows Metafile (EMF,WMF)

导出方法：图片。

当导出图片文件时，对话窗口将提示设置参数：



导出参数：

- “解决方案”- 图形图像的分辨率。使用 96DPI 显示，分辨率为 300dpi 的打印。当导出成 TIFF 格式，你将能够为水平和垂直分辨率设置不同的值;
- “JPEG 品质”- JPG 文件压缩级别。只有当导出格式为 JPEG 时，这个选项才可用;
- “独立文件的每一页”- 如果启用该选项，则每个报表页面将被导出到一个单独的文件中，该文件的名称将在所选择的页面与给定页码基础上形成;
- “多帧 TIFF”- 这个选项会产生多帧 TIFF 文件。只有导出格式为成 TIFF 时，它才可用;
- “单色 TIFF”- 这个选项会产生单色 TIFF 文件。只有导出格式为 TIFF 时，它才可用。

当在一个文件中输出多页时，导出将占用大量的 CPU/内存资源。

Report design recommendations

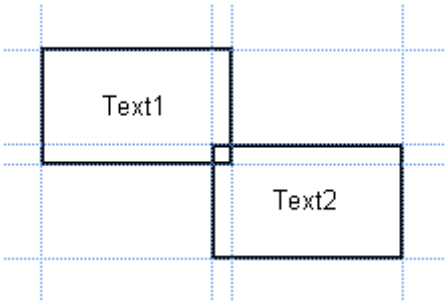
在本章，我们将讨论导出成其他数据格式的特殊设计要求。

Report 允许在创建报表的过程中操纵对象一个很大的数字。这是任何报表及其进一步的印刷快速发展的优势。打印的文档看起来就像展出。这是 Report 报表生成器使用的主要意图。

这种自由发展的不足之处是将 Report 文件以不同的数据格式导出的复杂性，它们有自己的限制和信息，有时是相当复杂的。多种格式，如 HTML，XLS 或 RTF，使用表中的数据呈现。这些格式不允许表格标记时单元格交叉和分类。

导出过滤器，作为一项规则，需考虑到这些要求。这是通过特殊的算法，并优化它们放置。在交叉对象中，有新的列和行在输出结果表出现。这是必要的用于获取结果和原始报表之间的最大相似性。报表设计中大量的交叉对象，导致了输出表的行和列的数量增加，并影响文件大小和它的复杂性。

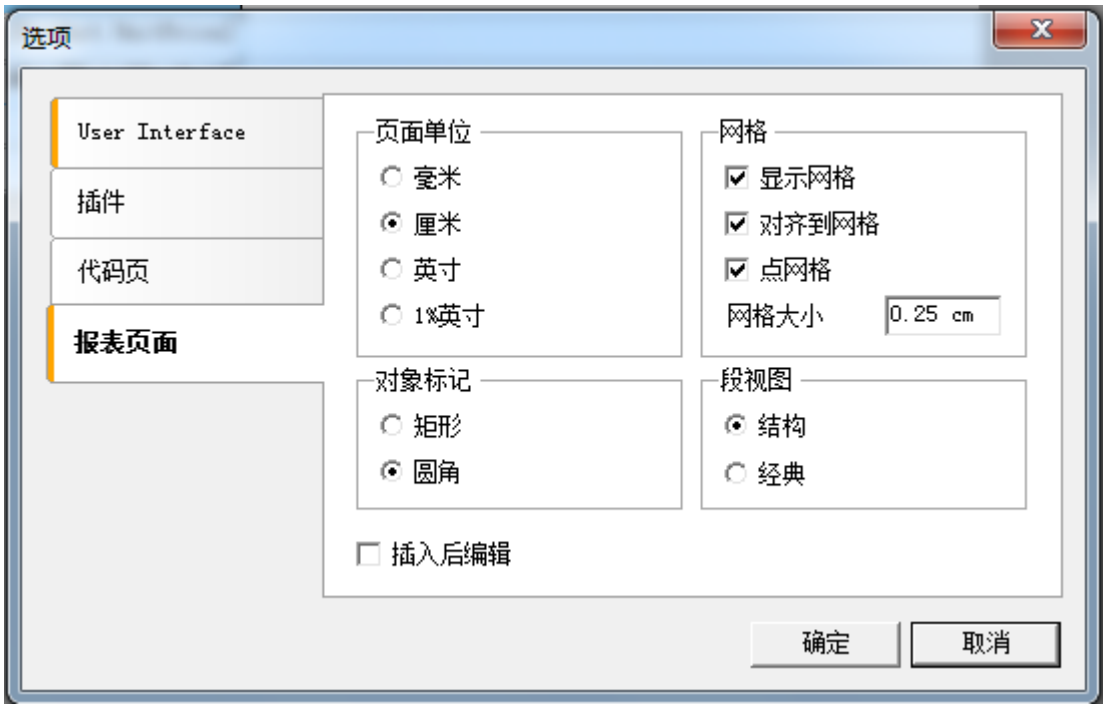
导出的质量在很大程度上取决于初始报表的设计。让我们看看下面的例子：



同一区域内的两个对象略有交叉。形成报表的记录数是 150。输出格式为 RTF 时，将创建 450 行。如果我们除去交叉，输出表中将只有 300 行。在大型报表和对象数量巨大的差异将是非常巨大的。当然，这会影响输出文件的大小。

当在报表中创建表时，要注意相邻单元格的边界。单元格不交叉，并按层次排序是非常重要的。导出过滤算法将切断单元格，但输出的结果可能是不可取的。对象被放置在垂直线或水平排列。向导可以帮助执行此操作。

网格对齐也助于防止单元格重叠。在设计器选择网格对齐。为了简化定位可以延长网格间距。设置网格间距和对齐可以在“视图|选项...”中操作：



对于文本框架最好是使用文本对象的边框，而不是单一的图形对象，如直线，矩形等，尽量不要在透明的文本对象上使用背景对象。

应用这些简单的规则将帮助你创建一个报表，当它以任意基于表格的形式输出时，会看起来很完美。

通过邮件发送报表

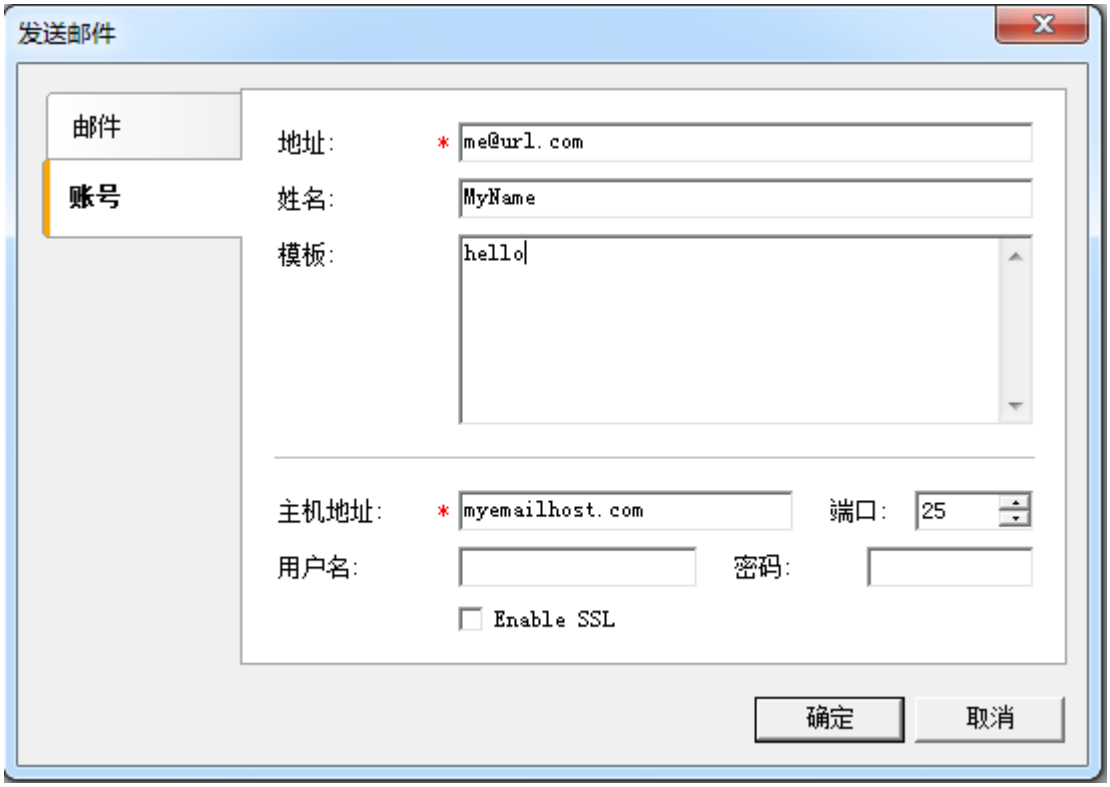
Report 允许你通过电子邮件发送准备好的报表。它可以两种模式工作：

Mode	Description
SMTP	这是默认模式。要发送电子邮件，你不需要任何外部程序。
MAPI	你可以打开此模式，编程方式。要做到这一点，设置 Config.EmailSettings.UseMAPI= true， 如果你使用 EnvironmentSettings 组件，其 EnvironmentSettings.EmailSettings.UseMAPI 属性设置为 true。 要发送电子邮件,Report 中使用默认的电子邮件客户端如 Outlook Express。此客户端必须支持 MAPI 协议。

要发送电子邮件，您需要指定一个收件人的电子邮件地址。还需要指定主题和邮件正文，但这不是必需的。在对话框底部，选择报表的格式- 它会被附加到消息框：



如果您使用 SMTP 方式，您需要建立一个帐号。它只需一次。一旦你创建完成，Report 会将参数保存在配置文件中。该参数可以在“帐户”选项卡上找到。所有的必填字段标为红色星号：



如果你的主机服务器需要身份验证，你还需要填写“用户名”和“密码”栏位。